



EKS

Elastic Kubernetes service

AWS Elastic Kubernetes Service EKS

Deploy Amazon EKS on AWS with Terraform

PREPARED BY

SIDNEY SMITH EBOT

ebotsmith@gmail.com

Table of Contents

1. Introduction	3
1.1. Terraform	3
1.2. Architecture	3
2. Build a Terraform Module for Deploying Amazon EKS on AWS	3
3. Create an S3 Bucket in AWS Using Terraform	24
4. Configure S3 State Locking and Migrate Local Terraform State to Remote Backend	36
4.1. Configure S3 State Locking	36
4.2. Migrate Local Terraform State to Remote Backend	40
5. Provision a VPC with Terraform for Amazon EKS	49
5.1. Configuring the Variables – “variables.tf”	49
5.1.1. Adding Variable for Region	51
5.1.2. Adding Variable for VPC CIDR block	51
5.1.3. Adding Variable for Public Subnet CIDR	51
5.1.4. Adding Variable for Private Subnet CIDR	51
5.1.5. Adding Variable for Availability Zones	51
5.1.6. Adding Variable for EKS Cluster	51
5.2. Configuring the VPC and its components – “main.tf”	52
5.2.1. Setting up VPC Configuration Block	53
5.2.2. Setting up Public Subnets block	54
5.2.3. Setting up the Private subnets Block	58
5.2.4. Setting up the Internet Gateway Block	59
5.2.5. Setting up the Elastic IP Block	60
5.2.6. Setting up the NAT Gateway Block	62
5.2.7. Setting up the Route Tables Block	64
5.2.8. Setting up Private Route Tables Block	67
5.2.9. Setting up Private Route Table Association Block	69
5.2.10. Setting up Public Route Table Association Block	71
5.3. Configuring the Outputs – “outputs.tf”	73
5.3.1. Setting up the output of the VPC	73
5.3.2. Setting up the output of the Public Subnet	74
5.3.3. Setting up the Output of the Private Subnet	75
6. Set Up an EKS Cluster on AWS with Terraform	76
6.1. Configuring the Variables – “variables.tf”	78
6.1.1. Adding Variable for Region	78

6.1.2.	Adding Variable for VPC ID.....	78
6.1.3.	Adding Variable for Subnet.....	78
6.1.4.	Adding Variable for EKS Cluster Name.....	78
6.1.5.	Adding Variable for Cluster Version.....	78
6.1.6.	Adding Variable for Node Groups.....	78
6.2.	Configuring the EKS Cluster – “main.tf”	79
6.2.1.	Define IAM Roles.....	79
6.2.2.	Create Control Plane.....	85
6.2.3.	Worker Nodes.....	88
6.3.	Setting the Outputs – “outputs.tf”	89
7.	How to Call and Use Terraform Modules from the Root Directory.....	91
7.1.	Defining Input Variables in the Root “variables.tf” file.....	91
7.2.	Configuring the Terraform Infrastructure with “main.tf”	92
7.3.	Configuring the Input Variables with “terraform.tfvars” file.....	95
7.4.	Defining Module Outputs in the Root directory – “outputs.tf” file.....	97
8.	Deploy an Amazon EKS Cluster with Terraform.....	98
8.1.	Terraform Init.....	98
8.2.	Terraform Format	99
8.3.	Terraform Validate.....	100
8.4.	Terraform Plan	100
8.5.	Terraform Apply.....	101
9.	How to Access and Connect to Your AWS EKS Cluster.....	102
9.1.	Access the EKS Cluster created	102
9.2.	Connect to the EKS cluster.....	104

1. Introduction

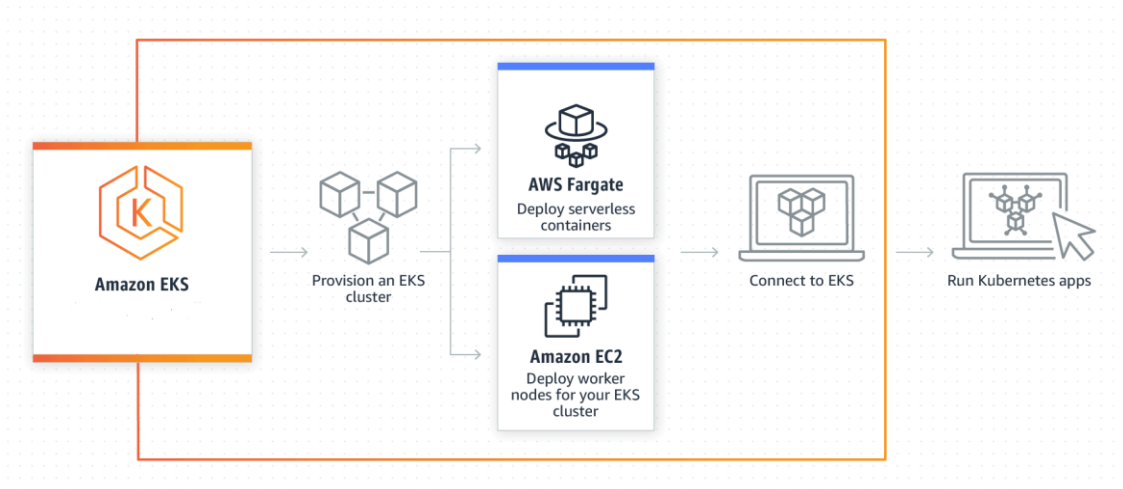
Amazon EKS (Elastic Container Service for Kubernetes) is a managed service that is used to run Kubernetes on AWS without the hassle of managing the Kubernetes control plane (Master Node).

Imagine, automating the deployment of a production grade Kubernetes environment for your CI/CD pipelines, microservice architecture or internal platform tools or with one terraform command.

In this tutorial, we will look at how to deploy an Amazon EKS cluster using Terraform. We will be using VSCode to write the code.

1.1. Terraform

1.2. Architecture

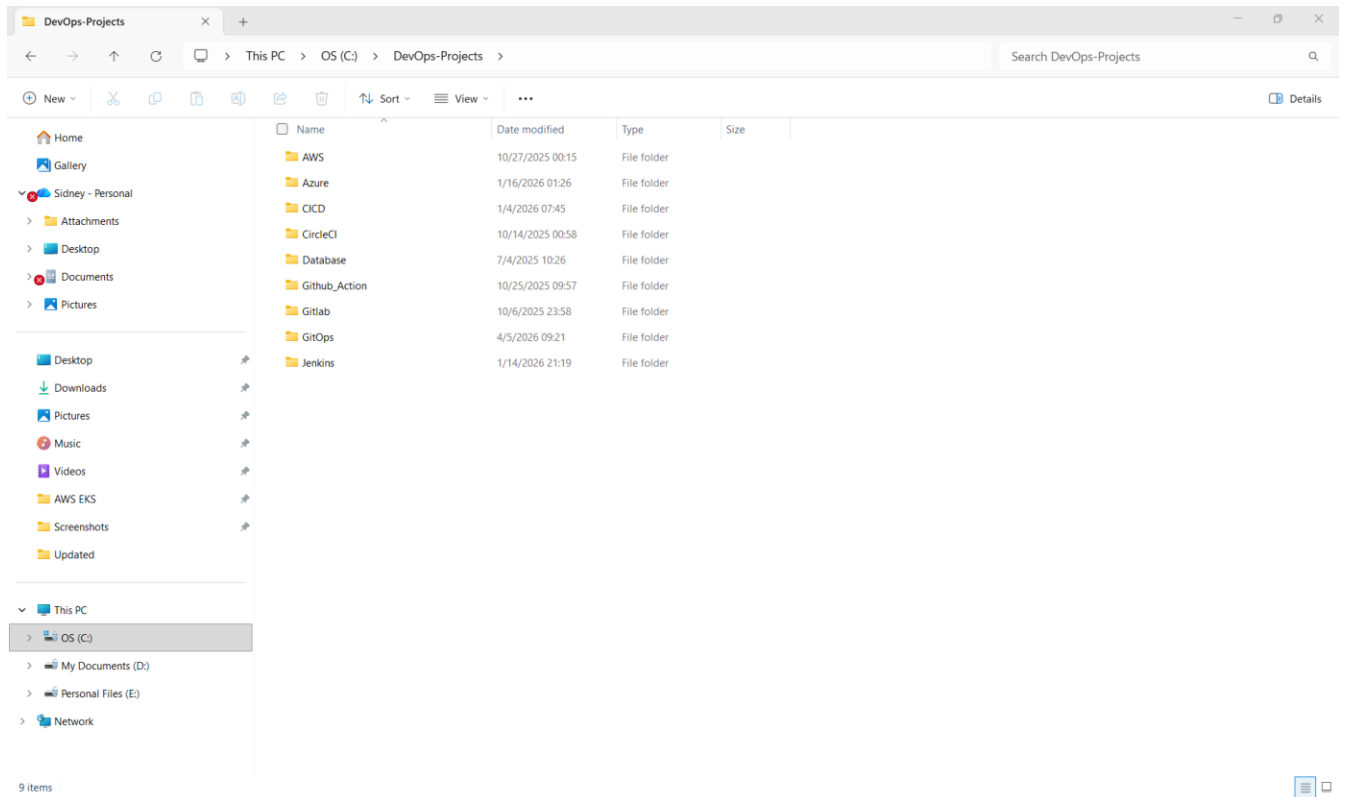


This is what happens when you deploy a Kubernetes cluster. You go to provision an EKS cluster. When you are provisioning the EKS cluster underlying infrastructure for the EKS cluster can either be a Fargate, which is serverless or you can have a Virtual Machine which is EC2 instance.

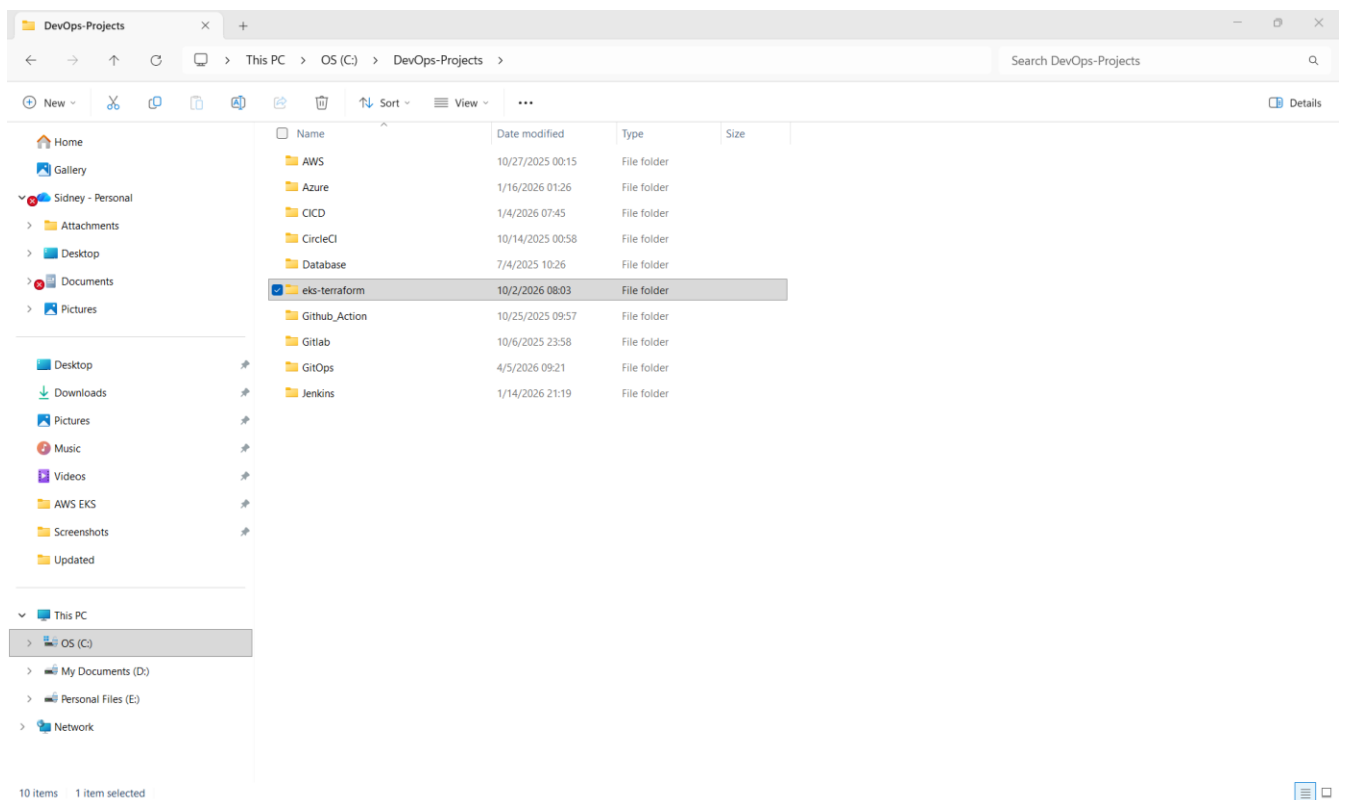
Once we have created a Master node or control plane and the data plane, then you connect to the EKS. And then deploy the applications.

2. Build a Terraform Module for Deploying Amazon EKS on AWS

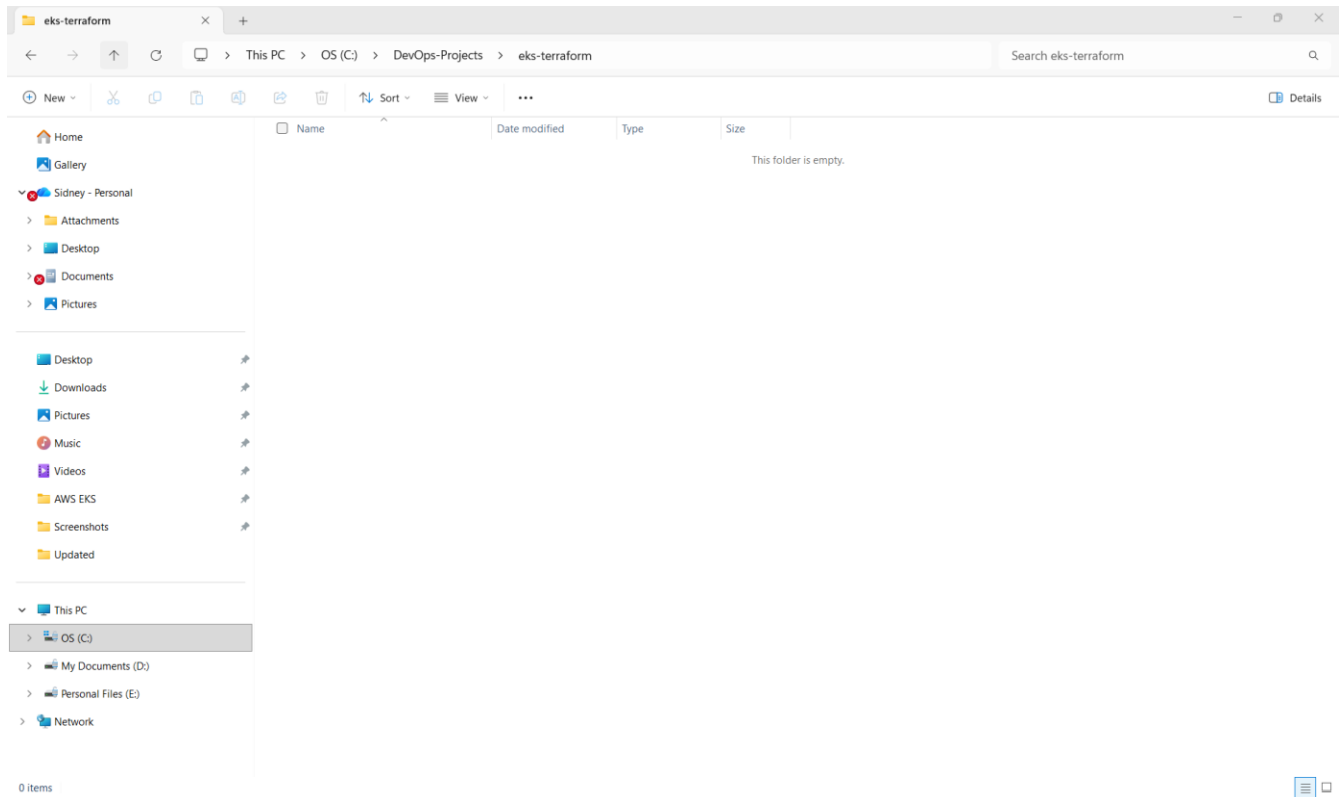
We will start by building terraform modules we will use in this demo. The codes will be written on VScode. We will start by creating the project folder at a suitable location.



We will create the folder here. We will call it **“eks-terraform”**

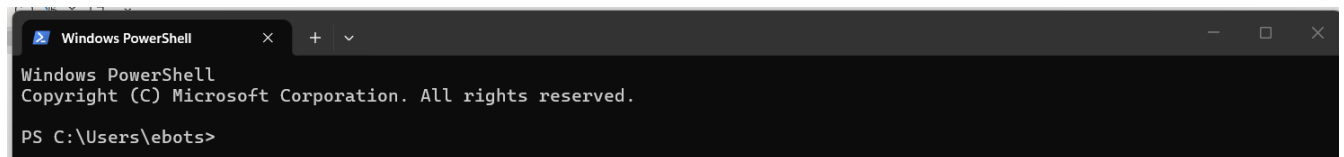


We will now open the created folder in VSCode. To do this, open the folder



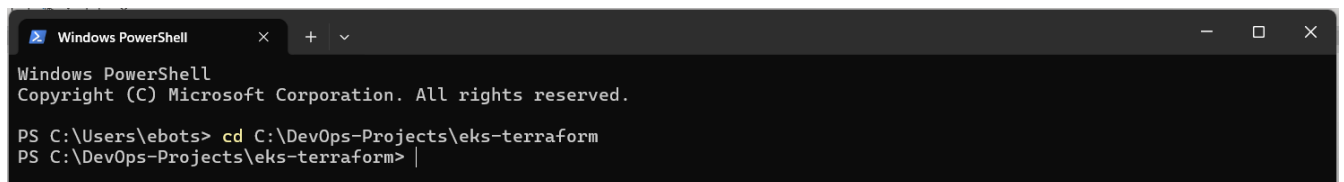
This is the path of the folder: C:\DevOps-Projects\eks-terraform

Open PowerShell command prompt



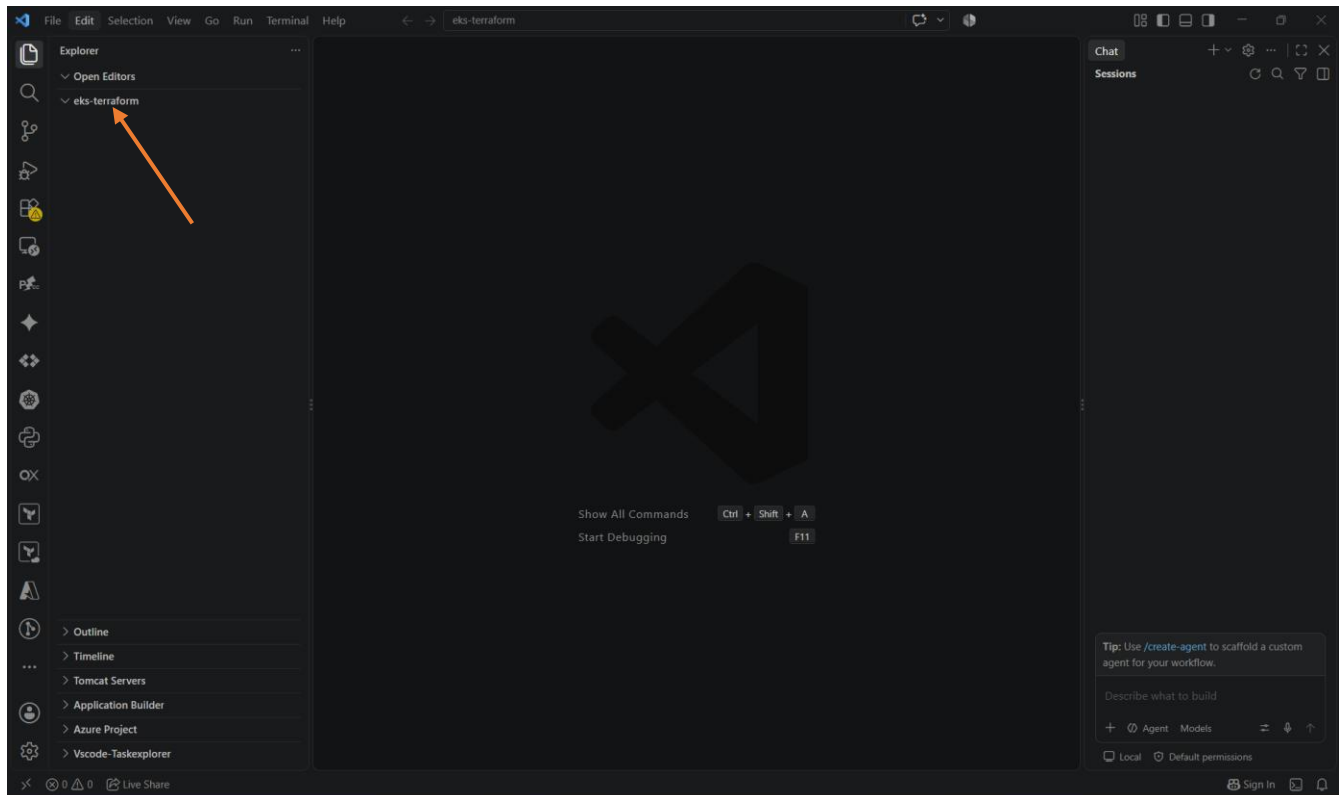
Navigate to the folder using the command:

`cd C:\DevOps-Projects\eks-terraform`



Then, run the command:

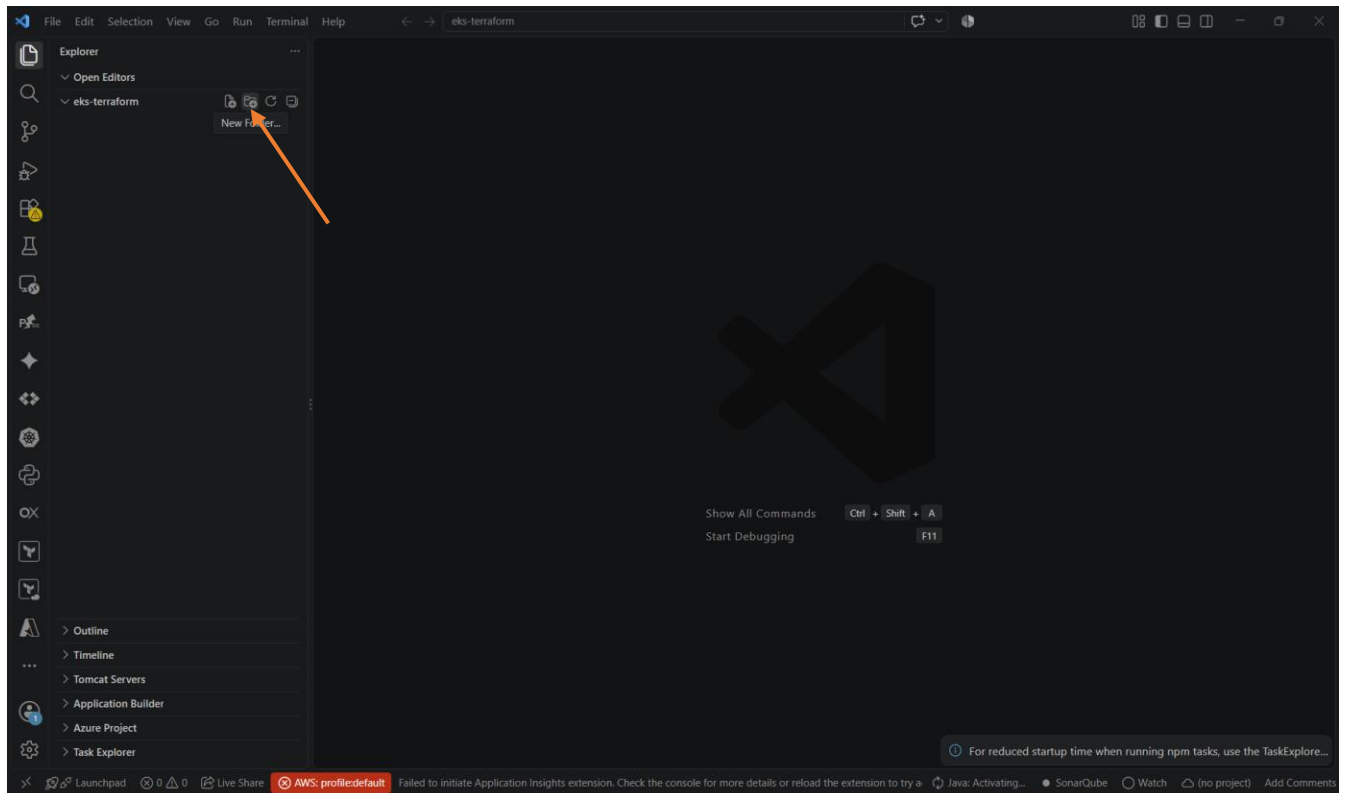
`code .`



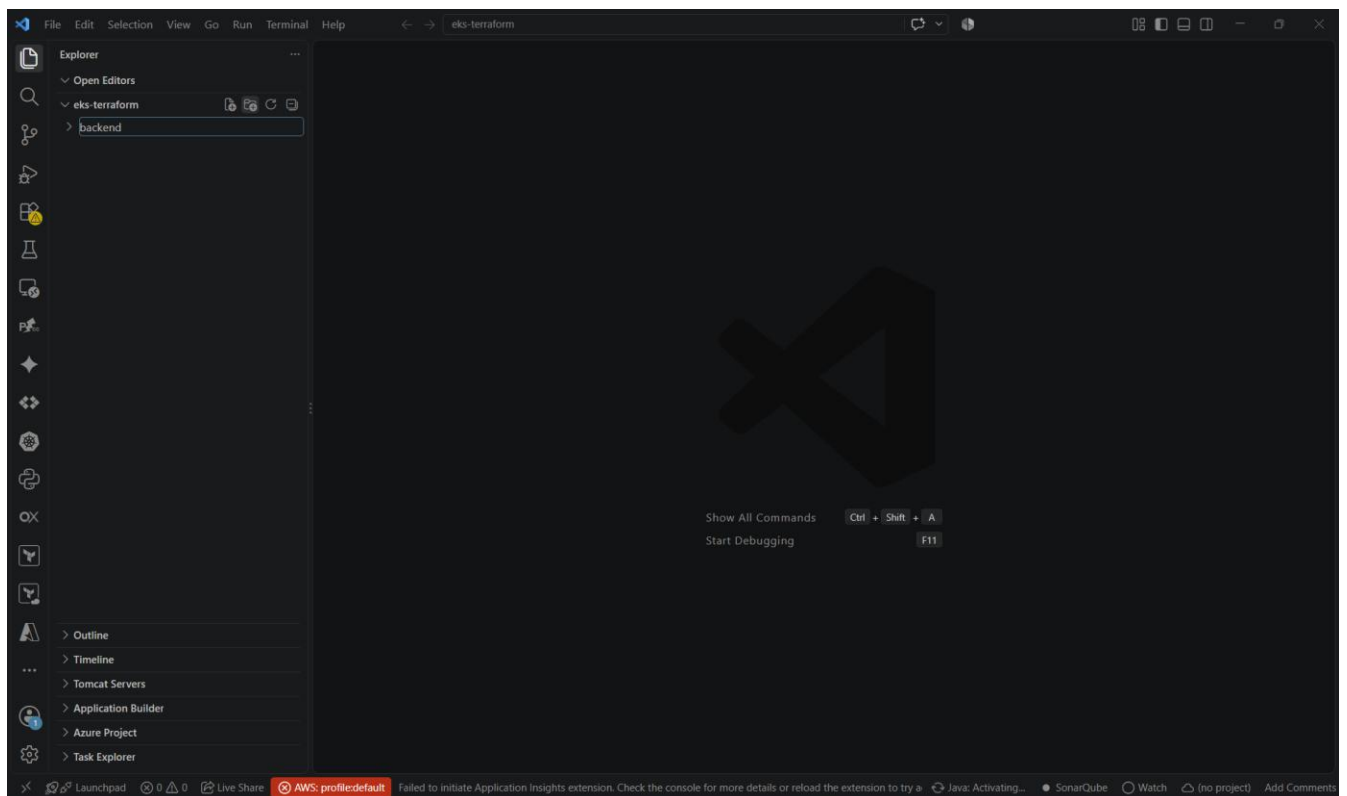
We have opened the folder on VScode. This will be our working directory. Now, we are going to walk-through the modular structure of our EKS deployment using Terraform.

Instead of creating “main.tf” file and having all our code there, it is actually very easy much better to use modules. This will help us to manage infrastructures more clearly. To do this we will create some directories and files.

Let us create the first directory called “backend”



Click on “New Folder” and name the folder “backend”, then press “Enter”

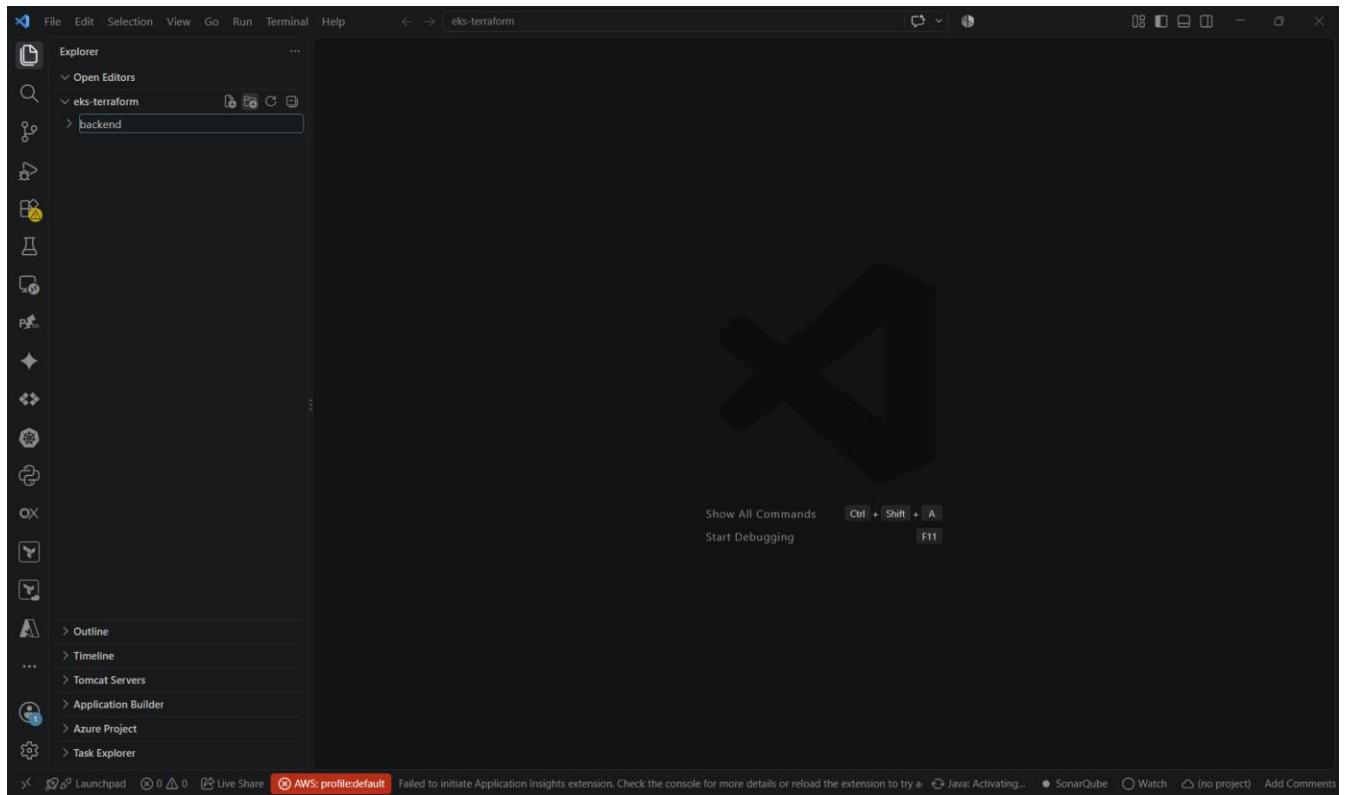


This is the folder where we will manage our remote state configuration. This is useful when you are working in teams. For example, you can store your state in S3 bucket and lock it using same S3 bucket. With S3 bucket, you can also lock your state file as well.

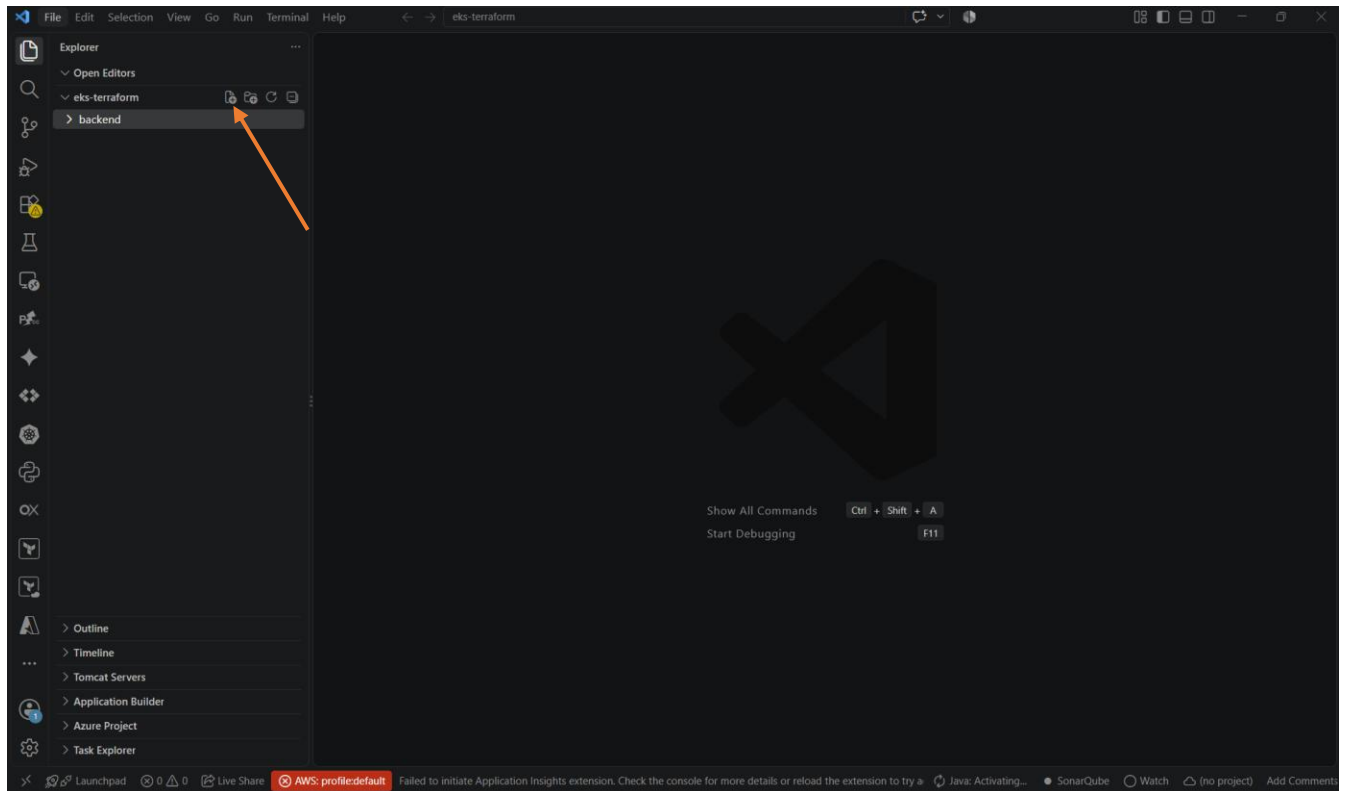
Prepared by Sidney Smith Ebot

In this backend folder, I am going to create a file called “main.tf”. This is where we will have our resources for S3 bucket and also for our state lock.

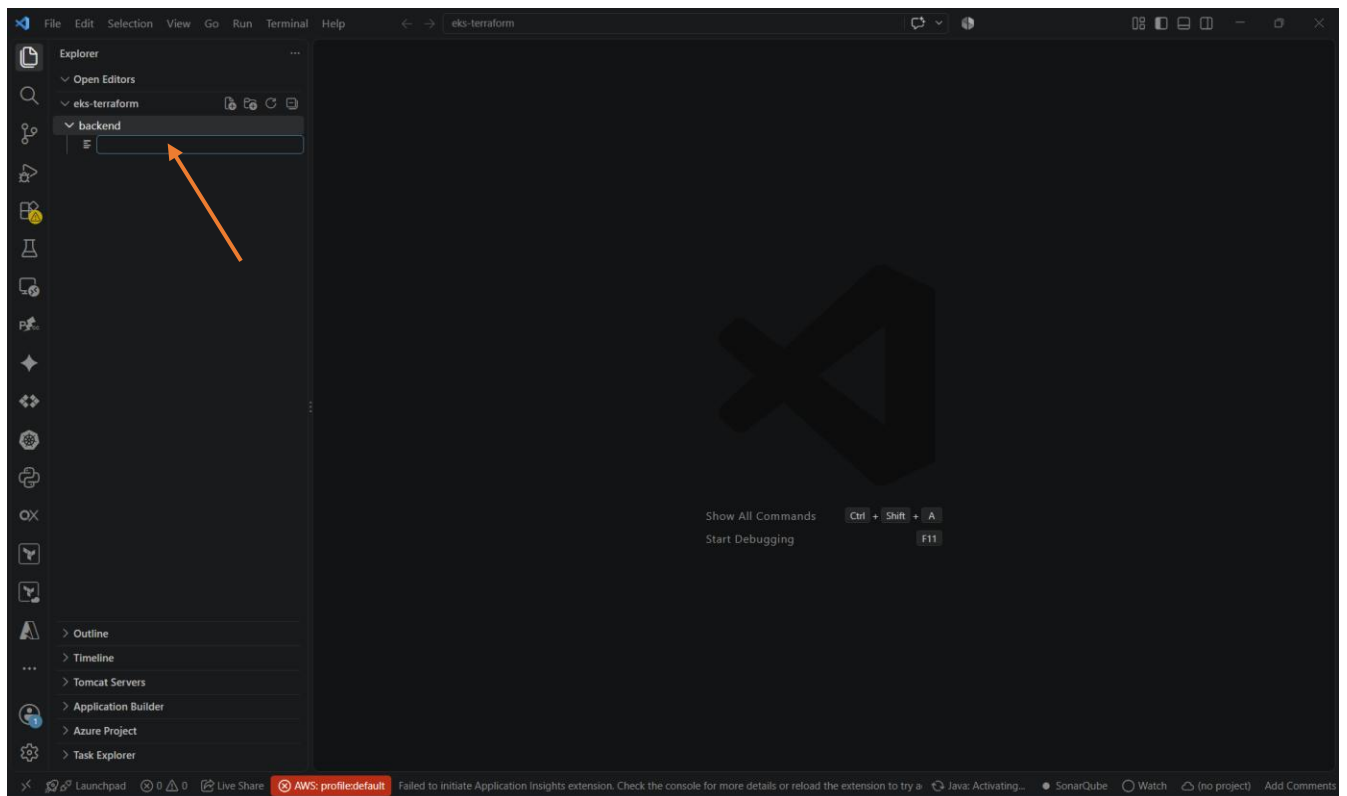
Let us create the file.



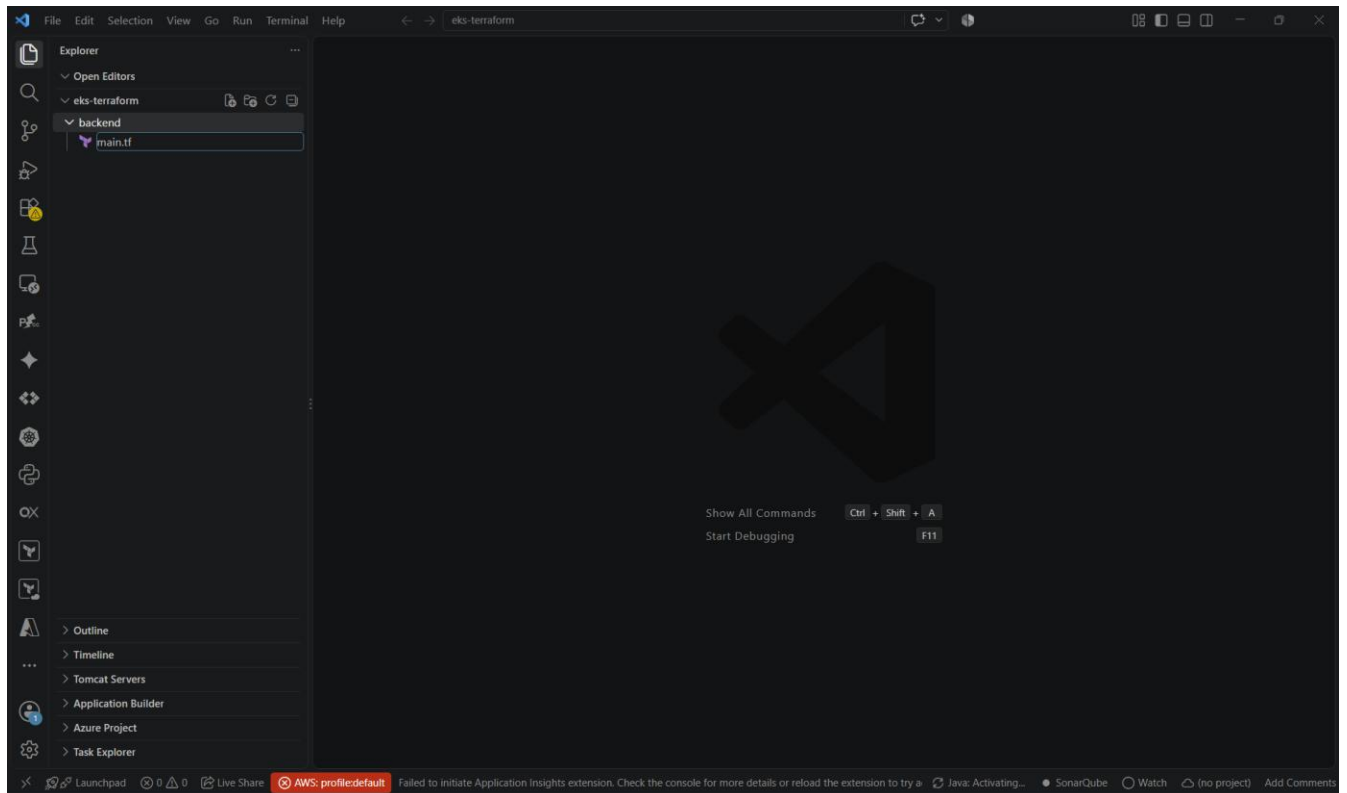
Select the “backend” folder by clicking on it.



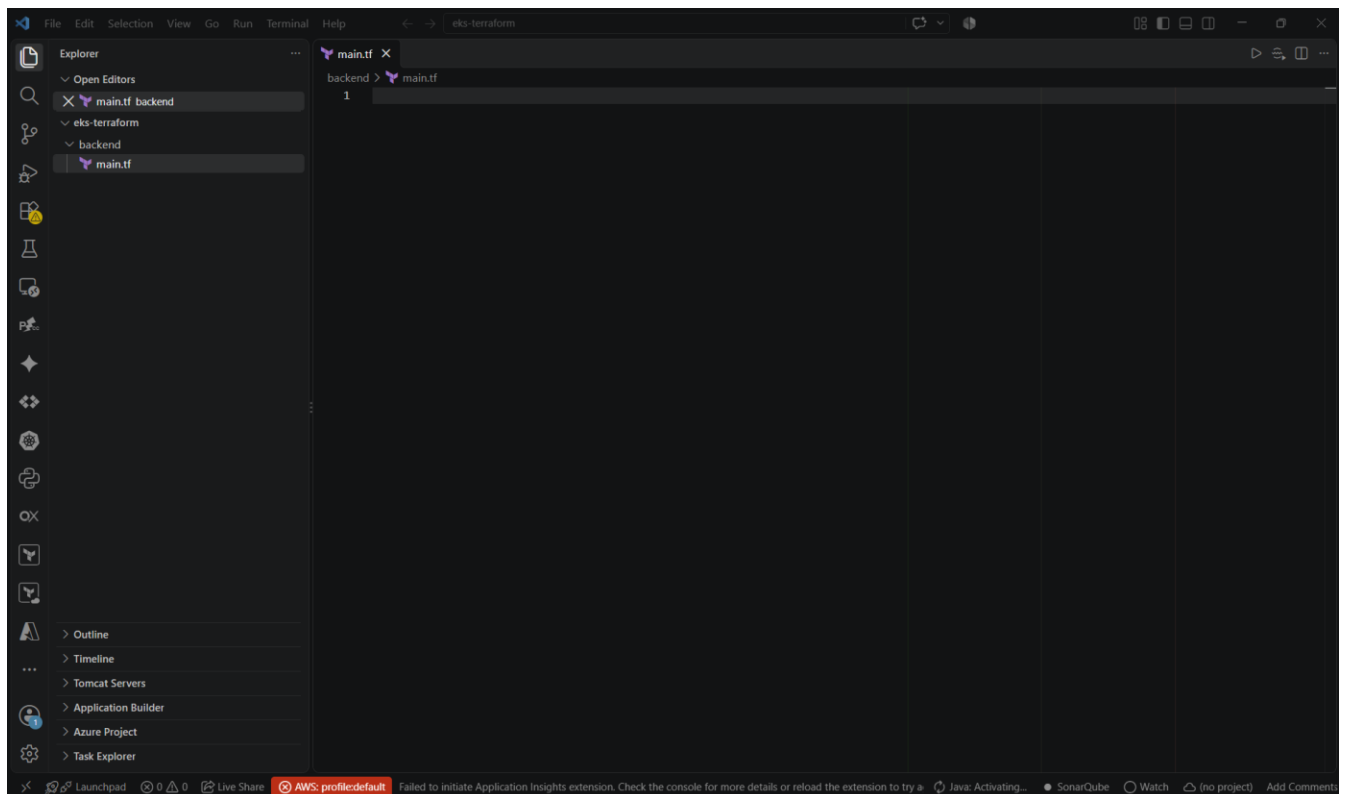
Then, click on the “New file” icon



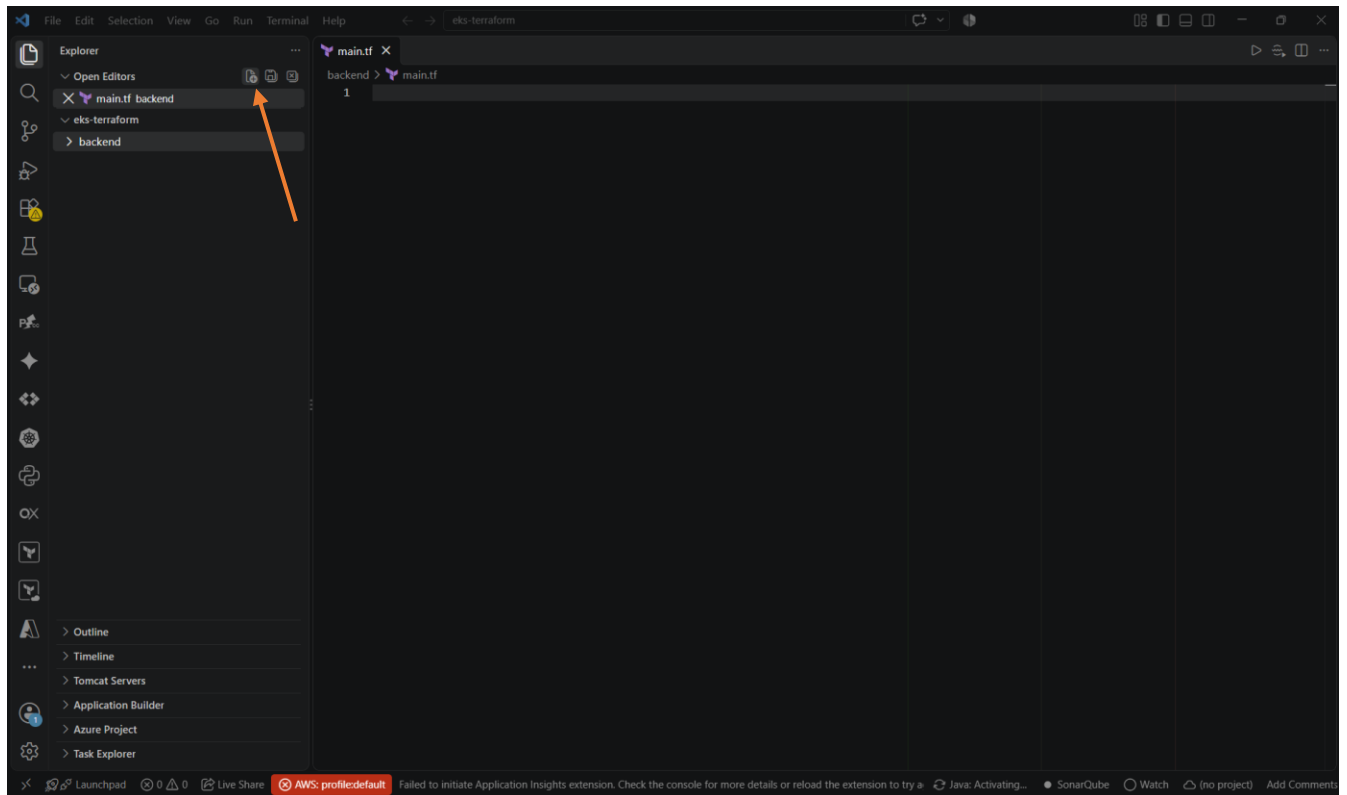
Enter the name of the file “**main.tf**”



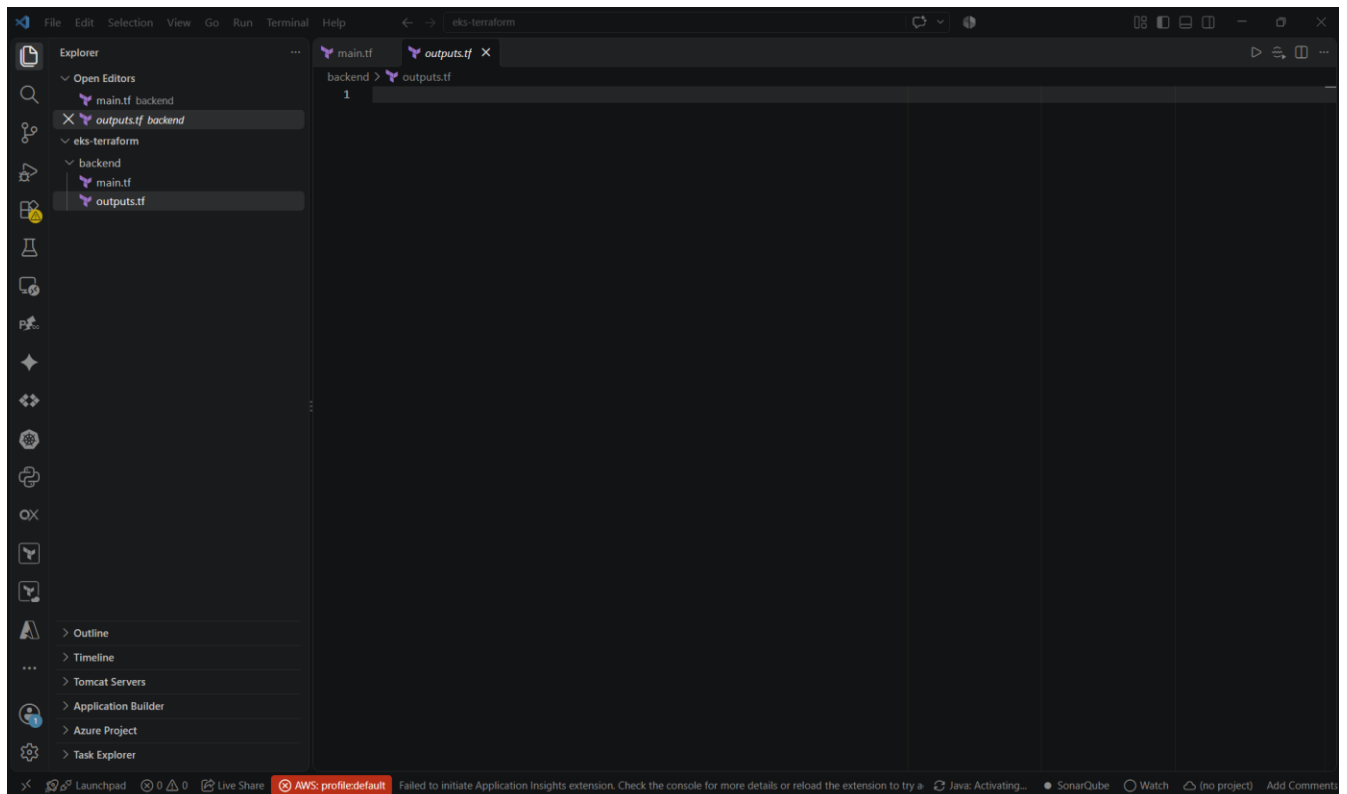
Press the “Enter” key



The file has been created. The next file we are going to create in the “backend” folder is to get our outputs. We will call it “outputs.tf”, select the “backend” folder again

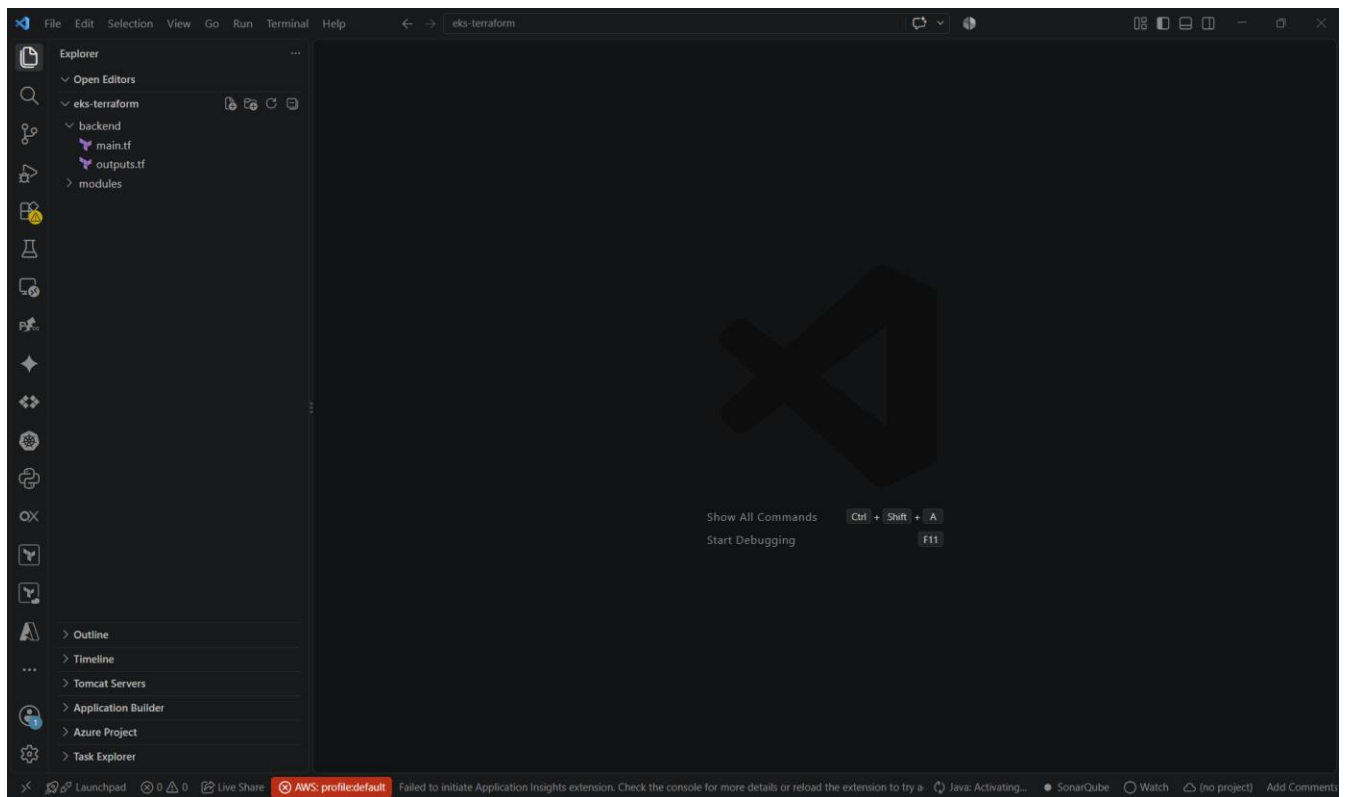


Click on “New File”

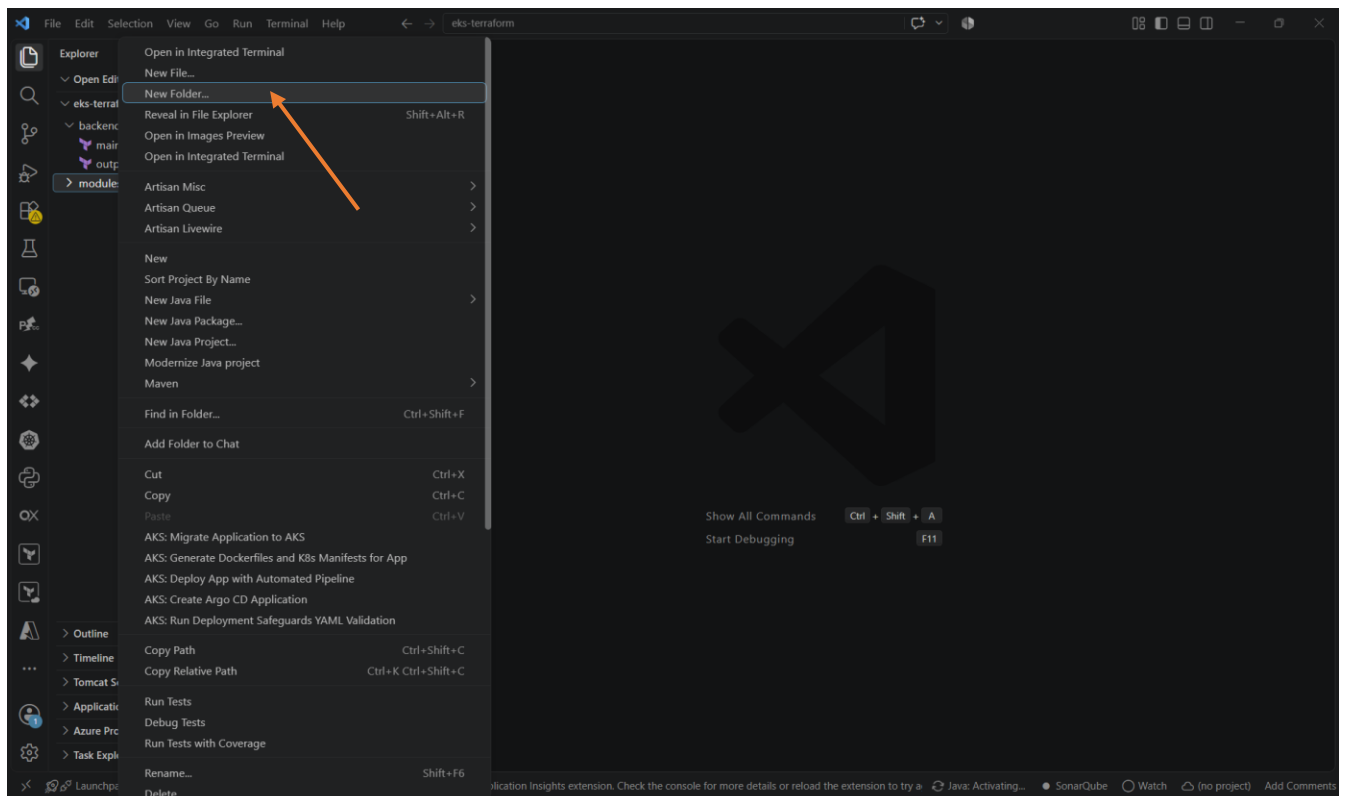


We have created the file for the outputs. These are the two files we need in our backend folder.

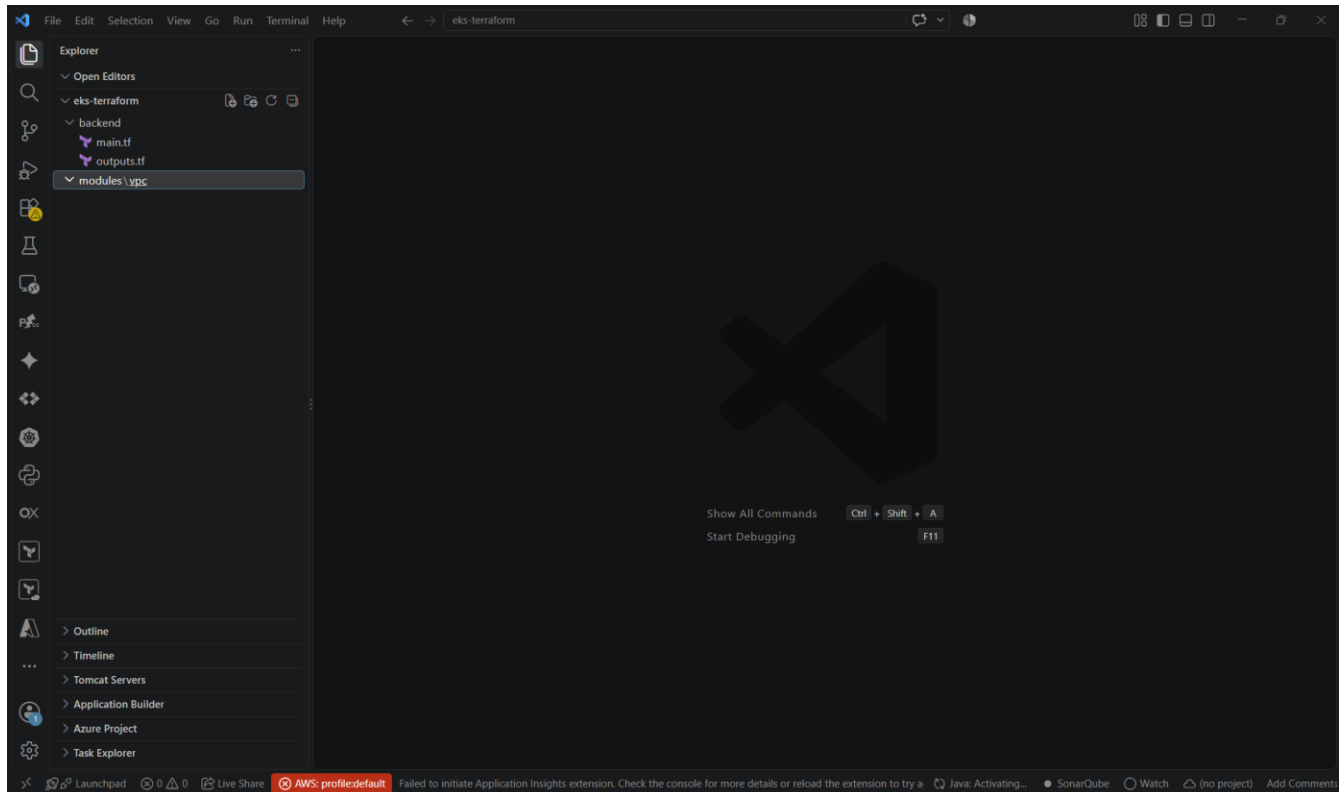
Let us go back to our project directory and create another folder called “**modules**”. This is where we will be having our EKS and VPC directories.



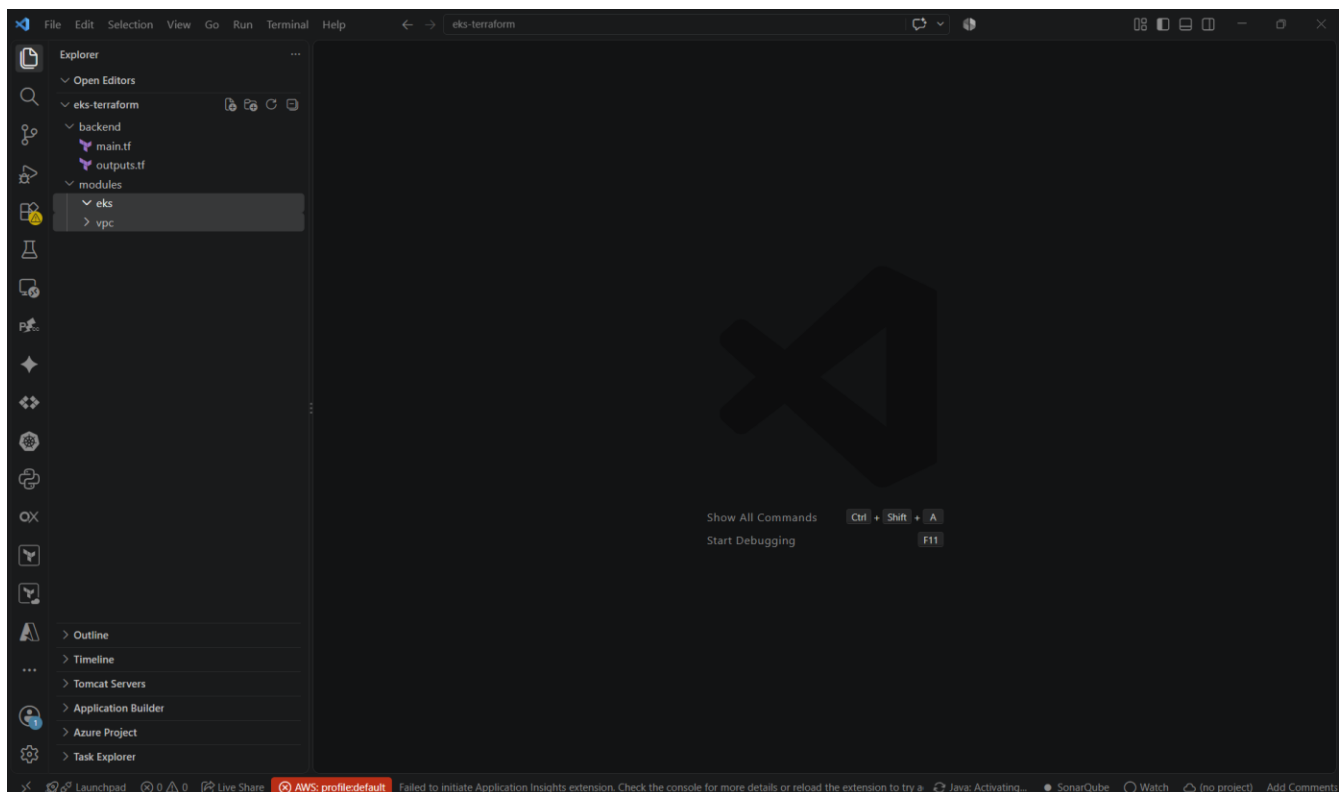
We have created the “modules” directory. Now, we have to create two folders in this directory, namely, “vpc” and “eks”. To do this, we can just right-click on the “modules” directory.



Select “New folder” and you will be requested to give a name, we will enter “vpc” and press “enter”

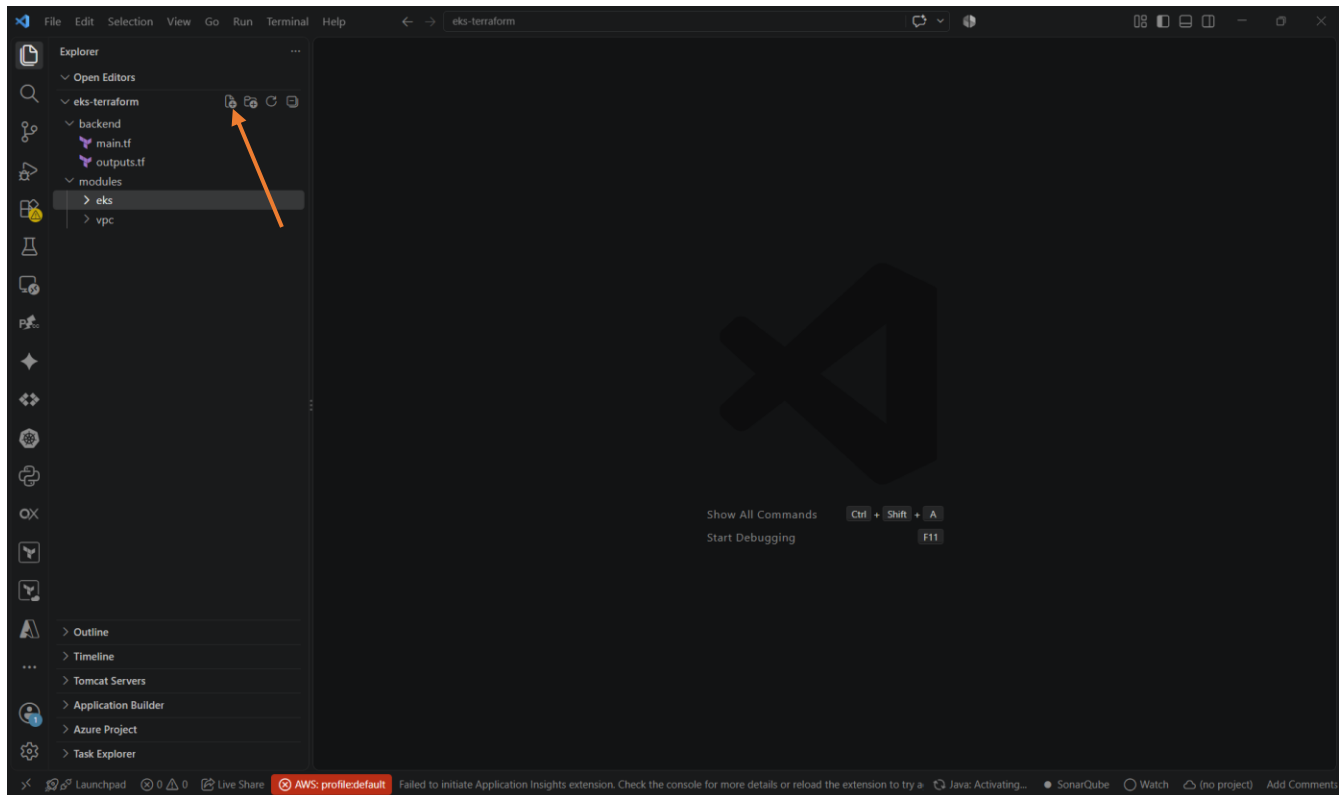


We have created the folder called “vpc”. The “vpc” folder will help us handle all the networking set up. Here we will create the VPC, Subnets (both private and public), route table and so on. Let is now create the folder called “eks”

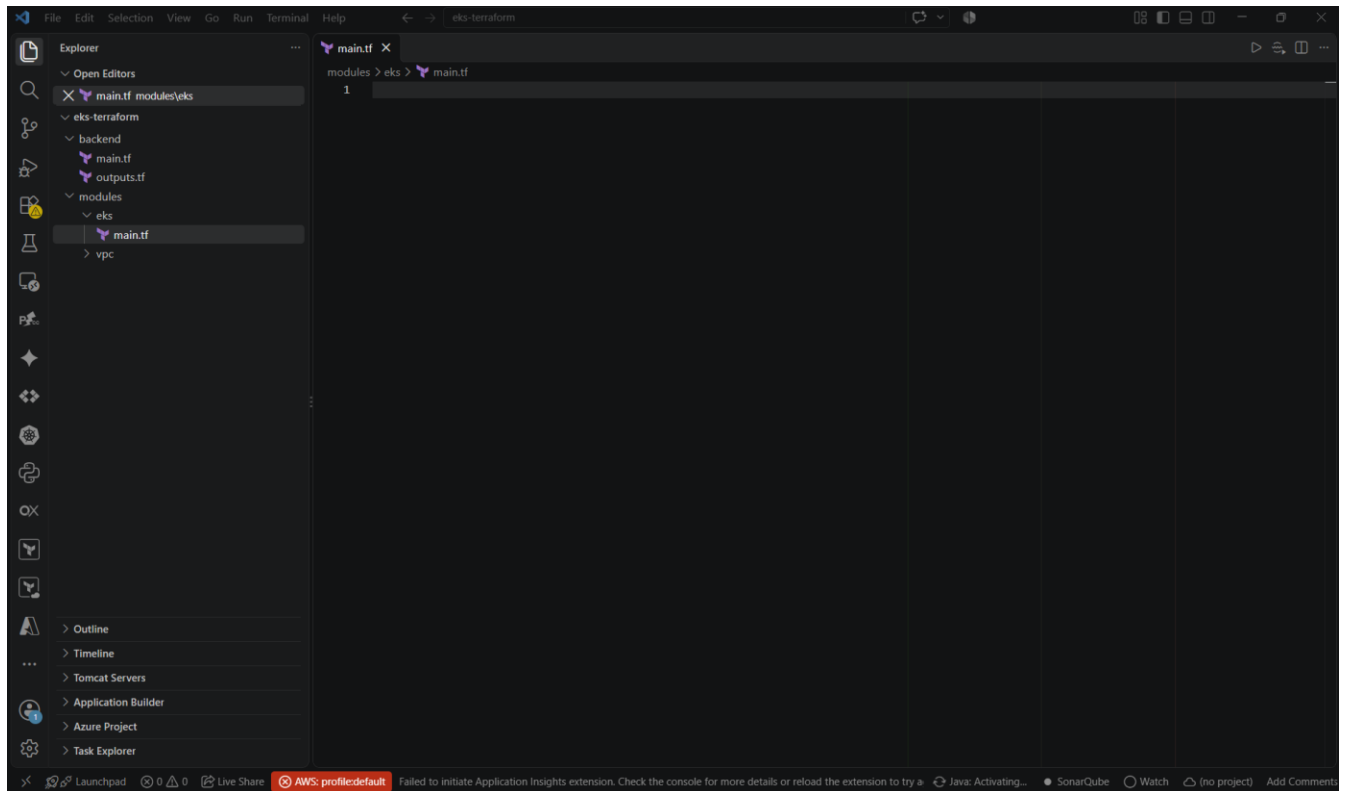


We have created the two folders in the “modules” directory. The “eks” folder will help us handle all the EKS creation like setting up the cluster, add our Node group configurations, IAM roles etc.

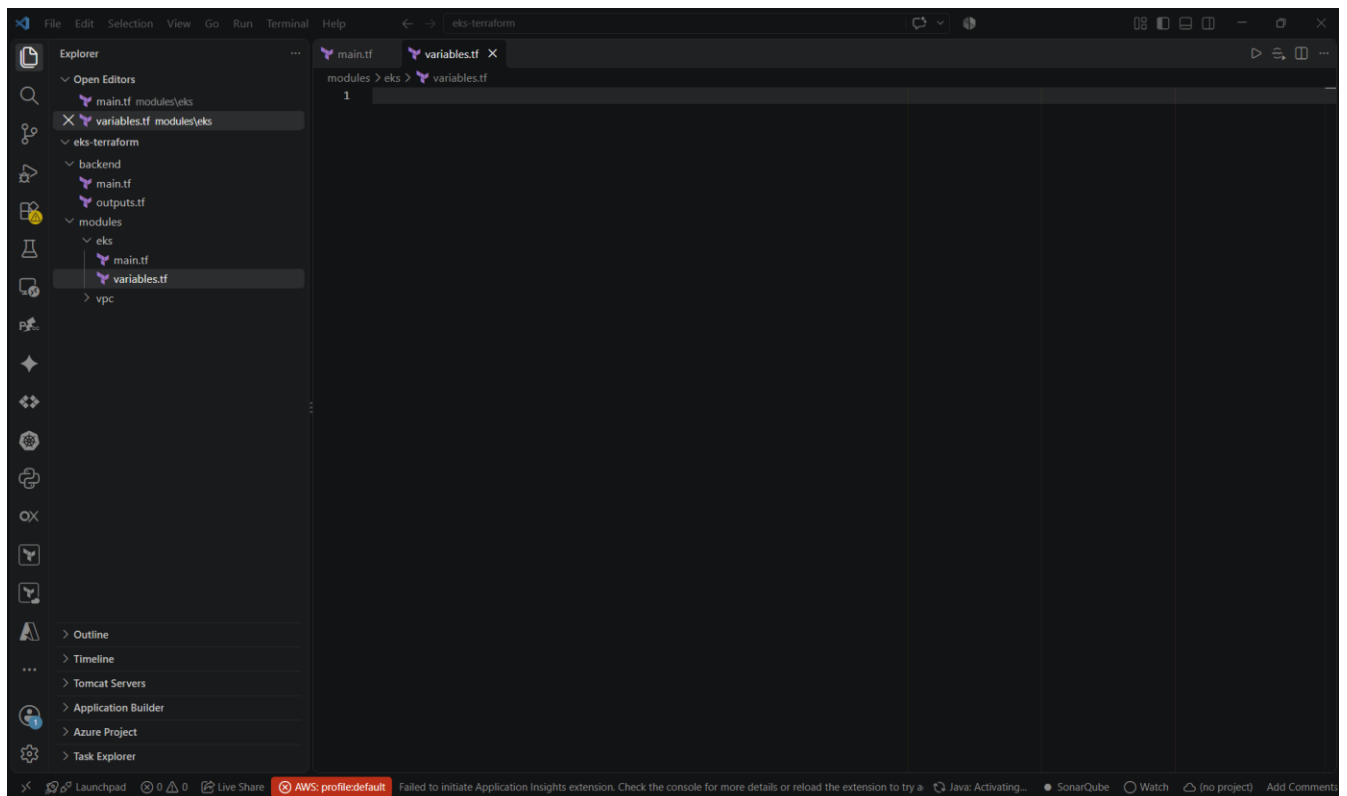
We have to now create the file in the “eks” and “vpc” folders. Let us start with the “eks”. Select the folder.



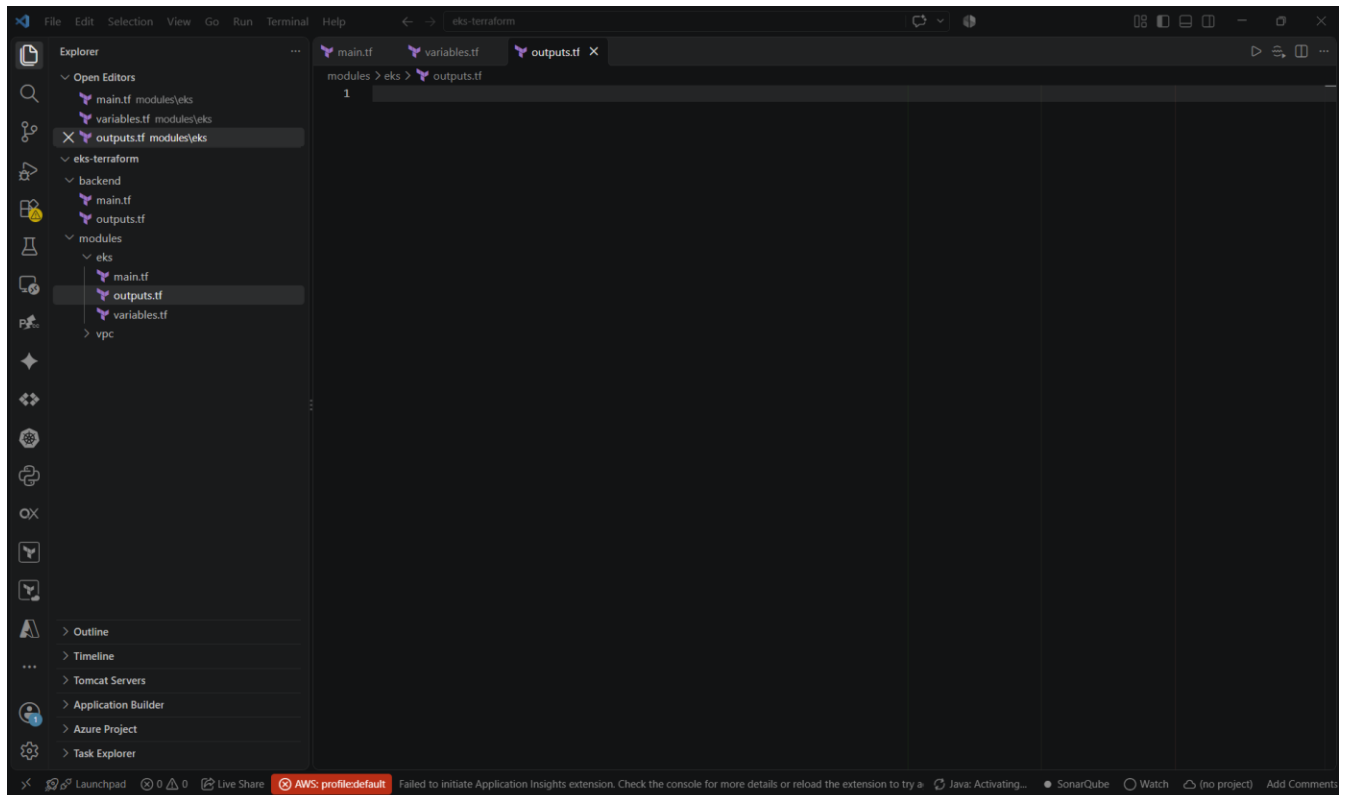
Click on “**New File**”. We will call the file “**main.tf**”



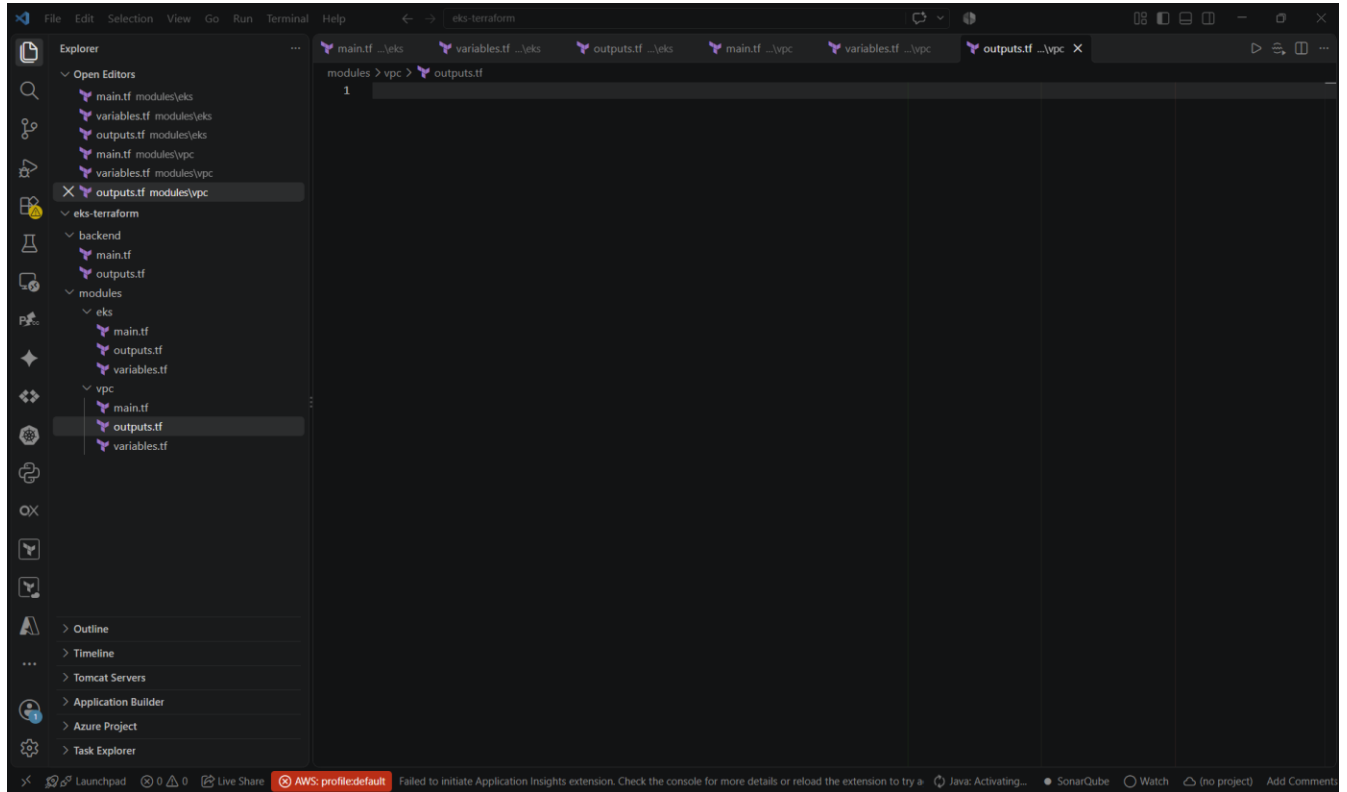
We have to create a second file in the “eks” folder called “**variables.tf**”



We have to create the third file in the “eks” folder called “**outputs.tf**”



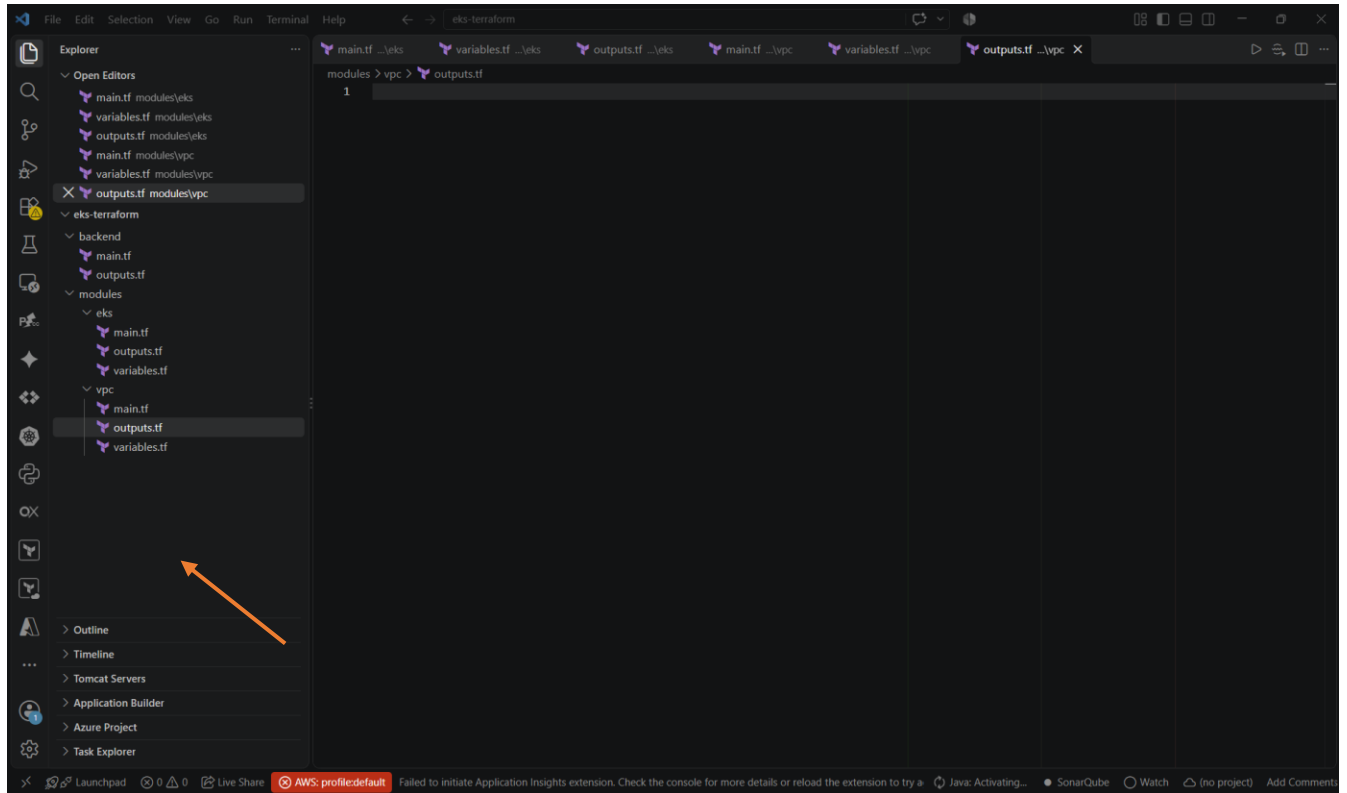
We have created the three files in the “eks” folder. We will create same three files in the “vpc” folder. Let us go ahead and do that.



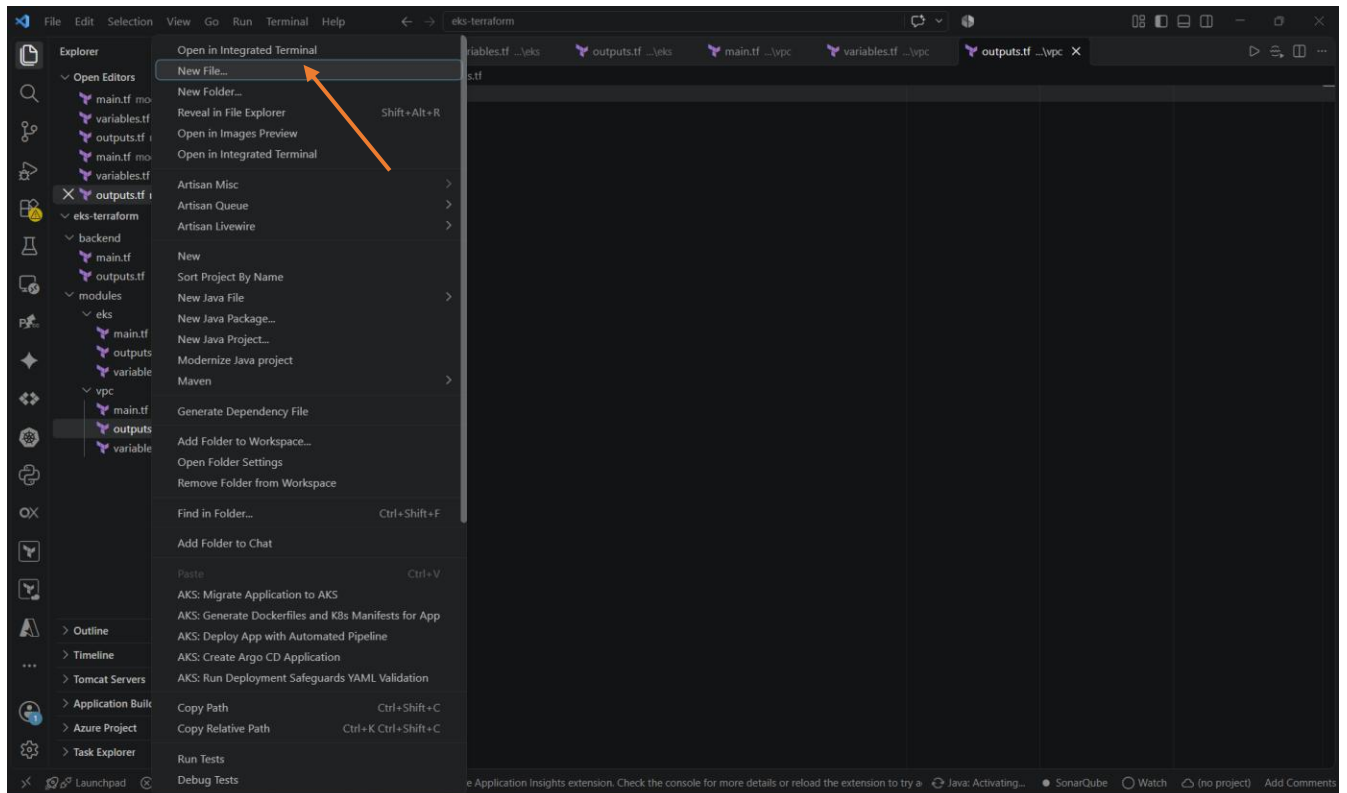
We have created the three files in the “vpc” folder.

We now have to create a “**main.tf**” file in the root directory (project directory), that is actually where we will be calling our modules from by the time, we are done building our configurations.

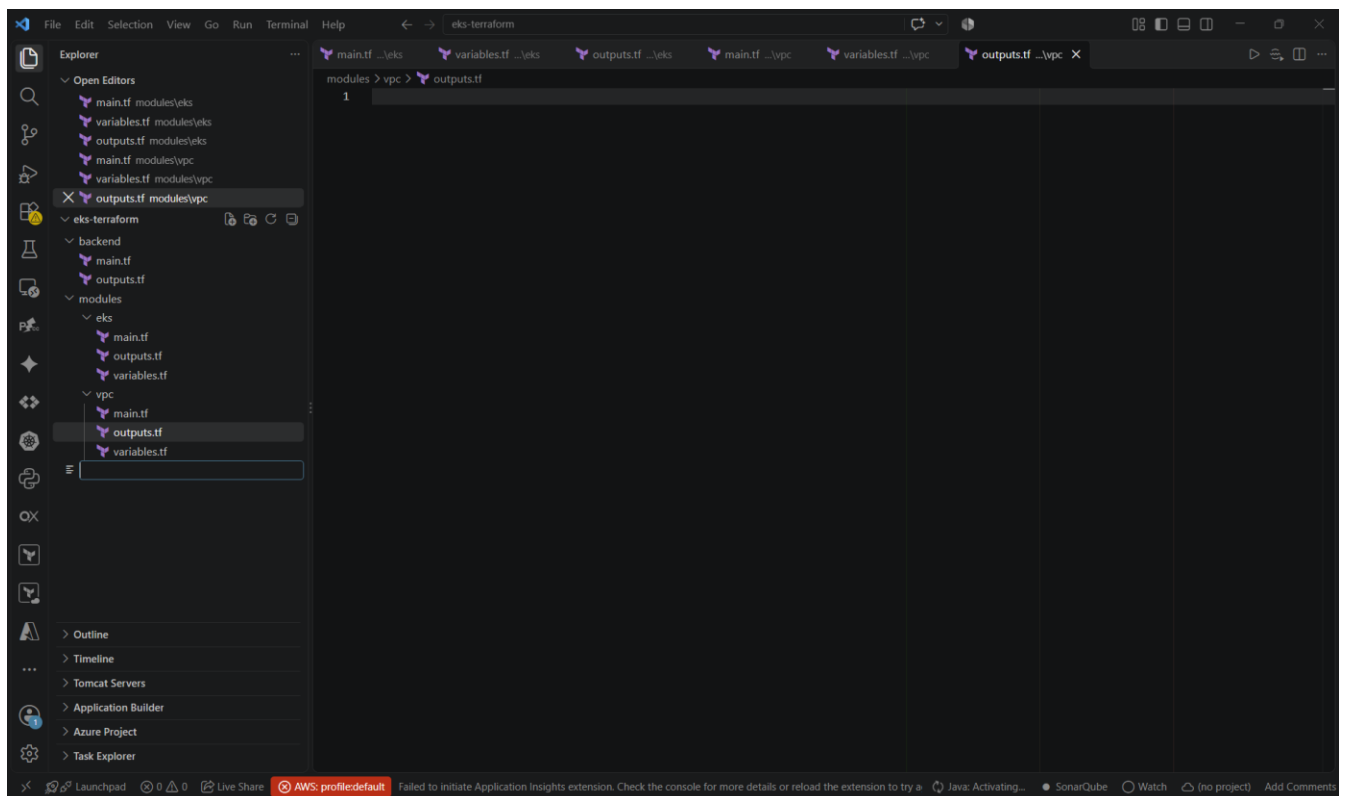
To do this, go to the VSCode



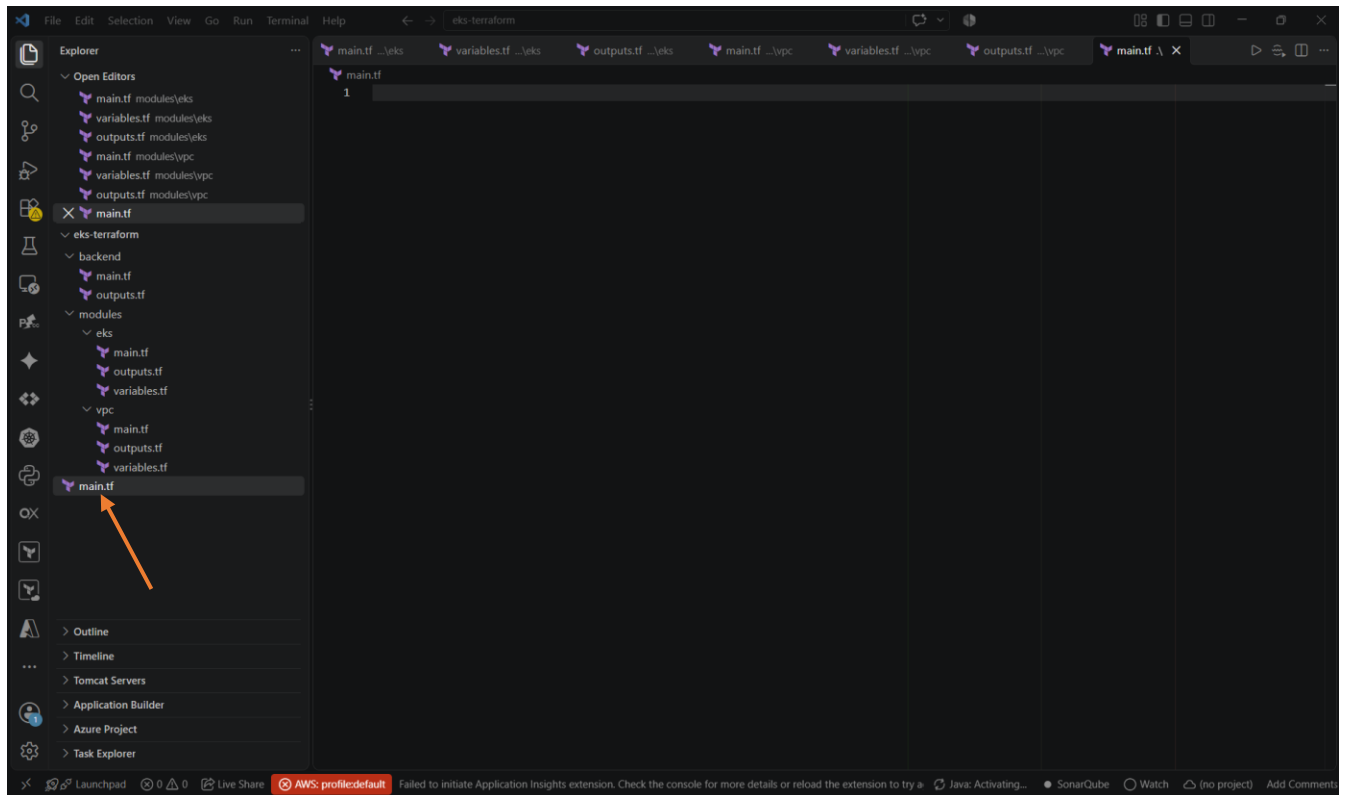
Right-click on the empty space



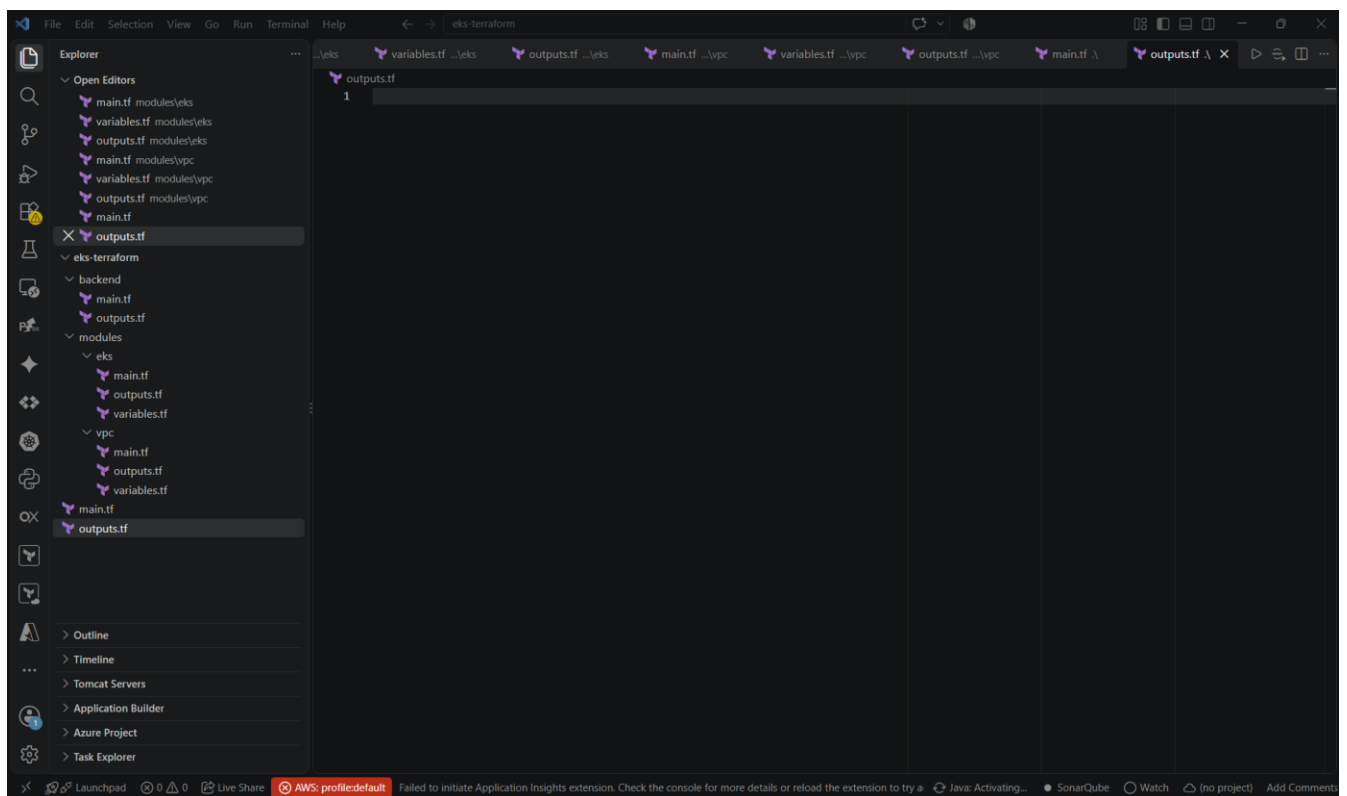
Select “New File”



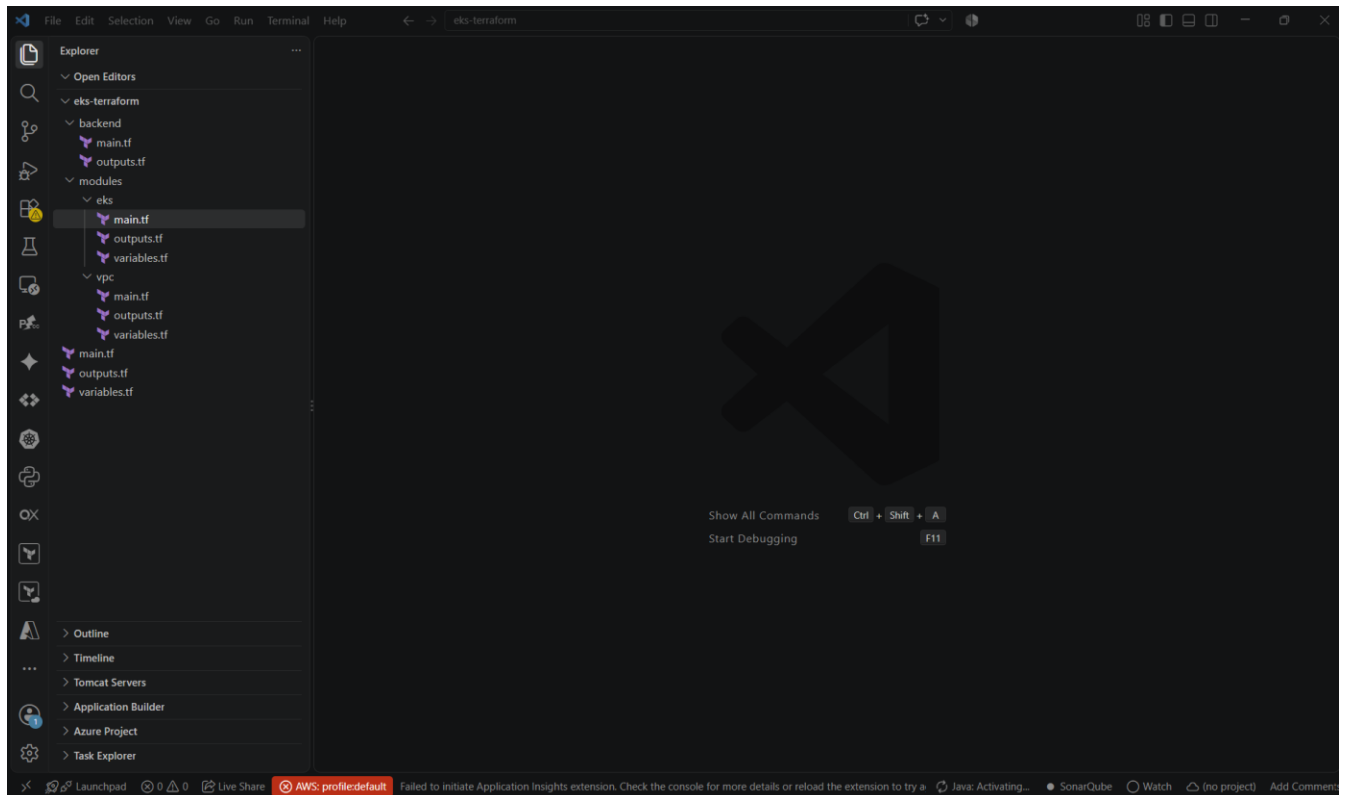
Enter the name “main.tf” and press “enter”



You can see that we have created the file “**main.tf**” in the root directory. We have to also create another file called “**outputs.tf**” in our root directory.

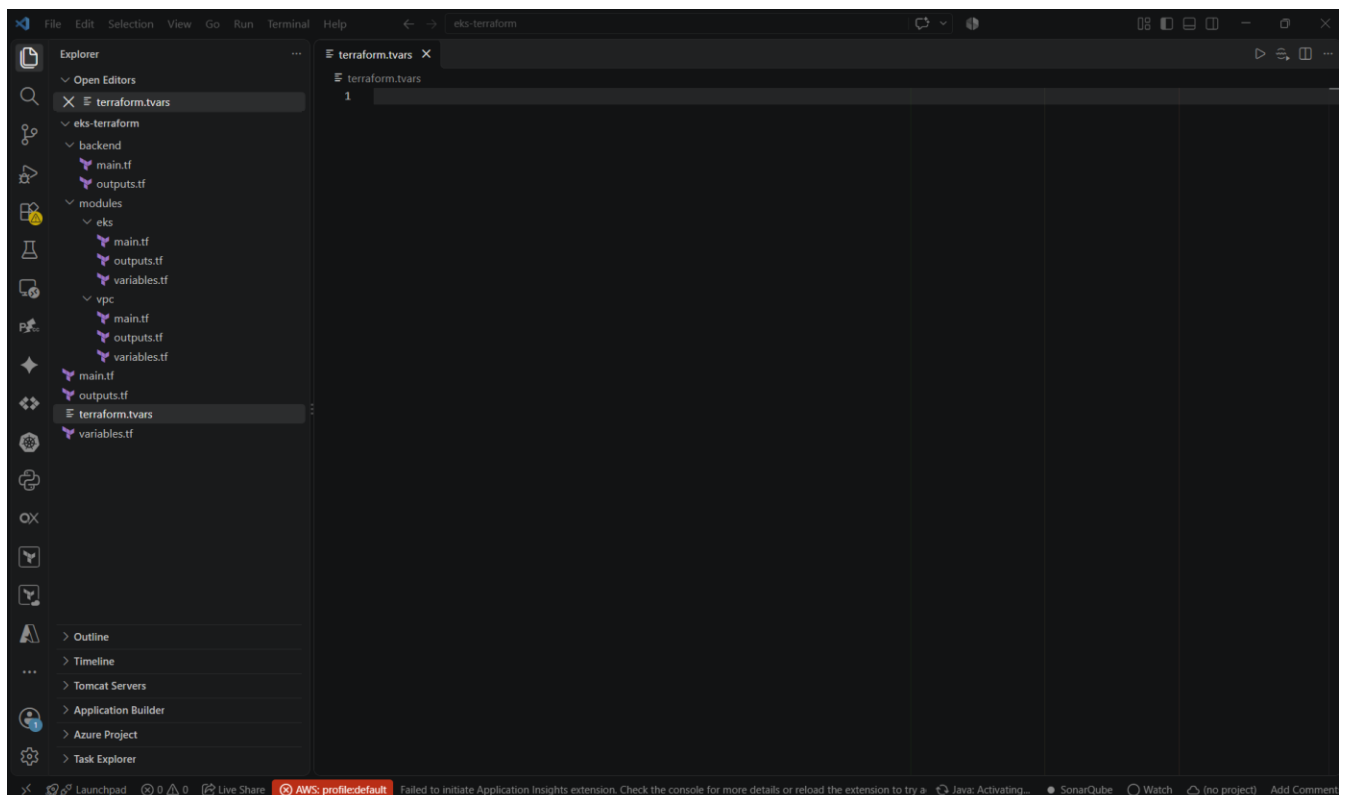


Then, let us create another file called “**variables.tf**” in our root directory.



We have created the file **“variables.tf”** in the root directory.

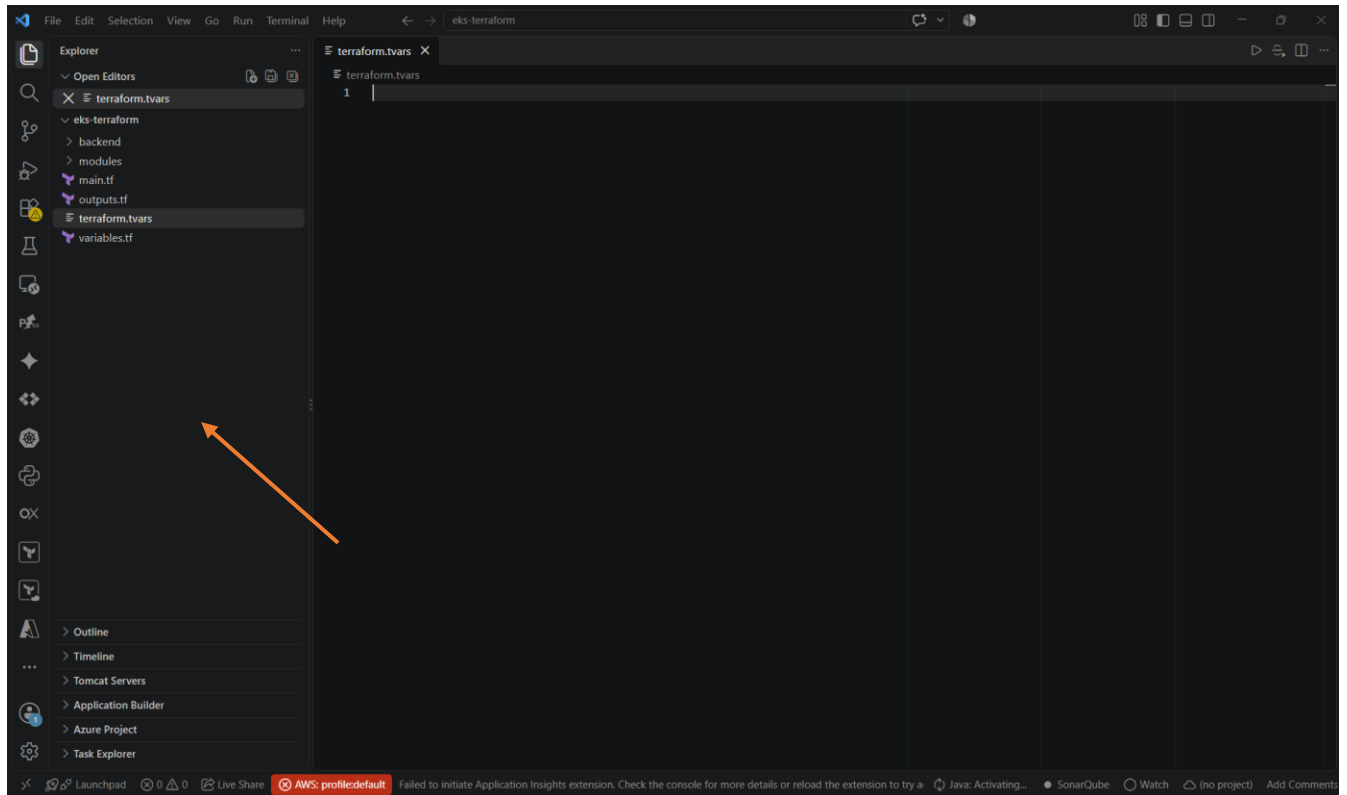
We also have to create a file called **“terraform.tvars”** in the root directory. This is where we have all the values of our variables.



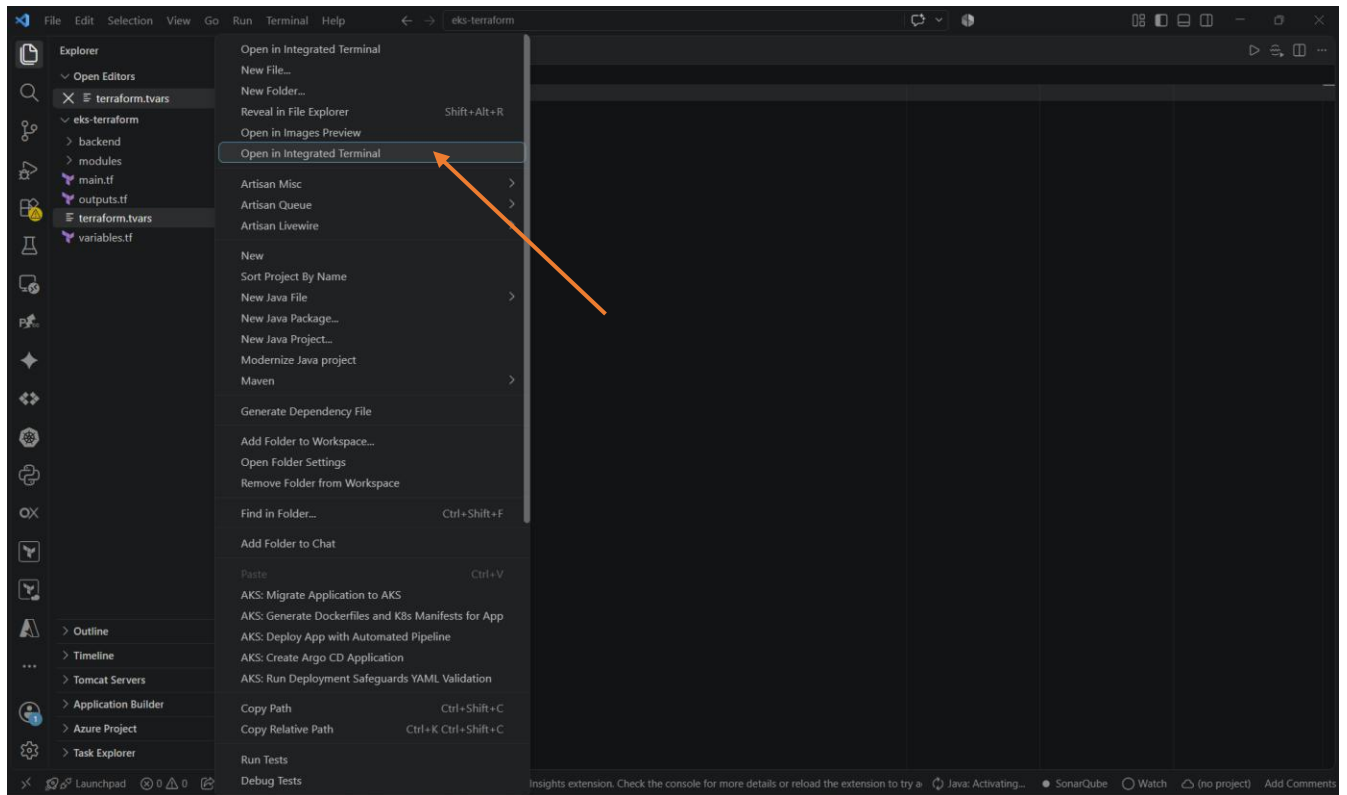
At the end, in our project folder, we have two directories, namely “backend” and “modules”. We also have four files namely “main.tf”, “outputs.tf”, “variables.tf” and “terraform.tfvars”.

In the “backend” directory, we have two files namely “main.tf” and “outputs.tf”. And finally, in the “modules” directory we have two folders namely “eks” and “vpc”, each containing three files namely “main.tf”, “outputs.tf” and “variables.tf”.

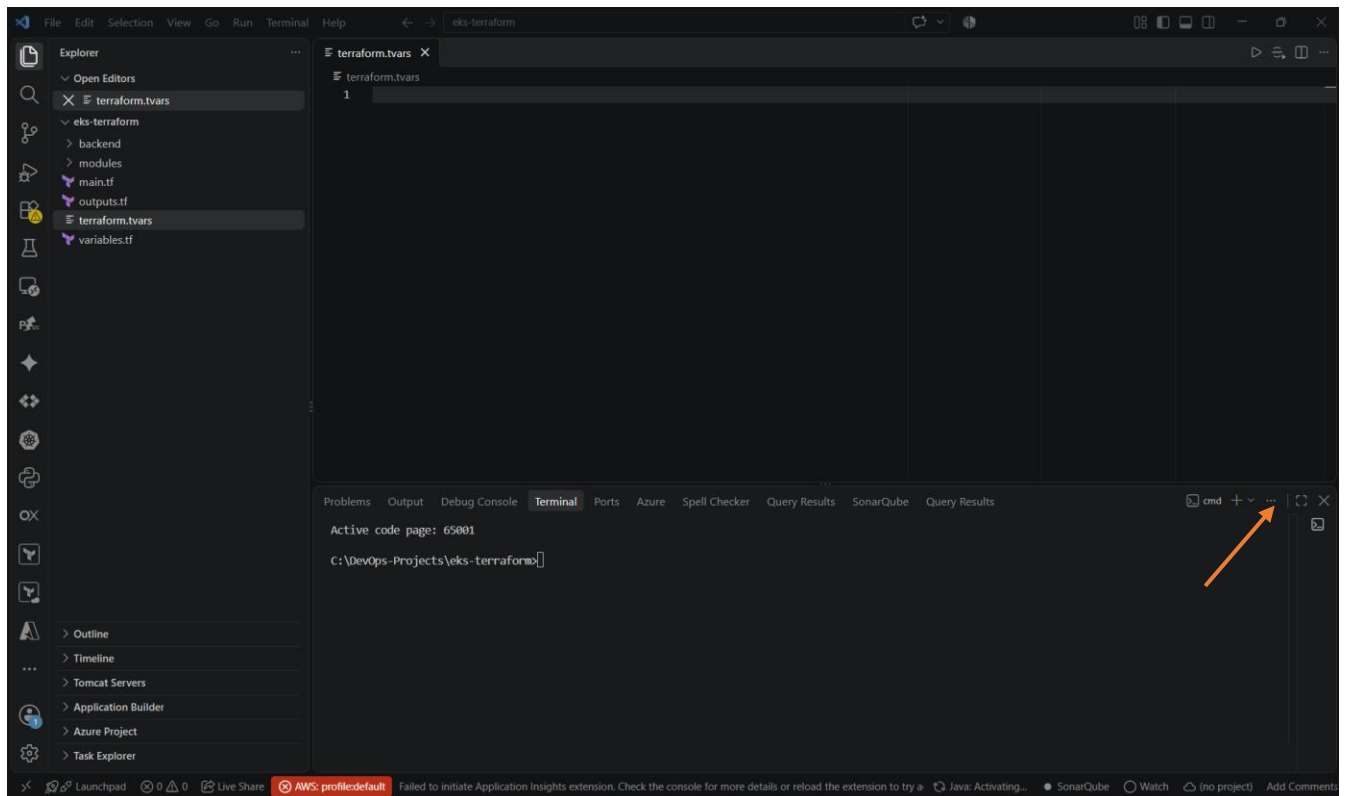
This is how our structure will look like. Let us open the terminal



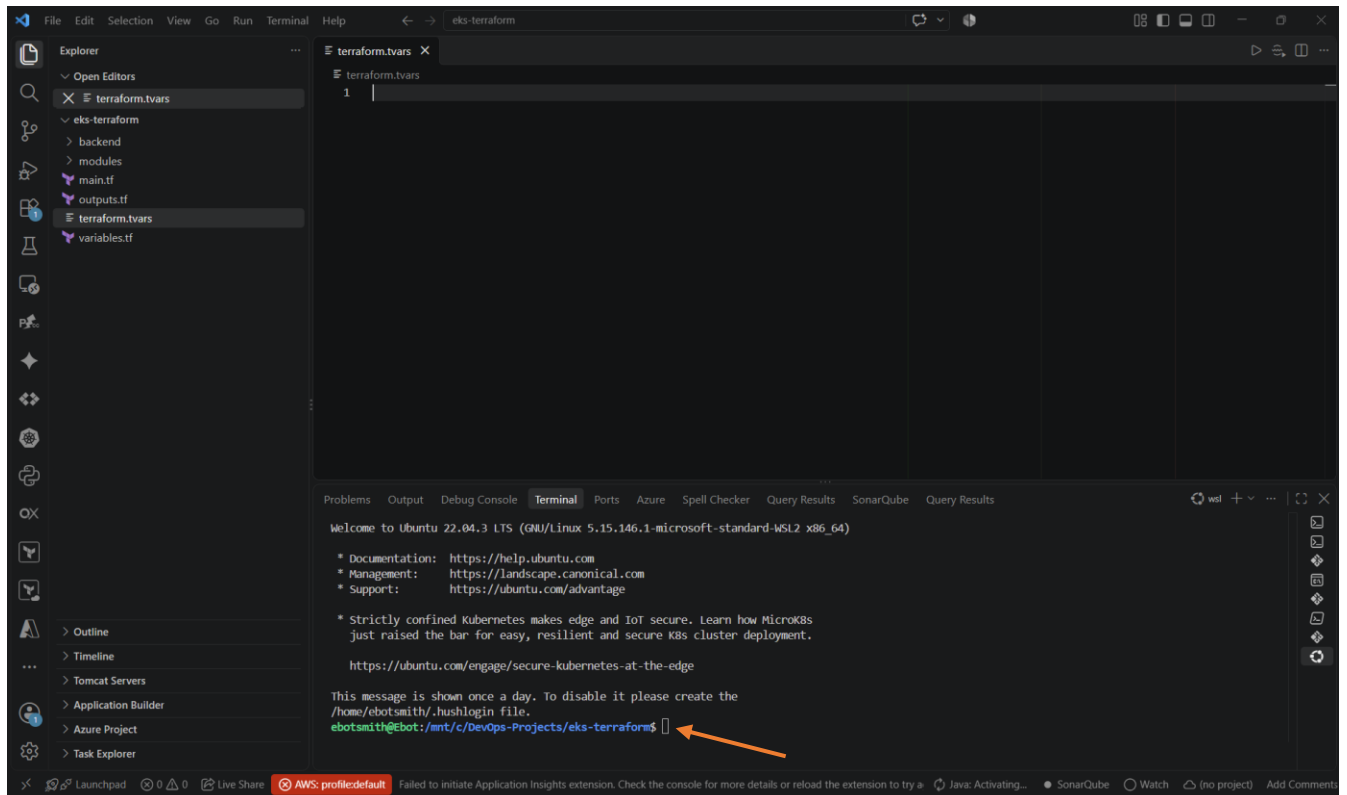
Right-click on the empty space



Select “Open in integrated terminal”

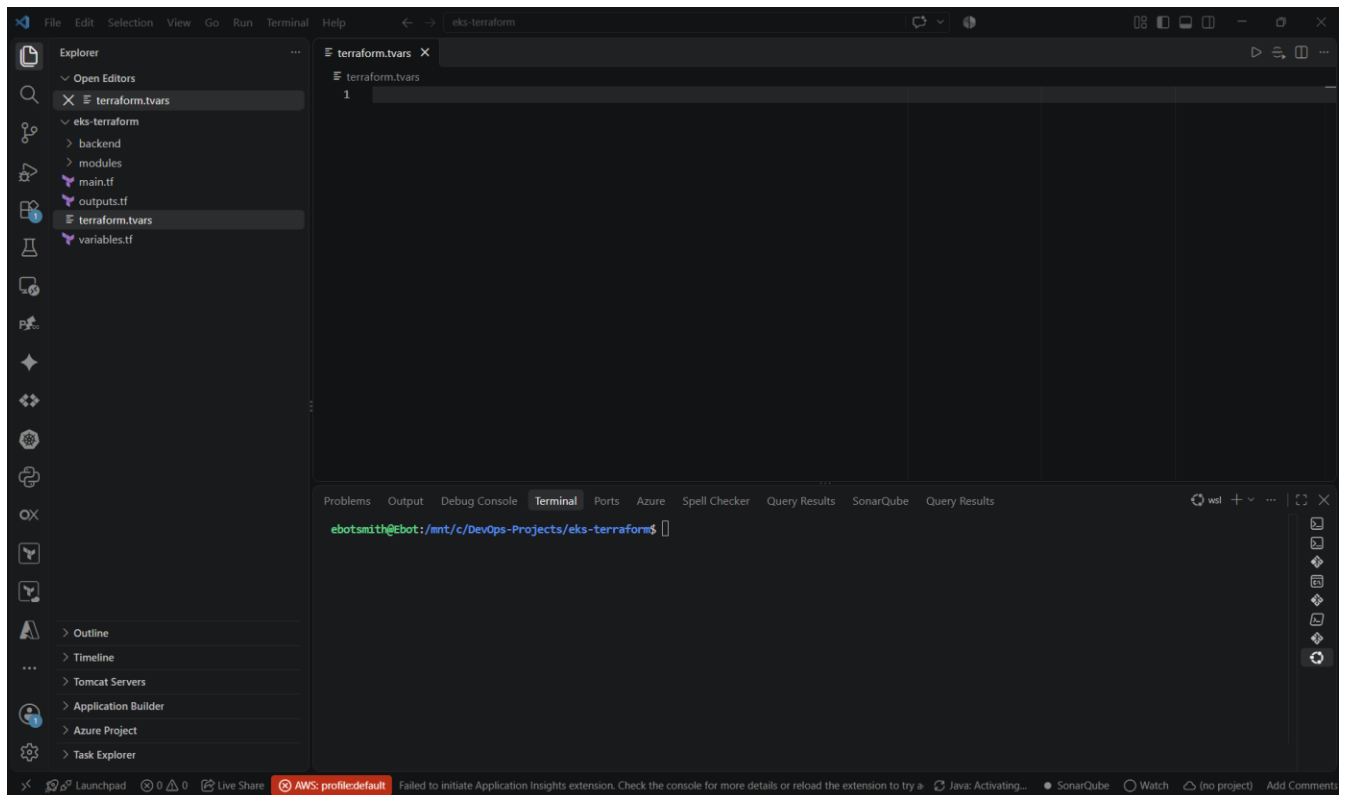


Click on the drop down and select “Ubuntu”



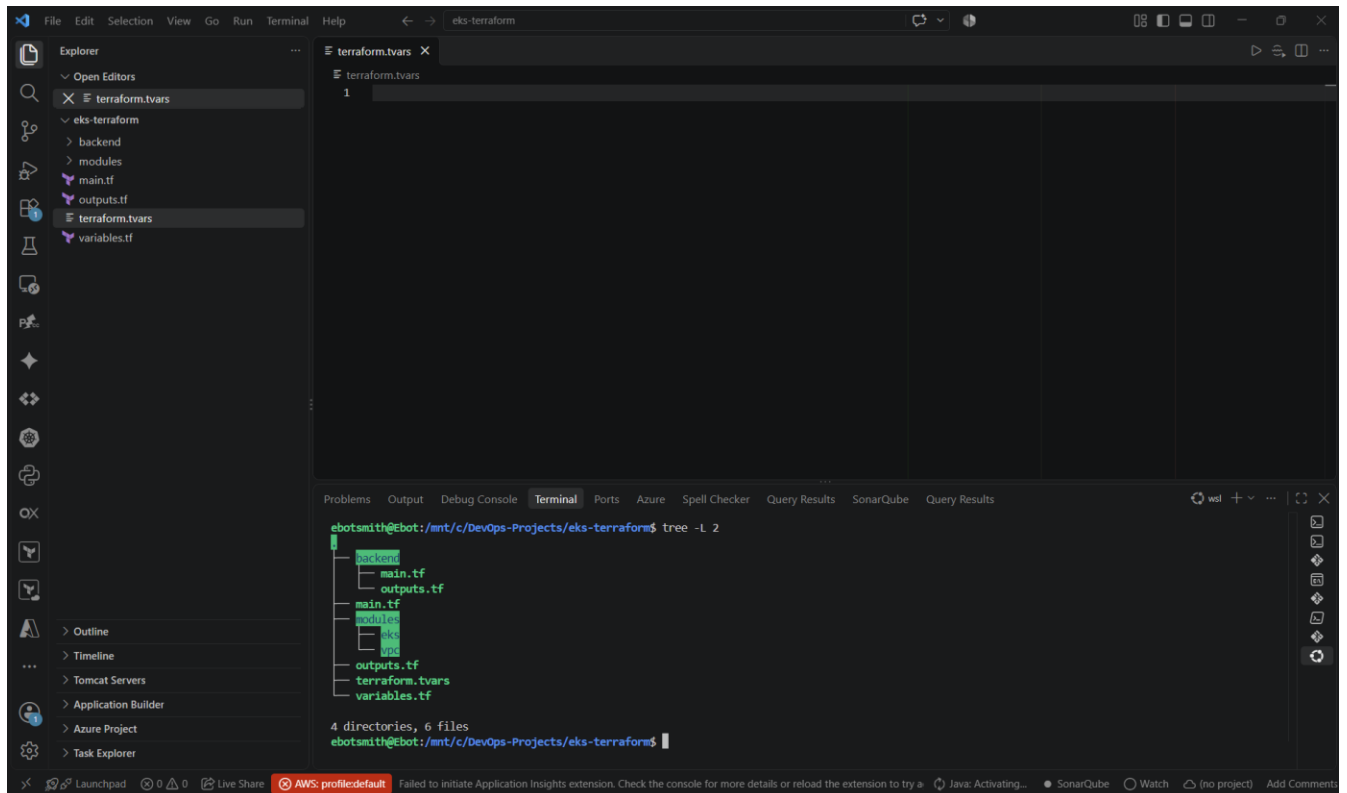
Clear the terminal using the command:

`clear`



Type the command:

```
tree -L 2
```



This is how the structure looks like.

3. Create an S3 Bucket in AWS Using Terraform

In this section we will write the Terraform configuration to create an S3 bucket. We will use the Terraform registry for assistance on how to do this.

Go to Terraform Registry on <https://registry.terraform.io/providers/hashicorp/aws/latest>

The screenshot shows the Terraform Registry page for the **aws** provider. The header includes the Terraform logo and 'Registry' text. A search bar is at the top. The main content area shows the provider details for **aws** (version 6.67.0), including a description, download statistics (8.0B, 54.5M this week), and a 'Public Cloud' badge. Below this, there are tabs for 'Overview' and 'Documentation'. The 'Top downloaded aws modules' section lists several modules like **s3-bucket**, **vpc**, **eks**, and **security-group**. On the right, a 'Provider Downloads' sidebar shows download statistics for the week, month, year, and all time.

Scroll down

This screenshot shows the 'How to use this provider' section of the Terraform Registry page. It includes instructions on how to install the provider by copying and pasting the configuration code into a Terraform configuration file. The code snippet is as follows:

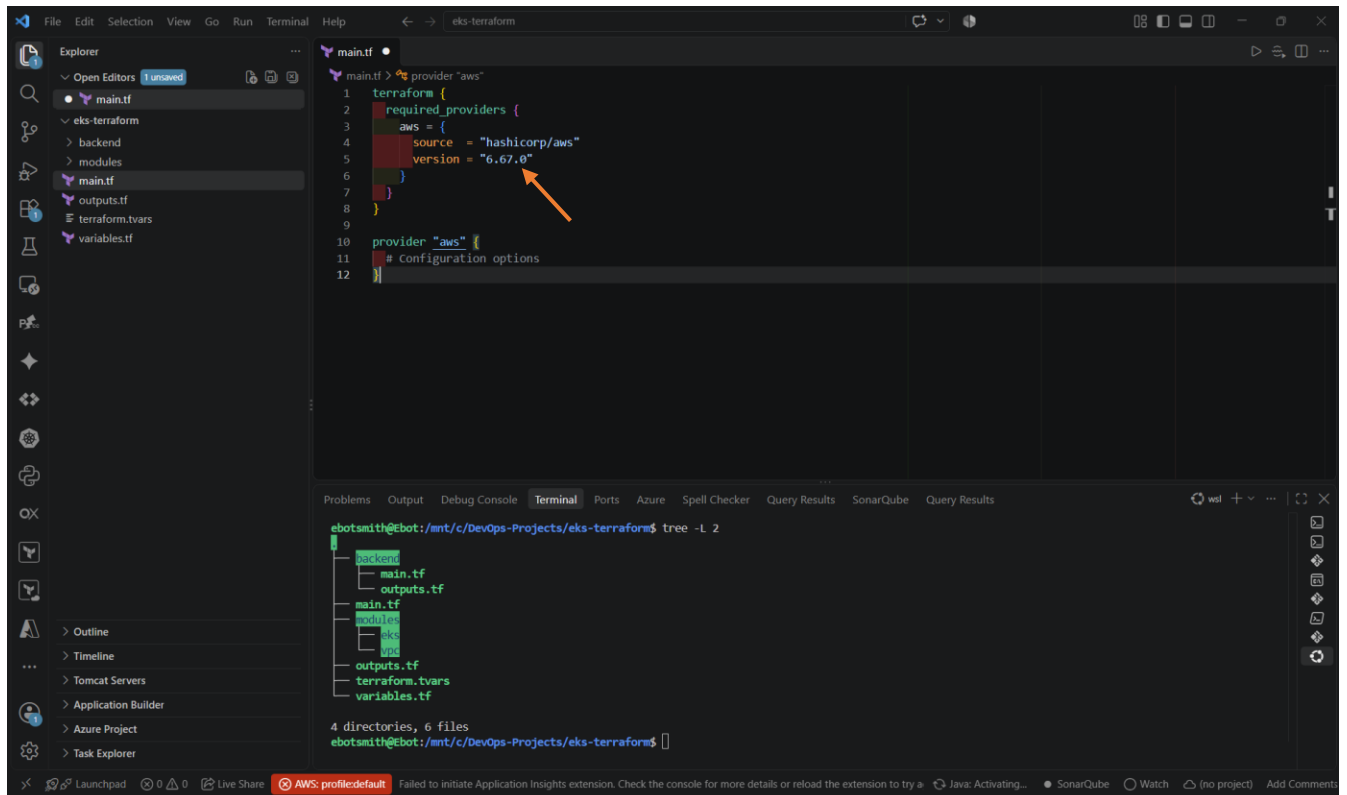
```
Terraform 0.13+

terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "6.67.0"
    }
  }
}

provider "aws" {
  # Configuration options
}
```

An orange arrow points to the copy icon in the top right corner of the code block.

Copy this and paste in the **"main.tf"** file in your root directory

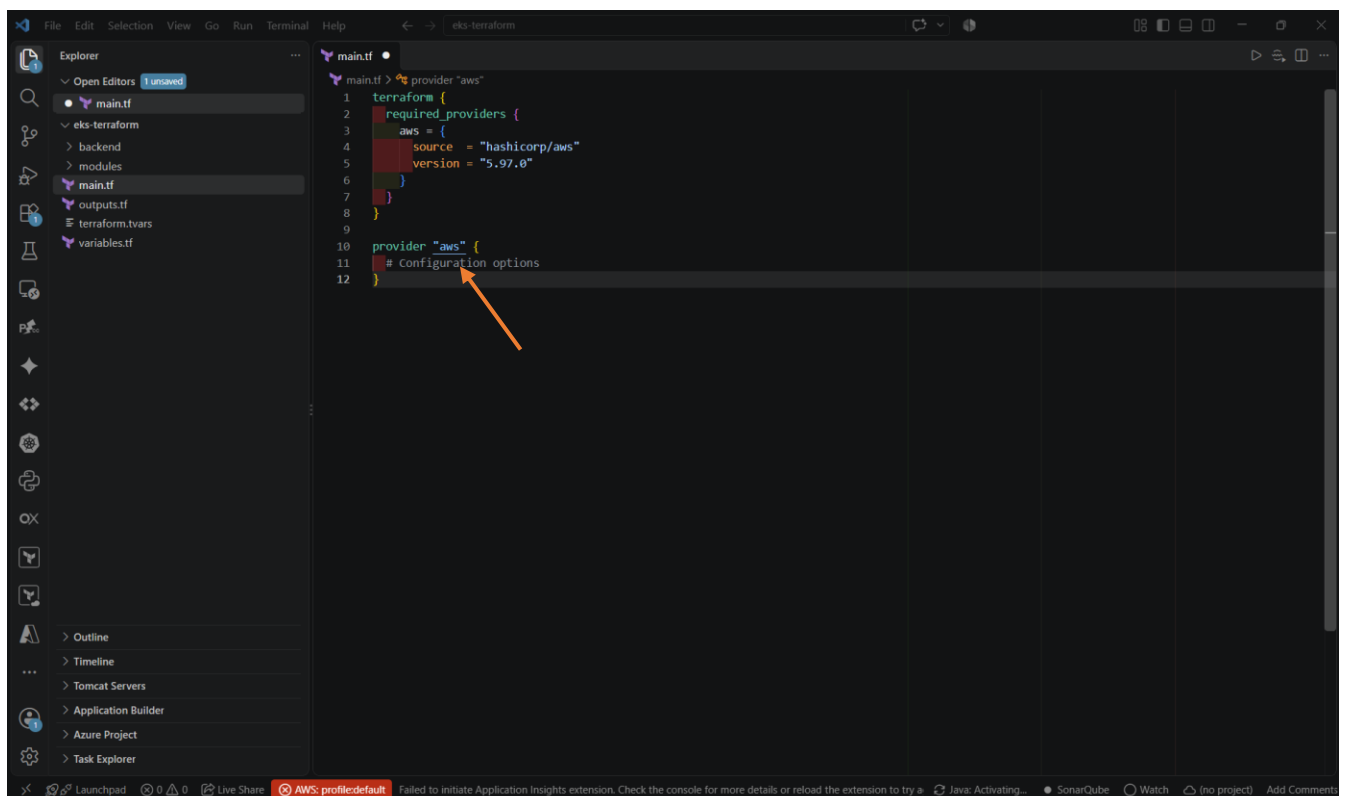


```
main.tf
1 terraform {
2   required_providers {
3     aws = {
4       source = "hashicorp/aws"
5       version = "6.67.0"
6     }
7   }
8 }
9
10 provider "aws" {
11   # configuration options
12 }
```

```
ebotsmith@ebot:/mnt/c/DevOps-Projects/eks-terraform$ tree -L 2
.
├── backend
│   ├── main.tf
│   └── outputs.tf
├── main.tf
├── modules
│   ├── eks
│   └── vpc
├── outputs.tf
├── terraform.tfvars
└── variables.tf

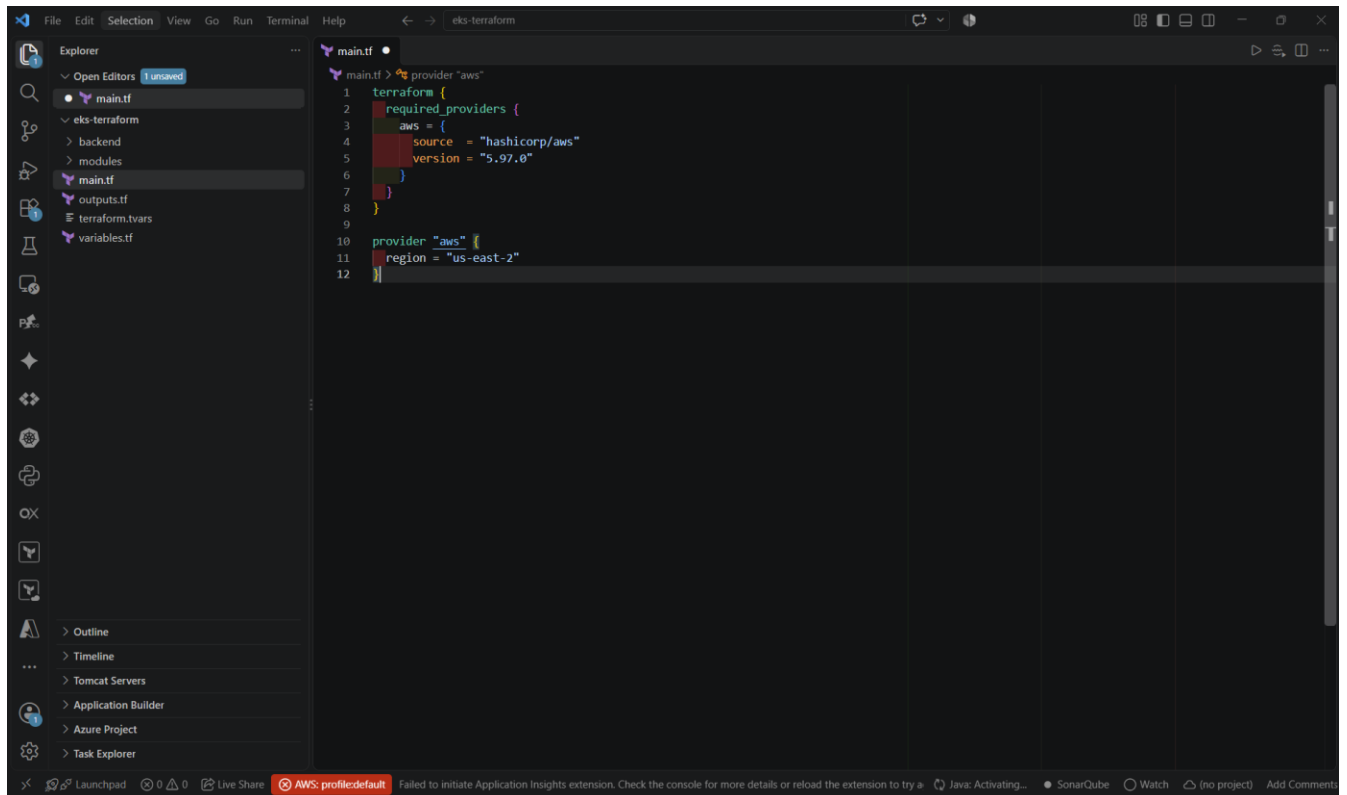
4 directories, 6 files
ebotsmith@ebot:/mnt/c/DevOps-Projects/eks-terraform$
```

You can see the version; we can change it to the version we want. Let us use version "5.97.0"

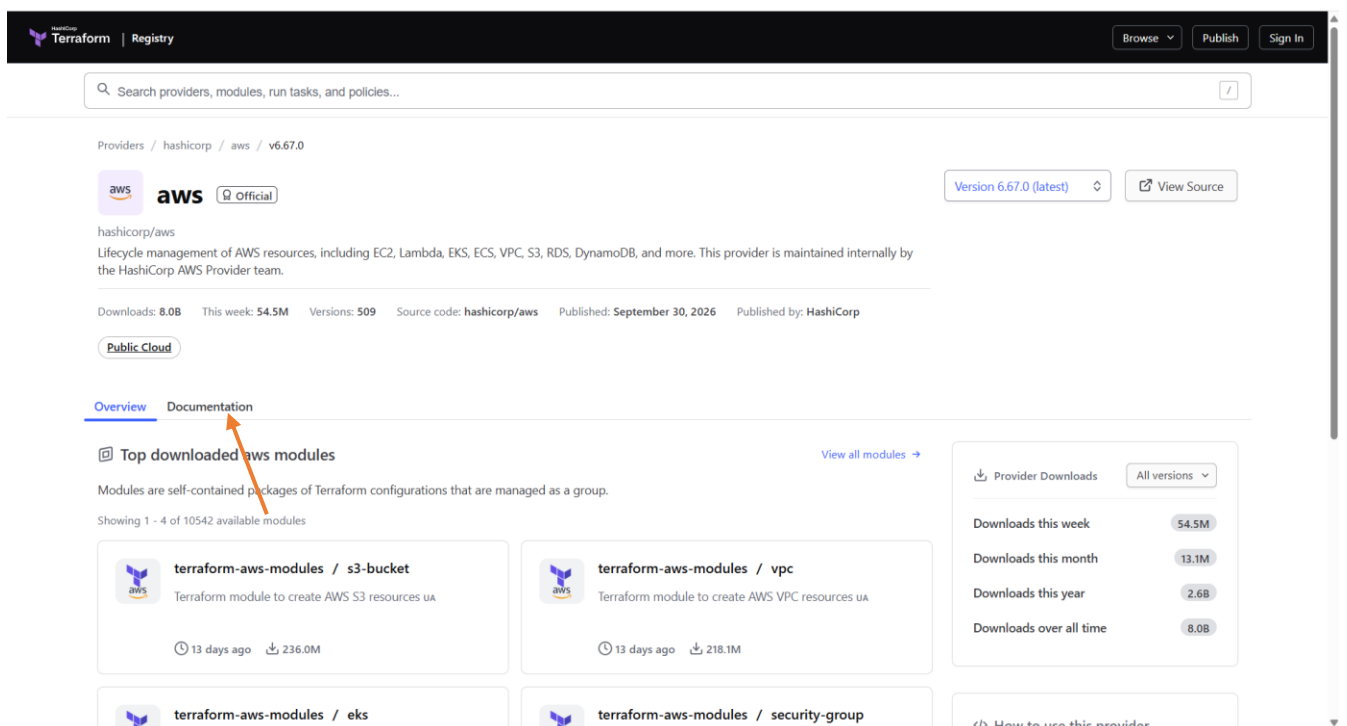


```
main.tf
1 terraform {
2   required_providers {
3     aws = {
4       source = "hashicorp/aws"
5       version = "5.97.0"
6     }
7   }
8 }
9
10 provider "aws" {
11   # configuration options
12 }
```

On the "Provider", we will enter our region. I am using "us-east-2"



The next resource we want to add here is the S3. To get the code for this, go to the Terraform registry.



Click on “documentation”

The screenshot shows the Terraform Registry page for the `aws` provider. The breadcrumb trail is `Providers / hashicorp / aws / v6.67.0`. The page header includes the Terraform logo, 'Registry', and buttons for 'Browse', 'Publish', and 'Sign In'. A search bar at the top contains the text 'Search providers, modules, run tasks, and policies...'. The main content area displays the `aws` provider details, including its version (6.67.0), source code, and publication date. A sidebar on the left shows a list of documentation topics under the heading 'AWS DOCUMENTATION'. An orange arrow points to the search bar in this sidebar, which contains the text 'Filter'. The main content area also features a search bar with the text 'Filter' and a list of documentation topics.

Type this in the search bar “aws s3 bucket”

This screenshot shows the same Terraform Registry page for the `aws` provider, but with search results displayed. The search bar in the sidebar now contains the text 'aws s3 bucket', and the results show 41 matching results. An orange arrow points to the first result, `aws_s3_bucket`, which is listed under the 'Resources' section. The main content area remains the same, showing the provider details and documentation topics.

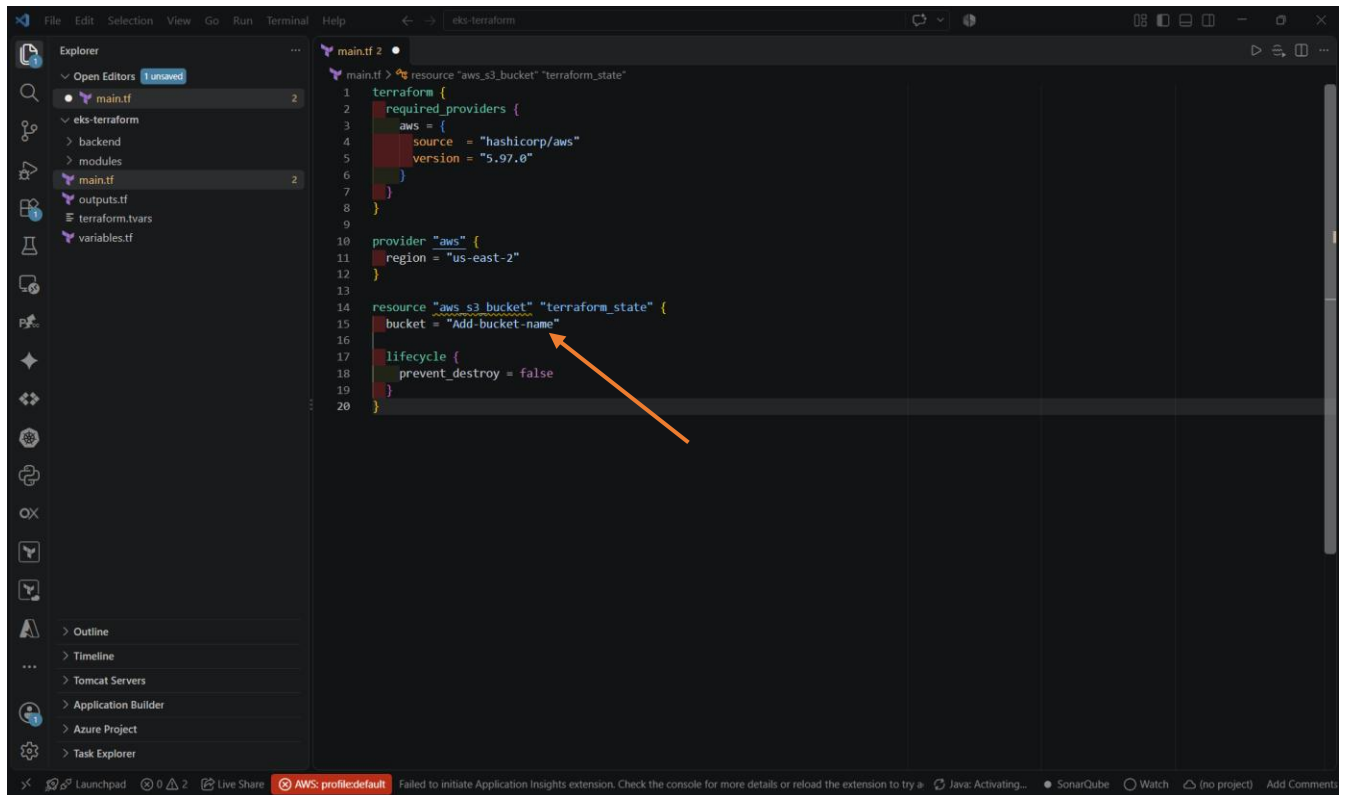
Click on “aws_s3_bucket”

The screenshot shows the Terraform Registry page for the `aws_s3_bucket` resource. The page is titled "Resource: aws_s3_bucket" and provides a description: "Provides a S3 bucket resource." It includes a "Note" section with information about managing S3 general purpose buckets and S3 Express directory buckets. The left sidebar shows a search for "aws s3 bucket" with 41 matching results, including `aws_s3_bucket` and various configuration resources. The right sidebar lists "ON THIS PAGE" links: Example Usage, Argument Reference, Attribute Reference, Timeouts, and Import.

Scroll down

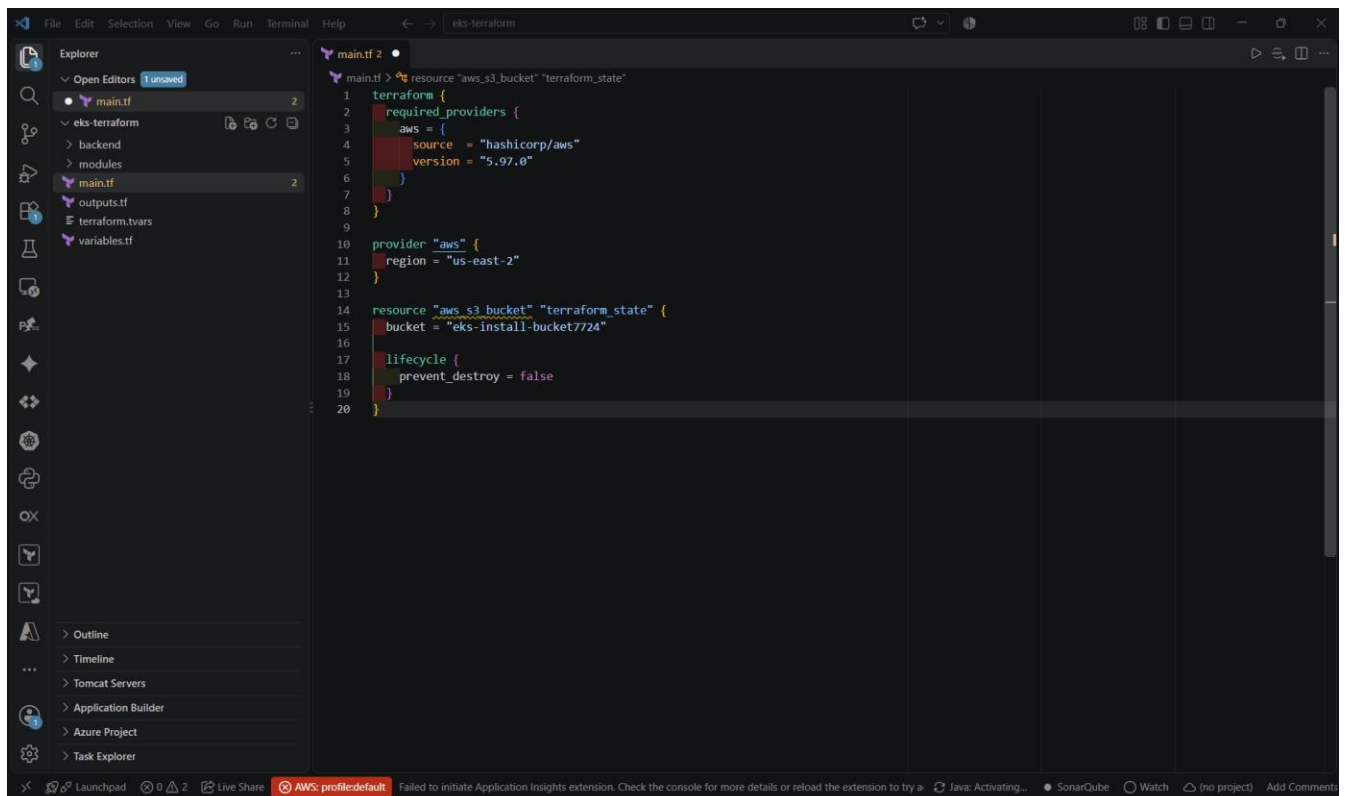
The screenshot shows the "Example Usage" section of the `aws_s3_bucket` resource page. It displays two examples: "Private Bucket With Tags" and "Bucket In Account-Regional Namespace". The "Private Bucket With Tags" example includes a `Copy` button, which is highlighted by an orange arrow. The "Bucket In Account-Regional Namespace" example also includes a `Copy` button. The left sidebar shows a search for "aws s3 bucket" with 41 matching results, including `aws_s3_bucket` and various configuration resources. The right sidebar lists "ON THIS PAGE" links: Attribute Reference, Timeouts, and Import.

Copy the S3 bucket resource and paste in the "main.tf" file.



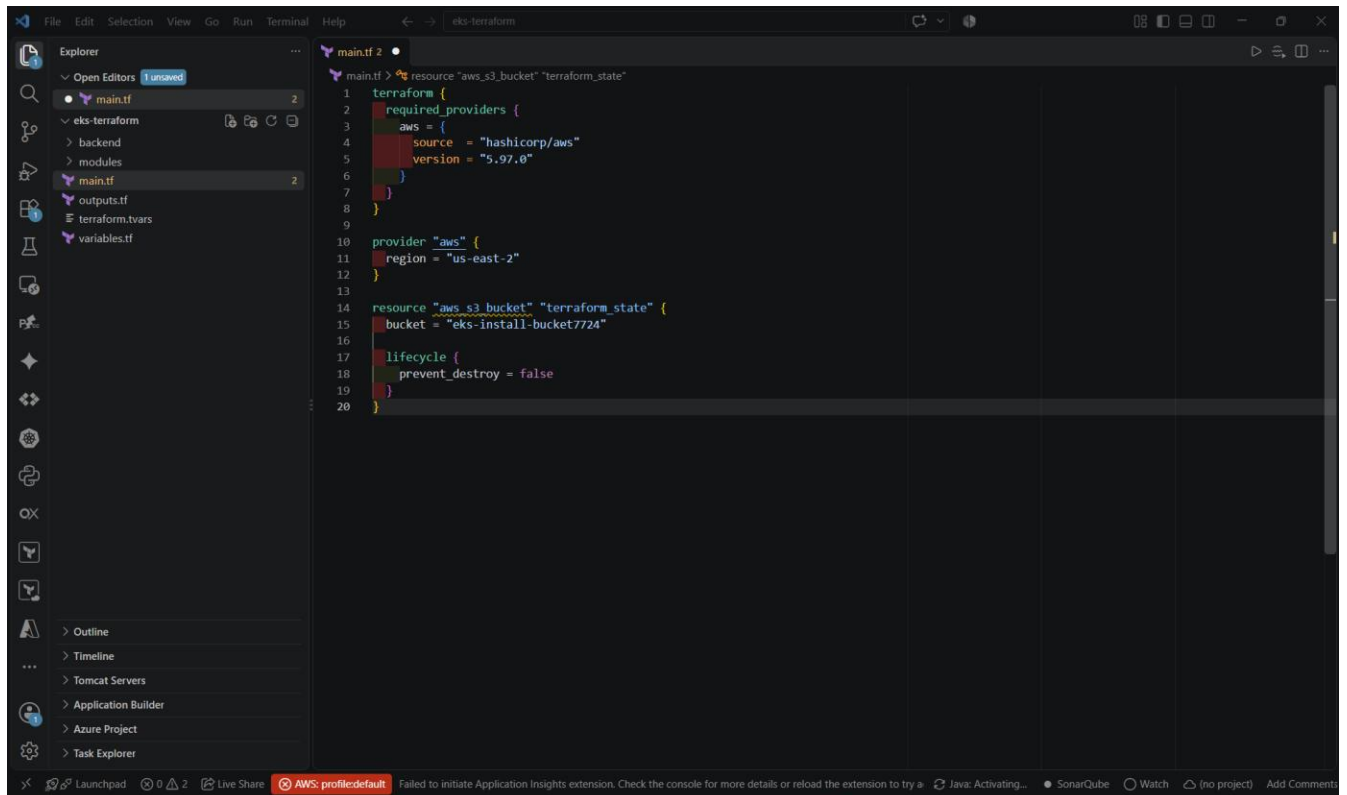
```
1 terraform {
2   required_providers {
3     aws = {
4       source = "hashicorp/aws"
5       version = "5.97.0"
6     }
7   }
8 }
9
10 provider "aws" {
11   region = "us-east-2"
12 }
13
14 resource "aws_s3_bucket" "terraform_state" {
15   bucket = "Add-bucket-name"
16
17   lifecycle {
18     prevent_destroy = false
19   }
20 }
```

We have to modify the code for the S3 bucket to give it a unique name. We will call it “**eks-install-bucket7724**”

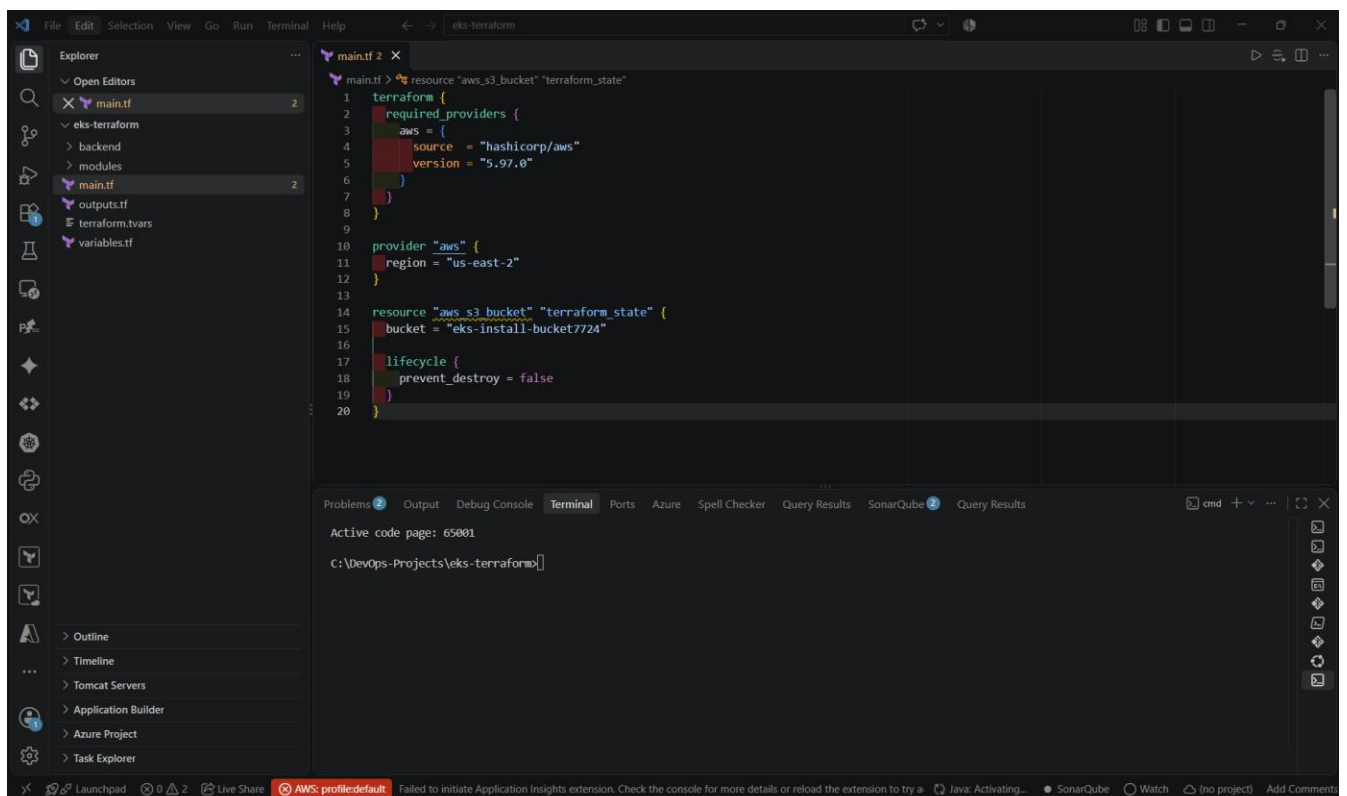


```
1 terraform {
2   required_providers {
3     aws = {
4       source = "hashicorp/aws"
5       version = "5.97.0"
6     }
7   }
8 }
9
10 provider "aws" {
11   region = "us-east-2"
12 }
13
14 resource "aws_s3_bucket" "terraform_state" {
15   bucket = "eks-install-bucket7724"
16
17   lifecycle {
18     prevent_destroy = false
19   }
20 }
```

This is all for the configuration of the S3 bucket. Let us try to create the bucket before adding the next resource.

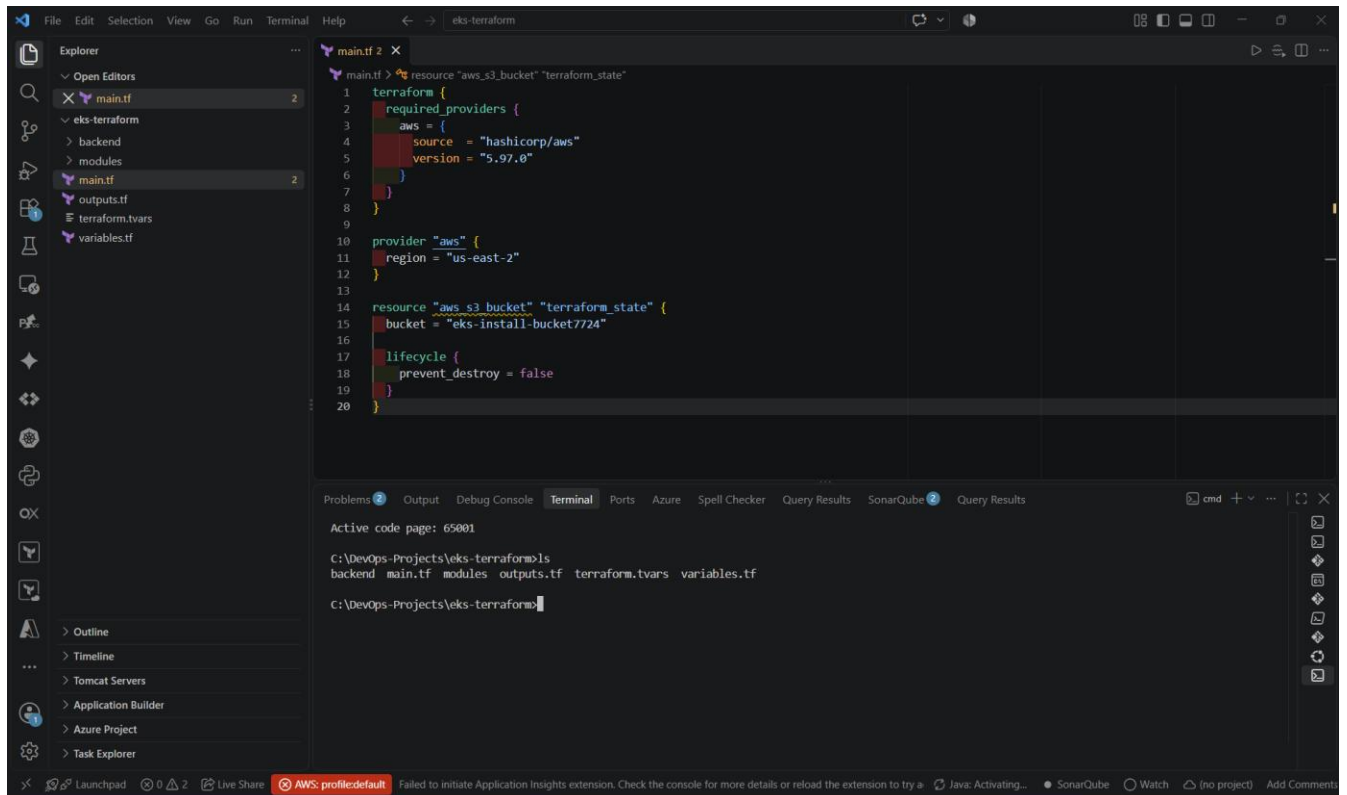


Save the changes. Click on the empty space and open the integrated terminal



Run the command to see the content of our root folder:

```
ls
```



The screenshot shows the Visual Studio Code interface with a Terraform configuration file named `main.tf` open. The file contains the following Terraform code:

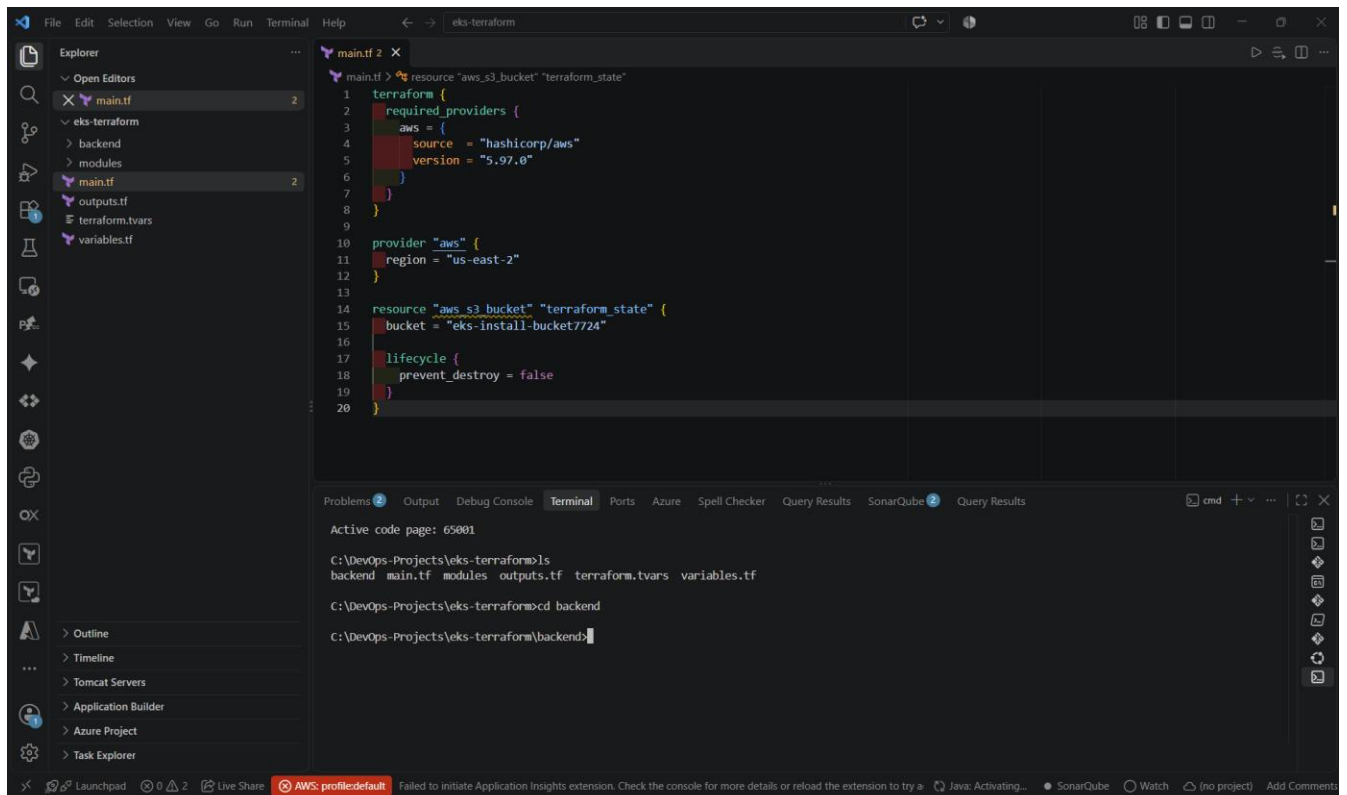
```
1 terraform {
2   required_providers {
3     aws = {
4       source = "hashicorp/aws"
5       version = "5.97.0"
6     }
7   }
8 }
9
10 provider "aws" {
11   region = "us-east-2"
12 }
13
14 resource "aws_s3_bucket" "terraform_state" {
15   bucket = "eks-install-bucket7724"
16
17   lifecycle {
18     prevent_destroy = false
19   }
20 }
```

The terminal window at the bottom shows the following commands and output:

```
Active code page: 65001
C:\DevOps-Projects\eks-terraform>ls
backend main.tf modules outputs.tf terraform.tvvars variables.tf
C:\DevOps-Projects\eks-terraform>
```

Navigate into the “**backend**” folder using the command:

```
cd backend
```

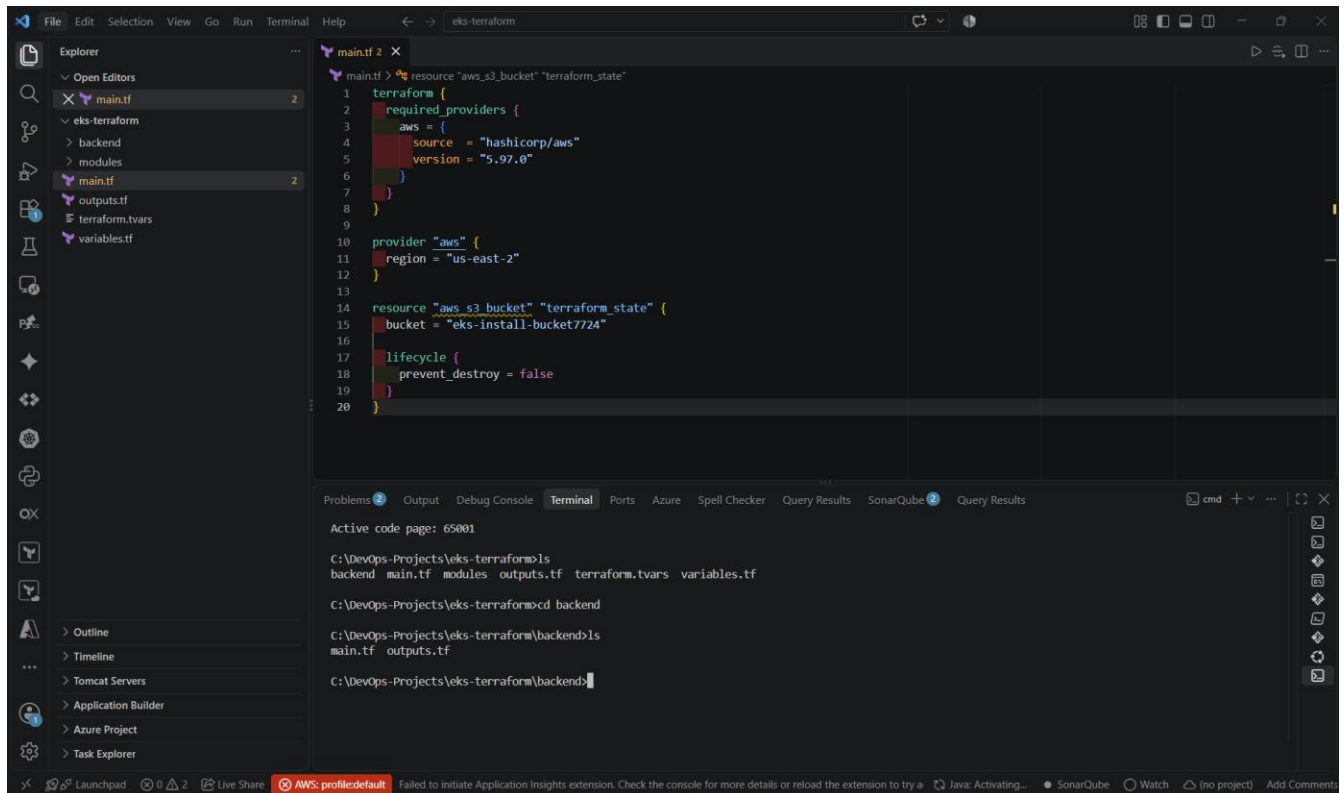


The screenshot shows the Visual Studio Code interface with the same Terraform configuration file `main.tf` open. The terminal window now shows the command `cd backend` being executed:

```
Active code page: 65001
C:\DevOps-Projects\eks-terraform>ls
backend main.tf modules outputs.tf terraform.tvvars variables.tf
C:\DevOps-Projects\eks-terraform>cd backend
C:\DevOps-Projects\eks-terraform\backend>
```

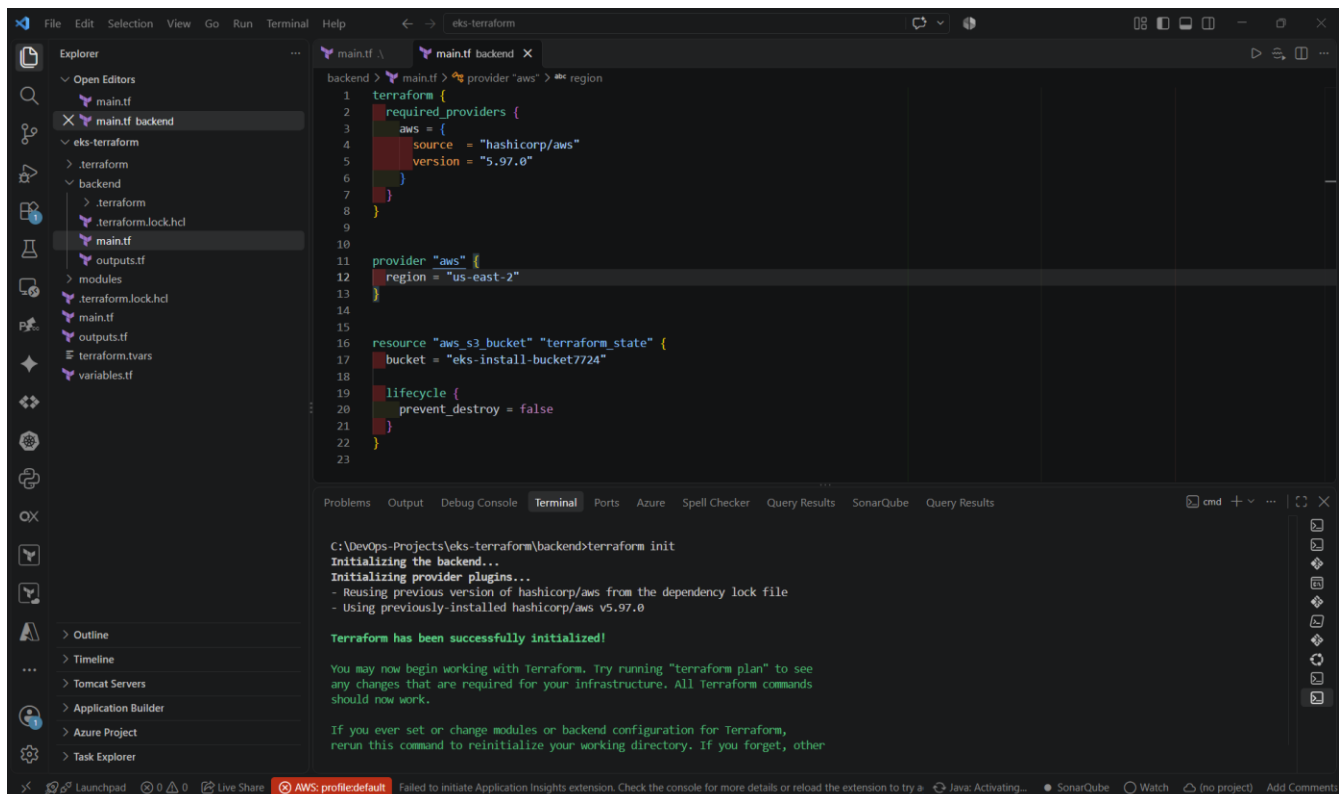
Then, check the content of the “**backend**” folder using the command:

ls



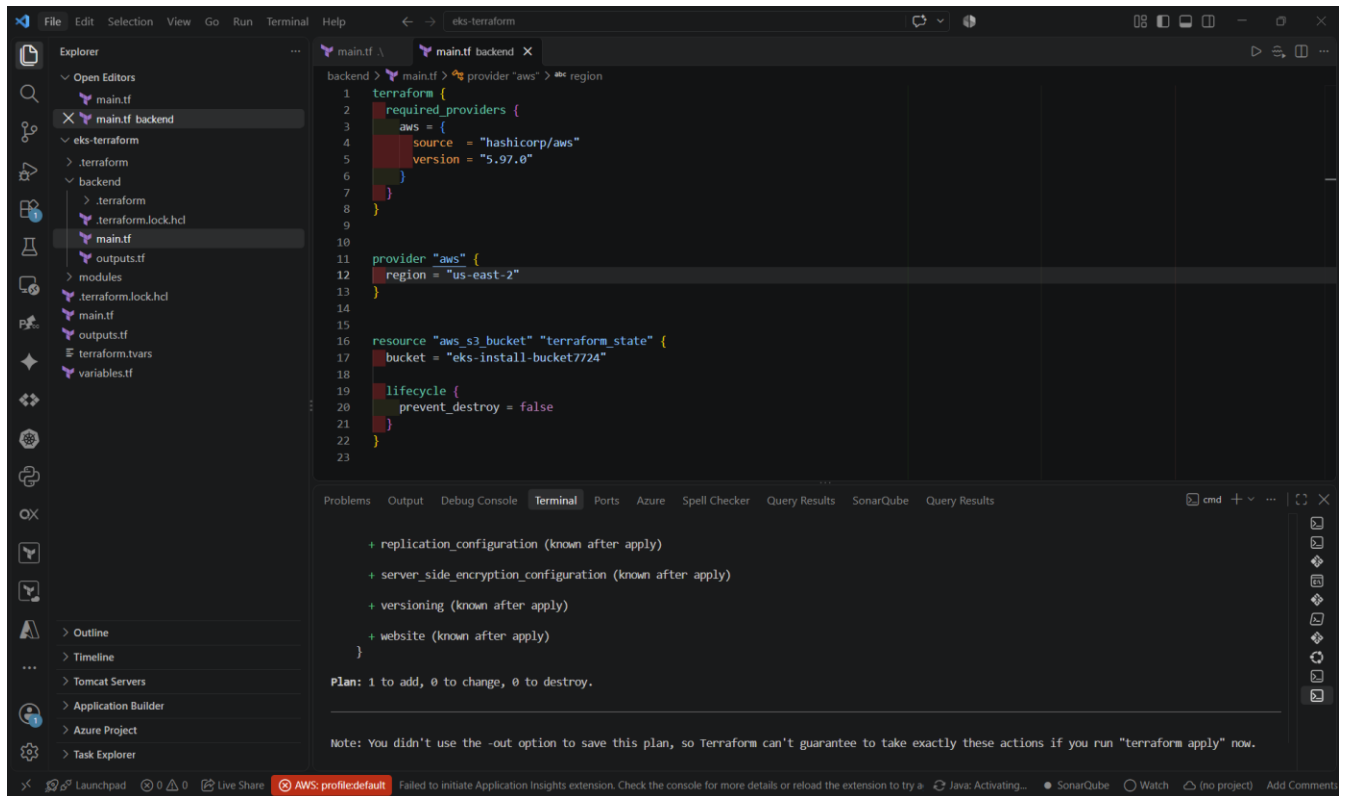
You can see the two files we created in our “backend” folder. Let us initialize this using the command:

```
terraform init
```



The “**backend**” has been initialized successfully. Let us run the command to review the resources we are about to provision:

```
terraform plan
```

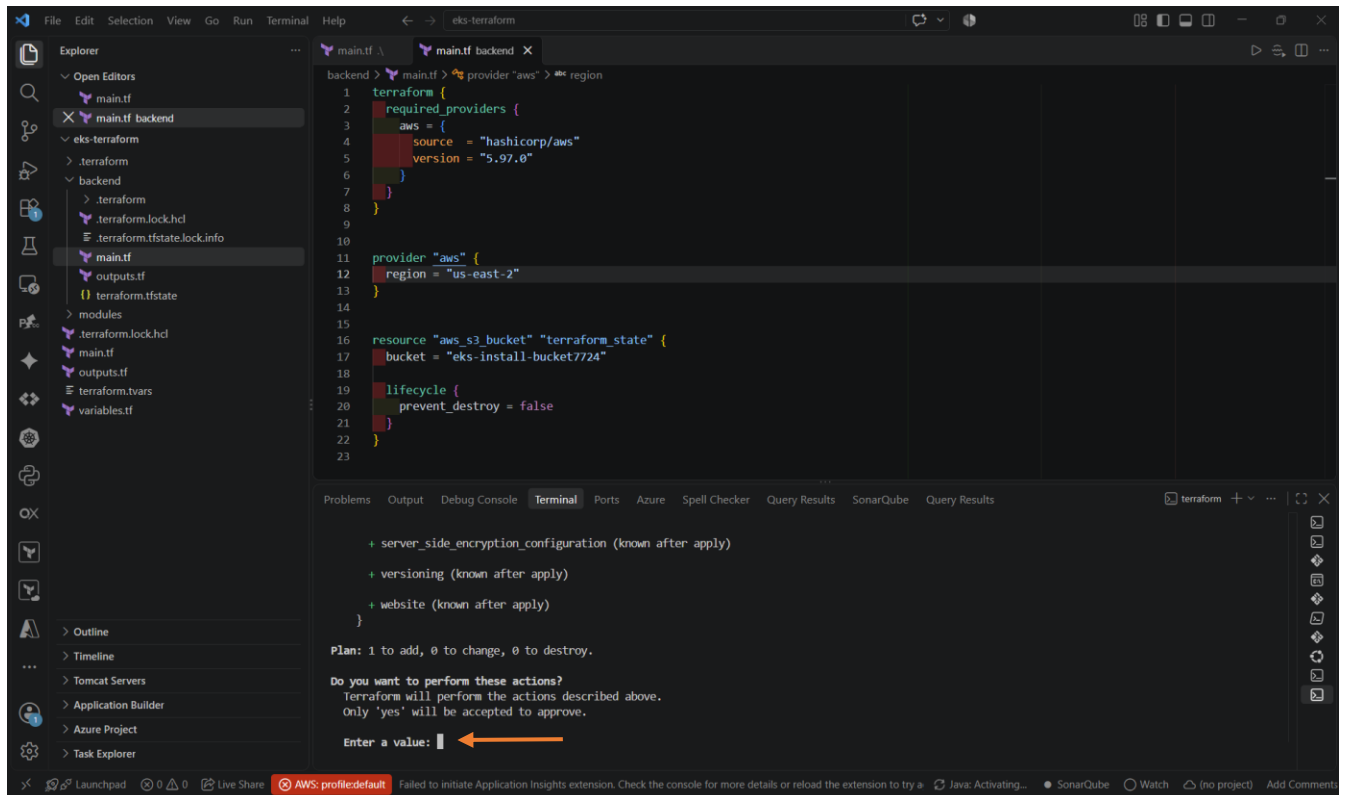


You might encounter some problem, that will require you to configure you AWS Credentials by using the command:

```
aws configure
```

Now, let us run the command to apply the changes:

```
terraform apply
```



The screenshot shows the Visual Studio Code interface with a Terraform configuration file open. The file is named `main.tf` and is located in the `eks-terraform` workspace. The configuration defines a Terraform provider for AWS, sets the region to `us-east-2`, and creates an `aws_s3_bucket` resource named `terraform_state` with the bucket name `eks-install-bucket7724`. The `lifecycle` block for the bucket has `prevent_destroy = false`.

The terminal output shows the Terraform plan and the prompt to confirm the actions:

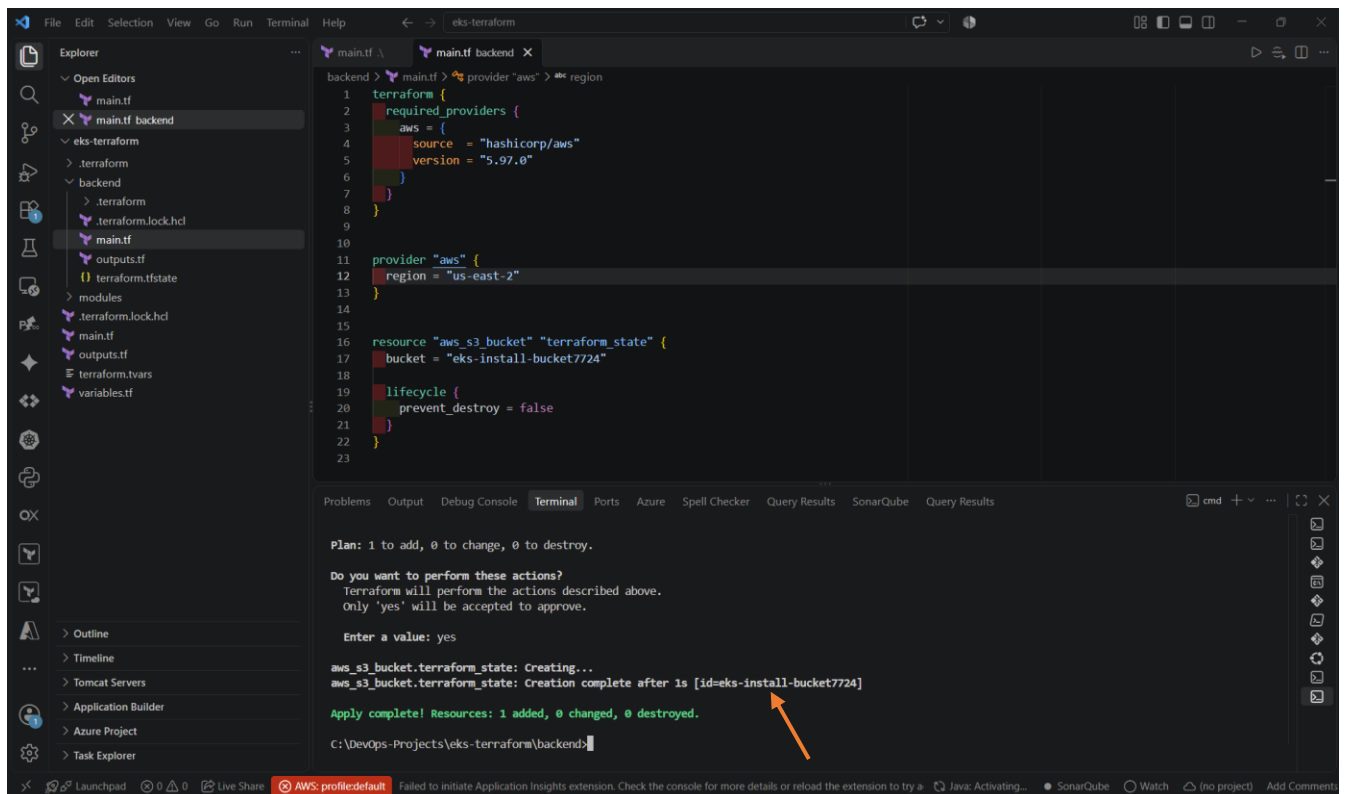
```
Plan: 1 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: 
```

An orange arrow points to the prompt `Enter a value:` .

Enter “yes” to confirm that you want to deploy the changes



The screenshot shows the Visual Studio Code interface with the same Terraform configuration file. The terminal output now shows the Terraform plan and the confirmation prompt, with the user entering `yes`:

```
Plan: 1 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

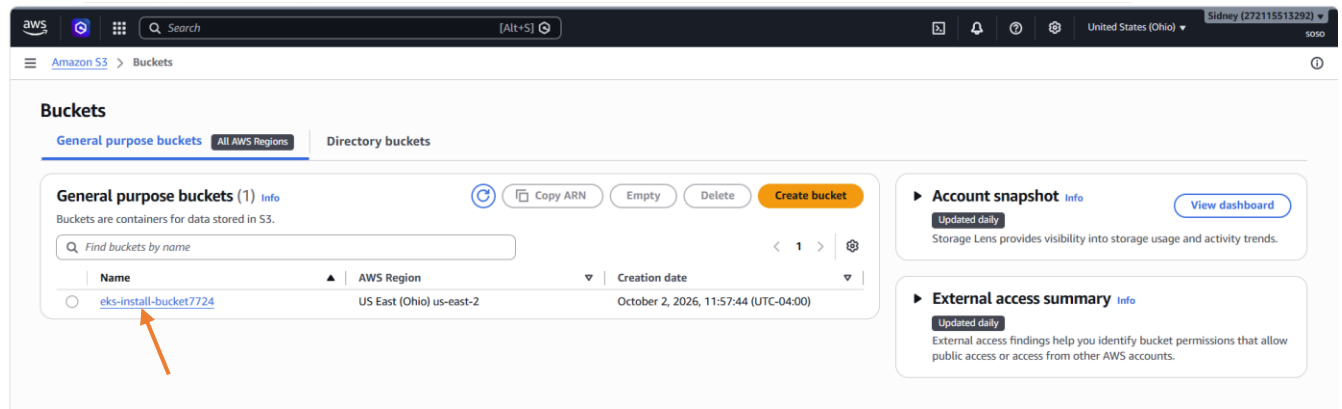
aws_s3_bucket.terraform_state: Creating...
aws_s3_bucket.terraform_state: Creation complete after 1s [id=eks-install-bucket7724]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

C:\DevOps-Projects\eks-terraform\backend>
```

An orange arrow points to the `aws_s3_bucket.terraform_state: Creation complete after 1s [id=eks-install-bucket7724]` line.

You can see that one resource has been added and you can see the name. Let us go to AWS management console to confirm that the S3 bucket has been created.



You can see that the bucket has been created.

4. Configure S3 State Locking and Migrate Local Terraform State to Remote Backend

In this section, we will be adding our S3 native state locking. You will see how S3 can actually lock our Terraform state file.

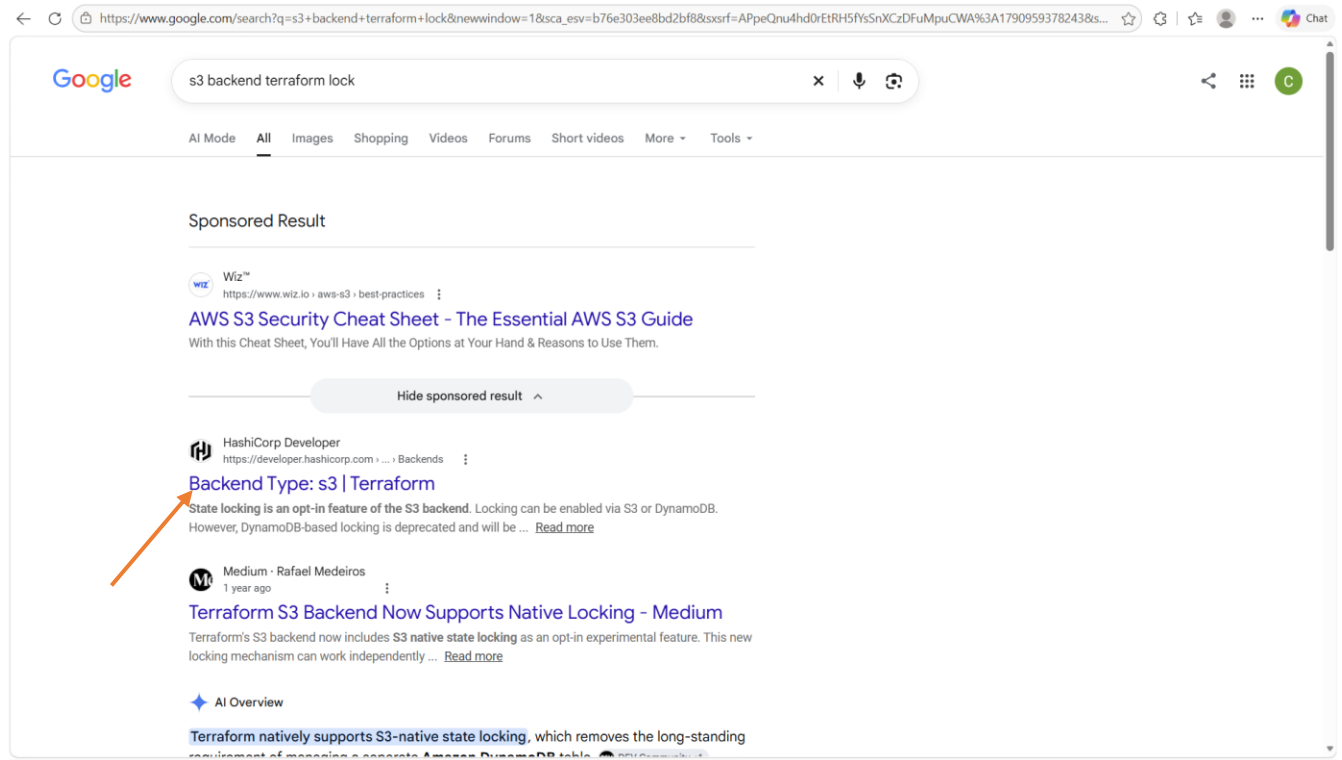
Traditionally, Terraform used DynamoDB for state locking before now. We are using it to avoid conflict when multiple users or pipelines are trying to access the same state file. It actually worked well, but it came with added cost and management overhead of maintaining a DynamoDB table.

Using S3 state locking, which is actually available in Terraform 1.9.0 and above. Terraform offers a built-in locking mechanism directly within S3. So, this helps to remove the need for a separate DynamoDB table. It will help to simplify our set up and it will also reduce cost.

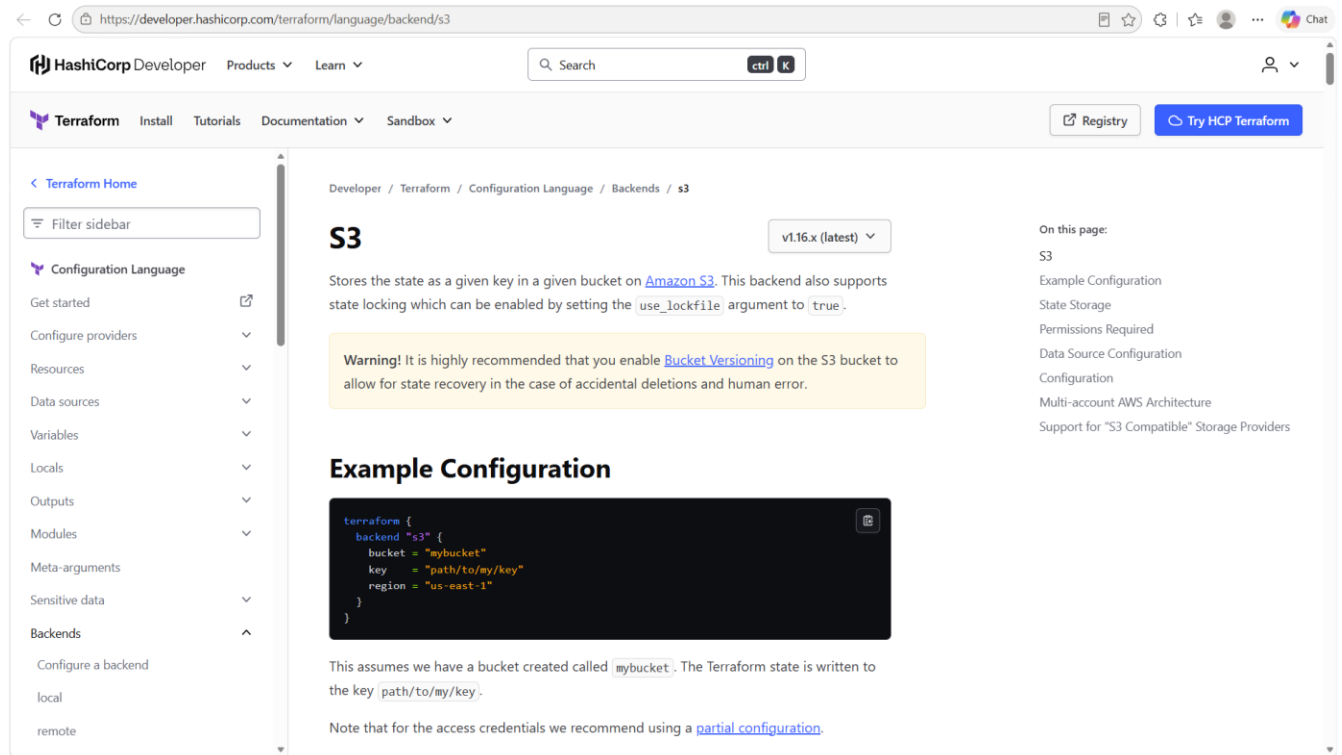
You can still use DynamoDB if needed but that could be duplicated very soon.

4.1. Configure S3 State Locking

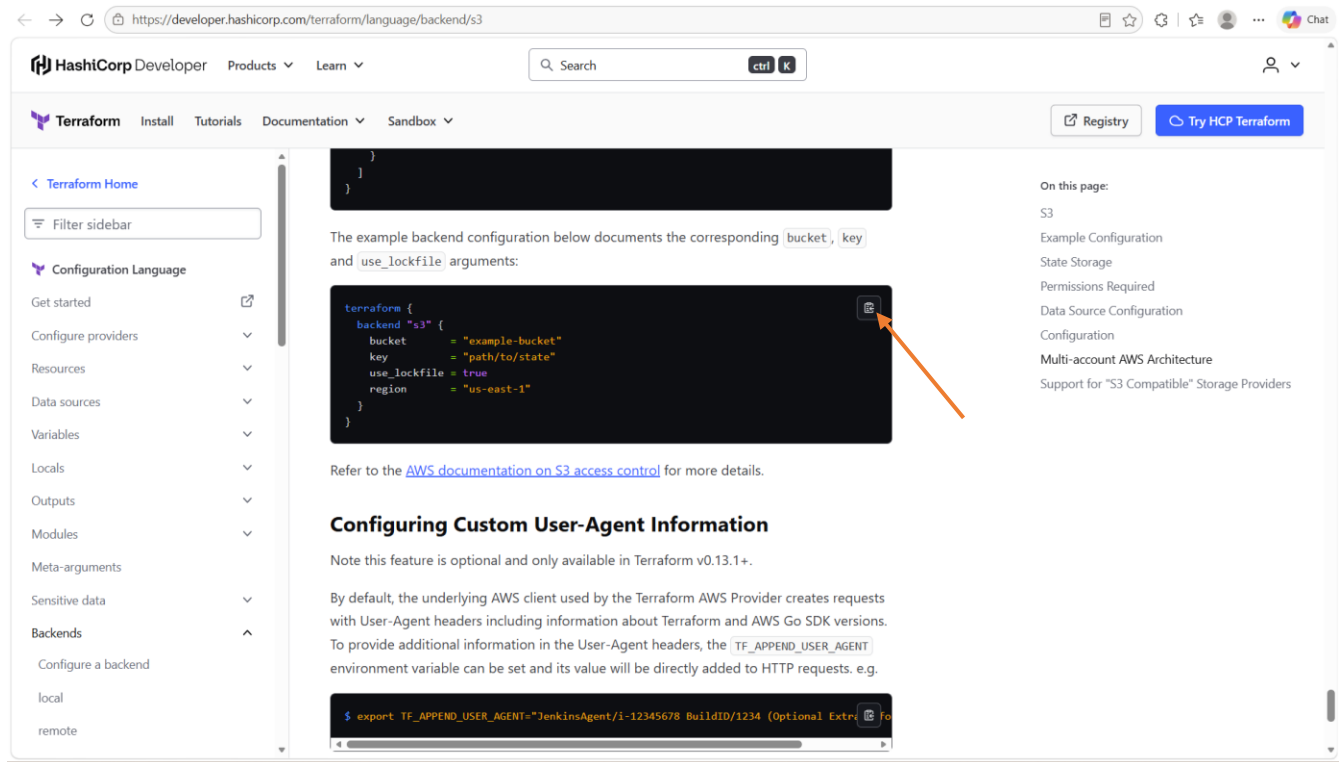
Let us go ahead with the set up. Go to google and search for “**backend S3 Terraform lock**”



Click on “Backend Type: s3 | Terraform”



Scroll down to “State Locking”



HashiCorp Developer Products Learn Search **ctrl K**

Terraform Install Tutorials Documentation Sandbox Registry Try HCP Terraform

< Terraform Home

Filter sidebar

Configuration Language

- Get started
- Configure providers
- Resources
- Data sources
- Variables
- Locals
- Outputs
- Modules
- Meta-arguments
- Sensitive data
- Backends
 - Configure a backend
 - local
 - remote

The example backend configuration below documents the corresponding `bucket`, `key` and `use_lockfile` arguments:

```
terraform {
  backend "s3" {
    bucket     = "example-bucket"
    key        = "path/to/state"
    use_lockfile = true
    region     = "us-east-1"
  }
}
```

Refer to the [AWS documentation on S3 access control](#) for more details.

Configuring Custom User-Agent Information

Note this feature is optional and only available in Terraform v0.13.1+.

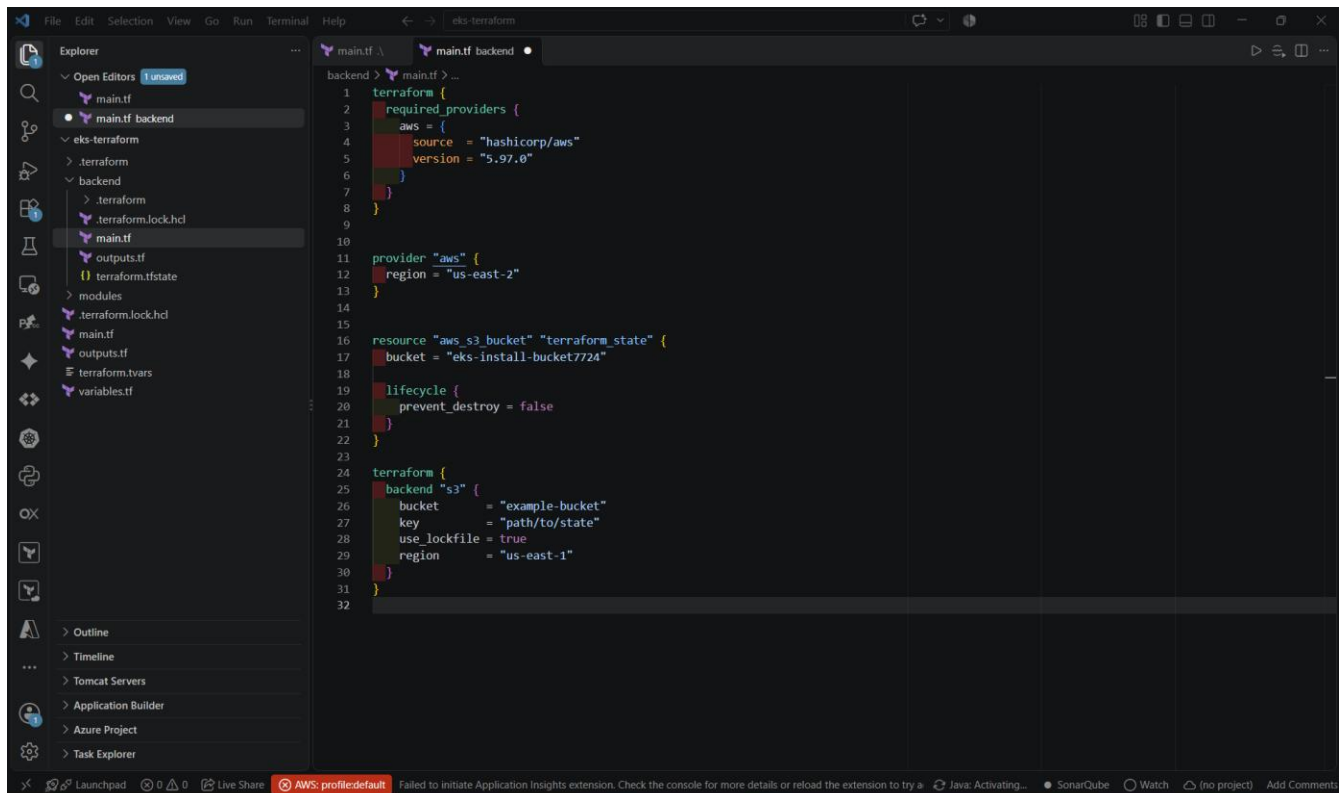
By default, the underlying AWS client used by the Terraform AWS Provider creates requests with User-Agent headers including information about Terraform and AWS Go SDK versions. To provide additional information in the User-Agent headers, the `TF_APPEND_USER_AGENT` environment variable can be set and its value will be directly added to HTTP requests. e.g.

```
$ export TF_APPEND_USER_AGENT="JenkinsAgent/i-12345678 BuildID/1234 (Optional Extra Info)"
```

On this page:

- S3
- Example Configuration
- State Storage
- Permissions Required
- Data Source Configuration
- Configuration
- Multi-account AWS Architecture
- Support for "S3 Compatible" Storage Providers

Copy this and paste on the "main.tf" file in the "backend" folder



```
1 terraform {
2   required_providers {
3     aws = {
4       source = "hashicorp/aws"
5       version = "5.97.0"
6     }
7   }
8 }
9
10
11 provider "aws" {
12   region = "us-east-2"
13 }
14
15
16 resource "aws_s3_bucket" "terraform_state" {
17   bucket = "eks-install-bucket7724"
18
19   lifecycle {
20     prevent_destroy = false
21   }
22 }
23
24 terraform {
25   backend "s3" {
26     bucket     = "example-bucket"
27     key        = "path/to/state"
28     use_lockfile = true
29     region     = "us-east-1"
30   }
31 }
32
```

Let us modify the added code

```
terraform {
  backend "s3" {
    bucket     = "Add-bucket-name"
```

```
    key          = "Add-path-of-terraform-state-file"
    region       = "us-east-2"
    encrypt      = true
    use_lockfile = true    #S3 native locking
  }
}
```

We have to add the name of our bucket and the path to the Terraform State file. The name of our S3 bucket is **"eks-install-bucket7724"**

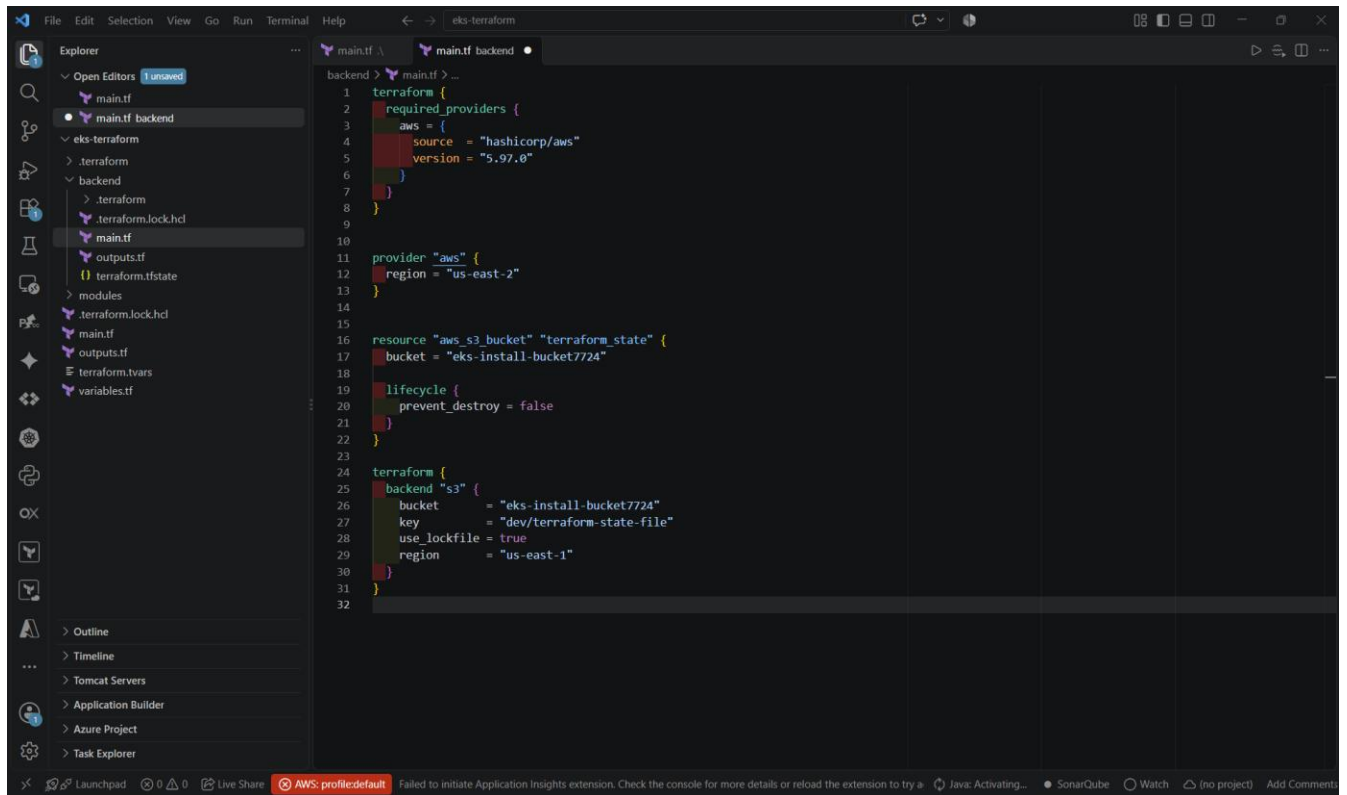
```
terraform {
  backend "s3" {
    bucket      = " eks-install-bucket7724"
    key         = "Add-path-of-terraform-state-file"
    region      = "us-east-2"
    encrypt     = true
    use_lockfile = true    #S3 native locking
  }
}
```

Then for the path of the Terraform state file, we will create a folder for the file. So, we will add this:

"dev/terraform-state-file"

```
terraform {
  backend "s3" {
    bucket      = " eks-install-bucket7724"
    key         = " dev/terraform-state-file"
    region      = "us-east-2"
    encrypt     = true
    use_lockfile = true    #S3 native locking
  }
}
```

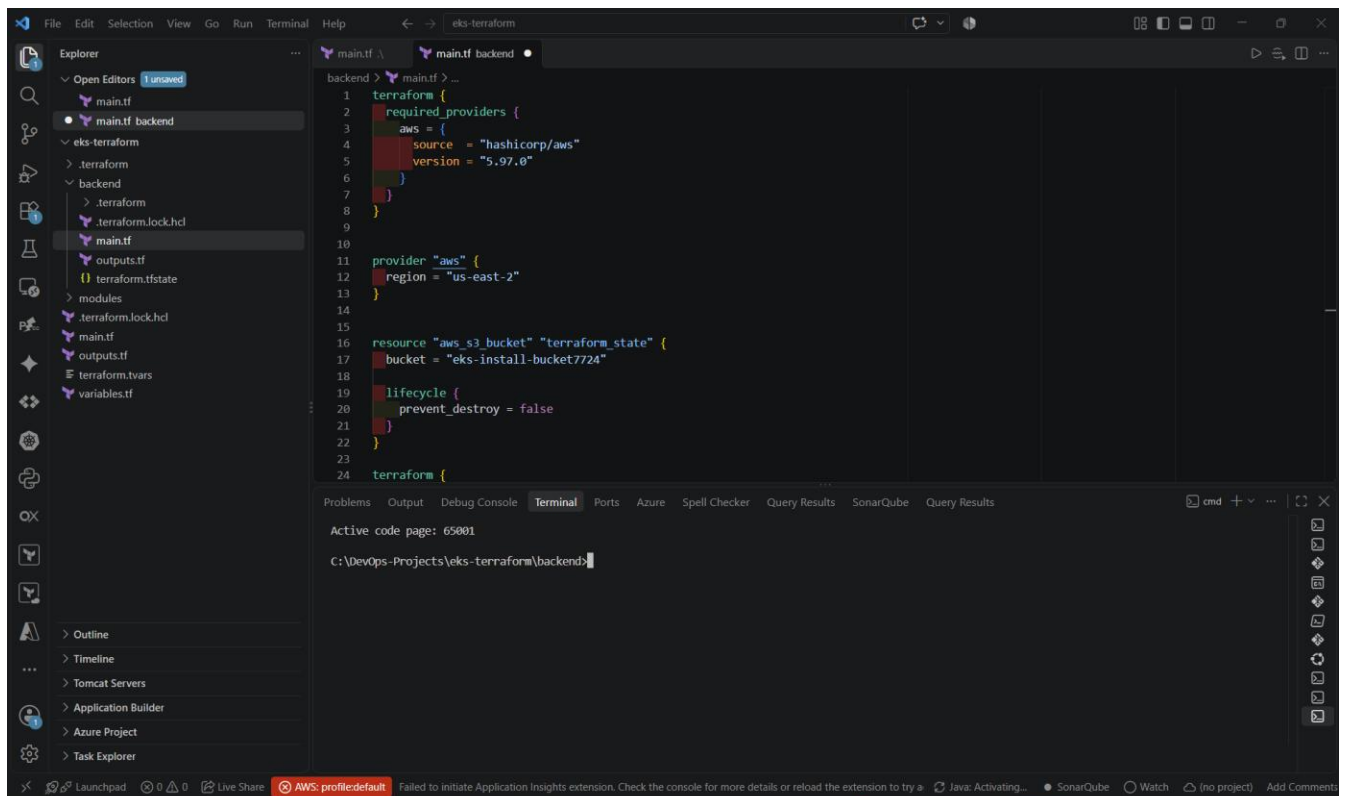
With **"use_lockfile"** set to true the setup is going to automatically use the native locking



This is all for the configuration of S3 state locking.

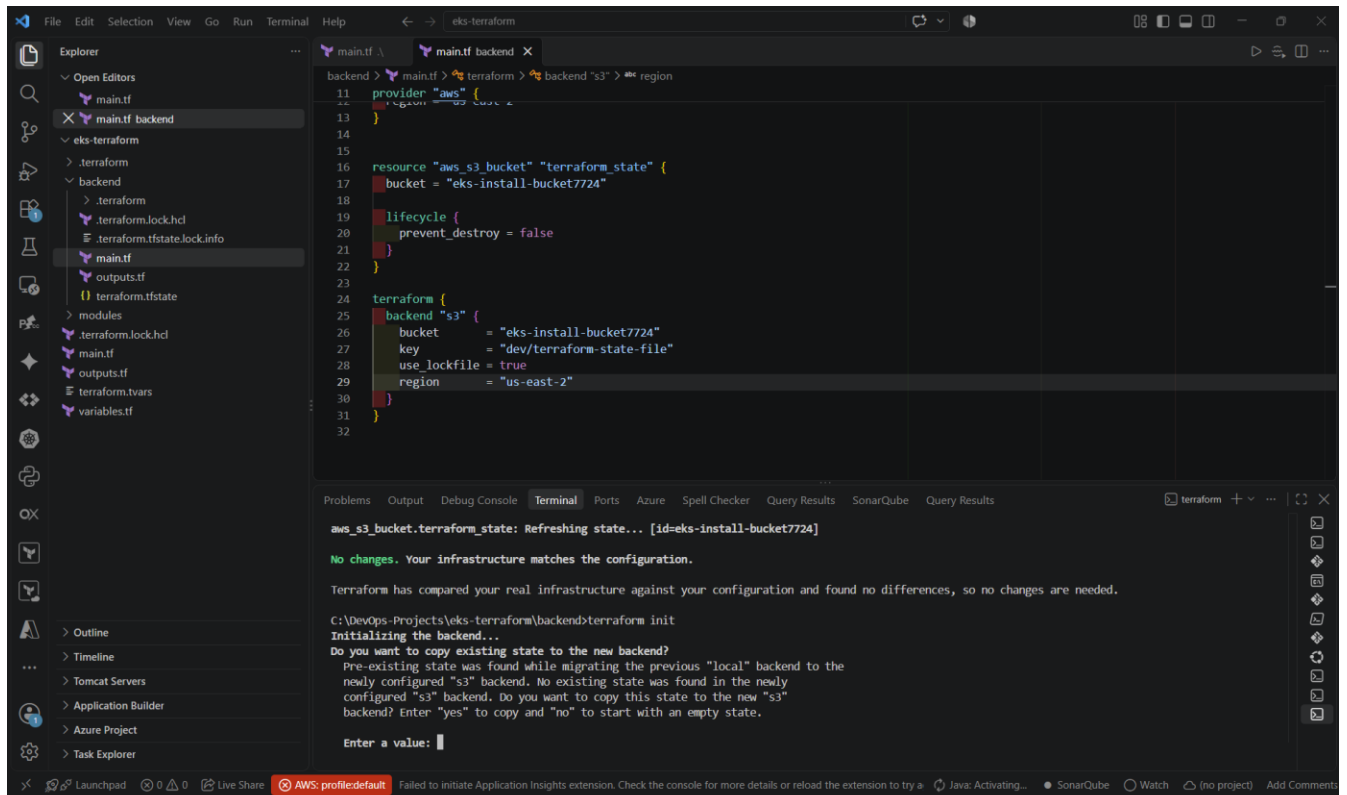
4.2. Migrate Local Terraform State to Remote Backend

We have to migrate the Terraform state file that is local to the remote “**backend**”. Let us open the terminal again



Make sure we are on the “**backend**” folder and run the command to initialize:

terraform init



```
File Edit Selection View Go Run Terminal Help
main.tf backend
main.tf backend
eks-terraform
.terraform
backend
.terraform
main.tf
outputs.tf
terraform.tfstate
modules
.terraform.lock.hcl
main.tf
outputs.tf
terraform.tfvars
variables.tf

11 provider "aws" {
12   region = "us-east-2"
13 }
14
15
16 resource "aws_s3_bucket" "terraform_state" {
17   bucket = "eks-install-bucket7724"
18
19   lifecycle {
20     prevent_destroy = false
21   }
22 }
23
24 terraform {
25   backend "s3" {
26     bucket = "eks-install-bucket7724"
27     key    = "dev/terraform-state-file"
28     use_lockfile = true
29     region = "us-east-2"
30   }
31 }
32

Problems Output Debug Console Terminal Ports Azure Spell Checker Query Results SonarQube Query Results
aws_s3_bucket.terraform_state: Refreshing state... [id=eks-install-bucket7724]

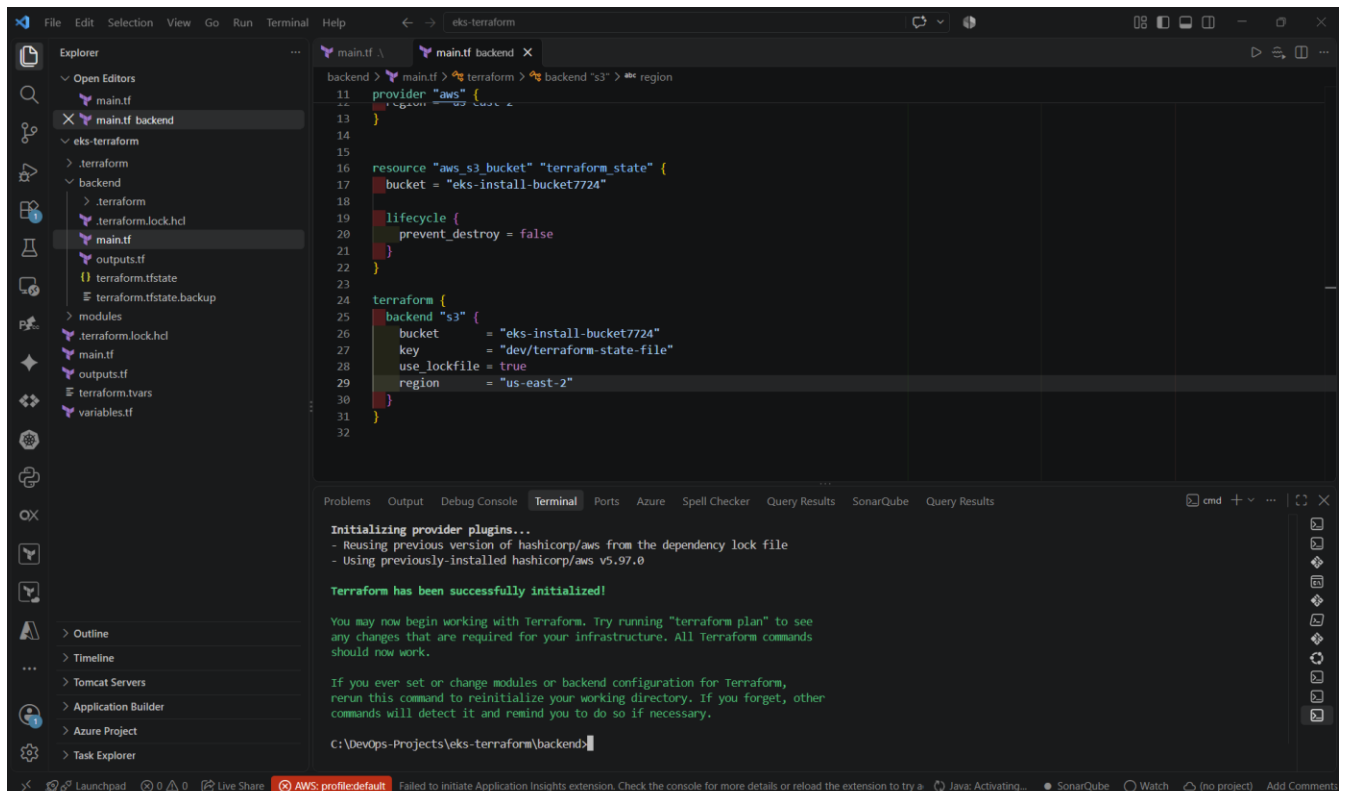
No changes. Your infrastructure matches the configuration.

Terraform has compared your real infrastructure against your configuration and found no differences, so no changes are needed.

C:\DevOps-Projects\eks-terraform\backend>terraform init
Initializing the backend...
Do you want to copy existing state to the new backend?
Pre-existing state was found while migrating the previous "local" backend to the
newly configured "s3" backend. No existing state was found in the newly
configured "s3" backend. Do you want to copy this state to the new "s3"
backend? Enter "yes" to copy and "no" to start with an empty state.

Enter a value: 
```

It is asking if you want to copy existing state to the new backend. Enter “**yes**” and press “**Enter**”



```
File Edit Selection View Go Run Terminal Help
main.tf backend
main.tf backend
eks-terraform
.terraform
backend
.terraform
main.tf
outputs.tf
terraform.tfstate
modules
.terraform.lock.hcl
main.tf
outputs.tf
terraform.tfvars
variables.tf

11 provider "aws" {
12   region = "us-east-2"
13 }
14
15
16 resource "aws_s3_bucket" "terraform_state" {
17   bucket = "eks-install-bucket7724"
18
19   lifecycle {
20     prevent_destroy = false
21   }
22 }
23
24 terraform {
25   backend "s3" {
26     bucket = "eks-install-bucket7724"
27     key    = "dev/terraform-state-file"
28     use_lockfile = true
29     region = "us-east-2"
30   }
31 }
32

Problems Output Debug Console Terminal Ports Azure Spell Checker Query Results SonarQube Query Results
Initializing provider plugins...
- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/aws v5.97.0

Terraform has been successfully initialized!

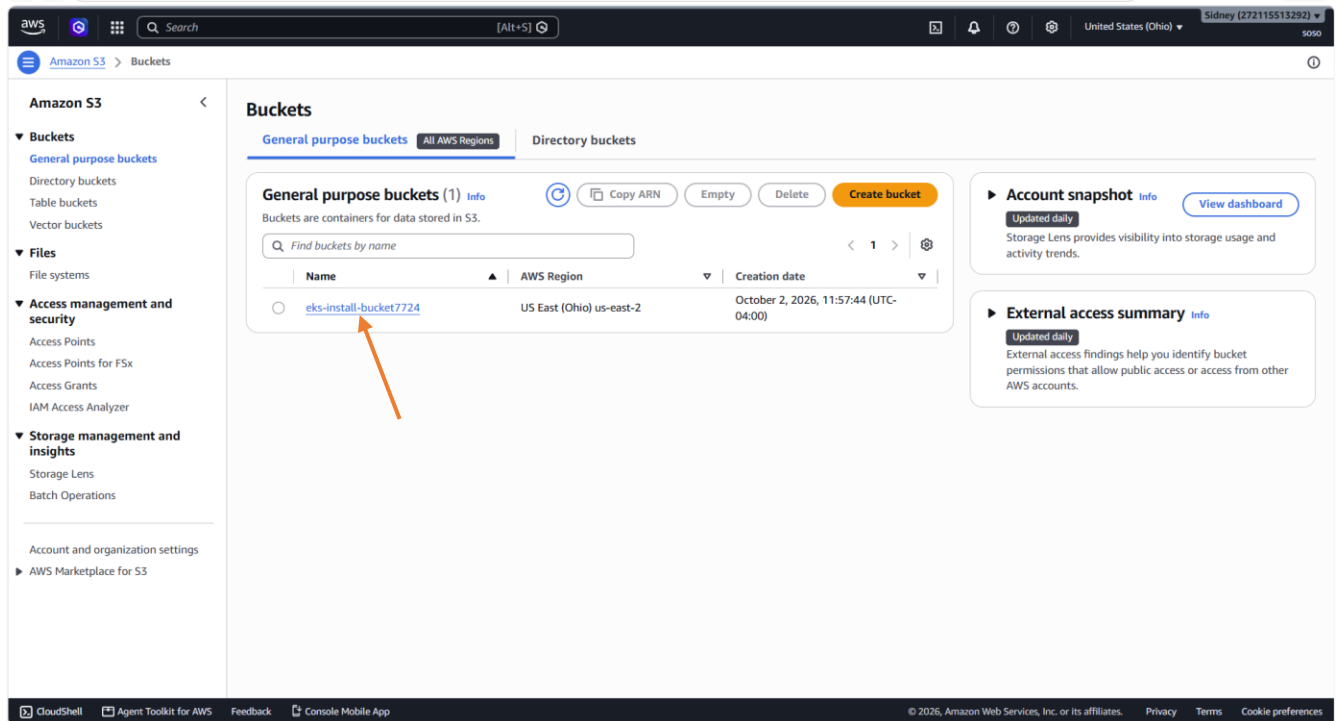
You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.

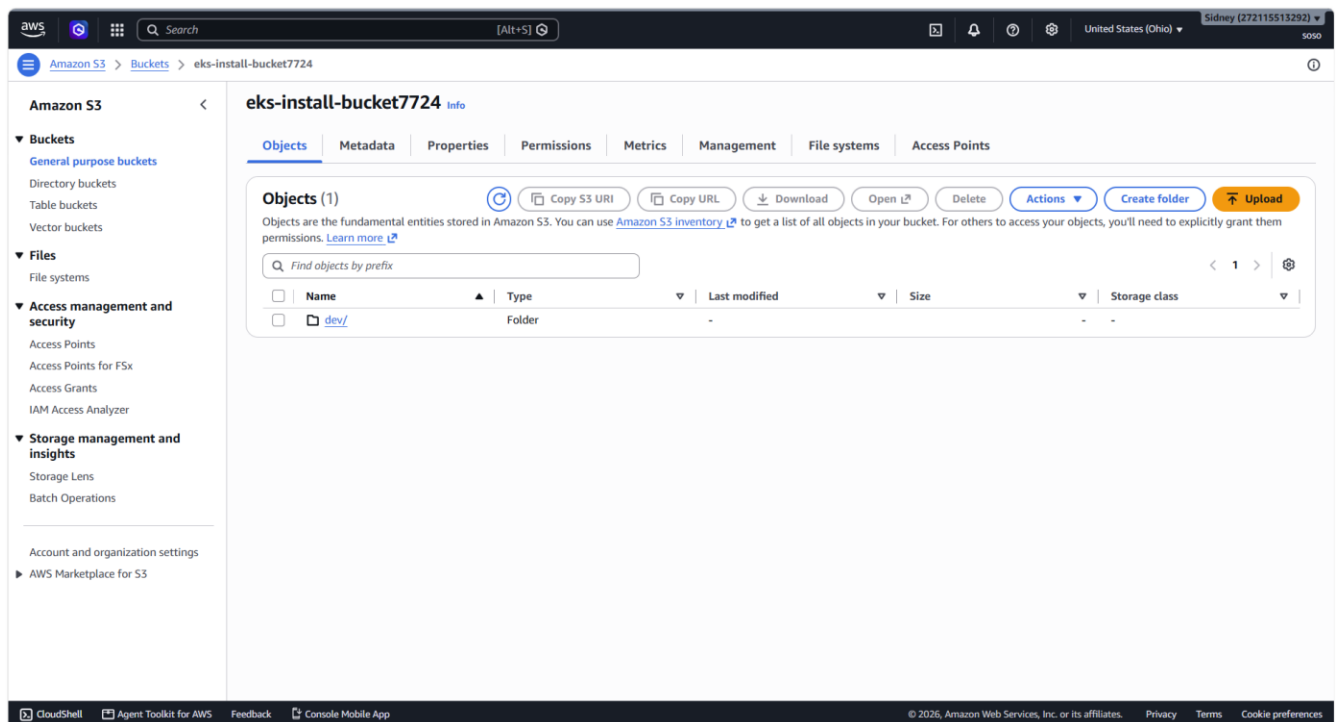
C:\DevOps-Projects\eks-terraform\backend>
```

Prepared by Sidney Smith Ebot

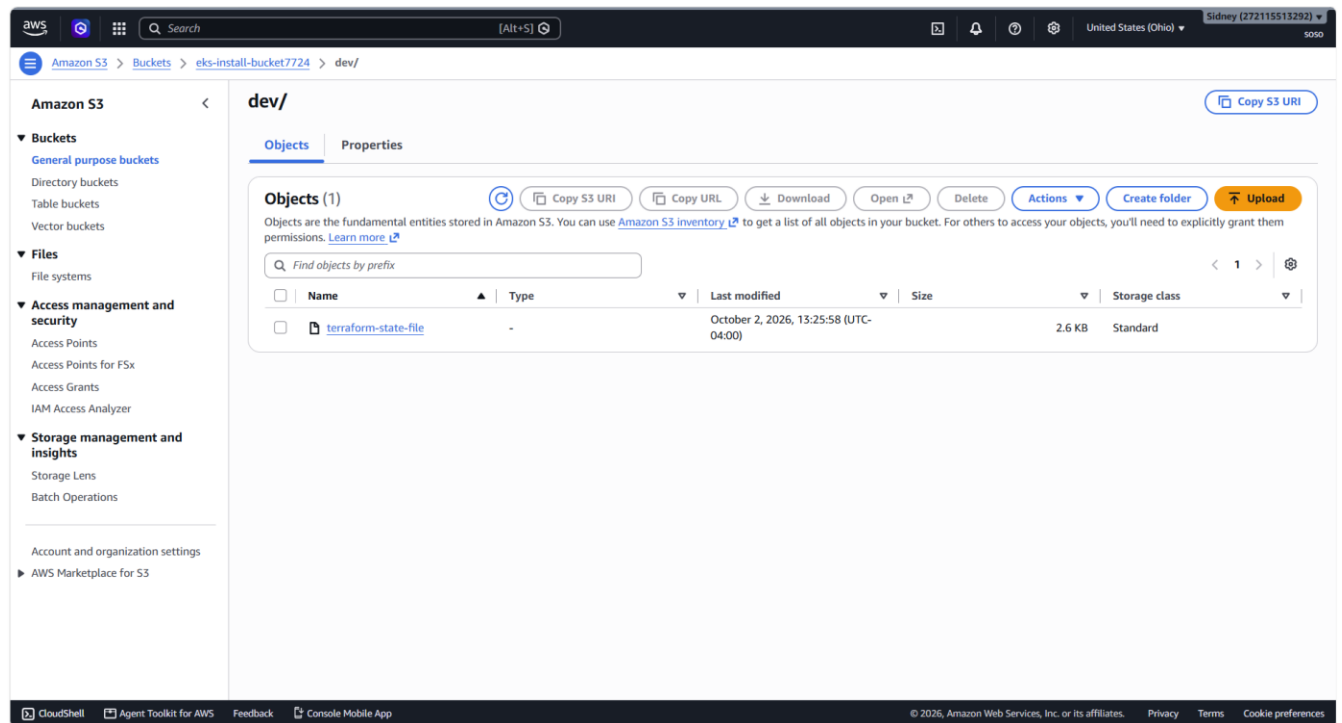
Now, go back to our S3 bucket and refresh it.



Open the bucket

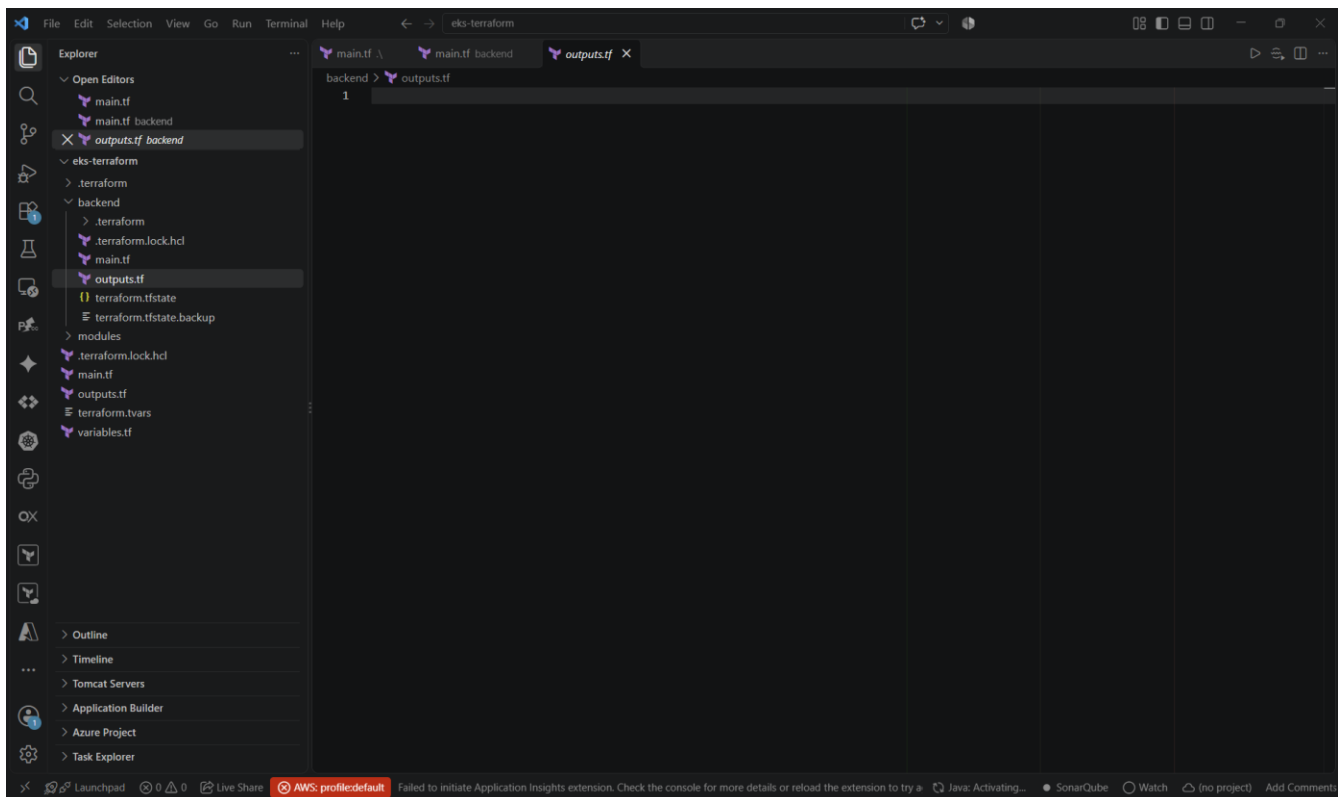


You can see the “dev” folder has been created. Click on the folder



You can now see our Terraform state file, this is successful. Terraform backend is now configured and pointing to our S3 bucket and we are now using S3 native state locking.

Before we go into the provisioning of the VPC, let us modify the “**outputs.tf**” file in the “**backend**” folder. Open the file.

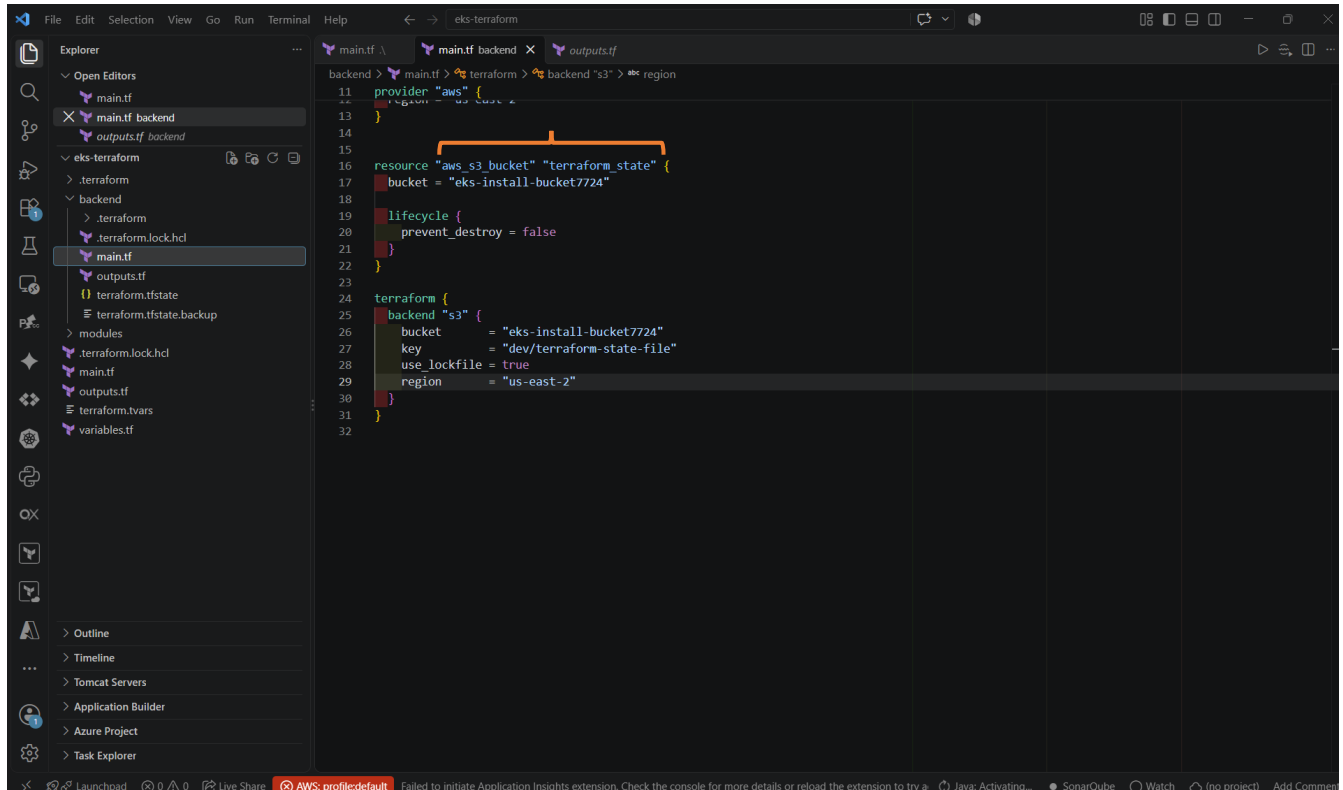


Let us add a block that where the name of the S3 bucket will be displayed. This is the template

```
output "s3_bucket_name" {  
  value      =  
  description = "The name of the S3 bucket"  
}
```

The name of the block is “**s3_bucket_name**” and the “**value**” is what we want to display on our terminal. The value will be the name of our S3 bucket. How do we get this value?

Go to the S3 block on our “**main.tf**” file in the “**backend**” folder.



Copy the resource name, that is “**aws_s3_bucket**” “**terraform_state**”. Remove the quotes and put a dot between the two parts:

```
aws_s3_bucket.terraform_state
```

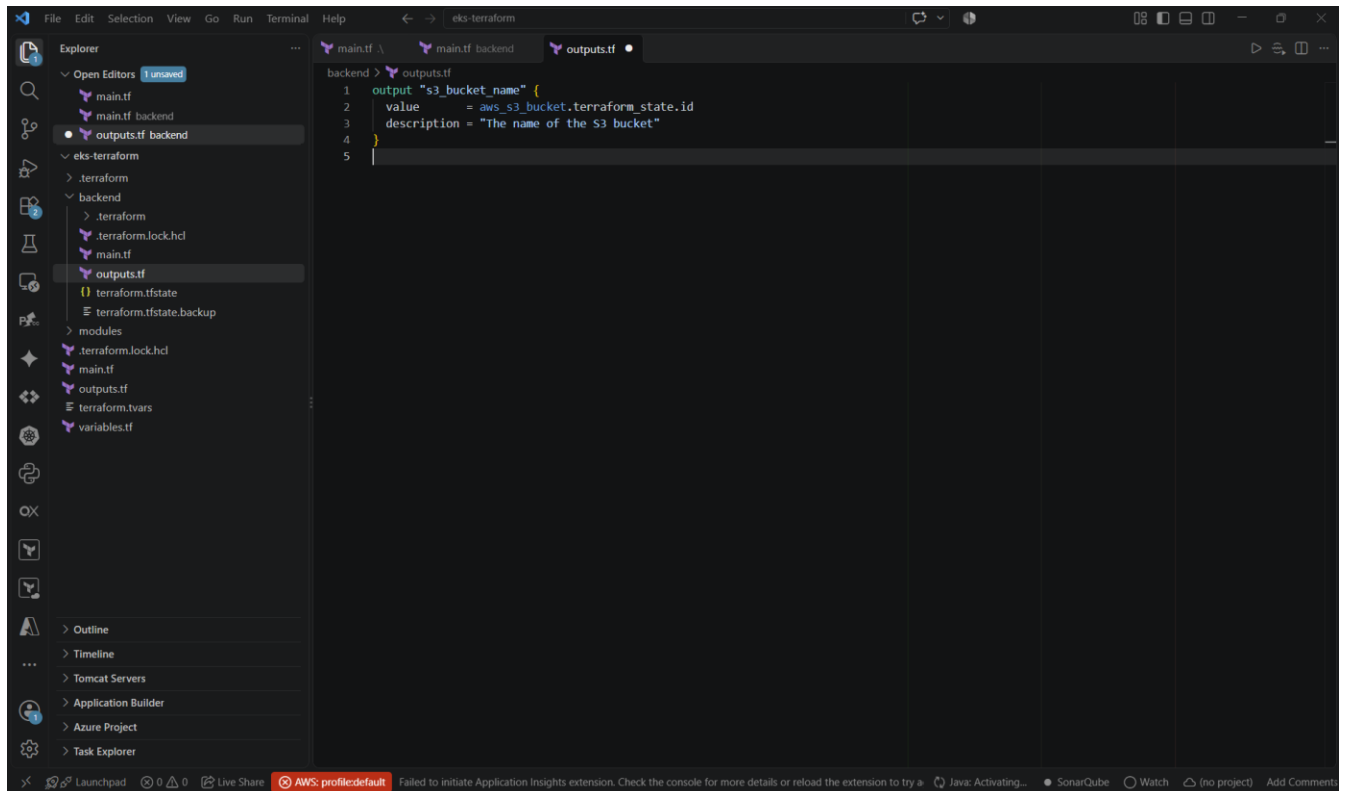
Then, add “**.id**” at the end to become

```
aws_s3_bucket.terraform_state.id
```

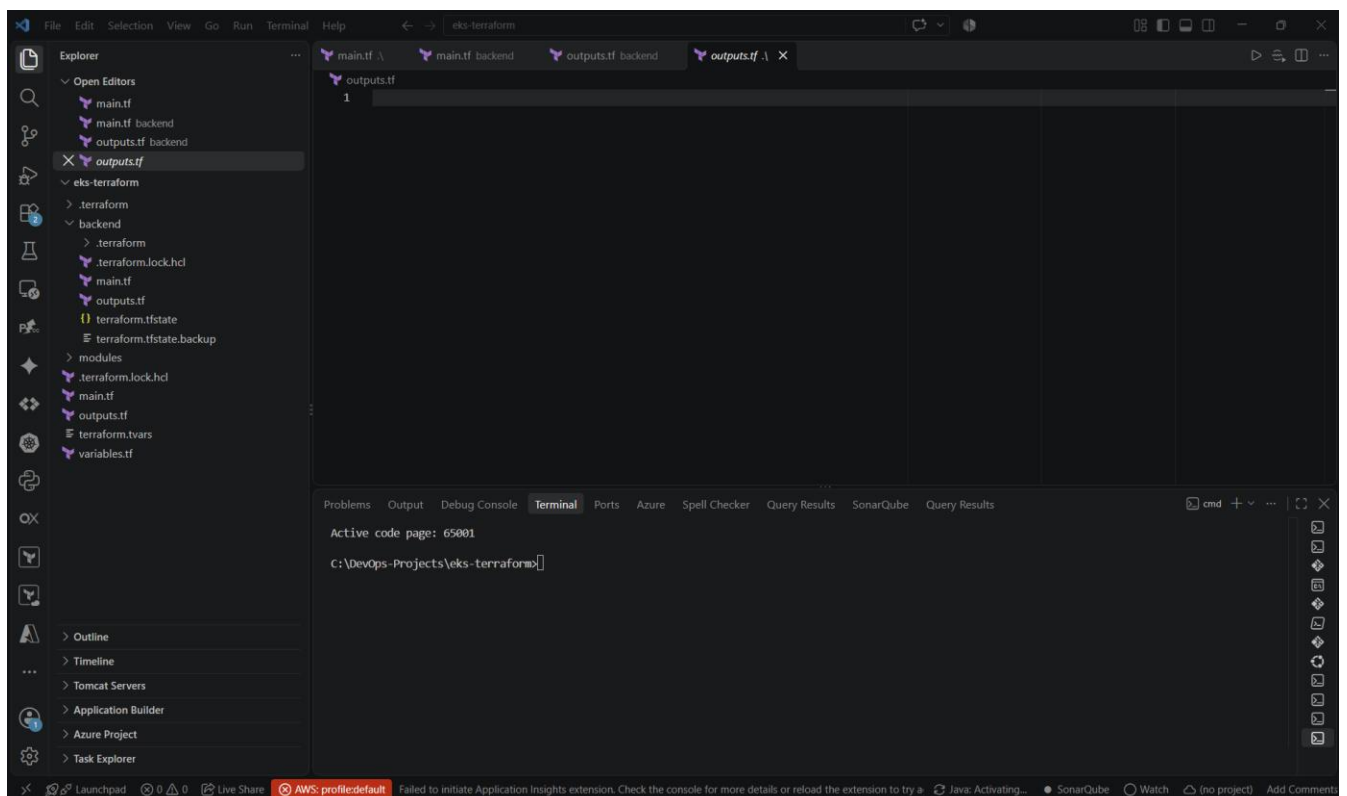
and paste this in the “**value**” of the “**output.tf**” file. Our code will now look like this

```
output "s3_bucket_name" {  
  value      = aws_s3_bucket.terraform_state.id  
  description = "The name of the S3 bucket"  
}
```

Copy the block of code and paste in the “**outputs.tf**” file on VSCode

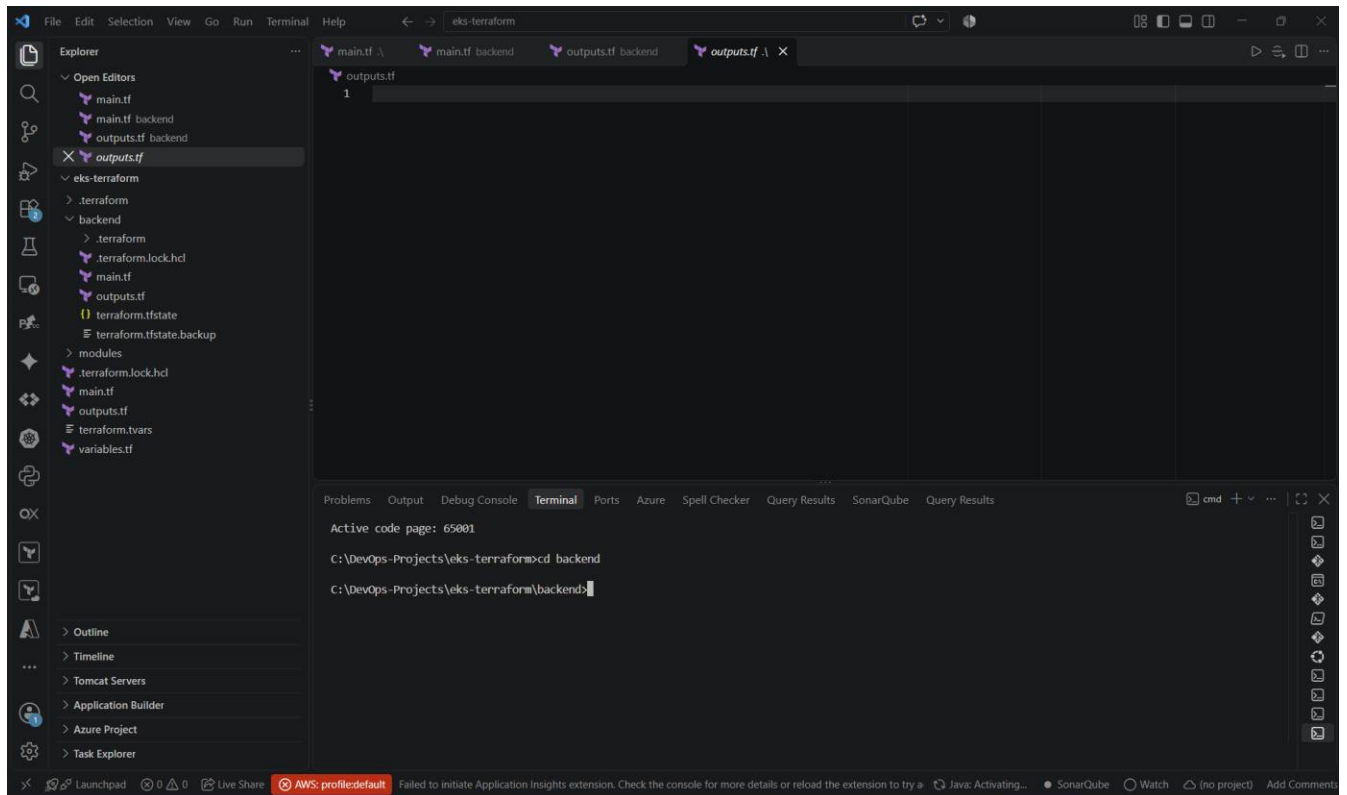


With this you will get the name of the S3 bucket displayed on the PowerShell terminal. Let us test it by opening the terminal



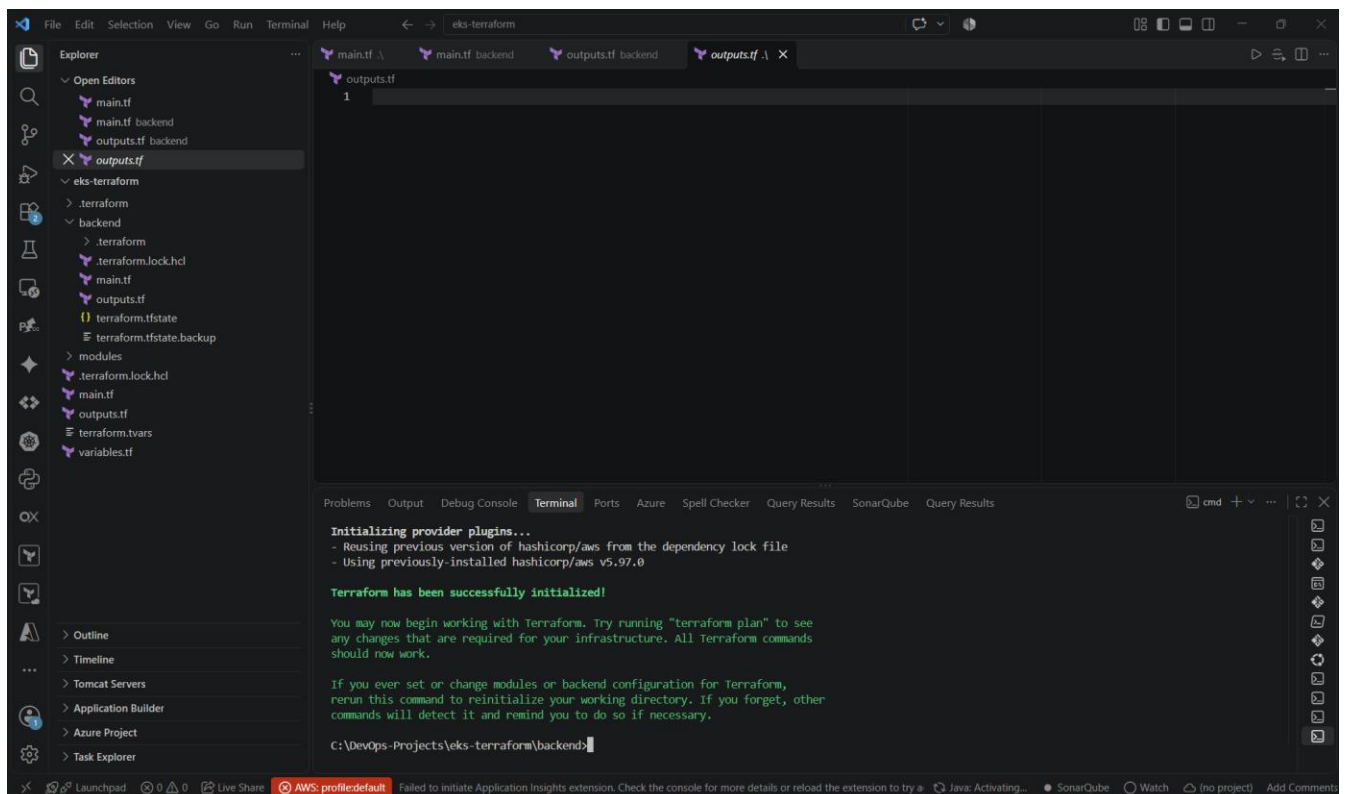
Navigate to the “**backend**” folder using the command:

cd backend



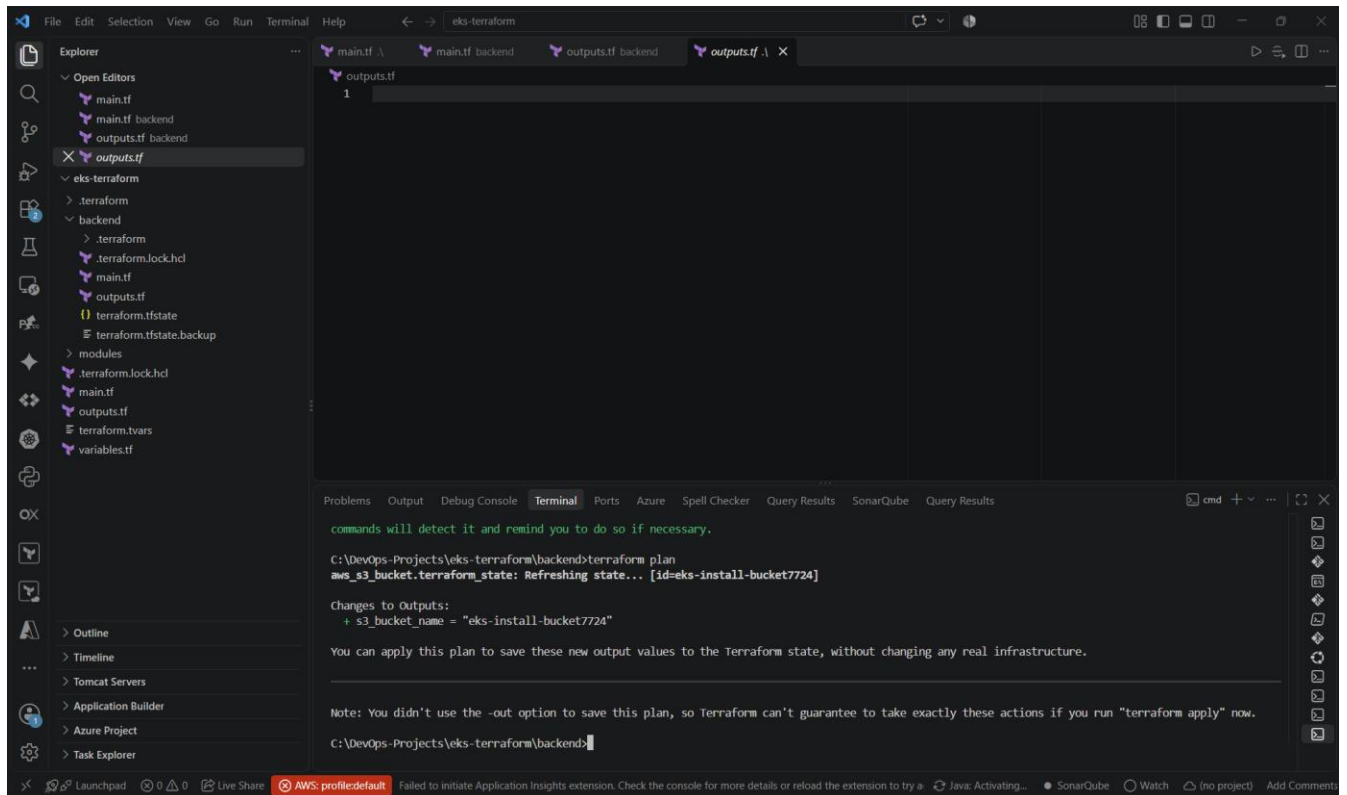
Run the command to initialize the module:

terraform init



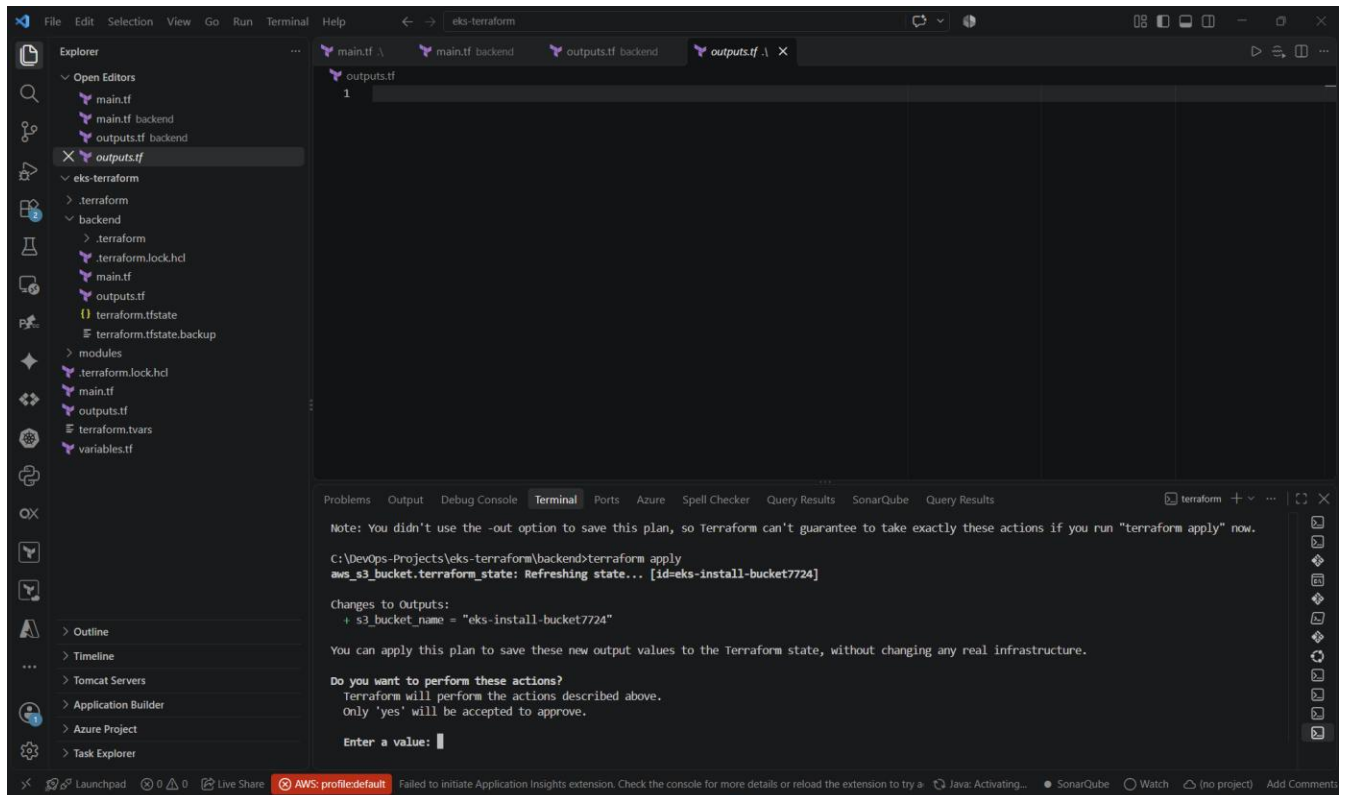
Then run the command:

```
terraform plan
```



Then run the command:

```
terraform apply
```



```
File Edit Selection View Go Run Terminal Help
eks-terraform
main.tf \ main.tf backend outputs.tf backend outputs.tf \
outputs.tf
1
Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform apply" now.
C:\DevOps-Projects\eks-terraform\backend>terraform apply
aws_s3_bucket.terraform_state: Refreshing state... [id=eks-install-bucket7724]

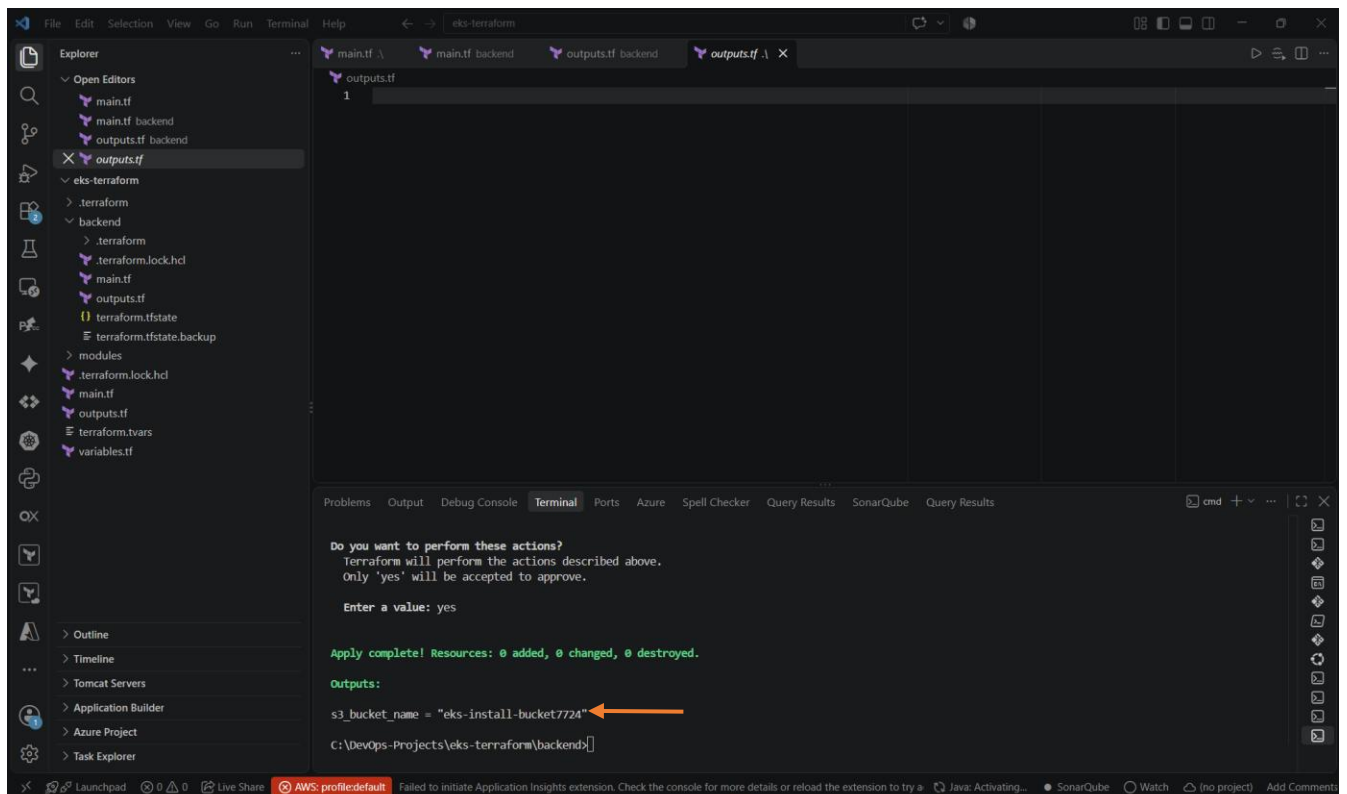
Changes to Outputs:
+ s3_bucket_name = "eks-install-bucket7724"

You can apply this plan to save these new output values to the Terraform state, without changing any real infrastructure.

Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: 
```

Enter “yes” and press “enter”



```
File Edit Selection View Go Run Terminal Help
eks-terraform
main.tf \ main.tf backend outputs.tf backend outputs.tf \
outputs.tf
1
Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

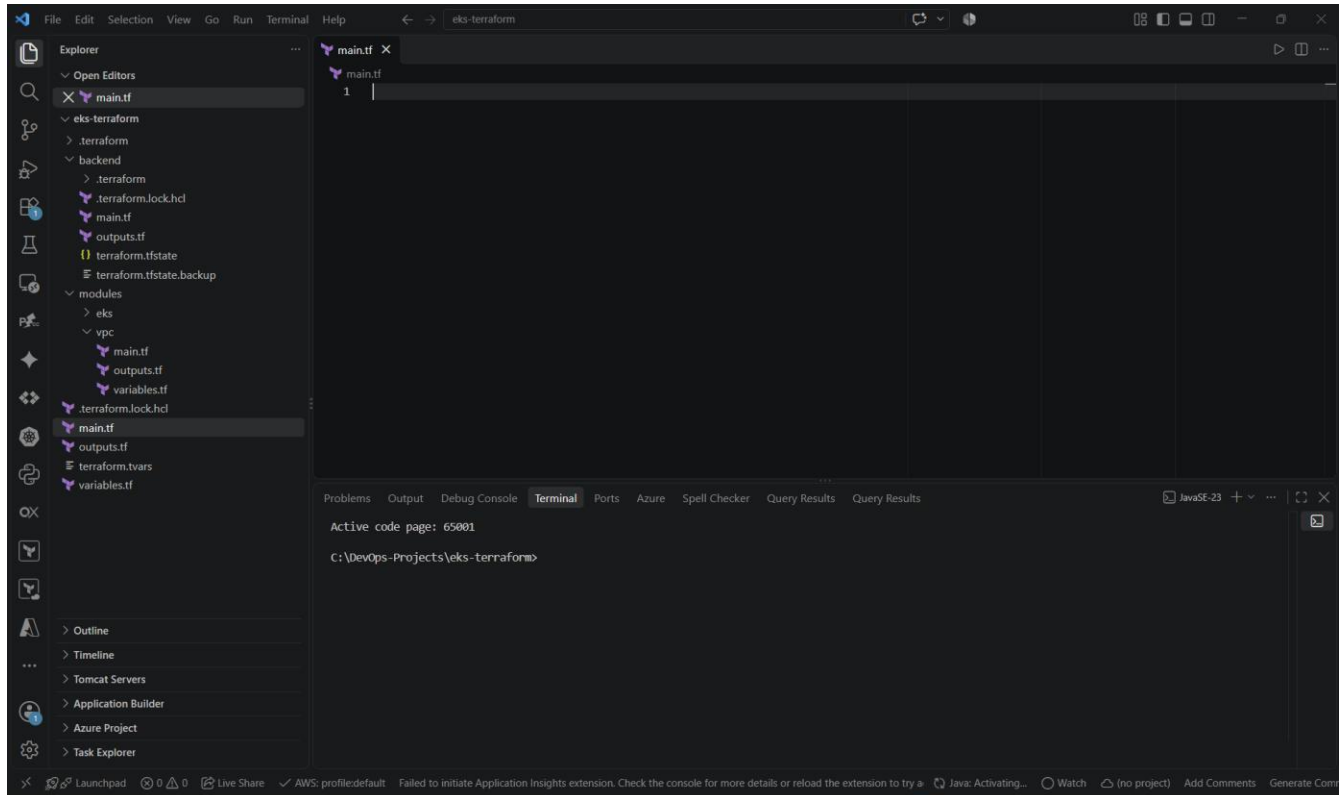
Outputs:
s3_bucket_name = "eks-install-bucket7724"
C:\DevOps-Projects\eks-terraform\backend>
```

You can see the name of our S3 bucket is displayed on the terminal.

5. Provision a VPC with Terraform for Amazon EKS

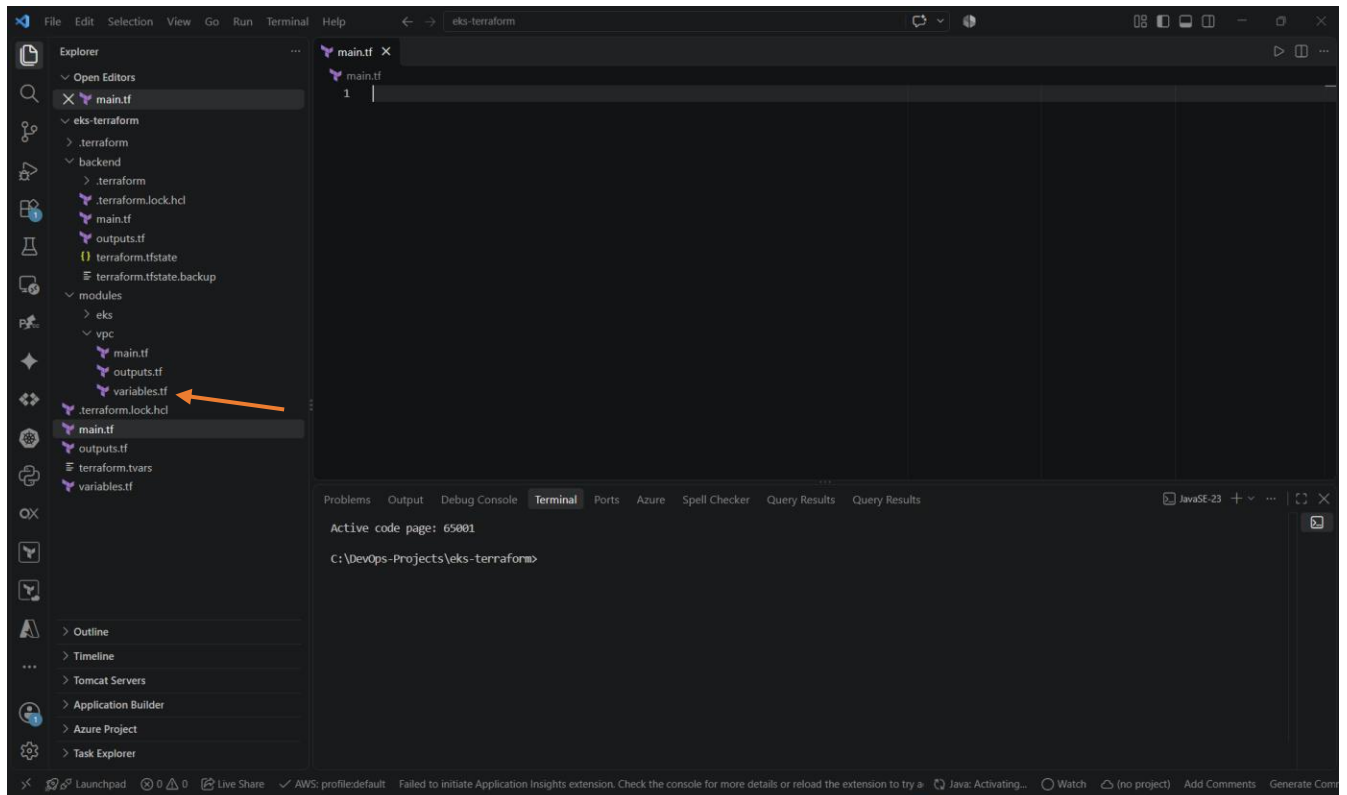
In this part, we are going to walkthrough on how to configure our VPC and its components. But let us start by defining our variables for VPC configuration.

Go to the “**vpc**” folder in the “**modules**” folder.

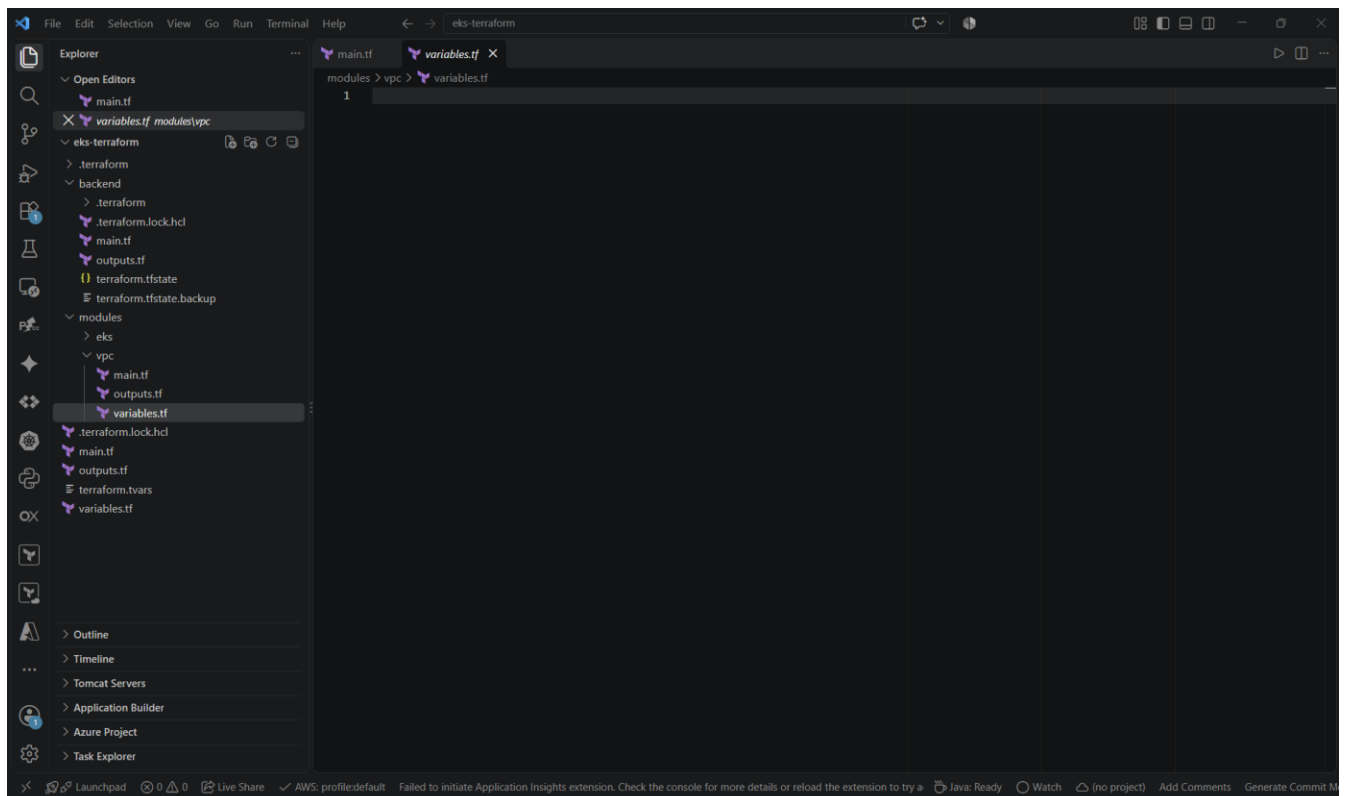


5.1. Configuring the Variables – “variables.tf”

We have to configure the variables for VPC. This will be done on the “**variables.tf**” file in the “**vpc**” folder.



Open the “**variables.tf**” files in the “**vpc**” folder.



Add the variable of all our resources, that is for “**region**”, “**vpc_cidr**”, “**public_subnet_cidr**”, “**private_subnet_cidr**”, “**availability_zones**” and “**eks_cluster_name**”.

5.1.1. Adding Variable for Region

Let us add the variable for the region. We will call this variable **“region”**. The type is **“string”** and the default region is **“us-east-2”**

```
variable "region" {  
  description = "AWS region"  
  type        = string  
  default     = "us-east-2"  
}
```

5.1.2. Adding Variable for VPC CIDR block

Let us add the variable for the VPC CIDR block. We will call this variable **“vpc_cidr”**

```
variable "vpc_cidr" {  
  description = "VPC CIDR block"  
  type        = string  
}
```

5.1.3. Adding Variable for Public Subnet CIDR

Let us add the variable for the public subnet CIDR. We will call this variable **“public_subnet_cidr”**

```
variable "public_subnet_cidr" {  
  description = "public subnet CIDR block"  
  type        = list(string)  
}
```

5.1.4. Adding Variable for Private Subnet CIDR

Let us add the variable for the private subnet CIDR. We will call this variable **“private_subnet_cidr”**

```
variable "private_subnet_cidr" {  
  description = "private subnet CIDR block"  
  type        = list(string)  
}
```

5.1.5. Adding Variable for Availability Zones

Let us add the variable for the availability zones. We will call this variable **“availability_zones”**

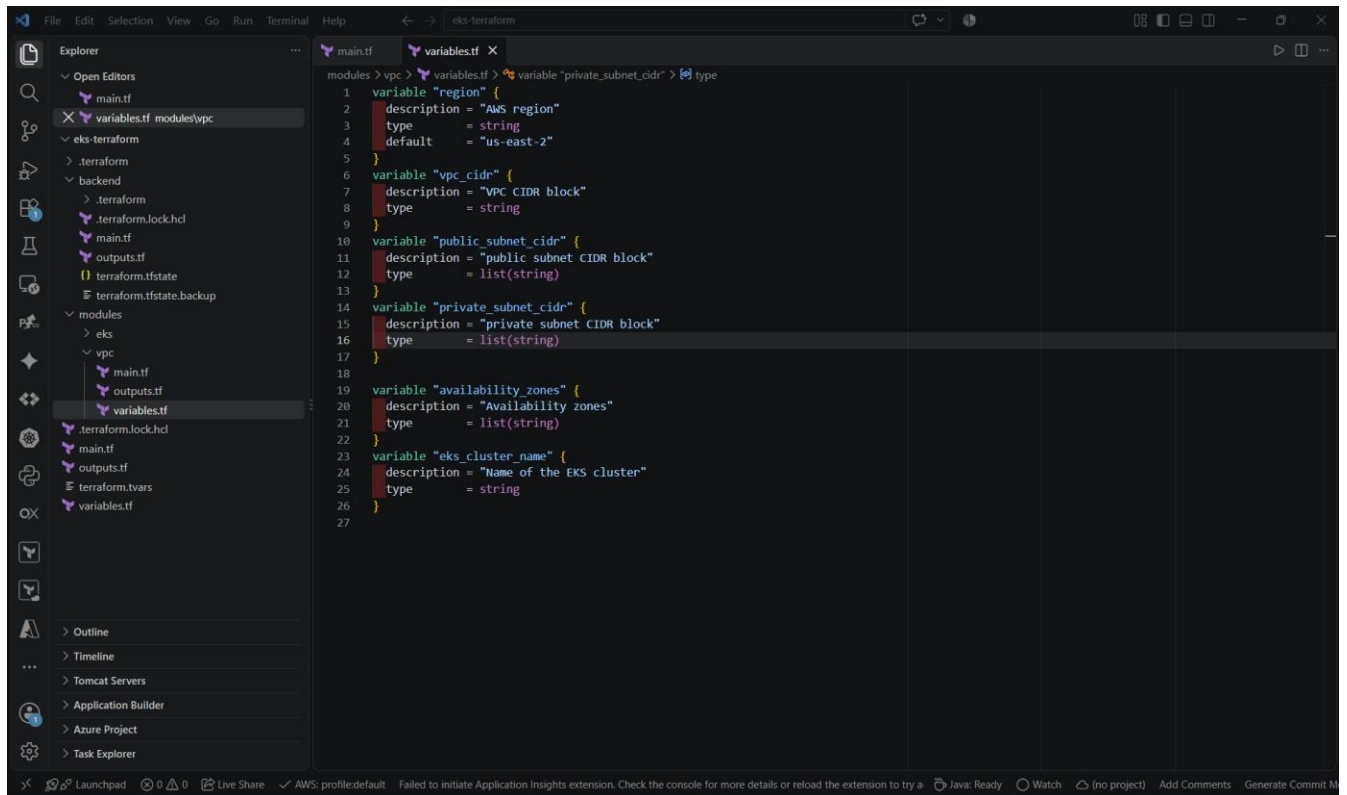
```
variable "availability_zones" {  
  description = "Availability zones"  
  type        = list(string)  
}
```

5.1.6. Adding Variable for EKS Cluster

Let us add the variable for the EKS cluster. We will call this variable **“eks_cluster_name”**

```
variable "eks_cluster_name" {  
  description = "Name of the EKS cluster"  
  type        = string  
}
```

Copy the blocks of all the variables and paste this in the **“variables.tf”** file in the **“vpc”** folder.



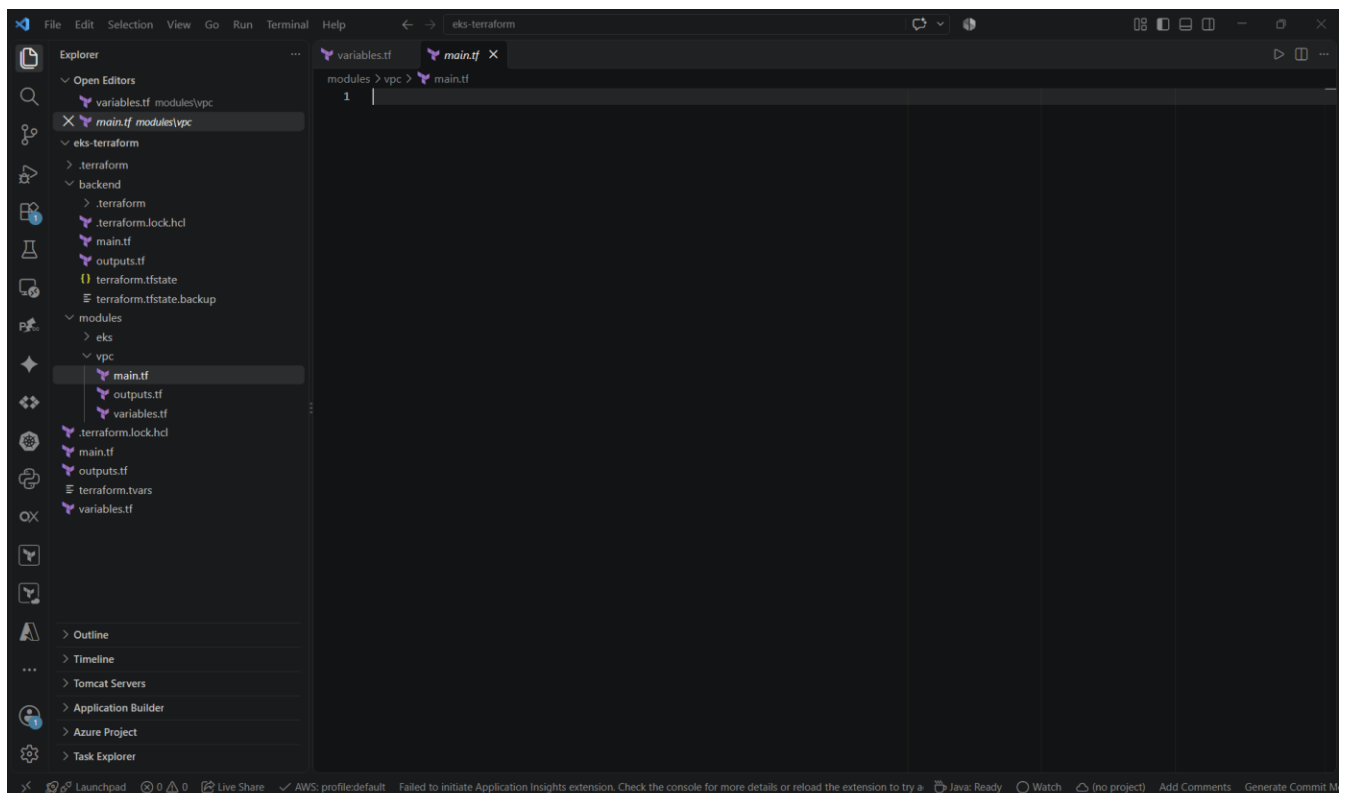
The screenshot shows the Visual Studio Code editor with the 'eks-terraform' project open. The Explorer sidebar on the left shows the file structure, with 'variables.tf' selected under the 'vpc' module. The main editor window displays the content of 'variables.tf' with the following code:

```
1 variable "region" {
2   description = "AWS region"
3   type        = string
4   default     = "us-east-2"
5 }
6 variable "vpc_cidr" {
7   description = "VPC CIDR block"
8   type        = string
9 }
10 variable "public_subnet_cidr" {
11   description = "public subnet CIDR block"
12   type        = list(string)
13 }
14 variable "private_subnet_cidr" {
15   description = "private subnet CIDR block"
16   type        = list(string)
17 }
18
19 variable "availability_zones" {
20   description = "Availability zones"
21   type        = list(string)
22 }
23 variable "eks_cluster_name" {
24   description = "Name of the EKS cluster"
25   type        = string
26 }
27
```

We have completed the code for the “**variables.tf**” file.

5.2. Configuring the VPC and its components – “**main.tf**”

We can now proceed with the code for the “**main.tf**” file



The screenshot shows the Visual Studio Code editor with the 'eks-terraform' project open. The Explorer sidebar on the left shows the file structure, with 'main.tf' selected under the 'vpc' module. The main editor window displays the content of 'main.tf' with the following code:

```
1
```

5.2.1. Setting up VPC Configuration Block

Let us add the first block for our VPC configuration. It will look like this:

```
# VPC Configuration
resource "aws_vpc" "eks_vpc" {
  cidr_block          = define_vpc_cidr # TODO: Define VPC CIDR in variables.tf
  enable_dns_hostnames = true
  enable_dns_support  = true

  tags = {
    Project      = "EKS-Cluster"
    Environment = define-tag # TODO: Modify the environment tag
  }
}
```

Let us modify the block to match our project goal. In the “**cidr_block**”, this is where we will be referencing the variable that we just added in the “**variable.tf**” file. The value of the “**cidr_block**” in our “**variables.tf**” file is “**vpc_cidr**”, then add “**var.**” in front to become “**var.vpc_cidr**”

So, the code becomes:

```
# VPC Configuration
resource "aws_vpc" "eks_vpc" {
  cidr_block          = var.vpc_cidr
  enable_dns_hostnames = true
  enable_dns_support  = true

  tags = {
    Project      = "EKS-Cluster"
    Environment = define-tag # TODO: Modify the environment tag
  }
}
```

The “**enable_dns_hostnames**” and “**enable_dns_support**” will be “**true**”.

```
# VPC Configuration
resource "aws_vpc" "eks_vpc" {
  cidr_block          = var.vpc_cidr
  enable_dns_hostnames = true
  enable_dns_support  = true

  tags = {
    Project      = "EKS-Cluster"
    Environment = define-tag # TODO: Modify the environment tag
  }
}
```

For the “**project**”, we will give it the value “**EKS-Cluster**”

```
# VPC Configuration
resource "aws_vpc" "eks_vpc" {
  cidr_block          = var.vpc_cidr
  enable_dns_hostnames = true
  enable_dns_support  = true

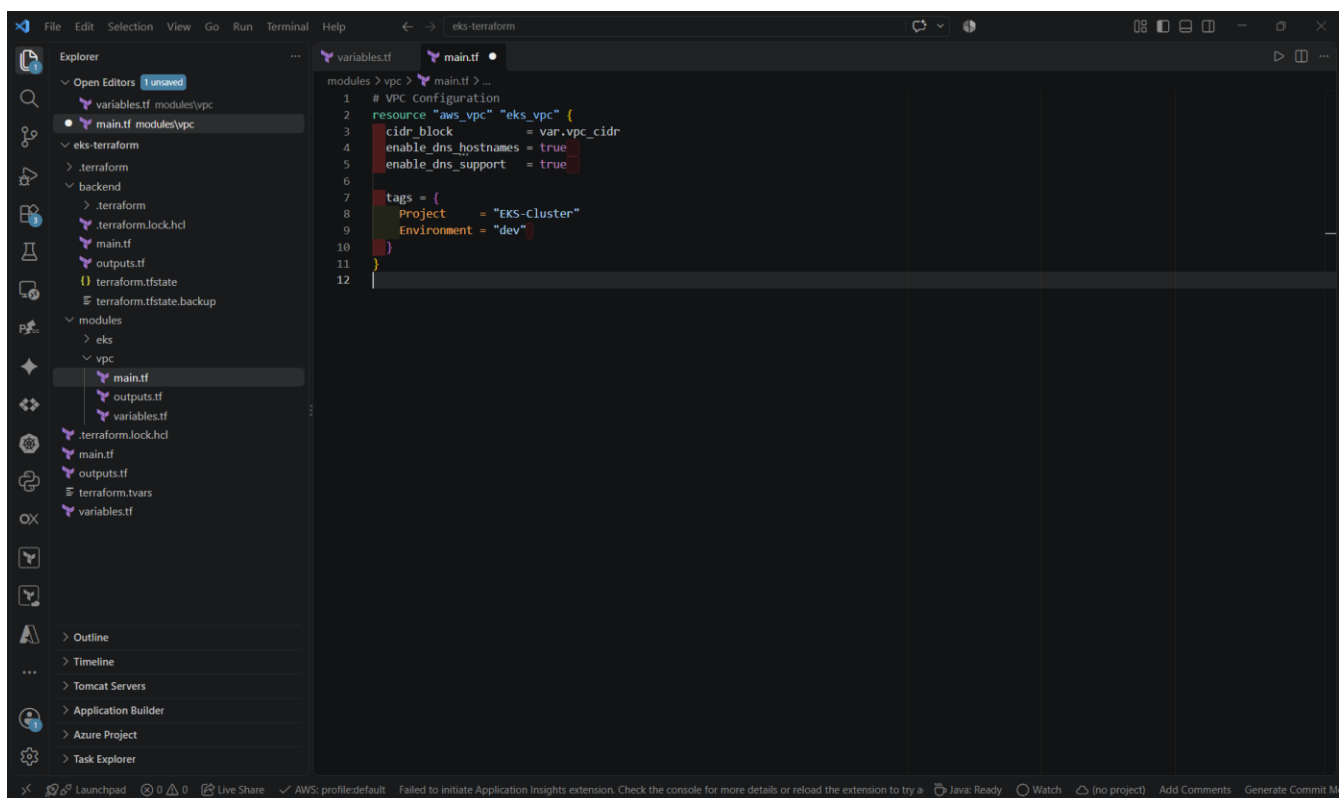
  tags = {
    Project      = "EKS-Cluster"
    Environment = define-tag # TODO: Modify the environment tag
  }
}
```

Lastly, for the **“Environment”**, we will use **“dev”** since we are doing this demo in the development environment. If you are at the production environment, you can use **“prod”**. So, this block will finally look like this:

```
# VPC Configuration
resource "aws_vpc" "eks_vpc" {
  cidr_block           = var.vpc_cidr
  enable_dns_hostnames = true
  enable_dns_support   = true

  tags = {
    Project      = "EKS-Cluster"
    Environment  = "dev"
  }
}
```

Let us copy this block and paste in our **“main.tf”** file in the **“vpc”** folder.



We have completed the first block to configure the VPC.

5.2.2. Setting up Public Subnets block

We will now move to the second block to configure the public subnets. For this we will use this resource block:

```
# Public Subnets
resource "aws_subnet" "public" {
  count                = length(define_cidrs)
  vpc_id              = vpc ID
  cidr_block          = public_subnet_cidrs[count.index] # TODO
  availability_zone    = add_availability_zones[count.index]

  map_public_ip_on_launch = true # Automatically assign a public IP address to instances
}
```

```
tags = {
  Name      = "public-subnet-${count.index + 1}" # TODO: Customize naming pattern
  Tier      = "public"
}
```

Let us modify this block to meet the requirement of the project. We will start with the **“count”**, we are using a count loop because we have to create three public subnets. In the bracket, we have to define the variable for the **“cidr”** of the public subnet.

In the **“variable.tf”** file in the **“vpc”** folder, we defined the variable for the public subnet as **“public_subnet_cidr”**. So, in the bracket we will use:

```
var.public_subnet_cidr
```

And the code becomes:

```
# Public Subnets
resource "aws_subnet" "public" {
  count          = length(var.public_subnet_cidr)
  vpc_id         = vpc_id
  cidr_block     = public_subnet_cidrs[count.index] # TODO
  availability_zone = add_availability_zones[count.index]

  map_public_ip_on_launch = true # Automatically assign a public IP address to instances

  tags = {
    Name      = "public-subnet-${count.index + 1}" # TODO: Customize naming pattern
    Tier      = "public"
  }
}
```

For the **“vpc_id”**, just go to the VPC configuration block

```
# VPC Configuration
resource "aws_vpc" "eks_vpc" {
  cidr_block         = var.vpc_cidr
  enable_dns_hostnames = true
  enable_dns_support = true

  tags = {
    Project      = "EKS-Cluster"
    Environment  = "dev"
  }
}
```

Copy the name of the resource and modify it.

```
"aws_vpc" "eks_vpc"
```

Remove the quotes, add a dot in the middle of the two parts and add **“.id”** at the end. This becomes

```
aws_vpc.eks_vpc.id
```

So, the code becomes

```
# Public Subnets
resource "aws_subnet" "public" {
  count          = length(var.public_subnet_cidr)
  vpc_id         = aws_vpc.eks_vpc.id
  cidr_block     = public_subnet_cidrs[count.index] # TODO
  availability_zone = add_availability_zones[count.index]

  map_public_ip_on_launch = true # Automatically assign a public IP address to instances
```

```
tags = {
  Name      = "public-subnet-${count.index + 1}" # TODO: Customize naming pattern
  Tier      = "public"
}
```

For the **“cidr_block”**, this is where we define the range of the IP addresses that will be in our subnet. We can get this from the **“variables.tf”** file.

```
variable "private_subnet_cidr" {
  description = "private subnet CIDR block"
  type        = list(string)
}
```

We will use the name of this variable **“private_subnet_cidr”** and modify it by adding **“var.”** in front to become:

```
"var.private_subnet_cidr"
```

So, the code becomes:

```
# Public Subnets
resource "aws_subnet" "public" {
  count          = length(var.public_subnet_cidr)
  vpc_id         = aws_vpc.eks_vpc.id
  cidr_block     = var.public_subnet_cidr[count.index] # TODO
  availability_zone = add_availability_zones[count.index]

  map_public_ip_on_launch = true # Automatically assign a public IP address to instances

  tags = {
    Name      = "public-subnet-${count.index + 1}" # TODO: Customize naming pattern
    Tier      = "public"
  }
}
```

For the **“availability_zone”**, we will do same. Get the name from the **“variable.tf”** file. That is

```
"availability_zones"
```

Remove the quotes and add **“var.”** in front. It becomes:

```
var.availability_zones
```

The Block will now look like this:

```
# Public Subnets
resource "aws_subnet" "public" {
  count          = length(var.public_subnet_cidr)
  vpc_id         = aws_vpc.eks_vpc.id
  cidr_block     = var.public_subnet_cidr[count.index] # TODO
  availability_zone = var.availability_zones[count.index]

  map_public_ip_on_launch = true # Automatically assign a public IP address to instances

  tags = {
    Name      = "public-subnet-${count.index + 1}" # TODO: Customize naming pattern
    Tier      = "public"
  }
}
```

For **“map_public_ip_on_launch”**, we will set it to **“true”**. This will automatically assign a public IP address to instances. The block will now look like this:

```
# Public Subnets
```

```
resource "aws_subnet" "public" {
  count          = length(var.public_subnet_cidr)
  vpc_id         = aws_vpc.eks_vpc.id
  cidr_block     = var.public_subnet_cidr[count.index] # TODO
  availability_zone = var.availability_zones[count.index]

  map_public_ip_on_launch = true # Automatically assign a public IP address to instances

  tags = {
    Name     = "public-subnet-${count.index + 1}" # TODO: Customize naming pattern
    Tier     = "public"
  }
}
```

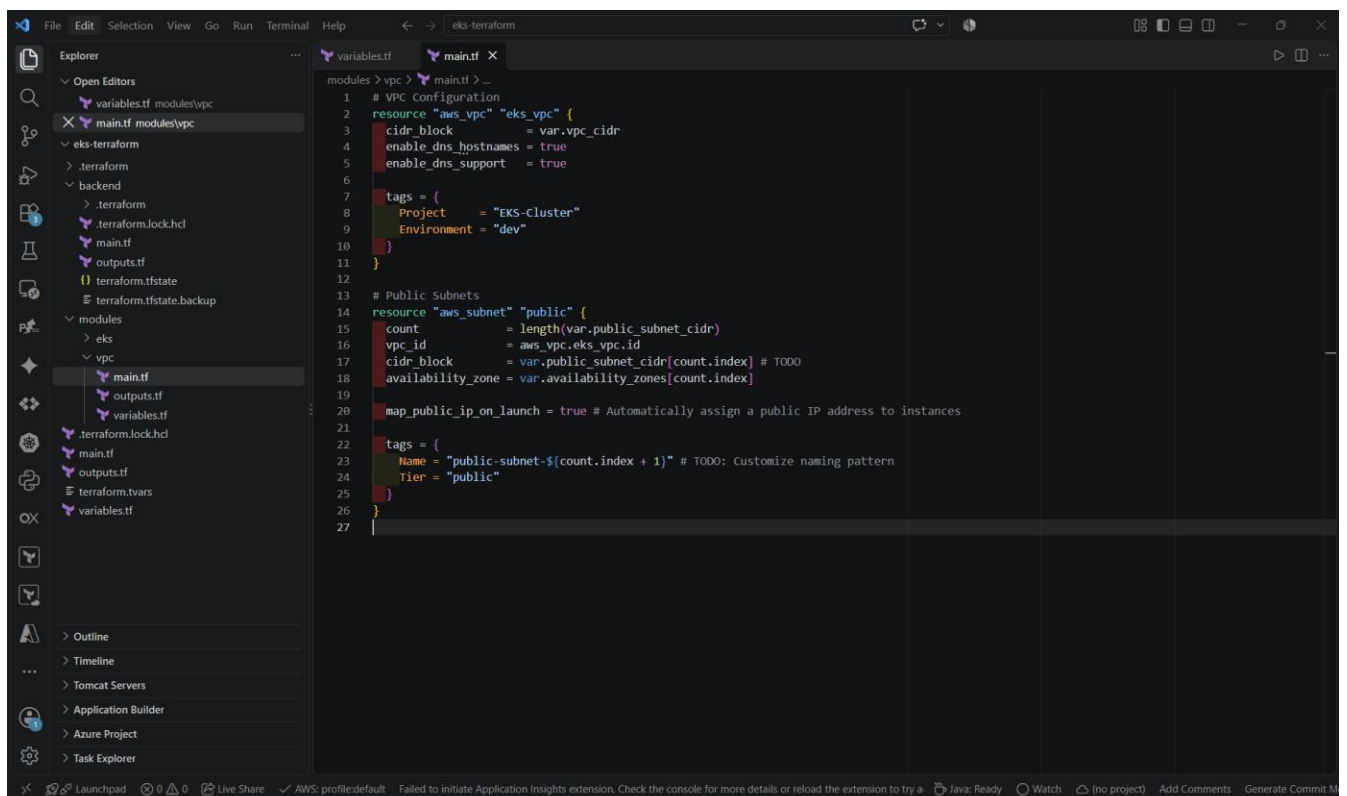
For “**tags**”, we will use “**public-subnet**” in the name since we are in the public subnet block. And for the “**Tier**”, we will just use “**public**”. The completed block will look like this:

```
# Public Subnets
resource "aws_subnet" "public" {
  count          = length(var.public_subnet_cidr)
  vpc_id         = aws_vpc.eks_vpc.id
  cidr_block     = var.public_subnet_cidr[count.index] # TODO
  availability_zone = var.availability_zones[count.index]

  map_public_ip_on_launch = true # Automatically assign a public IP address to instances

  tags = {
    Name     = "public-subnet-${count.index + 1}" # TODO: Customize naming pattern
    Tier     = "public"
  }
}
```

Copy the completed block and paste in the “**main.tf**” file in the “**vpc**” folder.



We have completed the block for the public subnet.

5.2.3. Setting up the Private subnets Block

We will now move to the third block to configure the private subnets with the name “**private**”. The private subnet is used to hold resources like database, worker nodes.

For this we will use this resource block:

```
# Private Subnets
resource "aws_subnet" "private" {
  count          = length(define_cidrs)
  vpc_id         = aws_vpc.eks_vpc.id
  cidr_block     = private_subnet_cidrs[count.index] # TODO: Add availability zone
  availability_zone = add_availability_zones[count.index] # TODO: Add availability zone

  map_public_ip_on_launch = true # Automatically assign a public IP address to instances

  tags = {
    Name     = "private-subnet-${count.index + 1}" # TODO: Customize naming pattern
    Tier     = "private"
  }
}
```

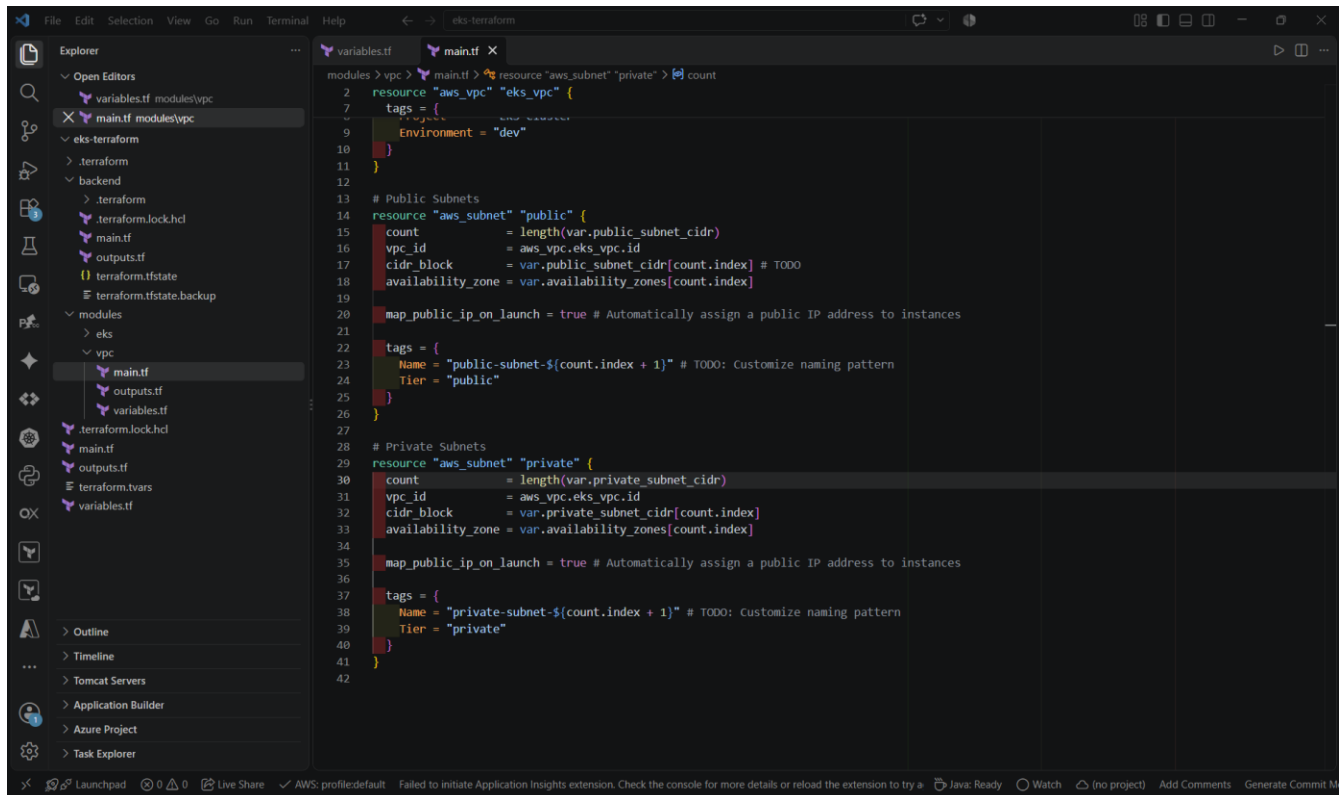
We will just follow the same steps used to modify the public subnet to modify this private subnet. The completed code will be like this:

```
# Private Subnets
resource "aws_subnet" "private" {
  count          = length(var.private_subnet_cidr)
  vpc_id         = aws_vpc.eks_vpc.id
  cidr_block     = var.private_subnet_cidr[count.index]
  availability_zone = var.availability_zones[count.index]

  map_public_ip_on_launch = true # Automatically assign a public IP address to instances

  tags = {
    Name     = "private-subnet-${count.index + 1}" # TODO: Customize naming pattern
    Tier     = "private"
  }
}
```

Copy this block and paste in the “**main.tf**” file in the “**vpc**” folder.



We have completed the block for the private subnets.

5.2.4. Setting up the Internet Gateway Block

We have to set up the block for the internet gateway with the name **“int-gw”**. The internet gateway is needed to provide internet traffic in and out of our public subnet. It has to be attached to the VPC, so we need the VPC ID.

We will use this sample block code:

```

# Internet Gateway
resource "aws_internet_gateway" "int-gw" {
  vpc_id = Add a VPC ID # TO DO: Add VPC ID

  tags = {
    Name = "Add a name" # TODO: Add tag name
  }
}

```

Let us modify the code to suit the requirement of our project. We will start by adding the value of the **“vpc_id”**. To do this, we have to get the name of the VPC resource on the VPC configuration block.

“aws_vpc” “eks_vpc”

Remove the quotes, add a dot between the two parts and add **“.id”** at the end. It will look like this:

aws_vpc.eks_vpc.id

The code becomes like this:

```

# Internet Gateway
resource "aws_internet_gateway" "int-gw" {

```

```
vpc_id = aws_vpc.eks_vpc.id

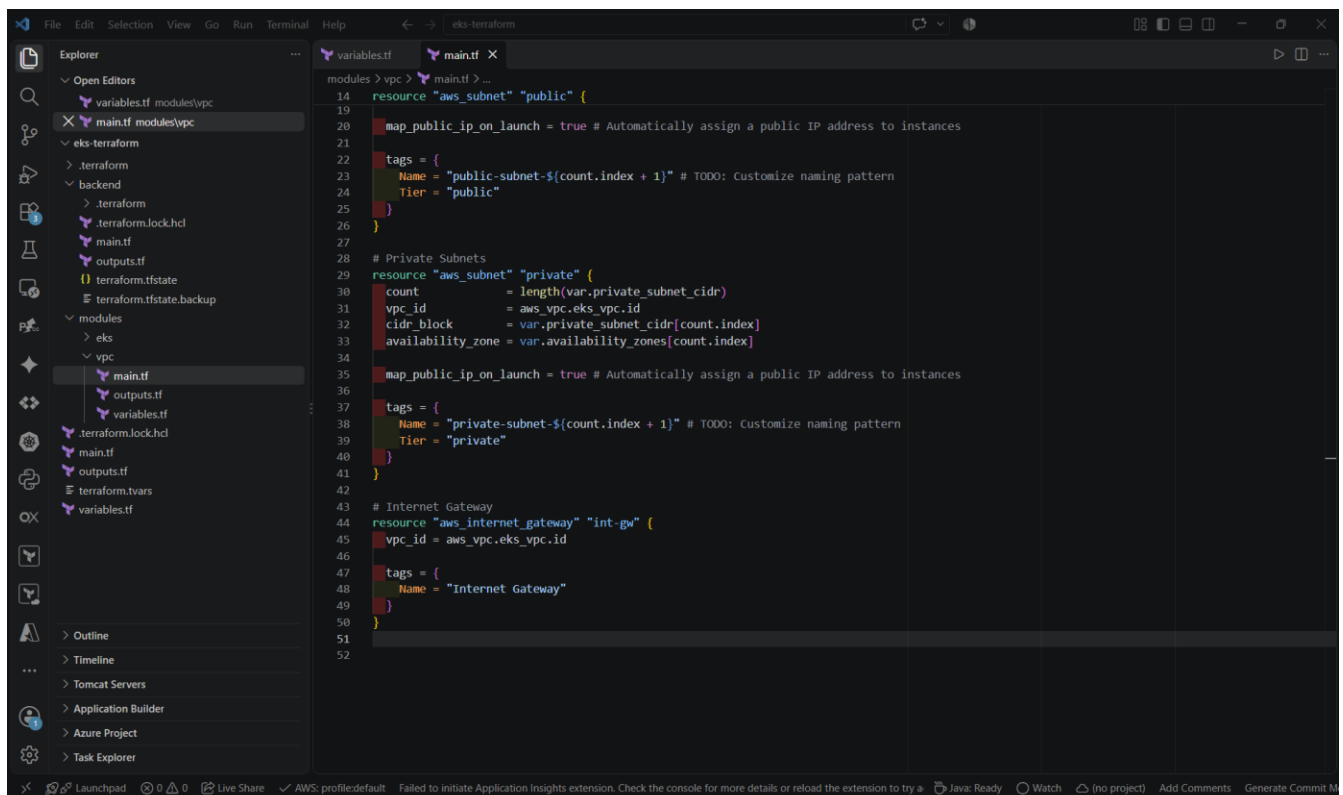
tags = {
  Name = "Add a name" # TODO: Add tag name
}
```

Then, for the “tags”, for the “Name”, we will enter the value “Internet Gateway”. And our completed code will look like this:

```
# Internet Gateway
resource "aws_internet_gateway" "int-gw" {
  vpc_id = aws_vpc.eks_vpc.id

  tags = {
    Name = "Internet Gateway"
  }
}
```

Copy this block and paste in the “main.tf” file in “vpc” folder.



We have completed the block for the Internet Gateway.

5.2.5. Setting up the Elastic IP Block

We have to add the block to create the Elastic IP called “eip-nat” with resource type as “aws_eip”. We will use this sample block.

```
# Elastic IPs
resource "aws_eip" "eip-nat" {
  count = length(add_public_subnet_cidrs) # TODO: Add public subnet cidr
  domain = "vpc"
```

```
    # TODO: Optionally add tags here for better resource tracking
}
```

For the **“count”**, we have to add a value in the bracket. We have to add the public subnet CIDR. To do this, we have to copy the name of the public subnet CIDR variable in the **“variable.tf”** file. That is

```
"public_subnet_cidr"
```

Remove the quotes and add **“var.”** in front. It becomes

```
var.public_subnet_cidr
```

The block will now look like this:

```
# Elastic IPs
resource "aws_eip" "eip-nat" {
    count = length(var.public_subnet_cidr)
    domain = "vpc"

    # TODO: Optionally add tags here for better resource tracking
}
```

Then for **“domain”**, the value is **“vpc”**

```
# Elastic IPs
resource "aws_eip" "eip-nat" {
    count = length(var.public_subnet_cidr)
    domain = "vpc"

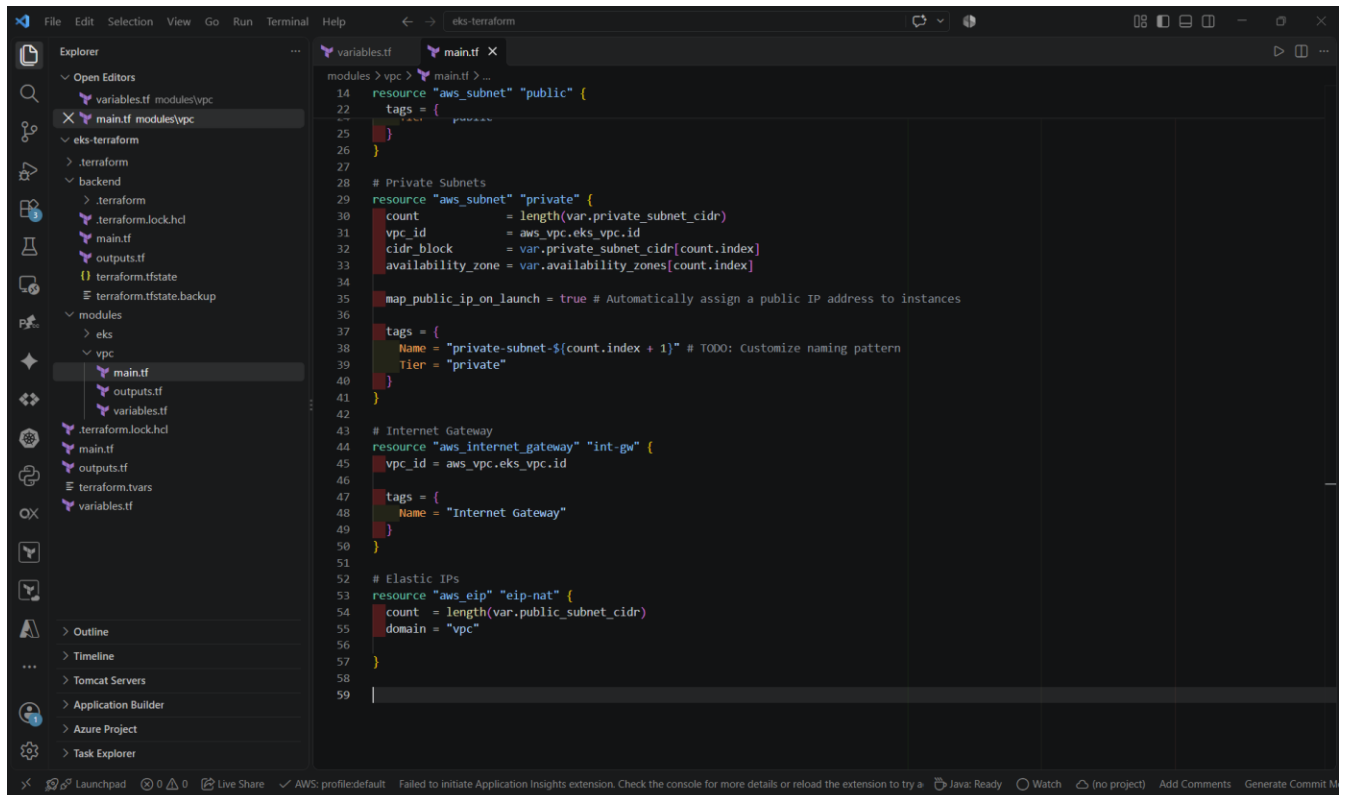
    # TODO: Optionally add tags here for better resource tracking
}
```

You can optionally add a tag, but we will not add any tag in this demo. The completed block will look like this:

```
# Elastic IPs
resource "aws_eip" "eip-nat" {
    count = length(var.public_subnet_cidr)
    domain = "vpc"

}
```

Copy this completed block and paste in the **“main.tf”** file in the **“vpc”** folder.



We have completed the Elastic IP block.

5.2.6. Setting up the NAT Gateway Block

We have to add the block to create the NAT Gateway called “**nat-gw**”. The NAT gateway is used to provide internet access to our resource in the private subnet without exposing them to the public or internet. We will use this sample block.

```
#NAT Gateway
resource "aws_nat_gateway" "nat-gw" {
  count          = length(add_public_subnet_cidrs) # TODO: Add public subnet cidr
  allocation_id = aws_eip.eip-nat[count.index].id
  subnet_id     = aws_subnet.public[count.index].id

  tags = {
    Name = "nat-gateway-${count.index + 1}"
  }
}
```

For the “count”, we will get the name of the public subnet CIDR variable in the “**variable.tf**” file and modify it.

“public_subnet_cidr”

Let us modify it by removing the quotes and adding “**var.**” in front. It will become

var.public_subnet_cidr

And the code will now look like this:

```
#NAT Gateway
resource "aws_nat_gateway" "nat-gw" {
  count = length(var.public_subnet_cidr)
```

```
allocation_id = aws_eip.eip-nat[count.index].id
subnet_id     = aws_subnet.public[count.index].id

tags = {
  Name = "nat-gateway-${count.index + 1}"
}
```

For the **“allocation_id”**, we are using the Elastic IP. We get the resource type and name of the Elastic IP in the **“main.tf”** file, then modify it.

```
"aws_eip" "eip-nat"
```

Remove the quotes, add a dot between the resource type and the name of the resource. Then add dot between them. It will look like this:

```
aws_eip.eip-nat
```

The code will look like this:

```
#NAT Gateway
resource "aws_nat_gateway" "nat-gw" {
  count          = length(var.public_subnet_cidr)
  allocation_id = aws_eip.eip-nat[count.index].id
  subnet_id     = aws_subnet.public[count.index].id

  tags = {
    Name = "nat-gateway-${count.index + 1}"
  }
}
```

For the **“subnet_id”**, we will be using the public subnet. We get the resource type and name of the Public Subnet in the **“main.tf”** file, then modify it.

```
"aws_subnet" "public"
```

Remove the quotes, add a dot between the resource type and the name of the resource. Then add dot between them. It will look like this:

```
aws_subnet.public
```

The code will look like this:

```
#NAT Gateway
resource "aws_nat_gateway" "nat-gw" {
  count          = length(var.public_subnet_cidr)
  allocation_id = aws_eip.eip-nat[count.index].id
  subnet_id     = aws_subnet.public[count.index].id

  tags = {
    Name = "nat-gateway-${count.index + 1}"
  }
}
```

For the **“tags”**, in the **“Name”**, we will add the value **“nat-gateway-\${count.index + 1}”**

The completed block will look like this:

```
#NAT Gateway
```

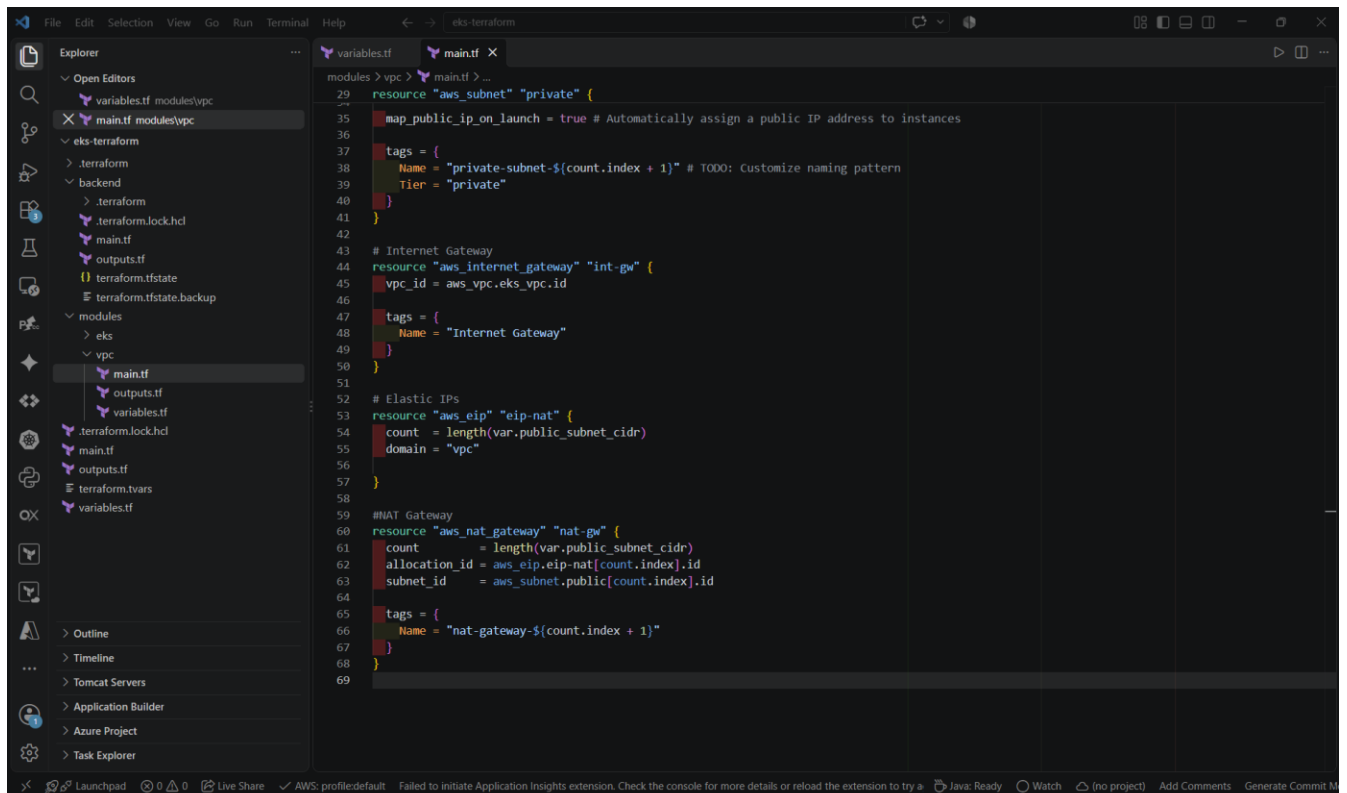
```

resource "aws_nat_gateway" "nat-gw" {
  count          = length(var.public_subnet_cidr)
  allocation_id = aws_eip.eip-nat[count.index].id
  subnet_id     = aws_subnet.public[count.index].id

  tags = {
    Name = "nat-gateway-${count.index + 1}"
  }
}

```

Copy this block and add in the “main.tf” file in the “vpc” folder



We have completed the NAT gateway block.

5.2.7. Setting up the Route Tables Block

We have to add the block to create the public route tables called “**public**” with resource type as “**aws_route_table**”. Here, we will make use of the internet gateway. We will use this sample block.

```

# Public Route Tables
resource "aws_route_table" "public" {
  vpc_id = Add VPC ID # TO DO: Add the VPC ID

  route {
    cidr_block = "Add CIDR Block" #TO DO: Allows all outbound traffic
    gateway_id = Add Internet Gateway ID #TO DO: Add Internet Gateway ID
  }

  tags = {
    Name = "Add a name" # TODO: Add tag name
  }
}

```

Let us modify the code to suit the requirement of our project. We will start by adding the value of the **“vpc_id”**. To do this, we have to get the name of the VPC resource on the VPC configuration block.

```
"aws_vpc" "eks_vpc"
```

Remove the quotes, add a dot between the two parts and add **“.id”** at the end. It will look like this:

```
aws_vpc.eks_vpc.id
```

The code becomes like this:

```
# Public Route Tables
resource "aws_route_table" "public" {
  vpc_id = aws_vpc.eks_vpc.id

  route {
    cidr_block = "Add CIDR Block" #TO DO: Allows all outbound traffic
    gateway_id = Add Internet Gateway ID #TO DO: Add Internet Gateway ID
  }

  tags = {
    Name = "Add a name" # TODO: Add tag name
  }
}
```

For the **“cidr_block”**, we will allow all outbound traffic, so we will add **“0.0.0.0/0”** for the value. The code will look like this:

```
# Public Route Tables
resource "aws_route_table" "public" {
  vpc_id = aws_vpc.eks_vpc.id

  route {
    cidr_block = "0.0.0.0/0" #TO DO: Allows all outbound traffic
    gateway_id = Add Internet Gateway ID #TO DO: Add Internet Gateway ID
  }

  tags = {
    Name = "Add a name" # TODO: Add tag name
  }
}
```

For the **“gateway_id”**, we will be using the internet gateway. We get the resource type and name of the Public Subnet in the **“main.tf”** file, then modify it.

```
"aws_internet_gateway" "int-gw"
```

Remove the quotes, add a dot between the resource type and the name of the resource. Then add dot between them. It will look like this:

```
aws_internet_gateway.int-gw
```

Add **“.id”** at the end

```
aws_internet_gateway.int-gw.id
```

The code will look like this:

```
# Public Route Tables
resource "aws_route_table" "public" {
  vpc_id = aws_vpc.eks_vpc.id

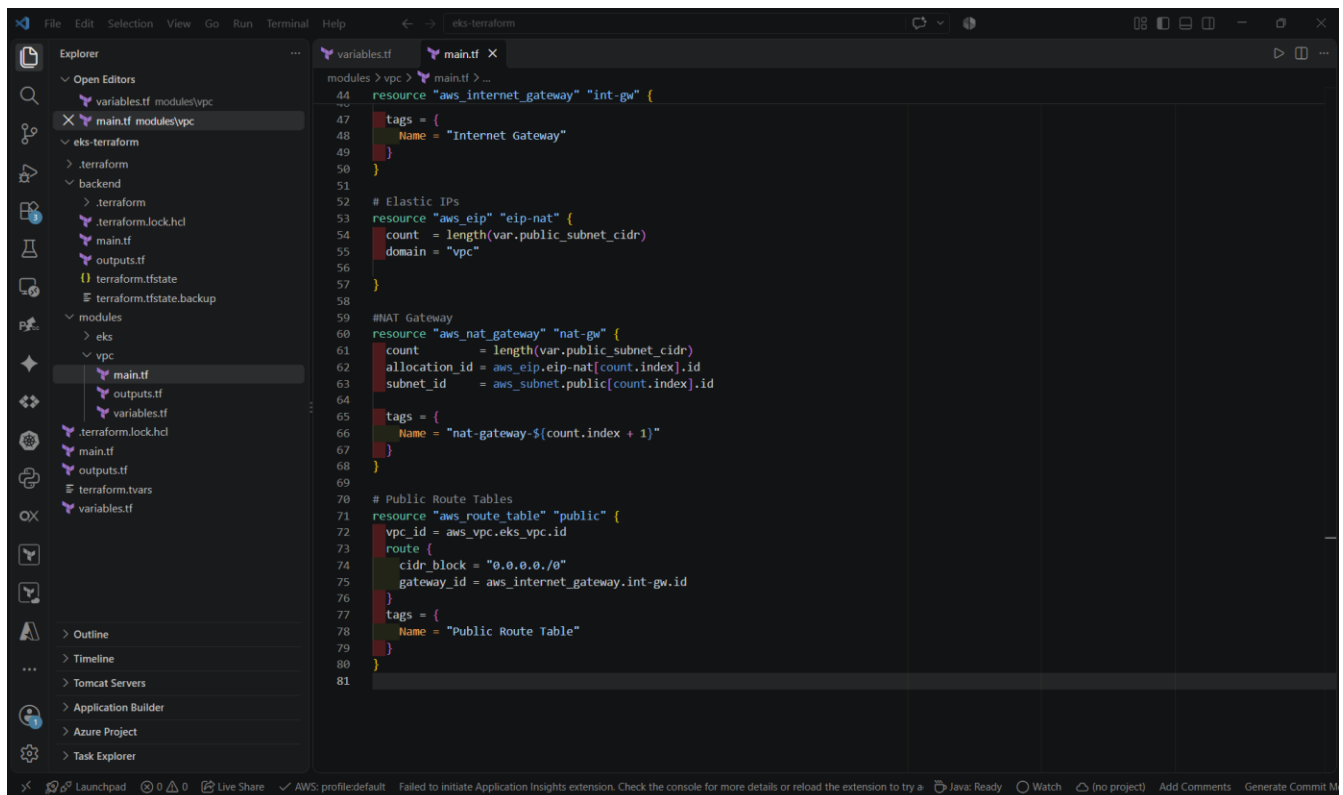
  route {
    cidr_block = "0.0.0.0/0" #TO DO: Allows all outbound traffic
    gateway_id = aws_internet_gateway.int-gw.id
  }
  tags = {
    Name = "Add a name" # TODO: Add tag name
  }
}
```

For the “tags”, we will enter “Public Route Table” for the value of “Name”. The completed code will look like this:

```
# Public Route Tables
resource "aws_route_table" "public" {
  vpc_id = aws_vpc.eks_vpc.id

  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.int-gw.id
  }
  tags = {
    Name = "Public Route Table"
  }
}
```

Copy this block and paste in the “main.tf” file in the “vpc” folder.



We have added the Public Route table block.

5.2.8. Setting up Private Route Tables Block

We have to add the block to create the private route tables called “**private**” with resource type as “**aws_route_table**”. Here, we will make use of the NAT gateway. Each private subnet will need a dedicated route table to help route outbound traffic through the corresponding NAT gateways. It will ensure that internal resources can access the internet without being publicly exposed.

We will use this sample block.

```
resource "aws_route_table" "private" {
  count = length(Add private_subnet_cidr)      # TO DO: Replace with private subnet cidr
  vpc_id = Add the VPC ID                      # TO DO: Add the VPC ID

  route {
    cidr_block      = "Add CIDR block"          # TO DO: Allows all outbound traffic
    nat_gateway_id = Add the NAT Gateway ID      # TO DO: Allows all outbound traffic
  }
  tags = {
    Name = "private-RT-${count.index + 1}"      # TO DO: Customize prefix or suffix if needed
  }
}
```

For the “**count**” parameter, we are going to define private subnet CIDR. Get the resource type and resource name of the private subnet CIDR variable in the “**variable.tf**” file.

```
"private_subnet_cidr"
```

Let us modify it. Remove the quotes and add “**var.**” in front. It will become

```
var.private_subnet_cidr
```

The code will look like this:

```
resource "aws_route_table" "private" {
  count = length(var.private_subnet_cidr)
  vpc_id = Add the VPC ID                      # TO DO: Add the VPC ID

  route {
    cidr_block      = "Add CIDR block"          # TO DO: Allows all outbound traffic
    nat_gateway_id = Add the NAT Gateway ID      # TO DO: Allows all outbound traffic
  }
  tags = {
    Name = "private-RT-${count.index + 1}"      # TO DO: Customize prefix or suffix if needed
  }
}
```

Let us add the value of the “**vpc_id**”. To do this, we have to get the name of the VPC resource on the VPC configuration block in the “**main.tf**” file.

```
"aws_vpc" "eks_vpc"
```

Remove the quotes, add a dot between the two parts and add “**.id**” at the end. It will look like this:

```
aws_vpc.eks_vpc.id
```

The code becomes like this:

```
resource "aws_route_table" "private" {
  count = length(var.private_subnet_cidr)
  vpc_id = aws_vpc.eks_vpc.id

  route {
    cidr_block      = "Add CIDR block"          # TO DO: Allows all outbound traffic
    nat_gateway_id = Add the NAT Gateway ID      # TO DO: Allows all outbound traffic
  }
}
```

```
}
tags = {
  Name = "private-RT-${count.index + 1}" # TO DO: Customize prefix or suffix if needed
}
}
```

For the **“cidr_block”**, we will allow all outbound traffic, so we will add **“0.0.0.0/0”** for the value. The code will look like this:

```
resource "aws_route_table" "private" {
  count = length(var.private_subnet_cidr)
  vpc_id = aws_vpc.eks_vpc.id

  route {
    cidr_block      = "0.0.0.0/0"
    nat_gateway_id = Add the NAT Gateway ID # TO DO: Allows all outbound traffic
  }
  tags = {
    Name = "private-RT-${count.index + 1}" # TO DO: Customize prefix or suffix if needed
  }
}
```

For the **“nat_gateway_id”**, we will be using the NAT gateway. We get the resource type and name of the Private Subnet in the **“main.tf”** file, then modify it.

```
"aws_nat_gateway" "nat-gw"
```

Remove the quotes, add a dot between the resource type and the name of the resource. Then add dot between them. It will look like this:

```
aws_nat_gateway.nat-gw
```

Add the index **“[count.index]”** followed by **“id”** at the end

```
aws_nat_gateway.nat-gw[count.index].id
```

The code will look like this:

```
resource "aws_route_table" "private" {
  count = length(var.private_subnet_cidr)
  vpc_id = aws_vpc.eks_vpc.id

  route {
    cidr_block      = "0.0.0.0/0"
    nat_gateway_id = aws_nat_gateway.nat-gw[count.index].id
  }
  tags = {
    Name = "private-RT-${count.index + 1}" # TO DO: Customize prefix or suffix if needed
  }
}
```

On **“tags”**, for the **“Name”**, we will use **“private-RT-\${count.index + 1}”** and the completed code will look like this:

```
resource "aws_route_table" "private" {
  count = length(var.private_subnet_cidr)
  vpc_id = aws_vpc.eks_vpc.id
  route {
    cidr_block      = "0.0.0.0/0"
    nat_gateway_id = aws_nat_gateway.nat-gw[count.index].id
  }
  tags = {
    Name = "private-RT-${count.index + 1}"
  }
}
```

}

Copy this and paste in the “**main.tf**” file in the “**vpc**” folder.

```

58
59 #NAT Gateway
60 resource "aws_nat_gateway" "nat-gw" {
61   count           = length(var.public_subnet_cidr)
62   allocation_id = aws_eip.eip-nat[count.index].id
63   subnet_id      = aws_subnet.public[count.index].id
64
65   tags = {
66     Name = "nat-gateway-${count.index + 1}"
67   }
68 }
69
70 # Public Route Tables
71 resource "aws_route_table" "public" {
72   vpc_id = aws_vpc.eks_vpc.id
73   route {
74     cidr_block = "0.0.0.0/0"
75     gateway_id = aws_internet_gateway.int-gw.id
76   }
77   tags = {
78     Name = "Public Route Table"
79   }
80 }
81
82 resource "aws_route_table" "private" {
83   count = length(var.private_subnet_cidr)
84   vpc_id = aws_vpc.eks_vpc.id
85   route {
86     cidr_block = "0.0.0.0/0"
87     nat_gateway_id = aws_nat_gateway.nat-gw[count.index].id
88   }
89   tags = {
90     Name = "private-RT-${count.index + 1}"
91   }
92 }
93

```

We have completed the private route table block.

5.2.9. Setting up Private Route Table Association Block

In this part, we will be associating each private subnet to its corresponding private route table we have defined in the “**private**” resource. We will give it the resource type as “**aws_route_table_association**” and resource name as “**private**”. We will make use of this template:

```

resource "aws_route_table_association" "private" {
  count           = length(define_cidrs) # TO DO: Define the Private Subnet CIDR
  subnet_id      = Add the Subnet ID    # TO DO: Add the Subnet ID
  route_table_id = Add the Route Table ID # TO DO: Add the route table ID
}

```

For the “**count**” parameter, we are going to define private subnet CIDR. Get the resource type and resource name of the private subnet CIDR variable in the “**variable.tf**” file.

“private_subnet_cidr”

Let us modify it. Remove the quotes and add “**var.**” in front. It will become

var.private_subnet_cidr

The code will look like this:

```

resource "aws_route_table_association" "private" {
  count           = length(var.private_subnet_cidr)
  subnet_id      = Add the Subnet ID    # TO DO: Add the Subnet ID
  route_table_id = Add the Route Table ID # TO DO: Add the route table ID
}

```

For the “**subnet_id**”, we will be using the private subnet. We get the resource type and name of the Private Subnet in the “**main.tf**” file, then modify it.

```
aws_subnet.private
```

Remove the quotes, add a dot between the resource type and the name of the resource. Then add dot between them. It will look like this:

```
aws_subnet.private
```

Add the index “[**count.index**]” followed by “.id” at the end to look like this

```
aws_subnet.private[count.index].id
```

The code becomes like this:

```
resource "aws_route_table_association" "private" {
  count          = length(var.private_subnet_cidr)
  subnet_id      = aws_subnet.private[count.index].id
  route_table_id = Add the Route Table ID # TO DO: Add the route table ID
}
```

For the “**route_table_id**”, we will be using the private route table. We get the resource type and name of the Private Route Table in the “**main.tf**” file, then modify it.

```
"aws_route_table" "private"
```

Remove the quotes, add a dot between the resource type and the name of the resource. Then add dot between them. It will look like this:

```
aws_route_table.private
```

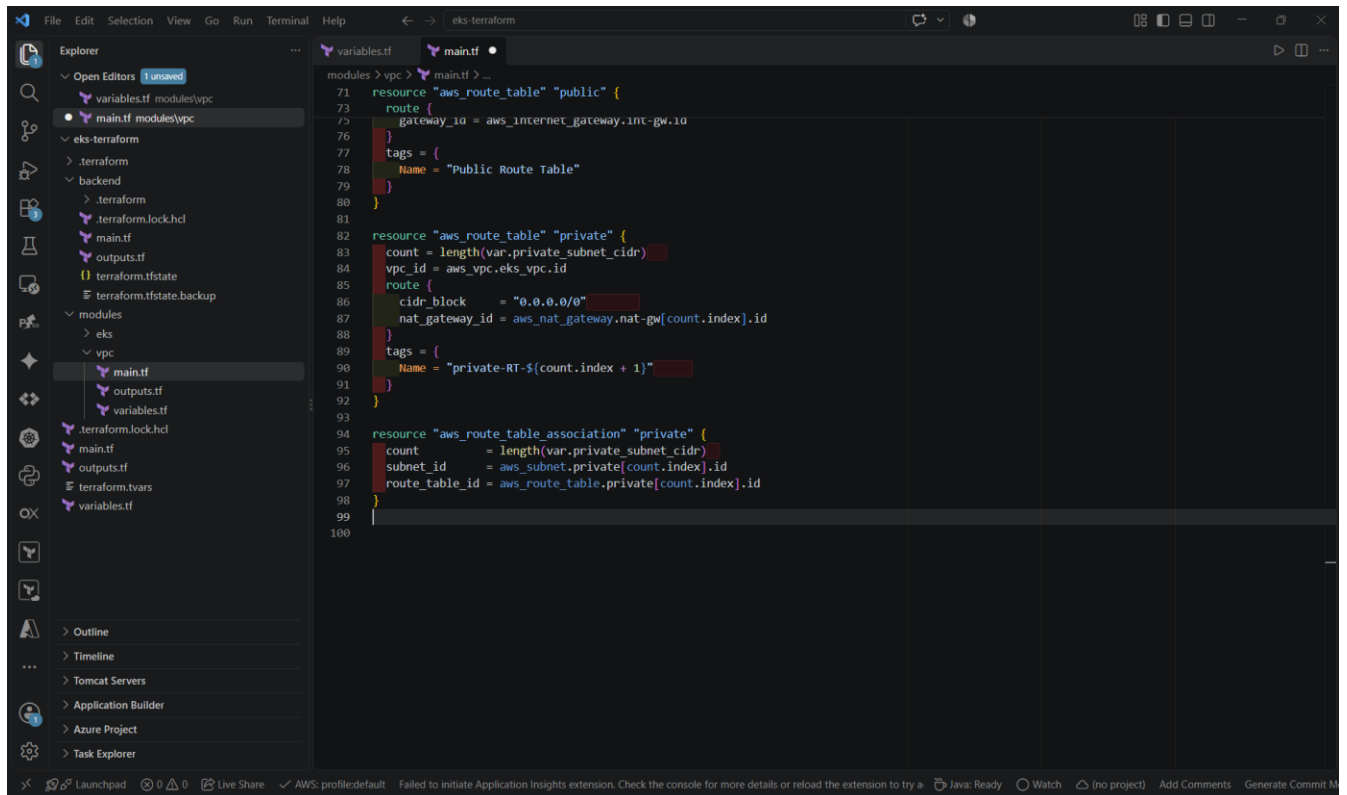
Add the index “[**count.index**]” followed by “.id” at the end to look like this

```
aws_route_table.private[count.index].id
```

The code becomes like this:

```
resource "aws_route_table_association" "private" {
  count          = length(var.private_subnet_cidr)
  subnet_id      = aws_subnet.private[count.index].id
  route_table_id = aws_route_table.private[count.index].id
}
```

Copy this code and paste in the “**main.tf**” file in the “**vpc**” folder.



We have completed the block for the association of the private route tables with their corresponding subnets.

5.2.10. Setting up Public Route Table Association Block

We will be associating each public subnet to its corresponding public route table we have defined in the **“public”** resource. We will give it the resource type as **“aws_route_table_association”** and resource name as **“public”**. We will make use of this template:

```
# Route Table Associations
resource "aws_route_table_association" "public" {
  count          = length(define_cidrs) # TO DO: Add Public Subnet CIDR
  subnet_id      = Add Subnet ID        # TO DO: Add Public Subnet ID
  route_table_id = Add Route Table ID   # TO DO: Add Public Route Table ID
}
```

For the **“count”** parameter, we are going to define public subnet CIDR. Get the resource type and resource name of the public subnet CIDR variable in the **“variable.tf”** file.

“public_subnet_cidr”

Let us modify it. Remove the quotes and add **“var.”** in front. It will become

var.public_subnet_cidr

The code will look like this:

```
# Route Table Associations
resource "aws_route_table_association" "public" {
  count          = length(var.public_subnet_cidr)
  subnet_id      = Add Subnet ID        # TO DO: Add Public Subnet ID
  route_table_id = Add Route Table ID   # TO DO: Add Public Route Table ID
}
```

For the “**subnet_id**”, we will be using the public subnet. We get the resource type and name of the Private Subnet in the “**main.tf**” file, then modify it.

```
"aws_subnet" "public"
```

Remove the quotes, add a dot between the resource type and the name of the resource. Then add dot between them. It will look like this:

```
aws_subnet.public
```

Add the index “[**count.index**]” followed by “**.id**” at the end to look like this

```
aws_subnet.public[count.index].id
```

The code becomes like this:

```
resource "aws_route_table_association" "public" {
  count          = length(var.public_subnet_cidr)
  subnet_id      = aws_subnet.public[count.index].id
  route_table_id = Add the Route Table ID # TO DO: Add the route table ID
}
```

For the “**route_table_id**”, we will be using the public route table. We get the resource type and name of the Public Route Table in the “**main.tf**” file, then modify it.

```
"aws_route_table" "public"
```

Remove the quotes, add a dot between the resource type and the name of the resource. Then add dot between them. It will look like this:

```
aws_route_table.public
```

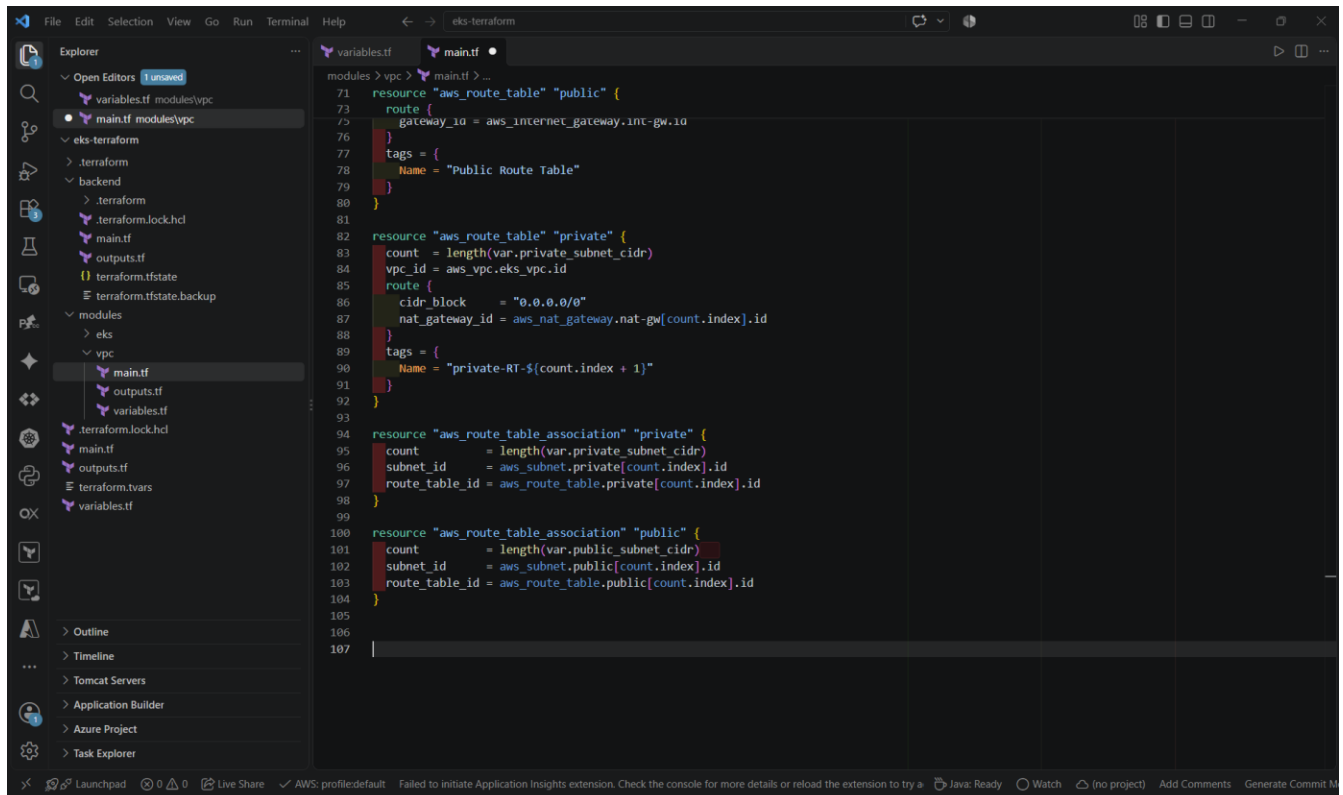
Add the index “[**count.index**]” followed by “**.id**” at the end to look like this

```
aws_route_table.public[count.index].id
```

The code becomes like this:

```
resource "aws_route_table_association" "public" {
  count          = length(var.public_subnet_cidr)
  subnet_id      = aws_subnet.public[count.index].id
  route_table_id = aws_route_table.public[count.index].id
}
```

Copy this code and paste in the “**main.tf**” file in the “**vpc**” folder.



We have completed the configuration of the VPC and its components.

5.3. Configuring the Outputs – “outputs.tf”

We also have to add some blocks in the “**outputs.tf**” file to display the outputs on the terminal.

5.3.1. Setting up the output of the VPC

Let us start with the VPC. We will use this sample template:

```
# VPC ID output is essential for associating other resources within the VPC.
output "vpc_id" {
  description = "VPC ID"
  value       = Add the VPC ID
}
```

For the “Description”, we will enter “VPC ID”. The code will look like this:

```
# VPC ID output is essential for associating other resources within the VPC.
output "vpc_id" {
  description = "VPC ID"
  value       = Add the VPC ID
}
```

For the “**value**”, we will enter the VPC ID. To get the VPC id, we will copy the resource type and name of the VPC configuration in the “**main.tf**” file.

```
"aws_vpc" "eks_vpc"
```

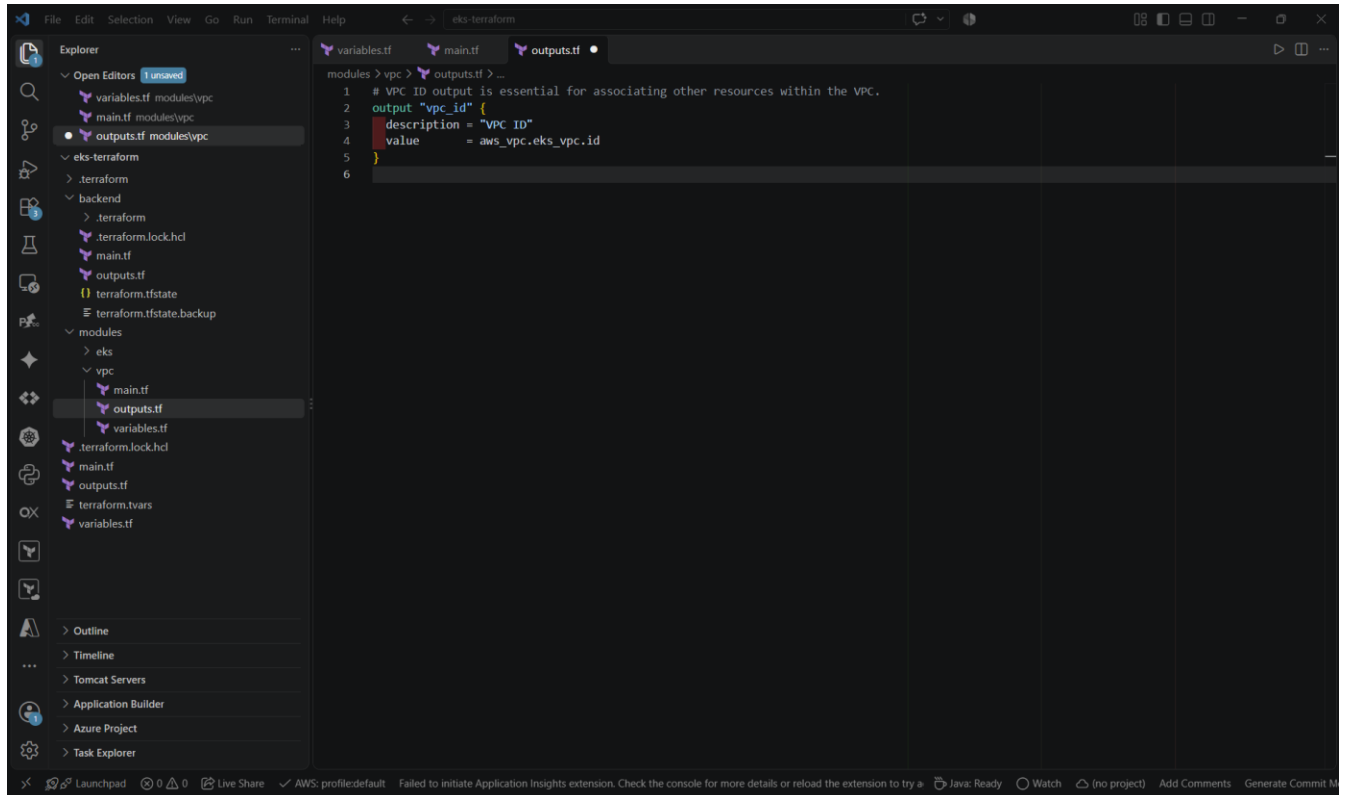
Let us modify it. Remove the quotes and add a dot between the resource type and resource name. Then add “**.id**” at the end.

```
aws_vpc.eks_vpc.id
```

This is the VPC id, the code will now look like this:

```
# VPC ID output is essential for associating other resources within the VPC.
output "vpc_id" {
  description = "VPC ID"
  value       = aws_vpc.eks_vpc.id
}
```

Copy and paste this block in the “**output.tf**” file in the “**vpc**” folder.



5.3.2. Setting up the output of the Public Subnet

We will now set up the output of the public subnet. We will use this template:

```
# Public subnet IDs are required for routing internet-facing traffic and public services.
output "public_subnet_ids" {
  description = "Add a Description"
  value       = Add a value
}
```

For the “**Description**”, we will enter “**public subnet IDs**”. The code will look like this:

```
# Public subnet IDs are required for routing internet-facing traffic and public services.
output "public_subnet_ids" {
  description = "public subnet IDs"
  value       = Add a value
}
```

For the “**value**”, we will enter the VPC ID. To get the VPC id, we will copy the resource type and name of the Public Subnet in the “**main.tf**” file.

```
"aws_subnet" "public"
```

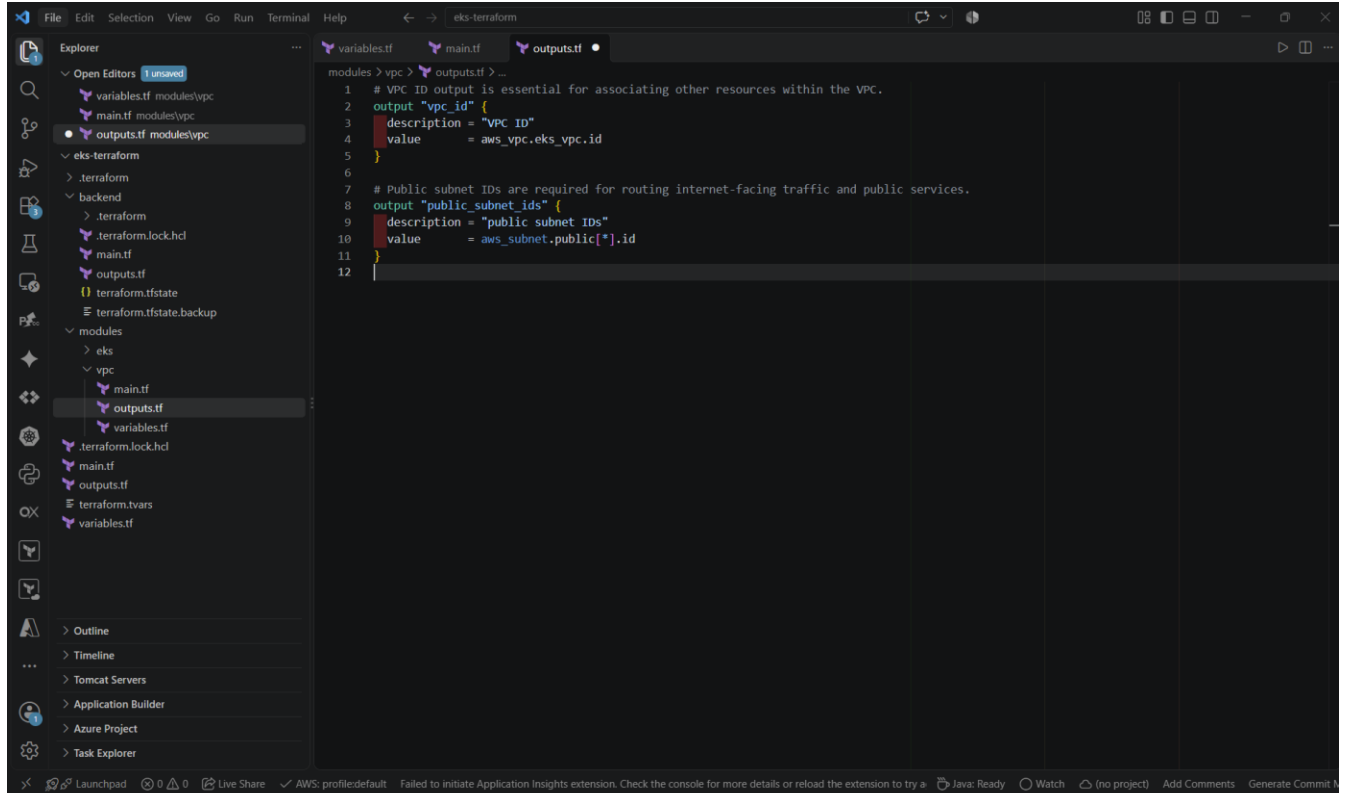
Let us modify it. Remove the quotes and add a dot between the resource type and resource name. Then add “[*]” and “.id” at the end.

```
aws_subnet.public[*].id
```

This is the public subnet id, the code will now look like this:

```
# Public subnet IDs are required for routing internet-facing traffic and public services.
output "public_subnet_ids" {
  description = "public subnet IDs"
  value       = aws_subnet.public[*].id
}
```

Copy this block and paste the in “**outputs.tf**” file in the “**vpc**” folder



We have added the block for the public subnet.

5.3.3. Setting up the Output of the Private Subnet

We will now set up the output of the private subnet. We will use this template:

```
# Private subnet IDs are necessary for isolating internal resources and worker nodes.
output "private_subnet_ids" {
  description = "Add Description"
  value       = Add a value
}
```

For the “**Description**”, we will enter “**private subnet IDs**”. The code will look like this:

```
# Private subnet IDs are necessary for isolating internal resources and worker nodes.
output "private_subnet_ids" {
  description = "private subnet IDs"
  value       = Add a value
}
```

For the “**value**”, we will enter the VPC ID. To get the VPC id, we will copy the resource type and name of the Private Subnet in the “**main.tf**” file.

```
"aws_subnet" "private"
```

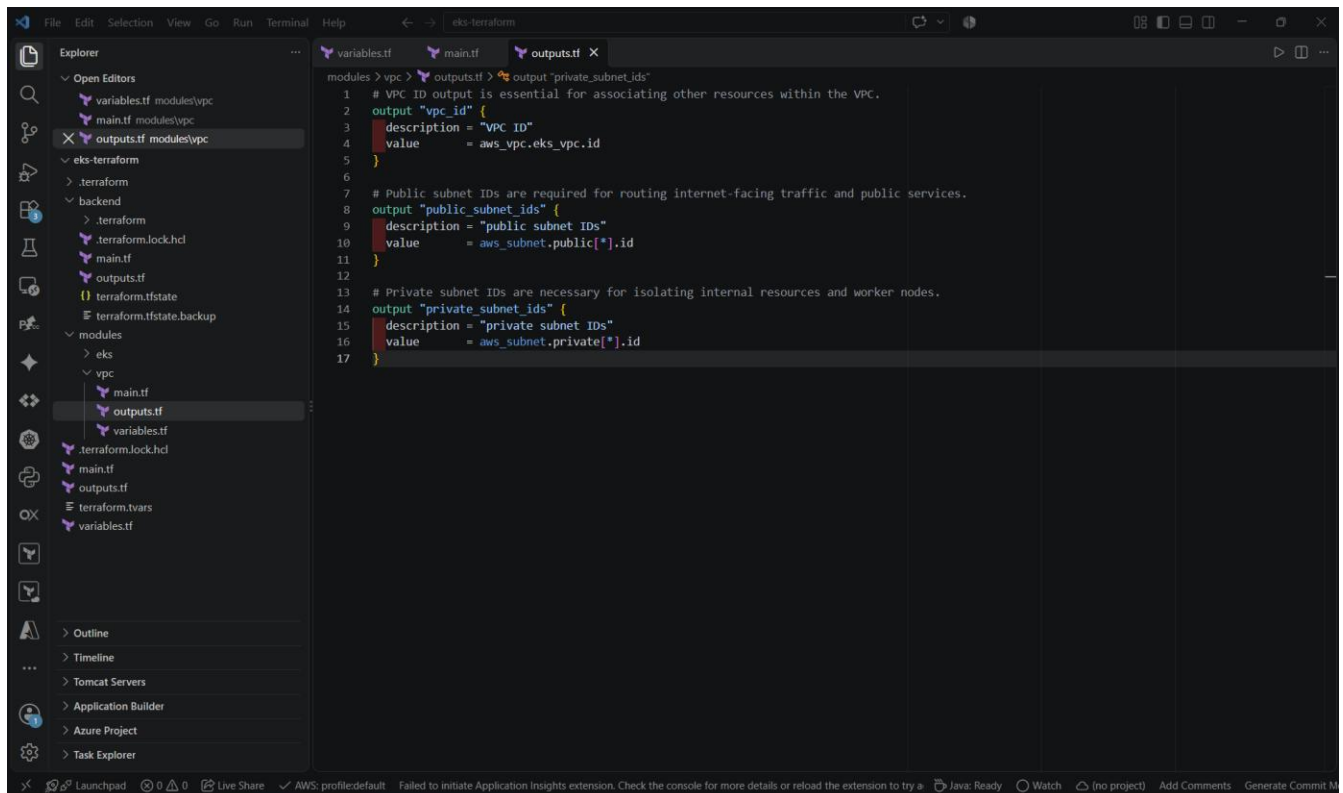
Let us modify it. Remove the quotes and add a dot between the resource type and resource name. Then add “[*]” and “.id” at the end.

```
aws_subnet.private[*].id
```

This is the private subnet ID, the code will now look like this:

```
# Private subnet IDs are necessary for isolating internal resources and worker nodes.
output "private_subnet_ids" {
  description = "private subnet IDs"
  value       = aws_subnet.private[*].id
}
```

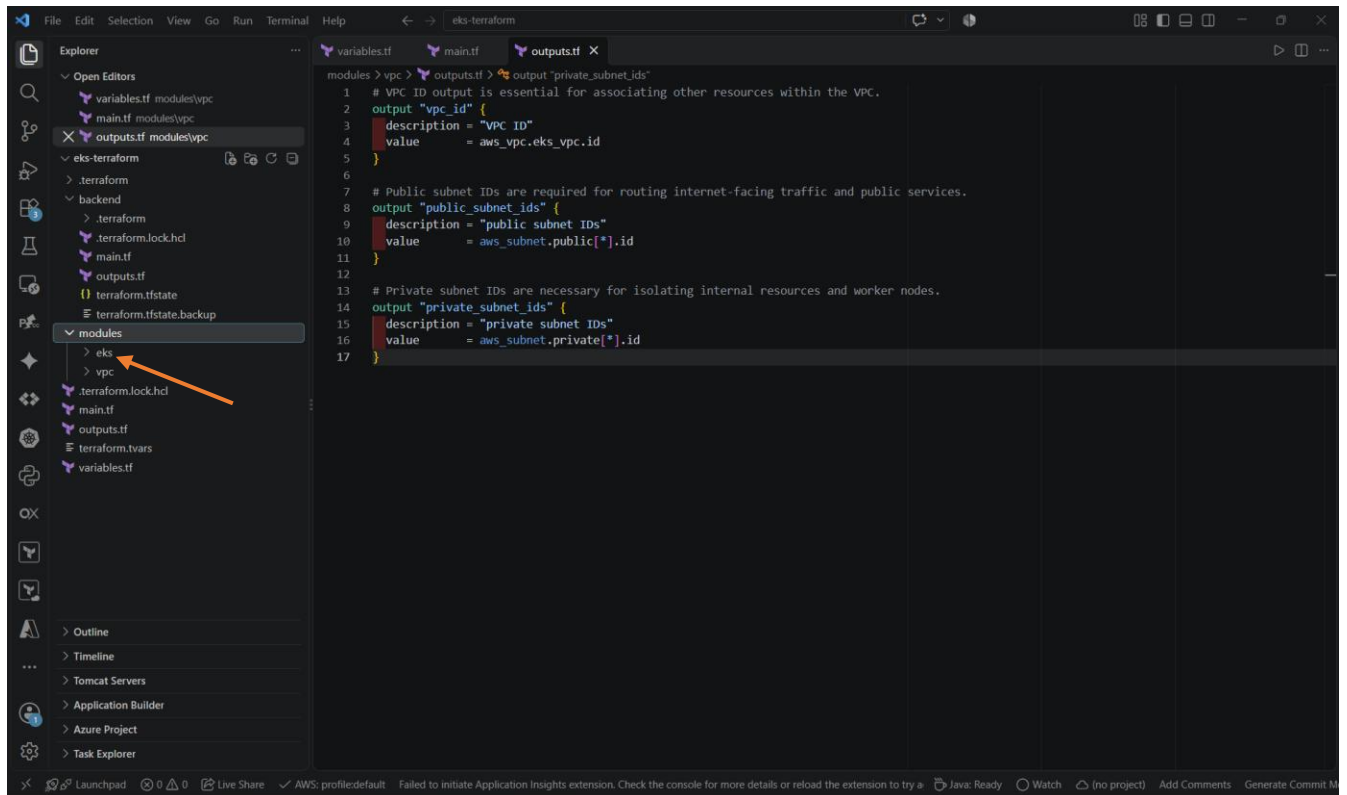
Copy this block and paste the in “outputs.tf” file in the “vpc” folder



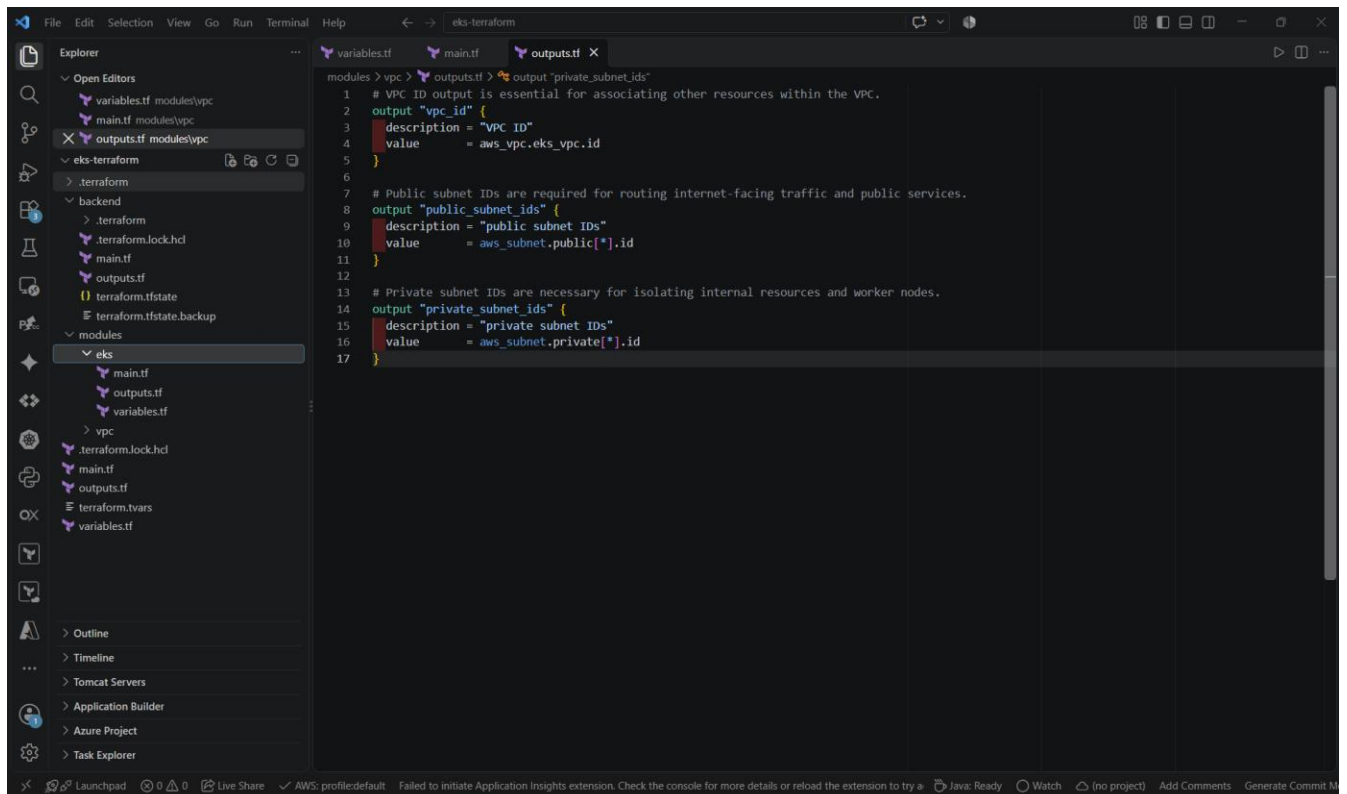
We have completed the blocks for the “outputs.tf” file

6. Set Up an EKS Cluster on AWS with Terraform

We have to configure the EKS cluster. We have to go to the “modules” directory.



Click on the “eks” folder



6.1. Configuring the Variables – “variables.tf”

We have to configure the variables for VPC. This will be done on the “**variables.tf**” file in the “**vpc**” folder.

6.1.1. Adding Variable for Region

Let us add the variable for the region. We will call this variable “**region**”. The type is “**string**” and the default region is “**us-east-2**”.

```
variable "region" {
  description = "AWS region"
  type        = string
  default     = "us-east-1"
}
```

6.1.2. Adding Variable for VPC ID

Let us add the variable for the VPC ID. We will call this variable “**vpc_id**”. The type is “**string**”.

```
variable "vpc_id" {
  description = "My VPC ID for EKS cluster"
  type        = string
}
```

6.1.3. Adding Variable for Subnet

Let us add the variable for the subnet ID. We will call this variable “**subnet_id**”. The type is a list of strings “**list(string)**”.

```
variable "subnet_id" {
  description = "Subnet ID"
  type        = list(string)
}
```

6.1.4. Adding Variable for EKS Cluster Name

Let us add the variable for the cluster name. We will call this variable “**eks_cluster_name**”. The type is “**string**”.

```
variable "eks_cluster_name" {
  description = "The name of my EKS cluster"
  type        = string
}
```

6.1.5. Adding Variable for Cluster Version

Let us add the variable for the cluster version. We will call this variable “**cluster_version**”. The type is “**string**”.

```
variable "cluster_version" {
  description = "My cluster version"
  type        = string
}
```

6.1.6. Adding Variable for Node Groups

Let us add the input variable for the Node Groups. We will call this variable “**node_groups**”. The types for the “**instance_types**” is a list of strings “**list(string)**”, the type for the “**capacity_type**” is “**string**”, the type of the auto-scaling variable “**scaling-config**” is an “**object**” containing the Maximum nodes, Desired nodes and Minimum nodes.

```
variable "node_groups" {
  description = "EKS node group configuration variable"

  type = map(
    object({
      instance_types = list(string)
      capacity_type  = string

      scaling_config = object({
        desired_size = number
        max_size     = number
        min_size     = number
      })
    })
  )
}
```

6.2. Configuring the EKS Cluster – “main.tf”

Let us configure the EKS cluster. This will be done on the “**main.tf**” file in the “**eks**” folder in the “**modules**” directory. Here we will configure the IAM roles, the control plane and the worker nodes.

6.2.1. Define IAM Roles

We will start by defining the IAM roles for our EKS cluster and its components. We will create IAM role for EKS cluster and worker nodes.

6.2.1.1. Configure IAM role for EKS Cluster and Attach Policy

We will start with the IAM role for the EKS cluster and attach the required policy. This role is very crucial in allowing our EKS cluster to make API request to AWS services. The resource type will be “**aws_iam_role**” and the resource name is “**eks-cluster-role**”. We will use this sample code:

```
# IAM Role for EKS cluster
resource "aws_iam_role" "eks-cluster-role" {
  name = "${define-cluster_name}-eks-cluster-role"

  assume_role_policy = jsonencode({
    Version = "2012-10-17",
    Statement = [{
      Action = "sts:AssumeRole",
      Effect = "Allow",
      Principal = {
        Service = "eks.amazonaws.com"
      }
    }]
  })
}
```

For the “**name**”, we have to define the EKS cluster name. We will get this from EKS cluster name in the “**variable.tf**” file.

```
"eks_cluster_name"
```

Remove the quotes and add “**var.**” at the beginning to become

```
var.eks_cluster_name
```

The code will now look like this:

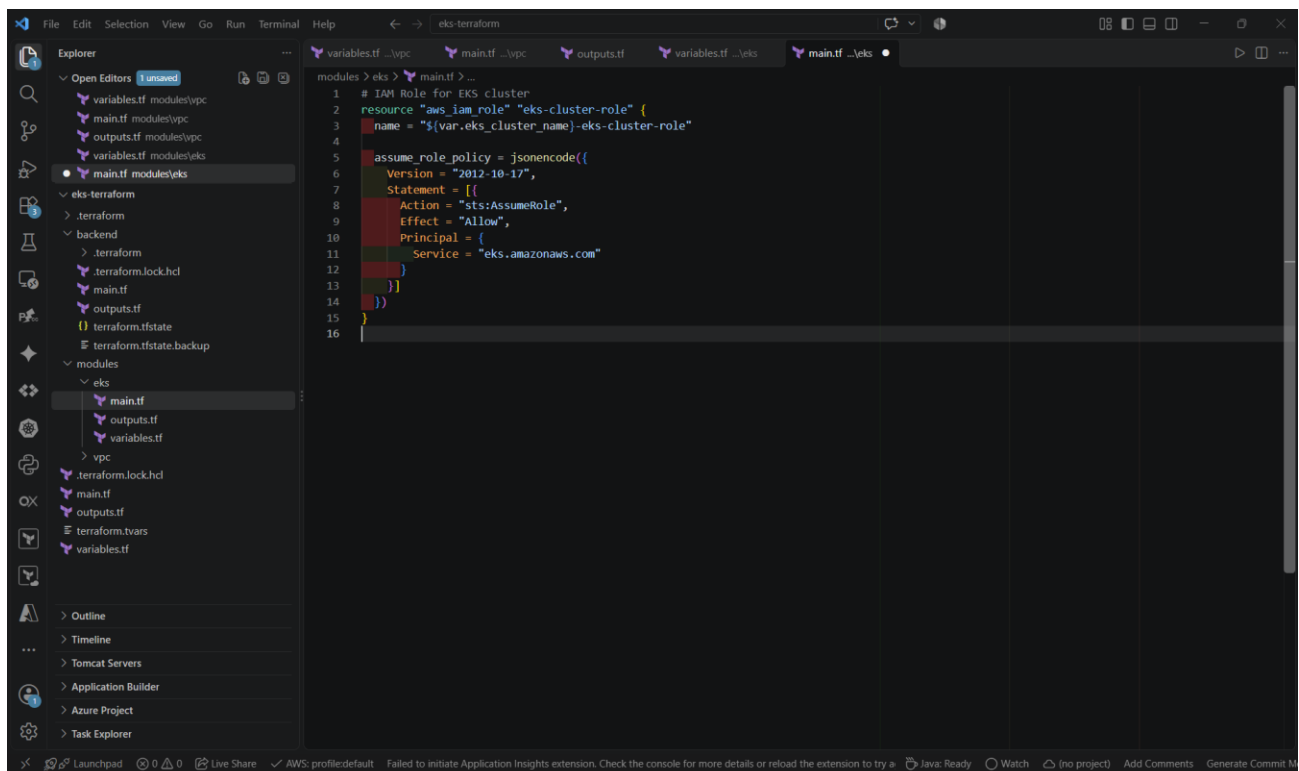
The important part is:

```
Principal = {  
  Service = "eks.amazonaws.com"  
}
```

This allows the Amazon EKS service to assume this IAM role. The modified code will look like this:

```
# IAM Role for EKS cluster  
resource "aws_iam_role" "eks-cluster-role" {  
  name = "${var.eks_cluster_name}-eks-cluster-role"  
  
  assume_role_policy = jsonencode({  
    Version = "2012-10-17",  
    Statement = [{  
      Action = "sts:AssumeRole",  
      Effect = "Allow",  
      Principal = {  
        Service = "eks.amazonaws.com"  
      }  
    }]  
  })  
}
```

Copy the completed code and paste in the “main.tf” file in the “eks” folder.



We have added the block for the IAM role of the EKS cluster. Now, let us attach the required EKS policy. The resource type will be “aws_iam_role_policy_attachment” and the resource name will be “cluster_policy”. We will use this sample template to attach the policy.

```
# Attach EKS Cluster Policy to Cluster Role
resource "aws_iam_role_policy_attachment" "cluster_policy" {
  policy_arn = "Attach_policy"          #To DO: Add policy
  role       = define-cluster-role name #TO DO: Add the role name
}
```

For “**policy_arn**”, we are attaching the policy “**AmazonEKSClusterPolicy**”. So, we will use:

```
"arn:aws:iam::aws:policy/AmazonEKSClusterPolicy"
```

The code will now look like this:

```
# Attach EKS Cluster Policy to Cluster Role
resource "aws_iam_role_policy_attachment" "cluster_policy" {
  policy_arn = "arn:aws:iam::aws:policy/AmazonEKSClusterPolicy"
  role       = define-cluster-role name          #TO DO: Add the role name
}
```

For the “**role**”, we will copy the resource type and resource name of the IAM role.

```
"aws_iam_role" "eks-cluster-role"
```

Let us modify it. Remove the quotes and add a dot between the resource type and resource name

```
aws_iam_role.eks-cluster-role
```

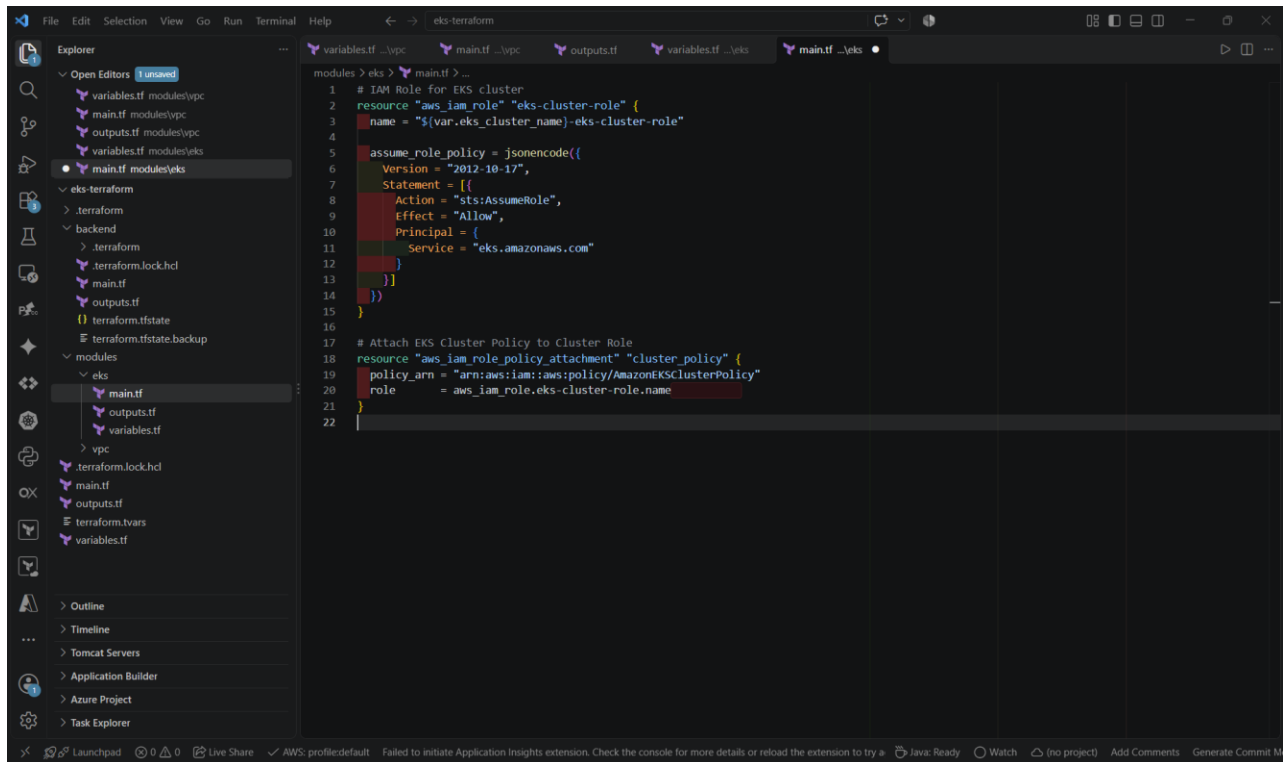
Add “**.name**” at the end to give

```
aws_iam_role.eks-cluster-role.name
```

The code will now look like this:

```
# Attach EKS Cluster Policy to Cluster Role
resource "aws_iam_role_policy_attachment" "cluster_policy" {
  policy_arn = "arn:aws:iam::aws:policy/AmazonEKSClusterPolicy"
  role       = aws_iam_role.eks-cluster-role.name
}
```

Copy this code and paste in the “**main.tf**” file in the “**eks**” folder



We have attached the policy to the IAM role for EKS cluster.

6.2.1.2. Configure the IAM Role for Worker Nodes and attach policy

We going to create the IAM role for our worker nodes. The resource type will be **“aws_iam_role”** and the resource name is **“eks-cluster-role”**. We will use this sample code:

```

# IAM Role for Worker Nodes
resource "aws_iam_role" "eks_node_role" {
  name = "${define-eks-cluster-name}-node-role"

  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Action = "sts:AssumeRole"
      Effect = "Allow"
      Principal = {
        Service = "ec2.amazonaws.com"
      }
    }]
  })
}

```

For the **“name”**, we have to define the EKS cluster name. We will get this from EKS cluster name in the **“variable.tf”** file.

“eks_cluster_name”

Remove the quotes and add **“var.”** at the beginning to become

var.eks_cluster_name

The code will now look like this:

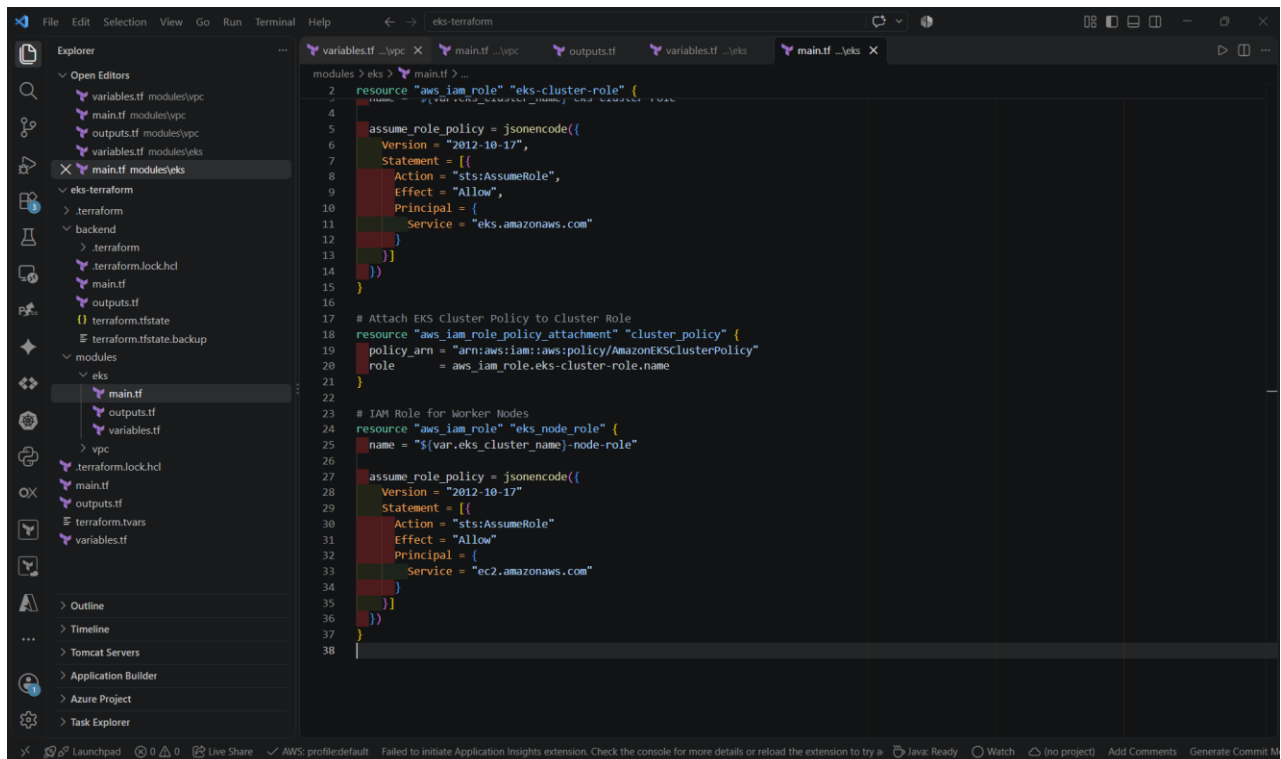
```
# IAM Role for Worker Nodes
resource "aws_iam_role" "eks_node_role" {
  name = "${var.eks_cluster_name}-node-role"

  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Action = "sts:AssumeRole"
      Effect = "Allow"
      Principal = {
        Service = "ec2.amazonaws.com"
      }
    }]
  })
}
```

For

```
Principal = {
  Service = "ec2.amazonaws.com"
}
```

This allows the Amazon EC2 service to assume this IAM role. Copy the completed code and paste in the “main.tf” file in the “eks” folder.



We have added the block for the IAM role of the worker nodes. Now, let us attach the required EKS policy. The resource type will be “aws_iam_role_policy_attachment” and the resource name will be “eks_node_policy”. We will use this sample template to attach the policy.

```
# Attach Required Policies to Worker Node Role
resource "aws_iam_role_policy_attachment" "eks_node_policy" {
  for_each = toset([
    Add the policies
  ])
}
```

```
  ])  
  
  policy_arn = each.value  
  role       = Add the role name  
}
```

For **“policy_arn”**, we are attaching the policy **“AmazonEKSWorkerNodePolicy”**, **“AmazonEKS_CNI_Policy”** and **“AmazonEC2ContainerRegistryReadOnly”**. So, we will use:

```
"arn:aws:iam::aws:policy/AmazonEKSWorkerNodePolicy",  
"arn:aws:iam::aws:policy/AmazonEKS_CNI_Policy",  
"arn:aws:iam::aws:policy/AmazonEC2ContainerRegistryReadOnly"
```

The code will now look like this:

```
# Attach Required Policies to Worker Node Role  
  
resource "aws_iam_role_policy_attachment" "eks_node_policy" {  
  for_each = toset([  
    "arn:aws:iam::aws:policy/AmazonEKSWorkerNodePolicy",  
    "arn:aws:iam::aws:policy/AmazonEKS_CNI_Policy",  
    "arn:aws:iam::aws:policy/AmazonEC2ContainerRegistryReadOnly"  
  ])  
  
  policy_arn = each.value  
  role       = Add the role name  
}
```

For the **“role”**, we will copy the resource type and resource name of the IAM role.

```
"aws_iam_role" "eks_node_role"
```

Let us modify it. Remove the quotes and add a dot between the resource type and resource name

```
aws_iam_role.eks_node_role
```

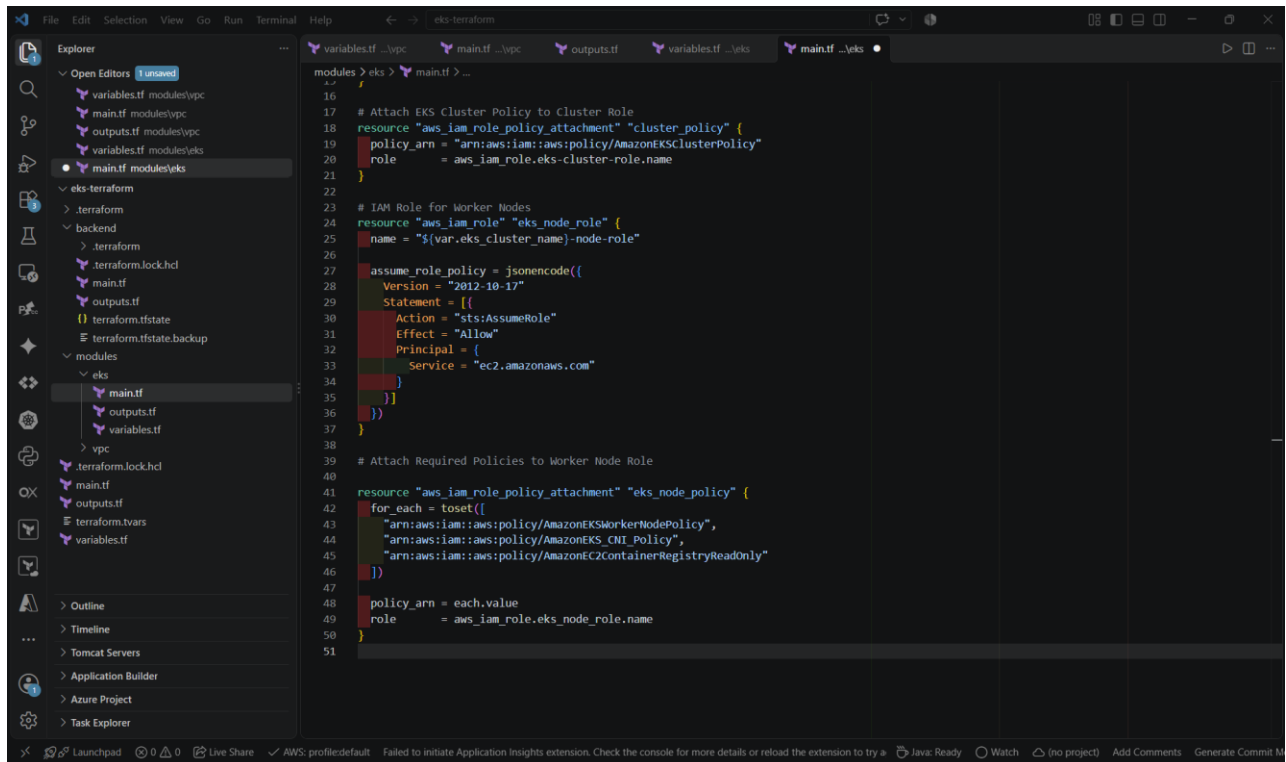
Add **“.name”** at the end to give

```
aws_iam_role.eks_node_role.name
```

The code will now look like this:

```
# Attach Required Policies to Worker Node Role  
  
resource "aws_iam_role_policy_attachment" "eks_node_policy" {  
  for_each = toset([  
    "arn:aws:iam::aws:policy/AmazonEKSWorkerNodePolicy",  
    "arn:aws:iam::aws:policy/AmazonEKS_CNI_Policy",  
    "arn:aws:iam::aws:policy/AmazonEC2ContainerRegistryReadOnly"  
  ])  
  
  policy_arn = each.value  
  role       = aws_iam_role.eks_node_role.name  
}
```

Copy this code and paste in the **“main.tf”** file in the **“eks”** folder



We have added the policy to be attached to the IAM role for worker node

6.2.2. Create Control Plane

In this part, we have to create the main configuration of the EKS cluster. The resource type will be **"aws_eks_cluster"** and the resource name will be **"main-eks-cluster"**.

```
# Create EKS Cluster
resource "aws_eks_cluster" "main-eks-cluster" {
  name      = Add the name of the Cluster
  version   = Add the version of cluster
  role_arn  = Add the arn of the role

  vpc_config {
    subnet_ids = Add subnet ID
  }

  depends_on = [
    aws_iam_role_policy_attachment.cluster_policy
  ]
}
```

For the **"name"**, copy the resource type and resource name of the cluster name variable in the **"variables.tf"** file.

"eks_cluster_name"

Let us modify this by removing the quotes and add **"var."** at the beginning.

var.eks_cluster_name

The code will now look like this:

```
# Create EKS Cluster
```

```
resource "aws_eks_cluster" "main-eks-cluster" {
  name      = var.eks_cluster_name
  version   = Add the version of cluster
  role_arn  = Add the arn of the role

  vpc_config {
    subnet_ids = Add subnet ID
  }

  depends_on = [
    aws_iam_role_policy_attachment.cluster_policy
  ]
}
```

For the “version”, copy the resource type and resource name of the cluster version variable in the “variables.tf” file.

```
"cluster_version"
```

Let us modify this by removing the quotes and add “var.” at the beginning.

```
var.cluster_version
```

The code will now look like this:

```
# Create EKS Cluster
resource "aws_eks_cluster" "main-eks-cluster" {
  name      = var.eks_cluster_name
  version   = var.cluster_version
  role_arn  = Add the arn of the role

  vpc_config {
    subnet_ids = Add subnet ID
  }

  depends_on = [
    aws_iam_role_policy_attachment.cluster_policy
  ]
}
```

For the “role_arn”, copy the resource type and resource name of the EKS IAM role

```
"aws_iam_role" "eks-cluster-role"
```

Remove the quotes, add a dot between the resource type and resource name and add “.arn” at the end. It will become:

```
aws_iam_role.eks-cluster-role.arn
```

The code will look like this:

```
# Create EKS Cluster
resource "aws_eks_cluster" "main-eks-cluster" {
  name      = var.eks_cluster_name
  version   = var.cluster_version
  role_arn  = aws_iam_role "eks-cluster-role.arn

  vpc_config {
```

```
    subnet_ids = Add subnet ID
  }

  depends_on = [
    aws_iam_role_policy_attachment.cluster_policy
  ]
}
```

For the “**subnet_ids**”, we will get the name of the subnet ID variable

```
subnet_id
```

Add “**var.**” at the beginning to become:

```
var.subnet_id
```

The code will look like this:

```
# Create EKS Cluster
resource "aws_eks_cluster" "main-eks-cluster" {
  name      = var.eks_cluster_name
  version   = var.cluster_version
  role_arn  = aws_iam_role.eks-cluster-role.arn

  vpc_config {
    subnet_ids = var.subnet_id
  }

  depends_on = [
    Add reference
  ]
}
```

For “**depends_on**”, we will add the argument to reference the EKS cluster. This means that we have to provision the IAM role before we can create the EKS cluster. We will get this by copying the resource type and resource name of the EKS cluster policy in the “**main.tf**” file.

```
"aws_iam_role_policy_attachment" "cluster_policy"
```

Remove the quotes, add a dot between the resource name and resource type. This looks like:

```
aws_iam_role_policy_attachment.cluster_policy
```

The code will now look like this:

```
# Create EKS Cluster
resource "aws_eks_cluster" "main-eks-cluster" {
  name      = var.eks_cluster_name
  version   = var.cluster_version
  role_arn  = aws_iam_role.eks-cluster-role.arn

  vpc_config {
    subnet_ids = var.subnet_id
  }

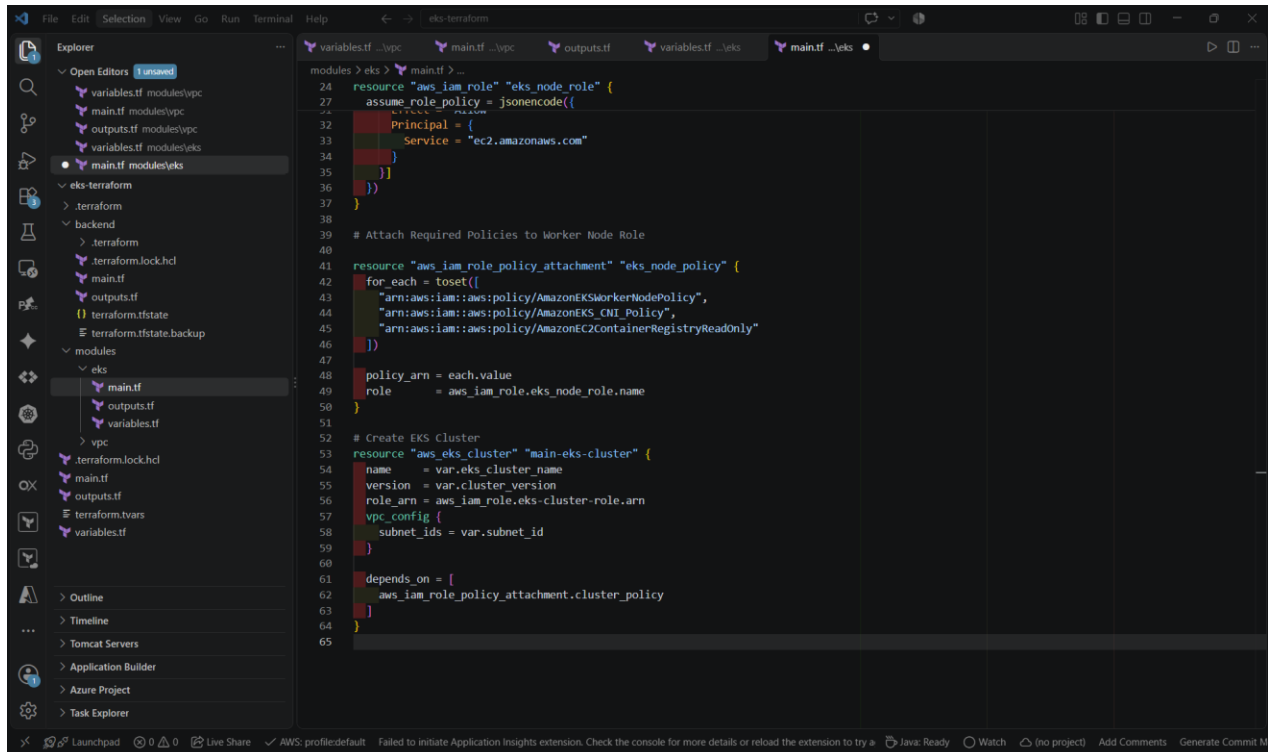
  depends_on = [
    aws_iam_role_policy_attachment.cluster_policy
  ]
}
```

```

    ]
  }
}

```

Copy this block and paste in the “**main.tf**” file in the “**eks**” folder.



We have added the block to create EKS cluster.

6.2.3. Worker Nodes

We have to create the Node group. We will use the resource type “**aws_eks_node_group**” and the resource name is “**eks-worker-node**”. We will use this sample template:

```

# Create EKS Managed Node Group
resource "aws_eks_node_group" "eks-worker-node" {
  for_each = var.node_groups

  cluster_name    = Add-cluster-name.name
  node_group_name = each.key
  node_role_arn   = aws_iam_role.eks_node_role.arn
  subnet_ids      = var.subnet_id

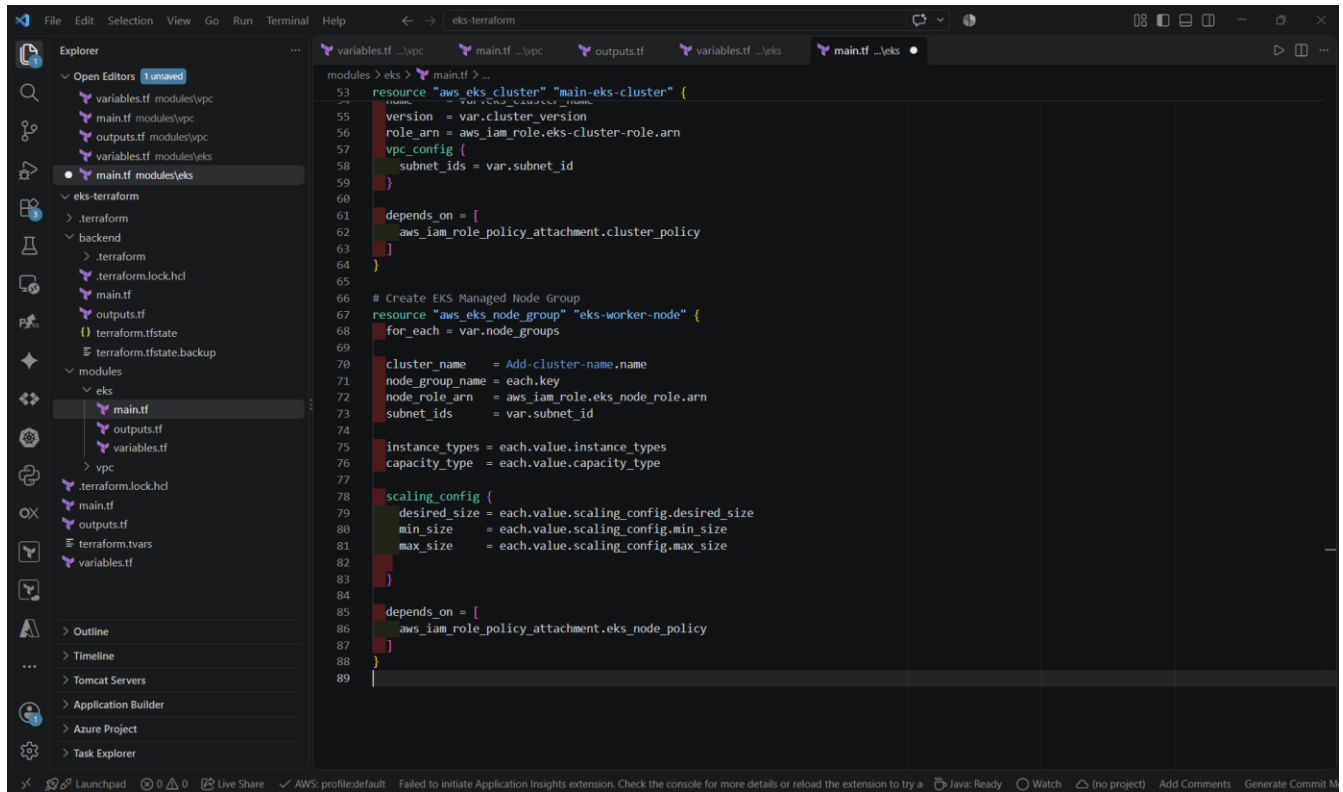
  instance_types = each.value.instance_types
  capacity_type  = each.value.capacity_type

  scaling_config {
    desired_size = each.value.scaling_config.desired_size
    min_size     = each.value.scaling_config.min_size
    max_size     = each.value.scaling_config.max_size
  }

  depends_on = [
    aws_iam_role_policy_attachment.eks_node_policy
  ]
}

```

Copy the code and paste in the “**main.tf**” file in the “**vpc**” folder.



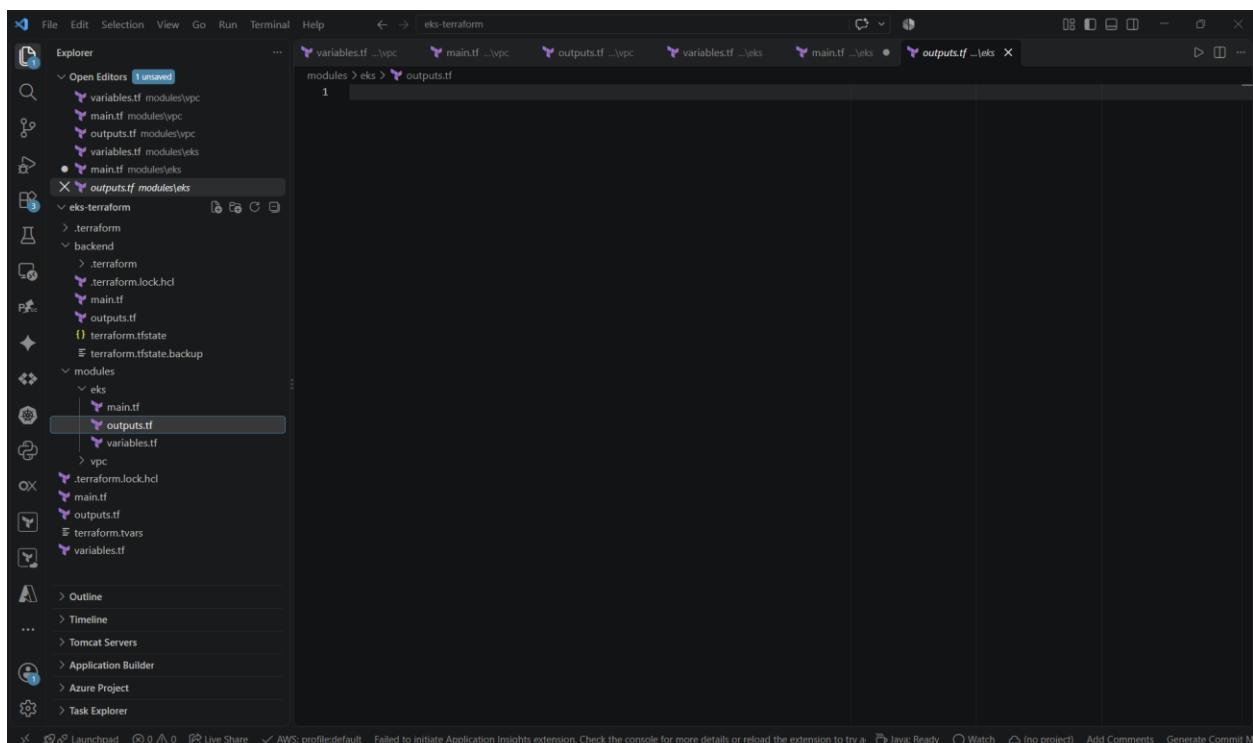
The screenshot shows the Visual Studio Code editor with the file explorer on the left and the main editor window displaying the `main.tf` file in the `eks` module. The file explorer shows the project structure with folders like `modules`, `eks`, and `vpc`. The `main.tf` file contains Terraform code for creating an EKS cluster and its components.

```
modules > eks > main.tf > ...
53 resource "aws_eks_cluster" "main-eks-cluster" {
54     name = var.eks_cluster_name
55     version = var.cluster_version
56     role_arn = aws_iam_role.eks-cluster-role.arn
57     vpc_config {
58         subnet_ids = var.subnet_id
59     }
60
61     depends_on = [
62         aws_iam_role_policy_attachment.cluster_policy
63     ]
64 }
65
66 # Create EKS Managed Node Group
67 resource "aws_eks_node_group" "eks-worker-node" {
68     for_each = var.node_groups
69
70     cluster_name = Add-cluster-name.name
71     node_group_name = each.key
72     node_role_arn = aws_iam_role.eks_node_role.arn
73     subnet_ids = var.subnet_id
74
75     instance_types = each.value.instance_types
76     capacity_type = each.value.capacity_type
77
78     scaling_config {
79         desired_size = each.value.scaling_config.desired_size
80         min_size = each.value.scaling_config.min_size
81         max_size = each.value.scaling_config.max_size
82     }
83 }
84
85     depends_on = [
86         aws_iam_role_policy_attachment.eks_node_policy
87     ]
88 }
89 }
```

We have completed the configuration of the EKS cluster and its components.

6.3. Setting the Outputs – “**outputs.tf**”

We will now configure the output in the file “**outputs.tf**” in the “**eks**” folder. Here, we just want to display the endpoint of the EKS cluster.



The screenshot shows the Visual Studio Code editor with the file explorer on the left and the main editor window displaying the `outputs.tf` file in the `eks` module. The file explorer shows the project structure with folders like `modules`, `eks`, and `vpc`. The `outputs.tf` file is currently empty, with only a line number '1' visible.

The resource name is **“cluster_endpoint”**. We will use the sample template:

```
output "cluster_endpoint" {
  description = "EKS cluster endpoint"
  value       = Add what you want to display
}
```

For the “description”, we will use “EKS cluster endpoint. The code will look like this:

```
output "cluster_endpoint" {
  description = "EKS cluster endpoint"
  value       = Add what you want to display
}
```

Let us add the argument for the **“value”**. Go to the resource to create EKS cluster. Copy the resource type and resource name:

```
"aws_eks_cluster" "main-eks-cluster"
```

Remove the quotes and add a dot between the resource type and resource name

```
aws_eks_cluster.main-eks-cluster
```

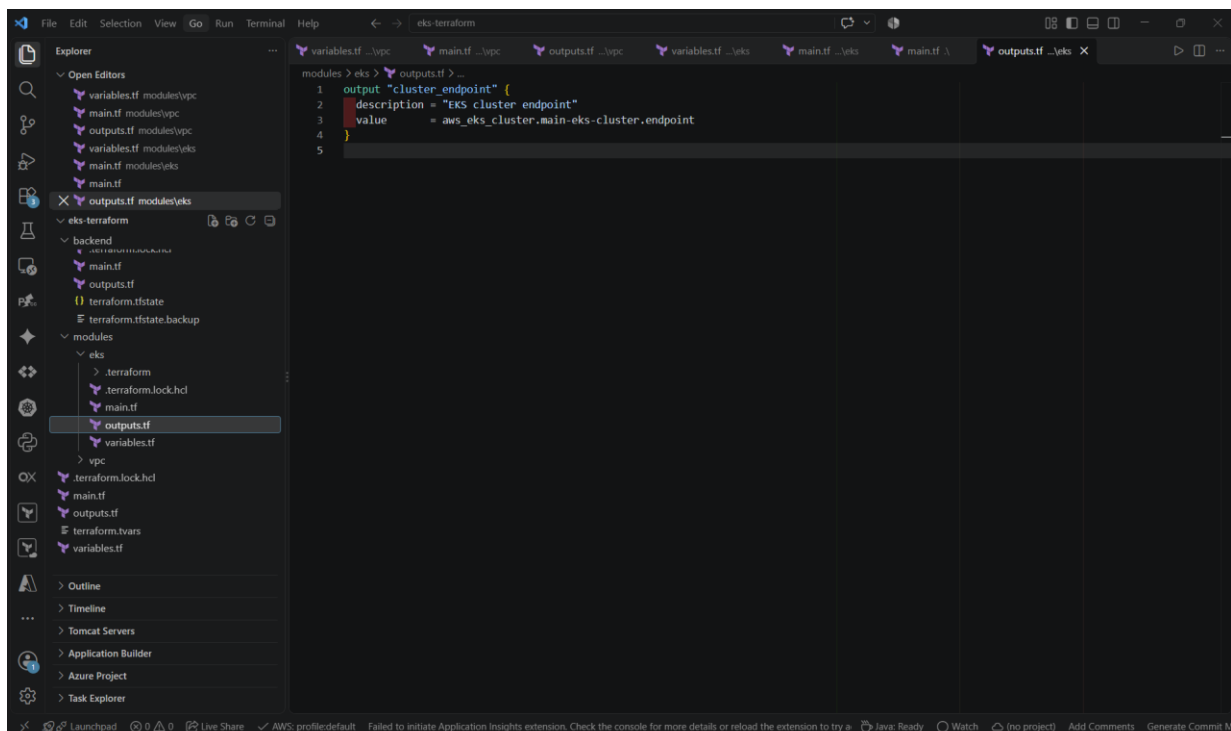
Add **“.endpoint”** at the end, It will become:

```
aws_eks_cluster.main-eks-cluster.endpoint
```

The completed code will be as follows:

```
output "cluster_endpoint" {
  description = "EKS cluster endpoint"
  value       = aws_eks_cluster.main-eks-cluster.endpoint
}
```

Copy this block and paste in the **“outputs.tf”** in the **“eks”** folder.



We have completed the configuration of the EKS cluster and its components.

7. How to Call and Use Terraform Modules from the Root Directory

In this part, we will start calling the modules. We have two modules, namely “**eks**” and “**vpc**”. We need to call the modules so that the resources can be provisioned for us. We will do the calls from the “**main.tf**” file in the root directory.

7.1. Defining Input Variables in the Root “variables.tf” file

Before starting the call, we have to first of all add the variables. The variables we are going to add here are not different from those we added in the “**variables.tf**” files in “**eks**” and “**vpc**” folders. It is just like we are combining the variables in the two “**variables.tf**” files in one.

It will look like this:

```
variable "region" {
  description = "AWS region"
  type        = string
}

variable "vpc_cidr" {
  description = "VPC CIDR block"
  type        = string
}

variable "availability_zones" {
  description = "Availability zones"
  type        = list(string)
}

variable "public_subnet_cidr" {
  description = "Public subnet CIDRs"
  type        = list(string)
}

variable "private_subnet_cidr" {
  description = "Private subnet CIDRs"
  type        = list(string)
}

variable "eks_cluster_name" {
  description = "EKS cluster name"
  type        = string
}

variable "cluster_version" {
  description = "EKS Kubernetes version"
  type        = string
}

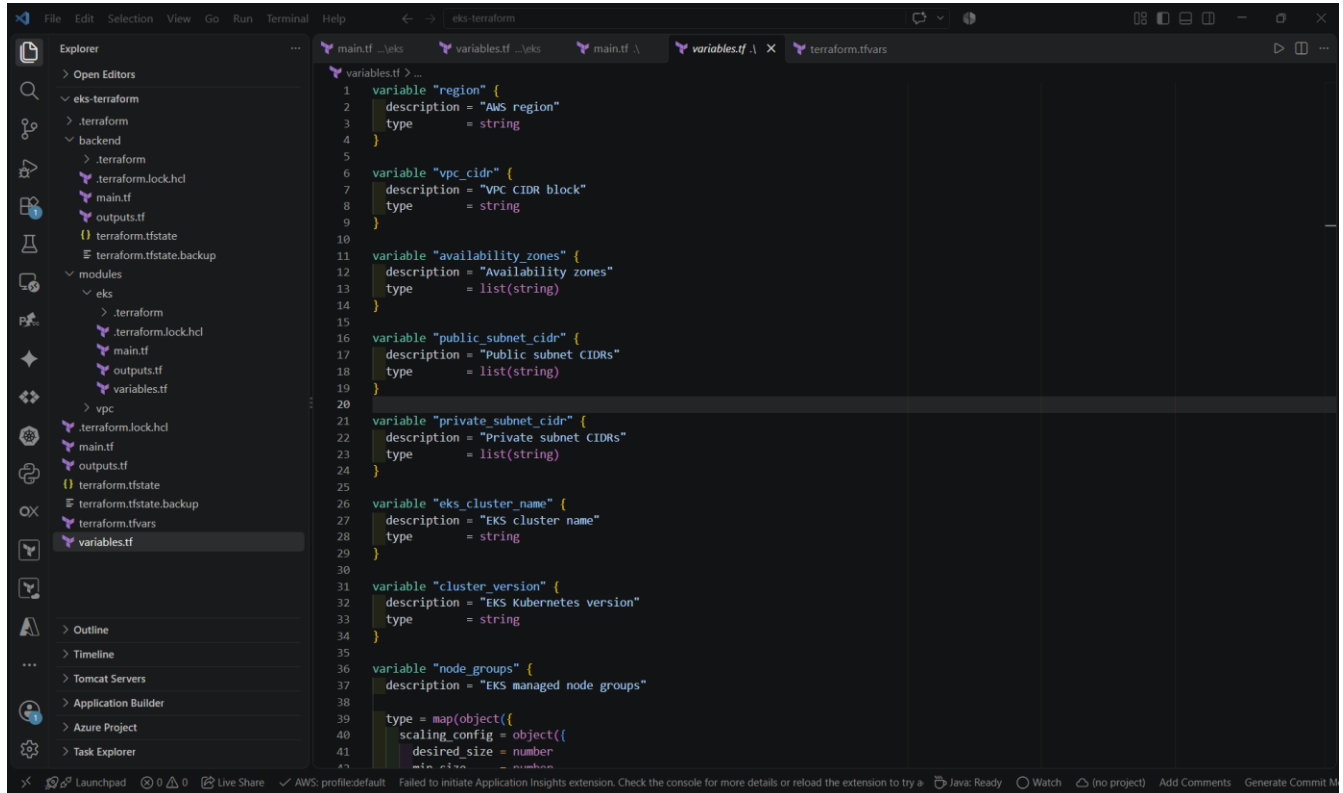
variable "node_groups" {
  description = "EKS managed node groups"

  type = map(object({
    scaling_config = object({
      desired_size = number
      min_size     = number
      max_size     = number
    })
  })
}
```

```
    })

    instance_types = list(string)
    capacity_type = string
  })
}
```

Copy this and paste in the “**variables.tf**” file in the root directory



We have completed with this part.

7.2. Configuring the Terraform Infrastructure with “main.tf”

We will now configure the “**main.tf**” file in our root directory. We will put the “**main.tf**” files in the “**eks**” and “**vpc**” together here.

Firstly, go to the “**main.tf**” file in the “**backend**”, copy it.

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "5.97.0"
    }
  }
}

provider "aws" {
  region = "us-east-2"
}
```

```

resource "aws_s3_bucket" "terraform_state" {
  bucket = "eks-install-bucket7724"

  lifecycle {
    prevent_destroy = false
  }
}

terraform {
  backend "s3" {
    bucket      = "eks-install-bucket7724"
    key         = "dev/terraform-state-file"
    use_lockfile = true
    region      = "us-east-2"
  }
}

```

Now, let us add the part to call the two modules, that is “**vpc**” and “**eks**”. We have to copy all the variables and assign their values using reference “**var.**”. It will look like:

```

module "vpc" {
  source = "../modules/vpc"

  region              = var.region
  vpc_cidr            = var.vpc_cidr
  availability_zones  = var.availability_zones # List of availability zones to distribute
subnets across
  private_subnet_cidr = var.private_subnet_cidr # CIDR blocks for private subnets
  public_subnet_cidr  = var.public_subnet_cidr  # CIDR blocks for public subnets
  eks_cluster_name    = var.eks_cluster_name    # Optional: used inside VPC module for
tagging or naming
}

module "eks" {
  source = "../modules/eks"

  region              = var.region # Name of the default region
  eks_cluster_name    = var.eks_cluster_name # Name of the EKS cluster to create
  cluster_version     = var.cluster_version # EKS version for the EKS control plane
  vpc_id              = module.vpc.vpc_id    # Use VPC ID output from the VPC module
  subnet_id           = module.vpc.private_subnet_ids # Use private subnet IDs from the VPC
module
  node_groups         = var.node_groups      # Map of node group configurations to
launch worker nodes
}

```

The completed code will become:

```

terraform {
  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "5.97.0"
    }
  }
}

provider "aws" {
  region = "us-east-2"
}

```

```
resource "aws_s3_bucket" "terraform_state" {
  bucket = "eks-install-bucket7724"

  lifecycle {
    prevent_destroy = false
  }
}

terraform {
  backend "s3" {
    bucket = "eks-install-bucket7724"
    key    = "dev/terraform-state-file"
    use_lockfile = true
    region = "us-east-2"
  }
}

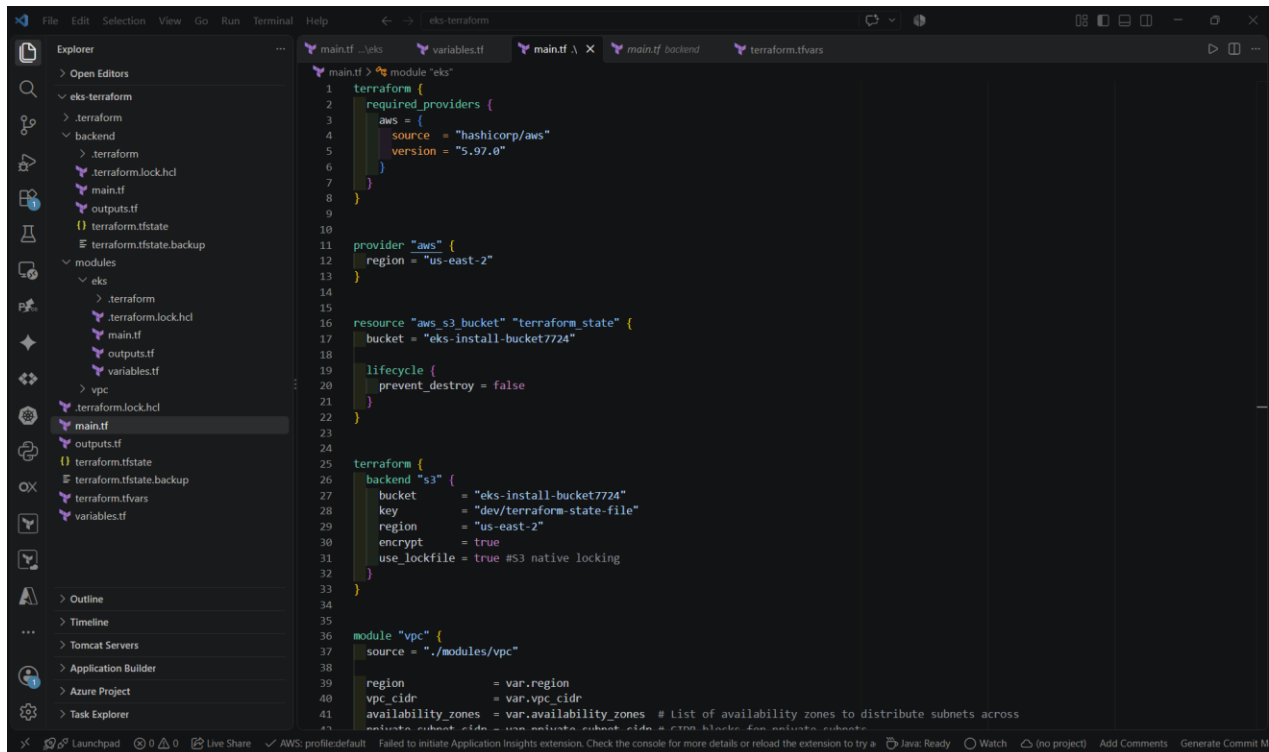
module "vpc" {
  source = "../modules/vpc"

  region          = var.region
  vpc_cidr        = var.vpc_cidr
  availability_zones = var.availability_zones # List of availability zones to distribute
subnets across
  private_subnet_cidr = var.private_subnet_cidr # CIDR blocks for private subnets
  public_subnet_cidr  = var.public_subnet_cidr  # CIDR blocks for public subnets
  eks_cluster_name    = var.eks_cluster_name    # Optional: used inside VPC module for
tagging or naming
}

module "eks" {
  source = "../modules/eks"

  region          = var.region # Name of the default region
  eks_cluster_name = var.eks_cluster_name # Name of the EKS cluster to create
  cluster_version = var.cluster_version # Kubernetes version for the EKS control
plane
  vpc_id          = module.vpc.vpc_id # Use VPC ID output from the VPC module
  subnet_id       = module.vpc.private_subnet_ids # Use private subnet IDs from the VPC
module
  node_groups     = var.node_groups # Map of node group configurations to
launch worker nodes
}
```

Copy this and paste on the “**main.tf**” file in the root directory.



We have configured the “**main.tf**” file.

7.3. Configuring the Input Variables with “**terraform.tfvars**” file

This is where we will add the values, we intend to assign to all the input variables. In this file, we will provide values for the variables defined in “**variables.tf**”.

The code will look like this:

```
# =====
# AWS REGION
# =====

region = "us-east-2"

# =====
# VPC CONFIGURATION
# =====

vpc_cidr = "10.0.0.0/16"

availability_zones = [
    "us-east-2a",
    "us-east-2b",
    "us-east-2c"
]

# =====
# PUBLIC SUBNETS
# =====

public_subnet_cidr = [
```

```
"10.0.1.0/24",
"10.0.2.0/24",
"10.0.3.0/24"
]

# =====
# PRIVATE SUBNETS
# =====

private_subnet_cidr = [
    "10.0.11.0/24",
    "10.0.12.0/24",
    "10.0.13.0/24"
]

# =====
# EKS CLUSTER
# =====

eks_cluster_name = "my-eks-cluster"

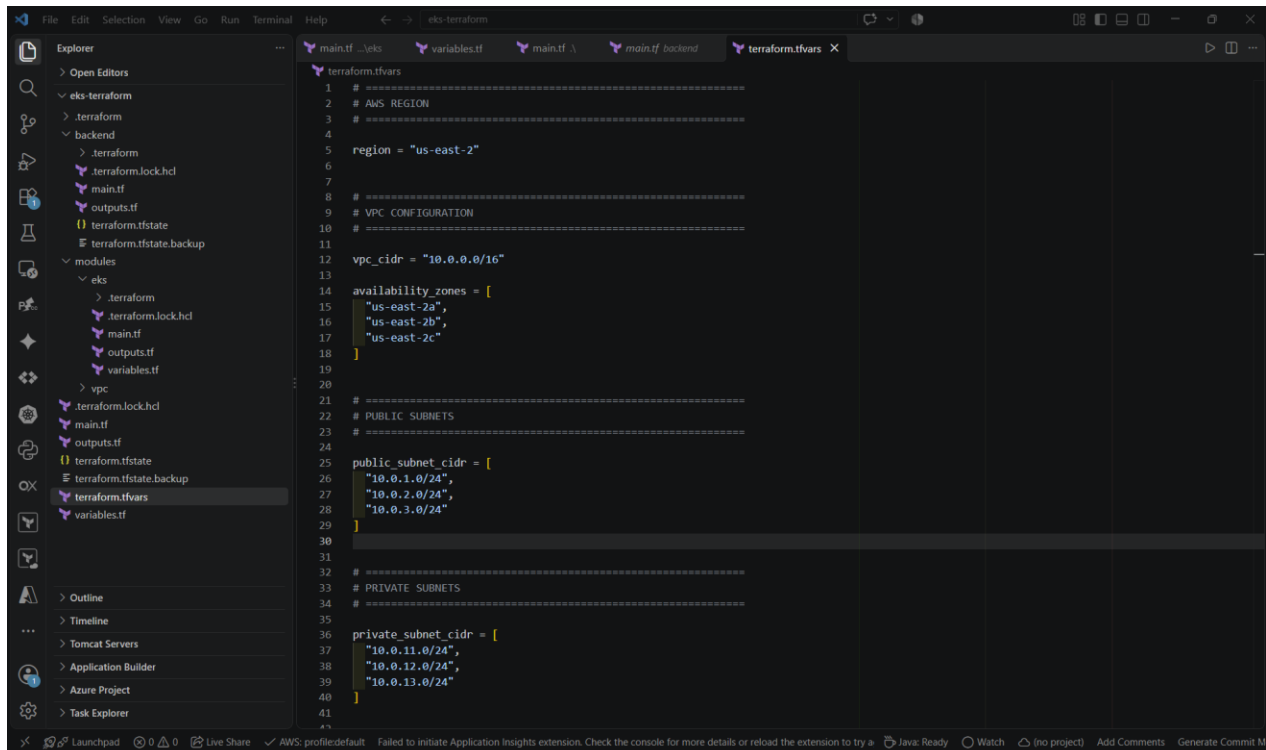
cluster_version = "1.34"

# =====
# EKS NODE GROUPS
# =====

node_groups = {
    default = {
        instance_types = ["t3.medium"]
        capacity_type   = "ON_DEMAND"

        scaling_config = {
            desired_size = 2
            max_size     = 3
            min_size     = 1
        }
    }
}
```

Copy this code and paste in the “**terraform.tfvars**” file in the root directory.



We have completed adding the values we intend to assign to our input variables.

7.4. Defining Module Outputs in the Root directory – “outputs.tf” file

The last thing to define the output values from our configuration in the “**outputs.tf**” file in the root directory. This file can expose values returned by those modules using the module reference:

```
module.<module_name>.<output_name>
```

Here, we will use the reference “**module.**” Since we are calling the modules.

We will add two blocks and the code will look like this:

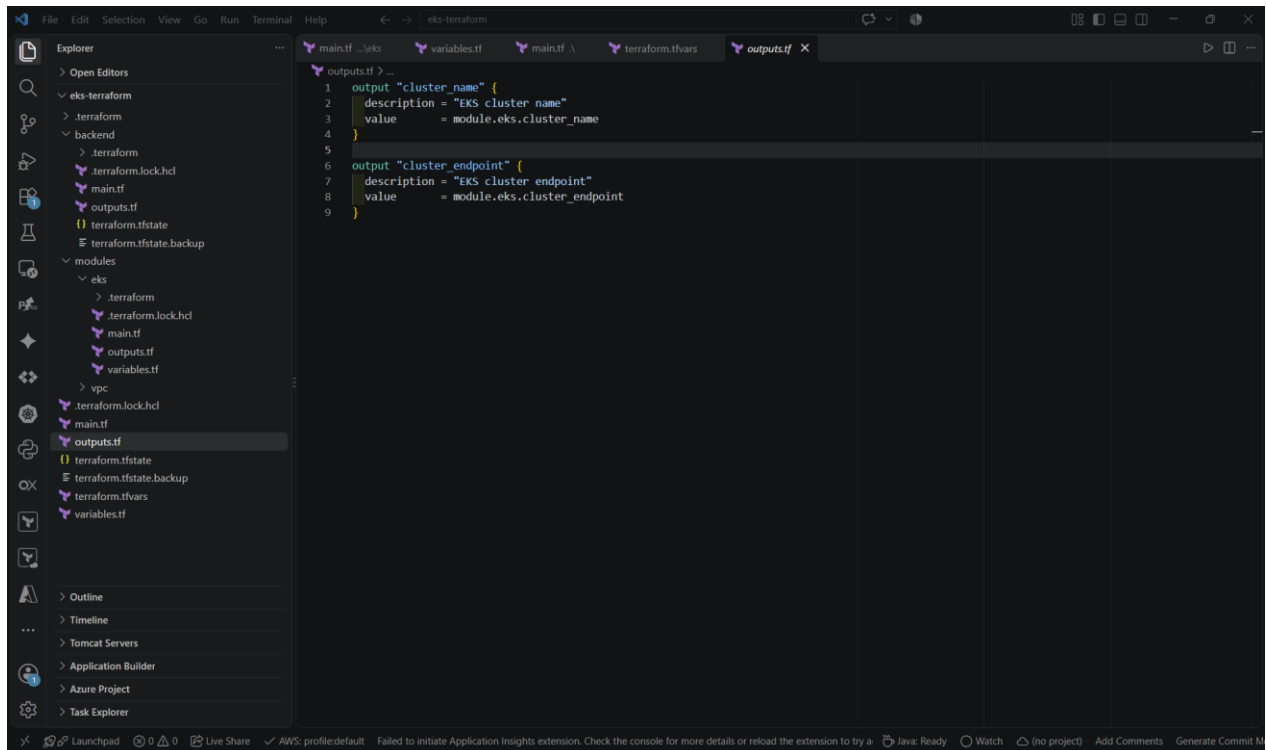
```

output "cluster_name" {
    description = "EKS cluster name"
    value      = module.eks.cluster_name
}

output "cluster_endpoint" {
    description = "EKS cluster endpoint"
    value      = module.eks.cluster_endpoint
}

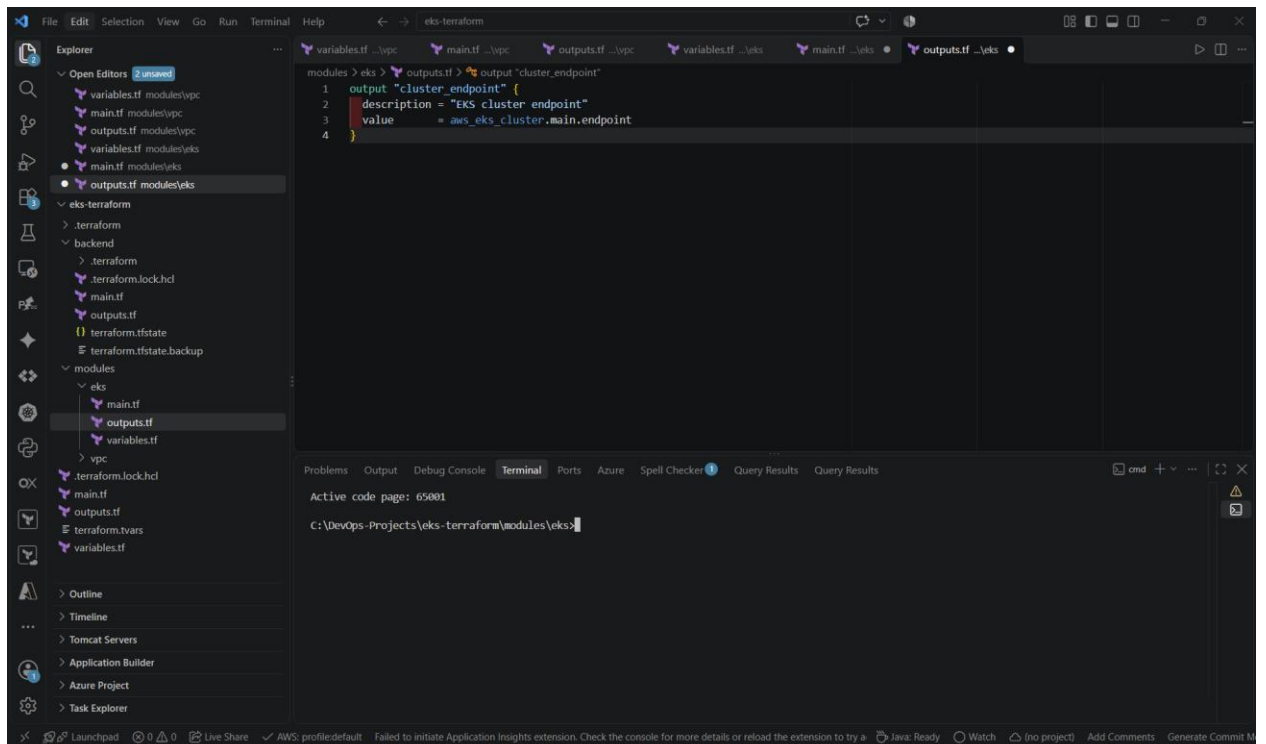
```

Copy this and paste in the “**outputs.tf**” file in the root directory.



8. Deploy an Amazon EKS Cluster with Terraform

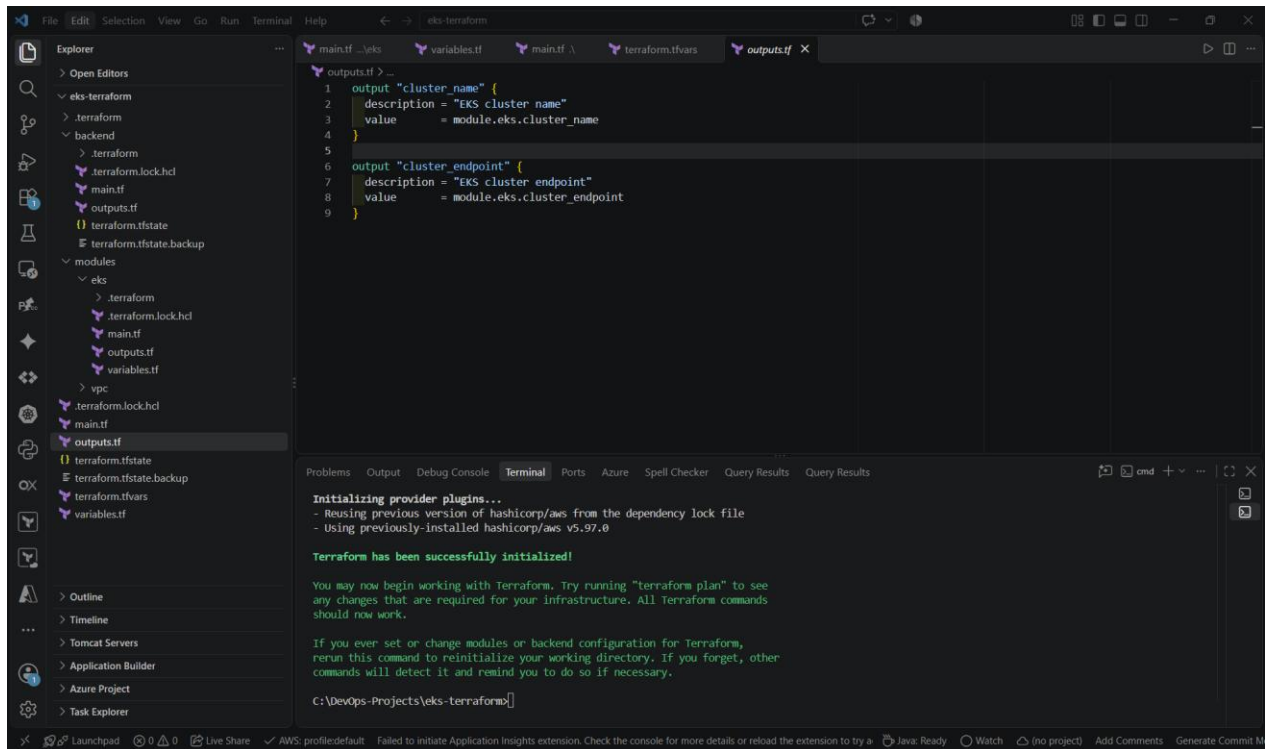
We will validate our configuration in this part. Click on the empty space and select “open integrated terminal”.



8.1. Terraform Init

We have to initialize our module. Let us run the command to initialize the Terraform working directory so Terraform can prepare to create and manage your infrastructure:

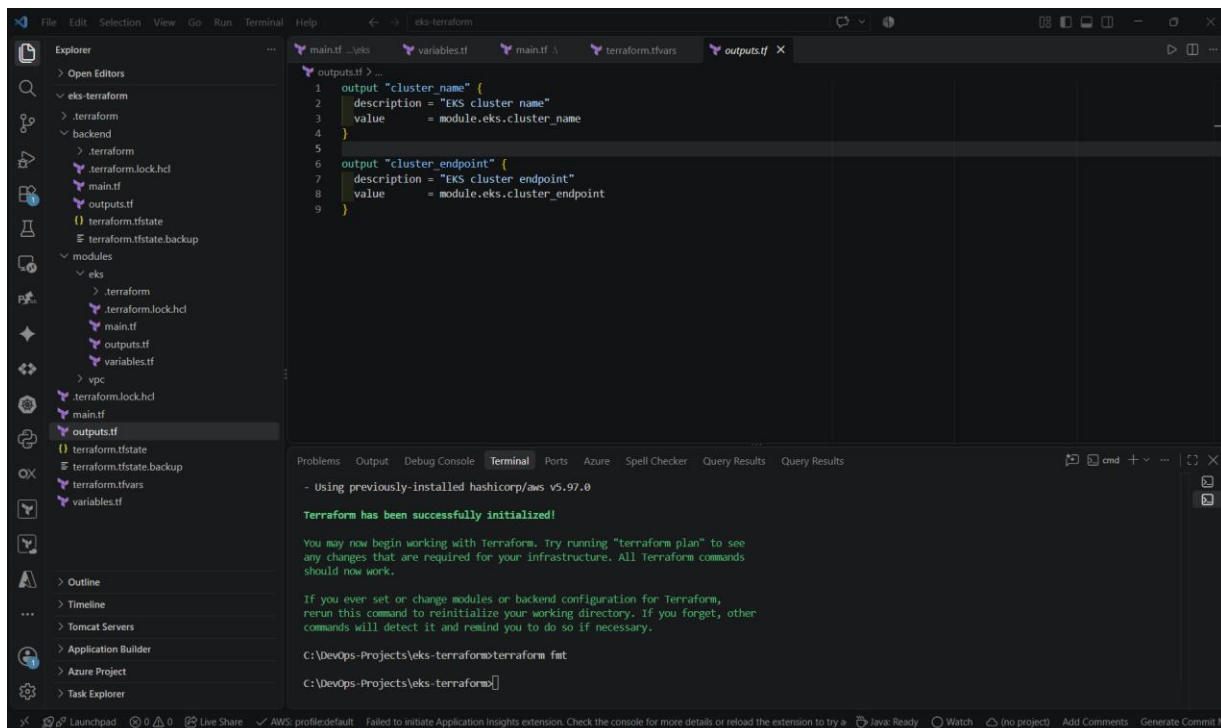
terraform init



8.2. Terraform Format

Let us run the command to format the Terraform configuration files so they follow Terraform's standard formatting style:

terraform fmt

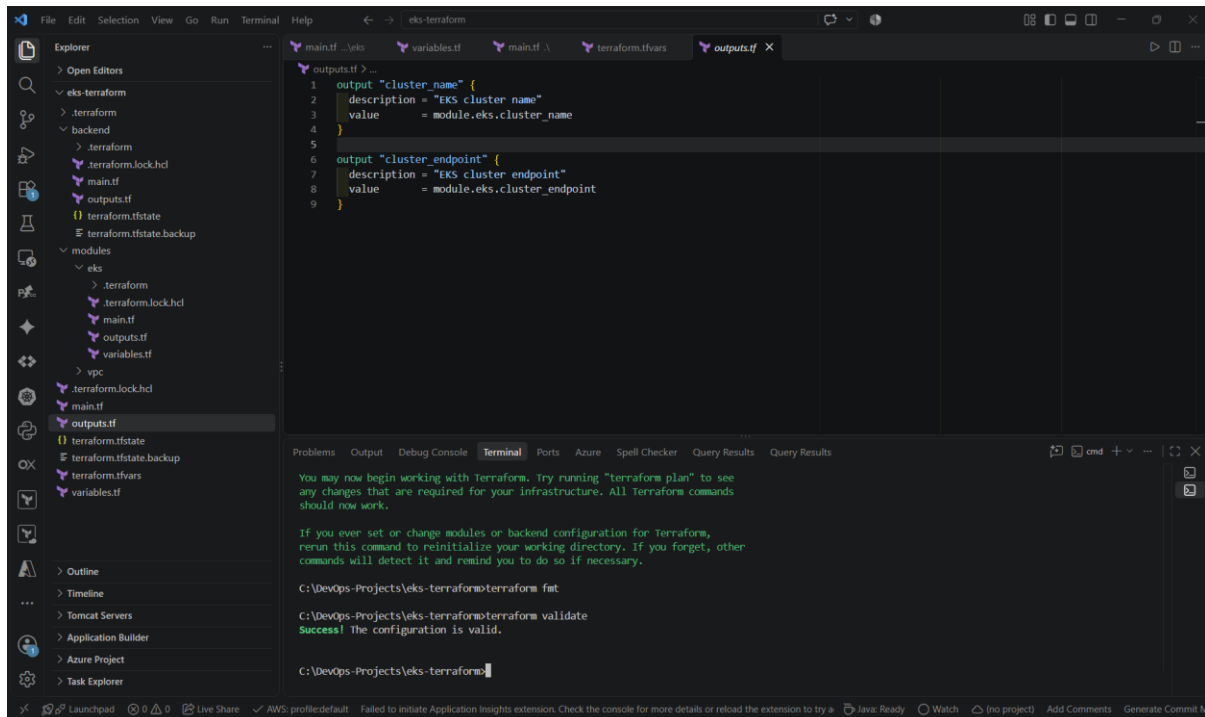


So, our code has been formatted correctly.

8.3. Terraform Validate

Let us run the command to check whether your Terraform configuration is syntactically correct and internally valid before you try to create or modify infrastructure:

```
terraform validate
```

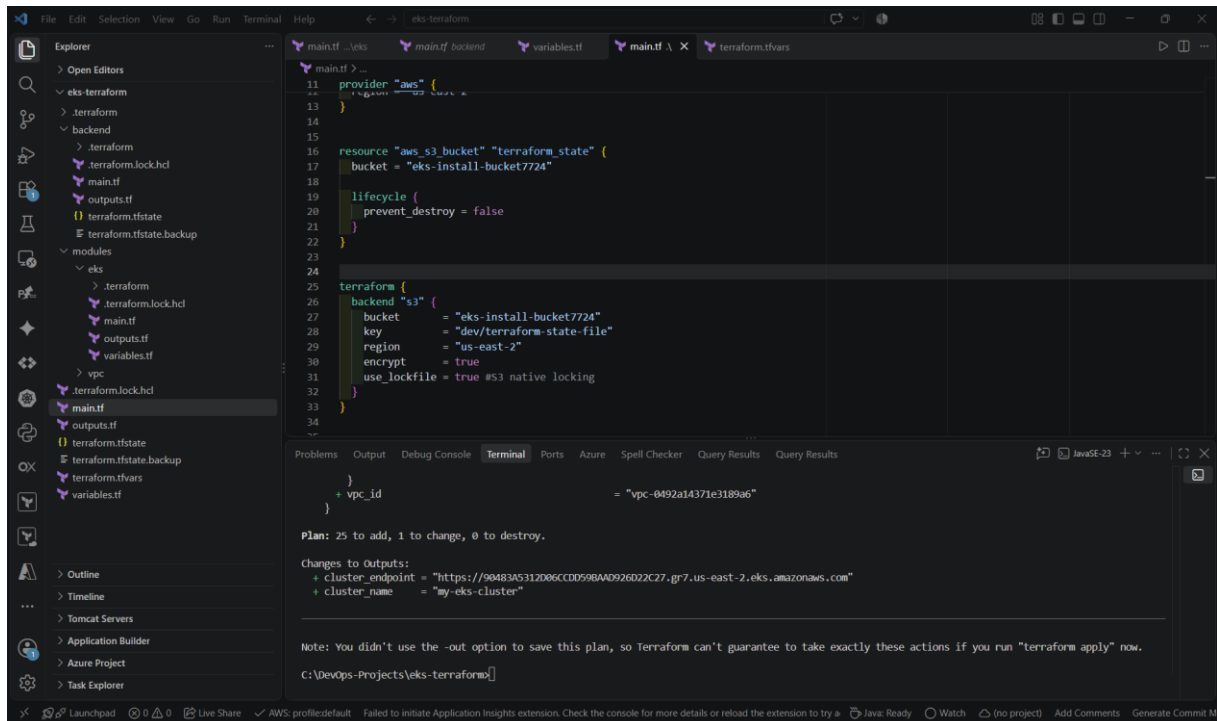


The validation is successful.

8.4. Terraform Plan

Let us run the command to show what Terraform intends to change in your infrastructure without actually making those changes.

```
terraform plan
```



```
11 provider "aws" {
12   region = "us-east-2"
13 }
14
15
16 resource "aws_s3_bucket" "terraform_state" {
17   bucket = "eks-install-bucket7724"
18
19   lifecycle {
20     prevent_destroy = false
21   }
22 }
23
24
25 terraform {
26   backend "s3" {
27     bucket = "eks-install-bucket7724"
28     key    = "dev/terraform-state-file"
29     region = "us-east-2"
30     encrypt = true
31     use_lockfile = true #s3 native locking
32   }
33 }
34
```

Problems Output Debug Console Terminal Ports Azure Spell Checker Query Results Query Results

```
+ vpc_id
= "vpc-0492a14371e3189a6"

Plan: 25 to add, 1 to change, 0 to destroy.

Changes to Outputs:
+ cluster_endpoint = "https://90483A5312D86CCDD59BAAD926D22C27.gr7.us-east-2.eks.amazonaws.com"
+ cluster_name     = "my-eks-cluster"

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform apply" now.

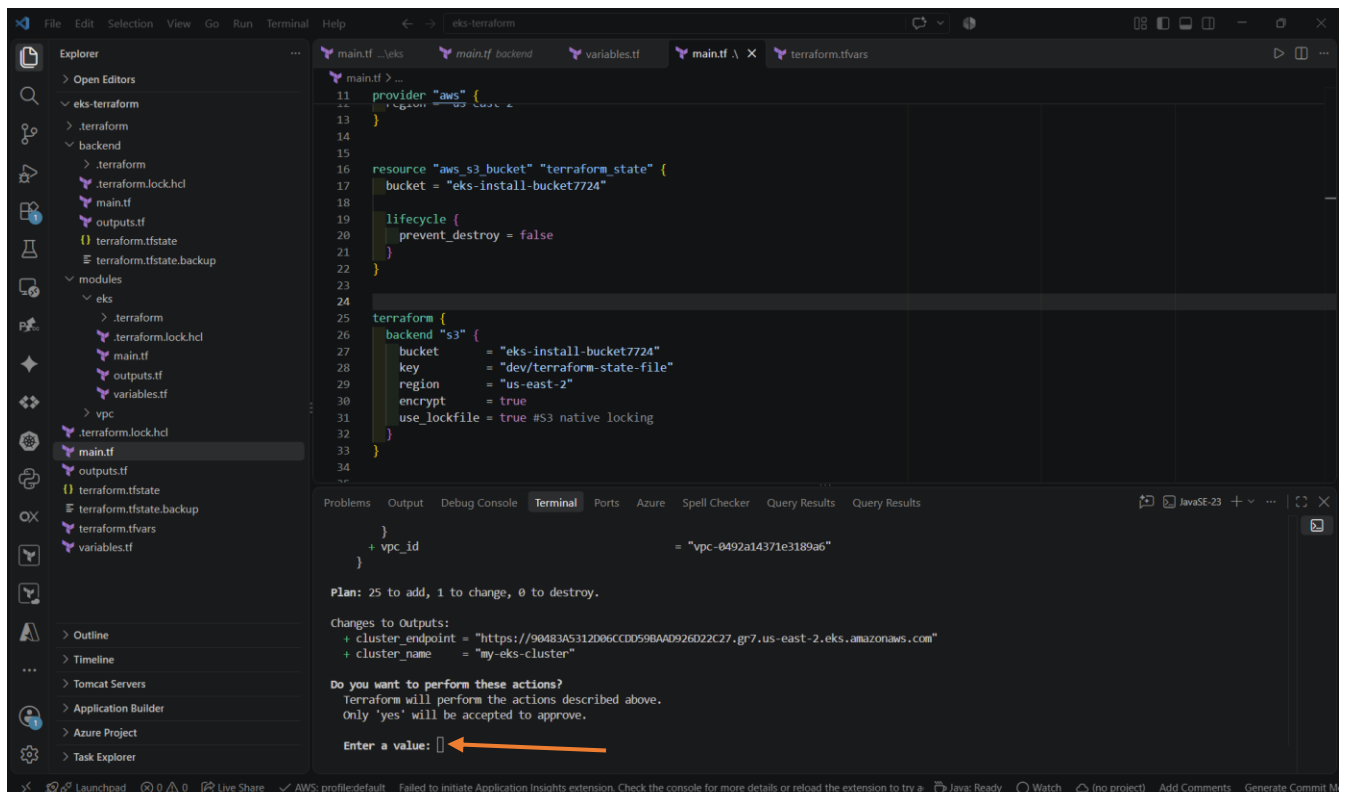
C:\DevOps-Projects\eks-terraform\
```

You can see our cluster endpoint and cluster name.

8.5. Terraform Apply

Let us now run the command that actually creates, changes, or deletes infrastructure according to your Terraform configuration:

terraform apply



```
11 provider "aws" {
12   region = "us-east-2"
13 }
14
15
16 resource "aws_s3_bucket" "terraform_state" {
17   bucket = "eks-install-bucket7724"
18
19   lifecycle {
20     prevent_destroy = false
21   }
22 }
23
24
25 terraform {
26   backend "s3" {
27     bucket = "eks-install-bucket7724"
28     key    = "dev/terraform-state-file"
29     region = "us-east-2"
30     encrypt = true
31     use_lockfile = true #s3 native locking
32   }
33 }
34
```

Problems Output Debug Console Terminal Ports Azure Spell Checker Query Results Query Results

```
+ vpc_id
= "vpc-0492a14371e3189a6"

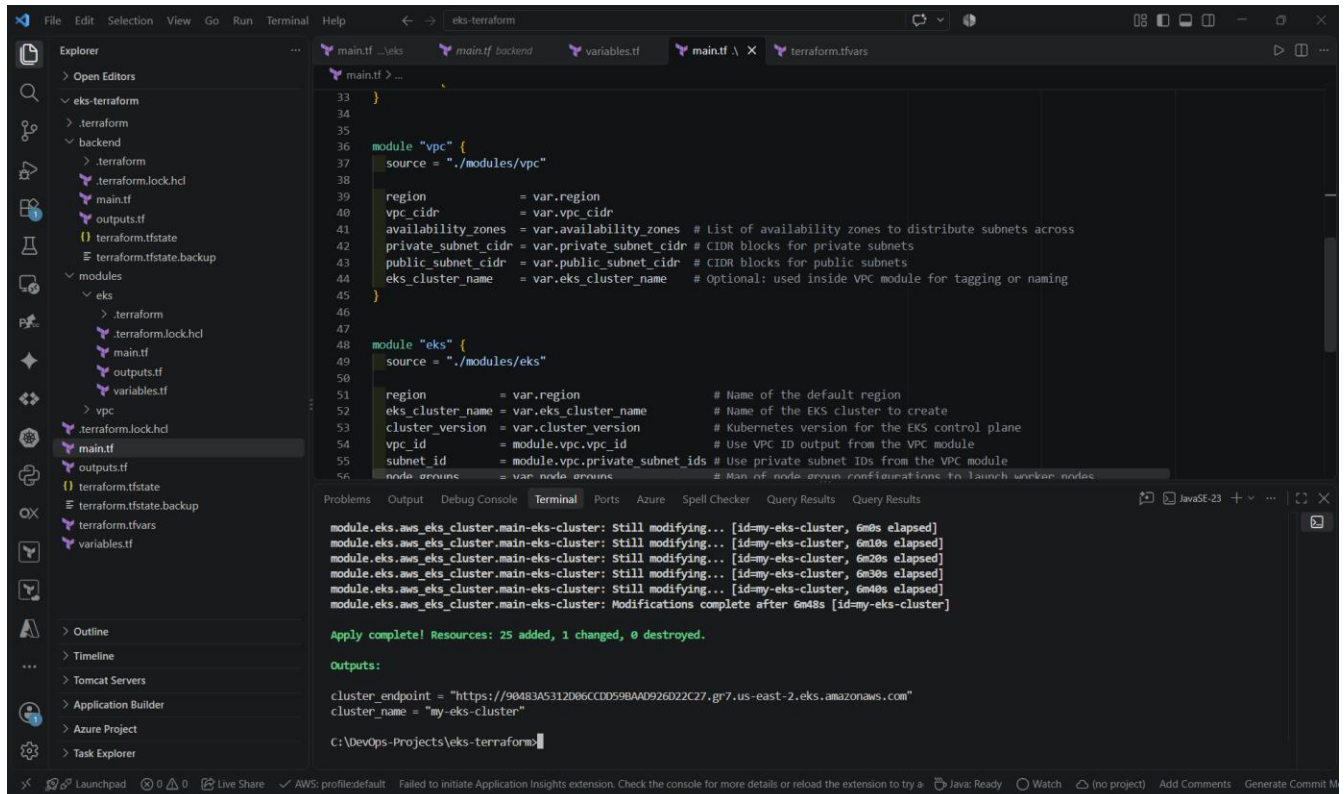
Plan: 25 to add, 1 to change, 0 to destroy.

Changes to Outputs:
+ cluster_endpoint = "https://90483A5312D86CCDD59BAAD926D22C27.gr7.us-east-2.eks.amazonaws.com"
+ cluster_name     = "my-eks-cluster"

Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: 
```

Type “yes” and press “Enter”



The screenshot shows a Visual Studio Code editor with a Terraform project. The Explorer sidebar on the left shows the project structure, including files like `main.tf`, `outputs.tf`, `terraform.tfstate`, and `modules`. The main editor displays the `main.tf` file, which contains two Terraform modules: `vpc` and `eks`. The `vpc` module is defined with variables for region, cidr, availability zones, and subnets. The `eks` module is defined with variables for region, cluster name, version, and subnets. The terminal at the bottom shows the output of the Terraform apply command, indicating that the resources were successfully provisioned. The output includes the following text:

```
module.eks.aws_eks_cluster.main-eks-cluster: Still modifying... [id=my-eks-cluster, 6m0s elapsed]
module.eks.aws_eks_cluster.main-eks-cluster: Still modifying... [id=my-eks-cluster, 6m10s elapsed]
module.eks.aws_eks_cluster.main-eks-cluster: Still modifying... [id=my-eks-cluster, 6m20s elapsed]
module.eks.aws_eks_cluster.main-eks-cluster: Still modifying... [id=my-eks-cluster, 6m30s elapsed]
module.eks.aws_eks_cluster.main-eks-cluster: Still modifying... [id=my-eks-cluster, 6m40s elapsed]
module.eks.aws_eks_cluster.main-eks-cluster: Modifications complete after 6m48s [id=my-eks-cluster]

Apply complete! Resources: 25 added, 1 changed, 0 destroyed.

Outputs:

cluster_endpoint = "https://904834531206CCDD59BA0926D22C27.gr7.us-east-2.eks.amazonaws.com"
cluster_name = "my-eks-cluster"
```

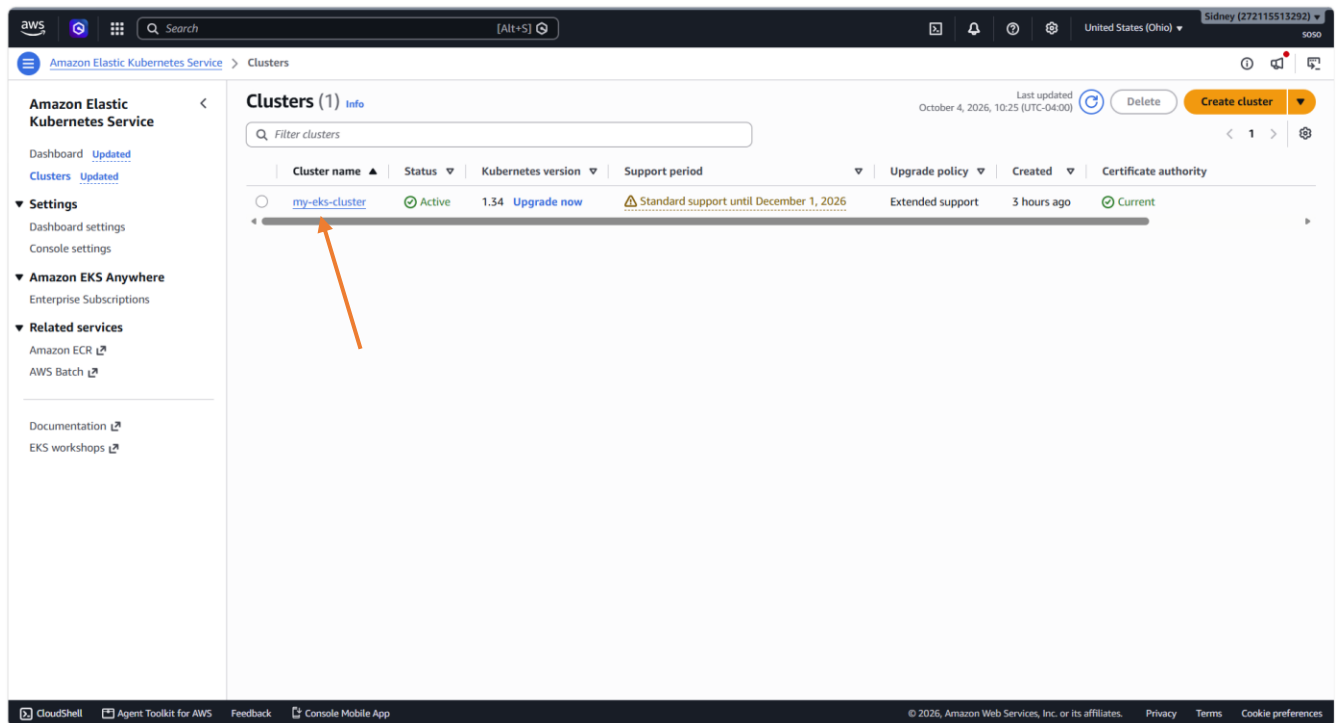
We have successfully provisioned our EKS cluster with the associated components and the worker nodes.

9. How to Access and Connect to Your AWS EKS Cluster

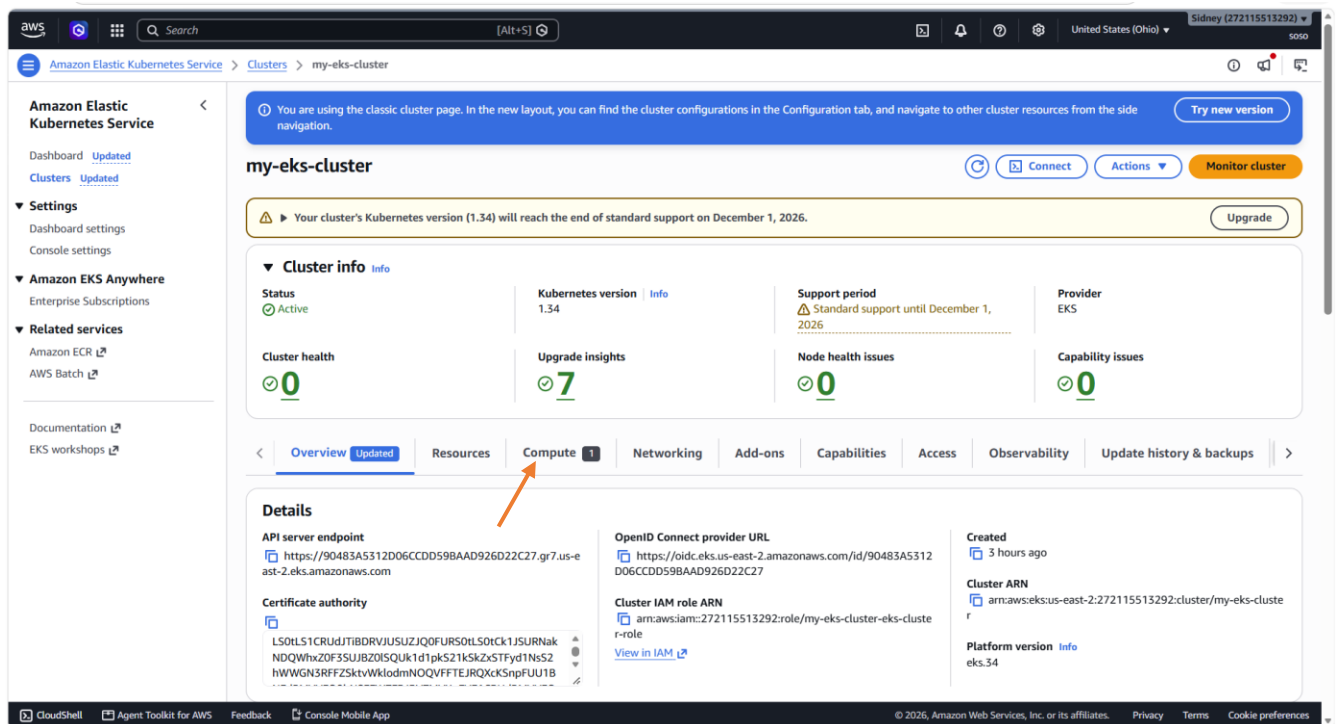
In this part, we will be accessing and connecting to the EKS cluster we have created. To do this, we have to go to the AWS Management console.

9.1. Access the EKS Cluster created

Let us access the EKS cluster created from Terraform. Go to AWS Management console



You can see the EKS cluster created. Click on the cluster



Click on “Compute” tab

The screenshot shows the Amazon Elastic Kubernetes Service (EKS) console for a cluster named 'my-eks-cluster'. The left sidebar contains navigation links for the Amazon Elastic Kubernetes Service, Settings, Amazon EKS Anywhere, and Related services. The main content area displays cluster information, including Status (Active), Kubernetes version (1.34), Support period (Standard support until December 1, 2026), and Provider (EKS). Below this, there are tabs for Overview, Resources, Compute, Networking, Add-ons, Capabilities, Access, Observability, and Update history & backups. The 'Compute' tab is selected, showing a table of nodes. An orange arrow points to the first node in the table.

Node name	Instance type	Capacity type	Compute	Managed by	Created	Status	CPU usage	Memory usage
ip-10-0-11-205.us-east-2.compute.internal	t3.medium	On-Demand	Node group	default	2 hours ago	Ready	22%	18%
ip-10-0-13-123.us-east-2.compute.internal	t3.medium	On-Demand	Node group	default	2 hours ago	Ready	12%	14%

You can see that we have two nodes. Scroll down

The screenshot shows the 'Node groups' section of the Amazon EKS console. It displays a table with one node group named 'default'. An orange arrow points to the 'Desired size' column, which has a value of 2. Below the table, there is a section for 'Fargate profiles' which is currently empty.

Group name	Desired size	AMI release version	Launch template	Status
default	2	1.33.13-20260930 Update now	-	Active

You can see our desired size is “2” as we stated in the “**terraform.tfvars**” file.

9.2. Connect to the EKS cluster

We will now connect to the EKS cluster. We will do this using PowerShell. Open PowerShell

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\ebots>
```

Check if AWS CLI has been installed by running the command:

```
aws --version
```

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\ebots> aws --version
aws-cli/2.28.6 Python/3.13.4 Windows/11 exe/AMD64
PS C:\Users\ebots>
```

You can see that AWS CLI has been installed. Let us check if kubectl has been installed by using the command:

```
kubectl version --client
```

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\ebots> aws --version
aws-cli/2.28.6 Python/3.13.4 Windows/11 exe/AMD64
PS C:\Users\ebots> kubectl version --client
Client Version: v1.30.0
Kustomize Version: v5.0.4-0.20230601165947-6ce0bf390ce3
PS C:\Users\ebots>
```

You can see that kubectl has been installed.

Let us run the command to connect your AWS CLI to an Amazon EKS cluster so that kubectl can manage that cluster:

```
aws eks --region <EKS_Region> update-kubeconfig --name <EKS_Cluster_Name>
aws eks --region us-east-2 update-kubeconfig --name my-eks-cluster
```

AWS retrieves information about my-eks-cluster and updates your local Kubernetes configuration file **“.kube/config”**.

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\ebots> aws --version
aws-cli/2.28.6 Python/3.13.4 Windows/11 exe/AMD64
PS C:\Users\ebots> kubectl version --client
Client Version: v1.30.0
Kustomize Version: v5.0.4-0.20230601165947-6ce0bf390ce3
PS C:\Users\ebots> aws eks --region us-east-2 update-kubeconfig --name my-eks-cluster
Added new context arn:aws:eks:us-east-2:272115513292:cluster/my-eks-cluster to C:\Users\ebots\.kube\config
PS C:\Users\ebots>
```

We have added a context and created a .kube/config file. So, it has created a new folder called **“.kube”**.

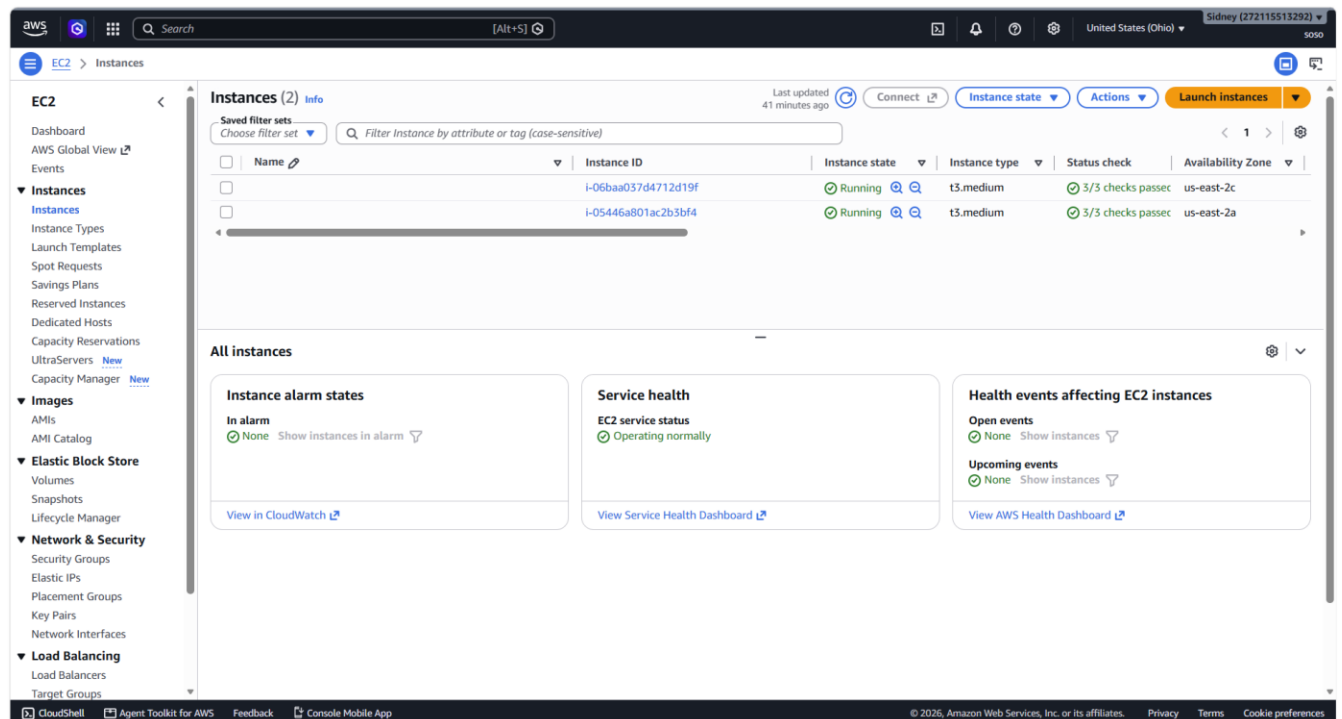
Let us run the command that lists the worker nodes in your cluster:

```
kubectl get nodes
```

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\ebots> aws --version
aws-cli/2.28.6 Python/3.13.4 Windows/11 exe/AMD64
PS C:\Users\ebots> kubectl version --client
Client Version: v1.30.0
Kustomize Version: v5.0.4-0.20230601165947-6ce0bf390ce3
PS C:\Users\ebots> aws eks --region us-east-2 update-kubeconfig --name my-eks-cluster
Added new context arn:aws:eks:us-east-2:272115513292:cluster/my-eks-cluster to C:\Users\ebots\.kube\config
PS C:\Users\ebots> kubectl get nodes
NAME                                STATUS    ROLES    AGE    VERSION
ip-10-0-11-205.us-east-2.compute.internal Ready    <none>    143m   v1.33.13-eks-3b4a6ca
ip-10-0-13-123.us-east-2.compute.internal Ready    <none>    143m   v1.33.13-eks-3b4a6ca
PS C:\Users\ebots>
```

You can see we have two worker nodes. These are the two nodes we provisioned. Let us check them in the EC2 instance dashboard.



You can see the two nodes running.

Let us try to run the command to list the Pods running in your current Kubernetes namespace:

```
kubectl get pods
```

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\ebots> aws --version
aws-cli/2.28.6 Python/3.13.4 Windows/11 exe/AMD64
PS C:\Users\ebots> kubectl version --client
Client Version: v1.30.0
Kustomize Version: v5.0.4-0.20230601165947-6ce0bf390ce3
PS C:\Users\ebots> aws eks --region us-east-2 update-kubeconfig --name my-eks-cluster
Added new context arn:aws:eks:us-east-2:272115513292:cluster/my-eks-cluster to C:\Users\ebots\.kube\config
PS C:\Users\ebots> kubectl get nodes
NAME                                STATUS    ROLES    AGE     VERSION
ip-10-0-11-205.us-east-2.compute.internal Ready    <none>    143m    v1.33.13-eks-3b4a6ca
ip-10-0-13-123.us-east-2.compute.internal Ready    <none>    143m    v1.33.13-eks-3b4a6ca
PS C:\Users\ebots> kubectl get pods
No resources found in default namespace.
PS C:\Users\ebots> |
```

The result is as expected because we have no created any pods.

Let us run the command to check whether your kubectl client can connect to a Kubernetes cluster and to display the addresses of important cluster services:

`kubectl cluster-info`

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\ebots> aws --version
aws-cli/2.28.6 Python/3.13.4 Windows/11 exe/AMD64
PS C:\Users\ebots> kubectl version --client
Client Version: v1.30.0
Kustomize Version: v5.0.4-0.20230601165947-6ce0bf390ce3
PS C:\Users\ebots> aws eks --region us-east-2 update-kubeconfig --name my-eks-cluster
Added new context arn:aws:eks:us-east-2:272115513292:cluster/my-eks-cluster to C:\Users\ebots\.kube\config
PS C:\Users\ebots> kubectl get nodes
NAME                                STATUS    ROLES    AGE     VERSION
ip-10-0-11-205.us-east-2.compute.internal Ready    <none>    143m    v1.33.13-eks-3b4a6ca
ip-10-0-13-123.us-east-2.compute.internal Ready    <none>    143m    v1.33.13-eks-3b4a6ca
PS C:\Users\ebots> kubectl get pods
No resources found in default namespace.
PS C:\Users\ebots> kubectl cluster-info
Kubernetes control plane is running at https://90483A5312D06CCDD59BAAD926D22C27.gr7.us-east-2.eks.amazonaws.com
CoreDNS is running at https://90483A5312D06CCDD59BAAD926D22C27.gr7.us-east-2.eks.amazonaws.com/api/v1/namespaces/kube-system/services/kube-dns:dns/proxy

To further debug and diagnose cluster problems, use 'kubectl cluster-info dump'.
PS C:\Users\ebots>
```